

Python Programlama

Kavramlar ve Algoritma Mantığı

Elektronik Yük. Müh.

İlhan ÖZKAN

İkinci Sürüm, Ağustos 2026, Ankara

Python Programlama

Yazar: Elektronik Yük. Müh. İlhan ÖZKAN

© Mart 2026, İlhan ÖZKAN. Tüm hakları saklıdır.

Bu kitap kâr amacı güdülmeden hazırlanmıştır ve **referans gösterilerek** her yerde kullanılabilir. Python programlama dilinin eğitimini veren ve alan tüm kişiler için serbestçe kullanılabilir.

Bu kitabın bir kısmı veya tamamı, **yazarın önceden yazılı izni alınarak** elektronik, mekanik, fotokopi veya herhangi bir kayıt sistemi ile çoğaltılabilir, yayımlanabilir veya depolanabilir.

ISBN: 978-625-92164-2-3

Baskı Tarihi: Eylül 2026

Baskı Yeri: Ağustos, Türkiye

Önsöz

Ülkemizde her alanda olduğu gibi yazılım alanında da bir terminoloji karmaşasının yaşandığı yazılan kitaplardan anlaşılmaktadır. Son dönemde yazılan kitaplar, adeta çeviri yapan programların bir gecede ürettiği çıktılar birleştirilerek hazırlanmaktadır. Bu kitaplar incelendiğinde, bilindiği varsayılan kavramların bile tanımlanırken birbirinin içerisine girdiği görülmektedir. 1989 yılından beri birçok programlama dili kullanarak yazılım geliştiriyor olmama rağmen, bu kitapların içeriğini anlama konusunda oldukça zorlanıyorum.

Yapısal programlamanın en çok uygulandığı Pascal dilidir. Pascal diline yakın zamanda ortaya çıkan C dili, kısa sürede sistem dili olarak tarihte yerini almıştır. Günümüzde sistem programlama ve yüksek hız gerektiren gömülü sistemlerde vazgeçilmez bir dil olmuştur.

Daha sonra C dilinden etkilenecek olan C++, Java, C# gibi birçok nesne yönelimli programlama dili ve yapısal programlama üzerine kurulmuş dillerdir. Birçok modern programlama diline esin kaynağı olan bu dilin özellikle bilgisayar mühendisleri tarafından bilinmesi gerekir.

Kitapta verilen konular çok sade olarak hazırlanmıştır. İşlenen kavramlar her bölümde üst üste konularak ilerlediğinden bir bölüm özümsemeden bir sonrakine geçilmemesi önerilir.

Yapısal programlama mantığını oluşturan kavramların programcıların kafasında aynı ışığı yakması için bu kitap hazırlanmıştır. Bu dilin eğitimini verecek tüm kişiler için serbestçe kullanılabilir.

Python'un sadeliği, okunabilirliği ve geniş kütüphane desteği, onu sadece öğrenilmesi kolay değil; aynı zamanda güçlü ve üretken bir dil yapar. Bu kitabın sonunda, kendi projelerinizi geliştirebilecek ve Python'un sağladığı zengin ekosistemden faydalanabilecek düzeye geleceksiniz.

İlk iki bölüm mühendis adaylarına yönelik temel ve derin bir giriş sağlamaktadır. Saf başlangıç seviyesindeki okuyucular doğrudan üçüncü bölümden başlayabilirler.

Bu kitabın güncel sürümüne <https://github.com/HoydaBre/python> adresinden erişilebilir. Amacım Türkçe düşünen ve konuşan öğrencilerimize kavramları doğru bir şekilde aktarmaktır. Katkılarınızı bekler bu yolculukta size rehberlik etmekten büyük bir mutluluk duyuyorum. Kodlama serüveninizde başarılar dilerim!

Elektronik Yüksek Mühendisi, İlhan ÖZKAN

ilhanozkan[at]outlook.com

[linkedin.com/in/ilhanozkan](https://www.linkedin.com/in/ilhanozkan)

Sürümler

İkinci Sürüm

İlk sürümden farklı olarak aşağıdaki değişiklikler yapılmıştır¹.

- ◊ Genel yazım hataları düzeltildi.
- ◊ Veri Bilimi, Yapay Zekâ ve Makine Öğrenmesi başlığı eklendi.
- ◊ Her bölüme “Düşün ve Çöz” başlığı eklendi

¹İlk sürüm, Şubat 2026 tarihinde yayımlanmıştır.

İçindekiler

Önsöz	i
Sürümler	ii
1 Temel Bilgisayar Kavramları	1
1.1 Mühendis ve Teknisyen	1
1.2 Bilgisayara Niçin İhtiyaç Duyulur?	1
1.3 Veri ve Bilgi	2
1.4 En Basit Bilgisayar	2
1.5 Emir Çevrimi	3
1.6 İşlemci ve Bellek İş Birliğinin Temeli	5
1.6.1 İşlem Nerede Yapılır?	5
1.6.2 Veri Nerede Durur?	5
1.7 İşlemci Tasarlama Stratejileri	5
1.8 Onaltılı Sayı Sisteminin Kullanılması	6
1.9 Program ve Programlama	7
1.10 Program Türleri	8
1.11 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri	9
1.12 Düşün ve Çöz	9
2 Temel Yazılım Kavramları	10
2.1 Düşük Düzey Programlama	10
2.2 Genel Amaçlı Yüksek Düzey Diller	11
2.3 Arapsaçı Kod	12
2.4 Değişken	13
2.4.1 Tam Sayıların Bellek Temsili	14
2.4.2 Gerçek Sayıların Bellek Temsili	14
2.5 Fonksiyon	15
2.6 Yapısal Programlama	16
2.7 Yapısal Programlama Genel Çerçevesi	16
2.8 Nesne Yönelimli Programlama İhtiyacı	17
2.9 Nesne Yönelimli Programlama	18
2.10 Python Programlama Dili	18
2.11 Soyutlama Merdiveni	19
2.12 Derleyici ve Yorumlayıcı	19
2.12.1 Derleyici	19
2.12.2 Yorumlayıcı	20
2.12.3 Python Dünyasında Durum	21
2.13 Python Sanal Makinesinin Kurulması	21
2.13.1 Etkileşimli Kabuk	22
2.13.2 Betik Tabanlı Çalışma Modu	22
2.14 Düşün ve Çöz	23
3 Python Diline Giriş	24
3.1 En Basit Python Programı	24
3.1.1 Kaynak Kodumuzu Oluşturacak Karakterler	24
3.1.2 Talimatlar ve Anahtar Kelimeler	25
3.1.3 Girintileme	26
3.1.4 Açıklamalar	26
3.2 Değişken Tanımlama	27
3.2.1 Python'da İlk Veri Tipleri Yanılgısı ve Dinamik Tiplendirme	27
3.2.2 Değişkenin Kapsamı (Variable Scope)	28
3.2.3 Değişken Kimliklendirme Kuralları	28
3.2.4 Birden Fazla Değişkeni Tek Satırda Tanımlama	29
3.2.5 Kod Okunabilirliği ve Yazım Stilleri	29

3.3	Değişmezler ve Sabit Değişkenler	30
3.3.1	Değişmezler	30
3.3.2	Sabit Değişken	31
3.4	Etkileşimli Kabuk Kullanımında Değişkenler	31
3.4.1	Değişkenlerin Bellek Adreslerini Öğrenme	31
3.4.2	Tanımlı Değişkenleri Görme	32
3.4.3	Değişkenleri Silme	32
3.4.4	Değişken Tiplerini Test Etme	33
3.5	Veri Tipleri	33
3.5.1	Temel Veri Tipleri	33
3.5.2	Diziler ve Koleksiyonlar	35
3.6	Düşün ve Çöz	37
4	Ardışık İşlemler	38
4.1	Ardışık İşlem Nedir?	38
4.2	Konsoldan Girdi Okuma ve Konsola Veri Yazma	38
4.3	Konsola Biçimlendirilmiş Metin Yazma	39
4.3.1	Biçim Belirleyicisi Kullanarak Biçimlendirme	39
4.3.2	str.format() Yöntemi	40
4.3.3	f-string	40
4.3.4	Hangisini Kullanmalıyız?	41
4.4	Tip Dönüşümleri	41
4.4.1	Örtük (Otomatik) Tip Dönüşümü	41
4.4.2	Açık Tip Dönüşümü	42
4.4.3	Taban Parametresi ile Gelişmiş Tam Sayı Dönüşümleri	42
4.5	İşleçler	42
4.5.1	Aritmetik İşleçler	43
4.5.2	İlişkisel İşleçler	43
4.5.3	Atama İşleçleri	44
4.5.4	Mantıksal İşleçler	45
4.5.5	Bit Düzeyi İşleçler	46
4.5.6	Üyelik İşleçleri	47
4.5.7	Kimlik İşleçleri	48
4.5.8	İşleç Öncelikleri	49
4.6	Kayan Noktalı Sayılarda Hassasiyet ve Yaklaşıklık Problemleri	50
4.6.1	İkili Sistemde Sonsuz Devretme Paradoksu	50
4.6.2	Python Perspektifi ve Kesin Çözüm: decimal Modülü	51
4.7	Düşün ve Çöz	51
5	Kontrol İşlemleri	52
5.1	Kontrol İşlemi Nedir?	52
5.2	Akış Diyagramları	52
5.3	Duruma Göre Seçimler	53
5.3.1	If Talimatı	53
5.3.2	If-Else Talimatı	55
5.3.3	Match Talimatı	57
5.4	İlişkisel Döngüler	58
5.4.1	While Talimatı	59
5.4.2	For Talimatı	59
5.4.3	Aralık Nesneleri	61
5.4.4	For-Else Talimatı	61
5.4.5	İç İçe Döngüler	62
5.4.6	Gözcü Kontrollü Döngüler	63
5.5	Dallanmalar	63
5.5.1	Break Talimatı	63
5.5.2	Continue Talimatı	64
5.5.3	Pass Talimatı	65
5.5.4	Return Talimatı	66
5.6	Düşün ve Çöz	66
6	Fonksiyonlar	67

6.1	Fonksiyon Nedir?	67
6.2	Hazır Fonksiyonlar	67
6.3	Kullanıcı Tanımlı Fonksiyonlar	69
6.3.1	Fonksiyon Nasıl Tanımlanır?	70
6.3.2	Fonksiyonlara Parametre Geçirme	71
6.3.3	Fonksiyon Açıklamaları	73
6.4	Değişken Kapsamı	73
6.4.1	Yerel Değişkenler	73
6.4.2	Küresel Değişkenler	74
6.5	Fonksiyonlarda Değişken Aktarımı: Değerle ve Referansla Çağırma	75
6.5.1	Nesne Referansıyla Çağırma	75
6.5.2	Değiştirilemez Türlerde Aktarım	75
6.5.3	Değiştirilebilir Veri Tiplerinde Aktarım	75
6.6	Özyinelemeli Fonksiyonlar	76
6.7	Kendi Modülümüzü Oluşturma	77
6.8	Paket Yönetimi	78
6.9	Lambda İfadeleri	80
6.10	Düşün ve Çöz	81
7	Nesne Yönelimli Programlama	84
7.1	Nesne Yönelimli Paradigma	84
7.2	Sınıf ve Nesne	84
7.3	Yöntemler	88
7.4	Erişim Belirleyiciler	89
7.5	Soyutlama	90
7.6	Sarmalama	91
7.7	Kalıtım	91
7.8	Aşırı Yükleme	93
7.9	Geçersiz Kılma	94
7.10	Çok Biçimlilik	94
7.11	Soyut Sınıf	95
7.12	Ara Yüzler / Roller	95
7.13	İç Sınıf	95
7.14	Tekil Nesne Sınıfı	96
7.15	Yansıma	96
7.15.1	type() fonksiyonu	96
7.15.2	isinstance() fonksiyonu	96
7.15.3	issubclass() fonksiyonu	97
7.15.4	callable() fonksiyonu	97
7.15.5	getattr() fonksiyonu	97
7.15.6	setattr() fonksiyonu	98
7.15.7	hasattr() fonksiyonu	98
7.15.8	dir() fonksiyonu	98
7.16	Dekoratifler	99
7.17	Düşün ve Çöz	100
8	Veri Yapıları	101
8.1	Dizi (array)	101
8.2	Liste (list)	104
8.3	Metin (str)	106
8.4	Demet (tuple)	108
8.5	Küme (set)	109
8.6	Sözlük (dict)	111
8.7	Düşün ve Çöz	115
9	İstisna Yönetimi	116
9.1	try-except Bloğu	116
9.2	İstisnai Duruma Düşme	117
9.3	Kendi İstisna Sınıflarımızı Oluşturma	118
9.4	Düşün ve Çöz	118

10 Python Standart Kütüphanesi	119
10.1 Zaman ve Tarih Yönetimi	119
10.1.1 Zamanın Temsili ve Temel Sınıflar	119
10.1.2 Sistem Zamanına Erişim	119
10.1.3 Zaman Aritmetiği	119
10.1.4 Biçimlendirme	119
10.1.5 Zaman Dilimleri	119
10.2 Metin İşleme ve Düzenli İfadeler	120
10.2.1 Temel Karakterler ve Belirleyiciler	120
10.2.2 re Modülü Temel Fonksiyonları	120
10.3 Dosya Yönetimi ve Bağlam Yöneticileri	121
10.3.1 Dosya Açma ve Erişim Modları	121
10.3.2 Bağlam Yöneticileri	121
10.3.3 Dosya Okuma Teknikleri	121
10.3.4 Yapılandırılmış Veri Saklama	122
10.3.5 Dosya ve Dizin Kontrolleri	122
10.4 Sistem Parametreleri ve Etkileşimi	122
10.4.1 Komut Satırı Argümanları	122
10.4.2 Bellek Yönetimi	122
10.4.3 Standart Giriş, Çıkış ve Hata Akışları	123
10.4.4 Dosya İşlemlerinde Özel Hata Türleri	123
10.4.5 Dosyaların Ön Tanımlı Olarak Aranacağı Diziner	123
10.4.6 Programdan Çıkış	123
10.4.7 Yorumlayıcı ve Platform Bilgileri	123
10.5 Matematiksel Hesaplamalar	123
10.5.1 Matematiksel Sabitler	123
10.5.2 Sayı Yuvarlama ve Sınır Değerler	124
10.5.3 Üs, Logaritmik ve Güç Fonksiyonları	124
10.5.4 Trigonometrik İşlemler	124
10.5.5 Özel Durumlar	124
10.6 Bilimsel Hesaplama	124
10.6.1 NumPy Dizileri	125
10.6.2 Özel Dizi Oluşturma Fonksiyonları	125
10.6.3 Dizi Özellikleri	126
10.6.4 Dizi İndisleri ve Dilimleme	126
10.6.5 Dizi Biçimlendirme	127
10.6.6 Matematiksel İşlemler	127
10.6.7 Yayın (Broadcasting)	129
10.6.8 Dizi Birleştirme ve Ayırma	129
10.6.9 Dizi Kopyalama ve View	129
10.6.10 Pratik Örnekler	130
10.6.11 Performans İpuçları	131
10.6.12 En İyi NumPy Uygulamaları	131
10.6.13 Bilimsel Hesaplama Özeti	131
10.7 Olasılık ve İstatistiksel Analiz Araçları	131
10.7.1 Rastlantısallık ve Simülasyon	131
10.7.2 Temel Veri Analizi	131
10.8 Yüksek Performanslı Döngü Yönetimi	131
10.8.1 Sonsuz ve Koşullu Yineleyiciler	132
10.8.2 Kombinasyonel Araçlar	132
10.9 İleri Seviye Veri Yapıları	132
10.10 JSON Veri Değişim Formatı	132
10.10.1 JSON Yapısı ve Python Nesneleriyle İlişkisi	132
10.10.2 Serileştirme	132
10.10.3 Dosya Tabanlı İşlemler	132
10.10.4 JSON ile Veri Bütünlüğü	132
10.10.5 Örnek Uygulama	132
10.11 Düşün ve Çöz	133
11 Test ve Hata Ayıklama	134
11.0.1 Yazılım Doğrulamada Birim Testi	134

11.0.2	unittest Modülü Temel Yapısı	134
11.0.3	pytest Modülü	135
11.0.4	assert Fonksiyonu	136
11.0.5	Birden Fazla Girdiyle Test	136
11.0.6	Test Demirbaşları	137
11.0.7	Sahte Nesneler	138
11.0.8	Veri Tabanı Testi	138
11.0.9	Test Kapsama Oranı	139
11.0.10	Testleri Gruplandırma	140
11.0.11	Paylaşımlı Demirbaşlar	140
11.0.12	Zaman Uyumsuz Fonksiyon Testleri	140
11.0.13	Test Odaklı Yazılım Geliştirme (TDD)	141
11.0.14	En İyi Uygulamalar	141
11.0.15	Test İçeren Örnek Proje Yapısı	141
11.1	Düşün ve Çöz	142
12	Zaman Uyumsuz Programlama	143
12.1	Zaman Uyumlu ve Uyumsuz Programlama	143
12.2	async ve await Anahtar Kelimeleri	144
12.3	Birden Fazla Görevi Birlikte Çalıştırma	144
12.4	Görev Oluşturma	144
12.5	Asenkron Dosya İşlemleri	145
12.6	Asenkron HTTP İstekleri	145
12.7	Asenkron Bağlam Yöneticileri	146
12.8	Asenkron Yineleyiciler	146
12.9	Zaman Aşımı	147
12.10	Asenkron Kuyruk	147
12.11	Hata Yönetimi	148
12.12	İlk Tamamlanan Görevi İşleme	148
12.13	Özet ve En İyi Uygulamalar	148
12.14	Düşün ve Çöz	149
13	Veri Tipi İpuçları	150
13.1	Veri Tipi İpuçlarının Temelleri	150
13.2	Temel Veri Tipleri	150
13.3	Gelişmiş Veri Tipi İpuçları	150
13.4	Python 3.10+ Modern Özellikler	151
13.5	Statik Tip Kontrolü	151
13.6	Generic Tipler ve TypeVar	152
13.7	Literal ve Final	152
13.8	Protocol ve Structural Subtyping	152
13.9	TypedDict	153
13.10	Callable	153
13.11	Özet ve En İyi Uygulamalar	153
13.12	Düşün ve Çöz	153
14	Veri Bilimi, Yapay Zekâ ve Makine Öğrenmesi	155
14.1	Veri Analizi ve Manipülasyonu (Pandas & Matplotlib)	155
14.1.1	Pandas Veri Yapıları: Series ve DataFrame	155
14.1.2	Eksik Veri ve Aykırı Değer Yönetimi	155
14.1.3	Veri Görselleştirme (Matplotlib & Seaborn)	155
14.2	Makine Öğrenmesine Giriş (Scikit-Learn)	156
14.2.1	Regresyon Problemleri: Doğrusal Regresyon	156
14.2.2	Sınıflandırma Problemleri ve Performans Metrikleri	156
14.3	Derin Öğrenme ve Yapay Sinir Ağlarına (ANN) Giriş	157
14.3.1	Bir Nöronun Anatomisi ve İleri Besleme	157
14.3.2	PyTorch ile İlk Yapay Sinir Ağı Modeli	157
14.4	Doğal Dil İşleme (NLP) ve Modern Dil Modelleri	158
14.4.1	Metin Ön İşleme ve Vektör Temsilleri	158
14.4.2	Transformer Mimarisi ve LLM Entegrasyonu	158
14.5	Bölüm Özeti ve Egzersizler	158

14.5.1 Bölüm Özeti	158
14.5.2 Düşün ve Çöz: Algoritmik ve Teorik Sorular	158
15 Web Geliştirme	159
15.1 İlk FastAPI Uygulaması	159
15.2 Path Parametreleri	159
15.3 Query Parametreleri	160
15.4 Request Body	160
15.5 HTTP Yöntemleri	161
15.6 Response Models	161
15.7 Hata Yönetimi	162
15.8 Dependency Injection	162
15.9 Asenkron Endpoint'ler	162
15.10 Dosya Yükleme	163
15.11 Çapraz Kaynak Paylaşımı	163
15.12 Arka Planda Görev Çalıştırma	164
15.13 Veritabanı Entegrasyonu (SQLAlchemy)	164
15.14 Authentication - JWT Token	165
15.15 Modüler Yapı	165
15.16 Gerçek Zamanlı İletişim	166
15.17 API'leri Test Etme	166
15.18 En İyi Uygulamalar	166
15.19 Web Proje Yapısı	166
15.20 Düşün ve Çöz	167
16 İyi Kodlama Uygulamaları ve Temiz Kod	168
16.1 Python Stil Rehberi	168
16.2 SOLID İlkeleri	170
16.3 Tekrarlardan Kaçınma İlkesi	173
16.4 KISS İlkesi (Keep It Simple, Stupid)	174
16.5 Code Smell'ler ve Çözümleri	174
16.6 Kod Kalitesi Araçları	175
16.7 İz Kayıtları Oluşturma (Logging)	175
16.8 Konfigürasyon Yönetimi	176
16.9 Bağlam Yöneticisi	176
16.10 Liste veya Sözlük Kavrama	177
16.11 En İyi Uygulamalar Özeti	177
16.12 Düşün ve Çöz	177
Dizin	179

Şekiller Listesi

1.1	Motorola 6800, Intel 8086 İşlemciler ile 2732 EPROM Belleği	2
2.1	IDLE Entegre Geliştirme ve Öğrenme Ortamı Kabuğu	23
5.1	Akış Diyagramları Genel Şekilleri	52
5.2	Dik Üçgenin Alan Hesabını Gösteren Akış Diyagramı	53
5.3	If Talimatı İcra Akışı	53
5.4	Bir Sayının İşaretini Bulan Akış Diyagramı	54
5.5	If-Else Talimatı İcra Akışı	55
5.6	Bir Sayının İşaretini Bulan If-Else Akış Diyagramı	56
5.7	Match Talimatı İcra Akışı	57
5.8	While Talimatı İcra Akışı	59
5.9	Sayaç Kontrollü Döngü İcra Akışı	60
5.10	For Talimatı İcra Akışı	60
5.11	For-Else Talimatı İcra Akışı	62
5.12	While Döngüsünde Break Talimatı İcra Akışı	63
5.13	For Döngülerinde Break Talimatı İcra Akışı	64
5.14	While Döngüsünde Continue Talimatı İcra Akışı	64
5.15	For Döngülerinde Continue Talimatı İcra Akışı	65

Tablolar Listesi

1.1	RISC ve CISC Mimarilerinin Temel Karşılaştırması	6
1.2	Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları	7
1.3	6800 Emir Seti	8
1.4	Yazılım Türleri ve Temel Farkları	9
2.1	Tam Sayı Türleri ve Donanımsal Sayı Sınırları	14
2.2	Kayan Noktalı Sayı Standartları ve Bit Dağılımları	15
2.3	İki ile Çarpma Fonksiyonu	15
2.4	Soyutlama Merdiveni	19
2.5	Derleyici ve Python Yorumlayıcı Mimarilerinin Karşılaştırılması	21
3.1	Anahtar Kelimeler	25
4.1	Geleneksel Biçim Belirleyici Karakterleri	39
4.2	Aritmetik İşleçler	43
4.3	İlişkisel İşleçler	44
4.4	Atama İşleçleri	45
4.5	Mantıksal İşleçleri	45
4.6	Python Programlama Dili Bit Düzeyi İşleçleri	46
4.7	Python Programlama Dili Üyelik İşleçleri	47
4.8	Python Programlama Dili Kimlik İşleçleri	48
4.9	Python Programlama Dili İşleç Öncelikleri	49
6.1	Fonksiyonlara Geçirilen Parametreler	76
8.1	Dizi Eleman Veri Tiplerini Belirleyen Parametreler	102
8.2	Dizi ve Liste Nesnesi Yöntemleri	104
8.3	Metin Yöntemleri	107
8.4	Kaçış Dizisi Karakterleri	108
8.5	Demet Yöntemleri	109
9.1	İstisna Türleri	117

1. Bölüm

Temel Bilgisayar Kavramları

1.1 Mühendis ve Teknisyen

Mühendis (**engineer**); matematiksel ve bilimsel ilkeleri pratik amaçlara uygulayarak, toplumun ihtiyaç duyduğu sistem, süreç veya ürünleri belirli kısıtlar altında optimize eden profesyoneldir. Bir mühendislik çıktısının başarısı; sistemin ekonomik (**cost**) olarak uygulanabilir, operasyonel açıdan güvenli (**safe**), yapısal olarak kullanışlı (**functional**) ve hedeflenen takvim doğrultusunda zamanında (**on time**) teslim edilmesine bağlıdır. Mühendisler bu hedeflere ulaşabilmek adına döngüsel bir metodoloji izlerler; bu süreç sırasıyla keşif (**invent**), kavramsal tasarım (**design**), matematiksel ve yapısal analiz (**analysis**), fiziksel veya dijital uygulama (**build**) ve doğrulama amacıyla yapılan sınama (**test**) aşamalarını kapsar¹.

Buna karşın **Teknisyenler** (**technician**), mühendisler tarafından geliştirilen teorik modellerin ve tasarımların sahada veya laboratuvar ortamında operasyonel hale getirilmesinden sorumludur. Teknisyenlerin temel işlevi; teknik ekipmanların kurulumu, kalibrasyonu, rutin bakımı, pratik uygulamaların yürütülmesi ve sistemlerin işler durumda tutulması gibi somut, operasyonel ve metodolojik süreçleri yönetmektir.

Ele alınan disiplin *Yazılım Mühendisliği* olduğunda bu kavramsal ayrım soyut bir düzleme taşınır:

- ♦ **Yazılım Mühendisleri:** IEEE ve ACM standartları doğrultusunda; sistematik, disiplinli ve ölçülebilir mühendislik yaklaşımlarını yazılım geliştirme, işletme ve bakım süreçlerine uygulayan kişilerdir. İstenilen karmaşık bilgisayar yazılımını ve sistem mimarisini; maliyet etkin (ekonomik), siber güvenlik standartlarına uygun (güvenli) ve kullanıcı gereksinimlerini eksiksiz karşılayacak (kullanışlı) şekilde, belirlenen proje takvimine sadık kalarak zamanında ortaya koyarlar. Yazılım mühendisi, problemin analizinden sistem mimarisinin tasarlanmasına kadar uzanan büyük resmi (**high-level design**) yönetir.
- ♦ **Programcılar (Yazılım Geliştiriciler / Teknisyenler):** Yazılım ekosisteminde geleneksel mühendislikteki teknisyen rolünün karşılığıdır. Programcılar, mühendisler tarafından soyut mimarisi ve algoritmik sınırları çizilmiş olan yazılım bileşenlerini, belirli programlama dillerinin sözdizimi (**syntax**) kurallarını kullanarak somut kod bloklarına dönüştürürler. Temel odakları; modüler kısımların kodlanması, mevcut kod tabanındaki hataların ayıklanması (**debugging**) ve geliştirilen yazılımın üretim ortamında kararlı ve çalışır durumda tutulmasıdır.

1.2 Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçinin tarımsal verimliliği artırmak adına traktöre yönelmesi veya bir berberin zanaatını optimize etmek için elektrikli tıraş makinesi kullanması, insanlık tarihinin temel mekanizasyon dürtüsünü temsil eder. Benzer bir analogiyle, bir mühendisin karmaşık analitik modelleri çözümlerken bilgisayara ihtiyaç duyması da aynı nedenselliğe dayanır.

Tüm bu araç, gereç ve endüstriyel makinelerin varlık sebebi; insan emeğinin kısıtlı zaman ve enerji parametrelerini aşarak, operasyonel süreçleri **daha hızlı** (**speed**) ve verimli bir şekilde yürütmektir. Bilgisayarlar da bu mekanizasyon zincirinin bilişsel (cognitive) bir halkasıdır. Fiziksel makineler insan kas gücünü çarpanlarken, bilgisayarlar insan zihnini ve hesaplama kapasitesini çarpanlar. En temelde bilgisayar; ham veriyi (**data**) hızla işleyerek anlamlı bilgiye (**information**) dönüştüren, deterministik algoritmalara dayalı ve tekrarlanan yüksek ölçekli hesaplamaları sıfır hata ile gerçekleştiren elektronik bir sistemdir.

Verinin bu dönüşüm süreci, endüstriyel alanda yeni kurumsal iş rollerinin doğmasına yol açmıştır. *Veri sorumlusu* (**chief data officer**) veya *veri hazırlama ve kontrol işletmeni* gibi spesifik unvanlar, tam olarak bu operasyonel optimizasyon ihtiyacından türemiştir. Bu uzmanlık alanlarına gereksinim duyulmasının temel nedeni; ham verinin bilgisayar mimarisinin işleyebileceği belirli bir veri yapısına (**data structure**) uygun olarak sisteme beslenmesi, işlenen bu verilerin ilişkisel veritabanlarında saklanması ve karar destek mekanizmaları için anlamlı şekilde raporlanmasının, modern bilişim dünyasının omurgasını oluşturmasıdır.

Tarihsel bir perspektiften bakıldığında, bilgisayarların erken dönem evriminde temel motivasyon, karmaşık cebirsel ve matematiksel ifadelerin hatasız ve yüksek süratle çözümlenmesi olmuştur. İlk bilgisayarlar, kelimenin tam anlamıyla büyük ölçekli birer "hesap makinesi" (**computer**) olarak kurgulanmıştır. Bu tarihsel gerçeklik, bilgisayar biliminin ilk yapı taşları olan erken dönem yüksek seviyeli programlama dillerinin isimlendirilmesinde de açıkça görülmektedir:

¹www.bls.gov/ooh/architecture-and-engineering

- ◇ Sonradan daha sistematik bir yapıya bürünerek **ALGOL** (**Algorithmic Language**) adını alacak olan **IAL** (**International Algebraic Language**), bilgisayarların uluslararası düzeyde cebirsel problemleri çözme vizyonunu yansıtır.
- ◇ Bilimsel ve mühendislik hesaplamalarında bir devrim yaratan **FORTTRAN** (**Formula Translation**), adından da anlaşılacağı üzere matematiksel formüllerin doğrudan makine diline tercüme edilmesi amacıyla tasarlanmıştır.

1.3 Veri ve Bilgi

Veri (**data**); nesnelerin, olayların veya durumların nitel ya da nicel özelliklerini temsil eden, ham, yapılandırılmamış ve tek başına bir anlam ifade etmeyen sembolik kayıtlardır. Genellikle ampirik gözlemler, fiziksel ölçümler veya sayım işlemleri sonucunda elde edilirler. Veri, bağlamdan (**context**) yoksun olduğu için kendi başına bir karar mekanizması oluşturamaz. Örneğin; anlık olarak ölçülen bir hava sıcaklığı değeri (örn. 24°C), bir derse giren anlık öğrenci sayısı veya hassas teraziyile tartılan bir nesnenin kütlesi, verilerin tipik birer örneğidir. Bu saf kısıntılar, henüz bir analize tabi tutulmadıkları için sistemin genel durumu hakkında kısıtlı bir öngörü sunarlar.

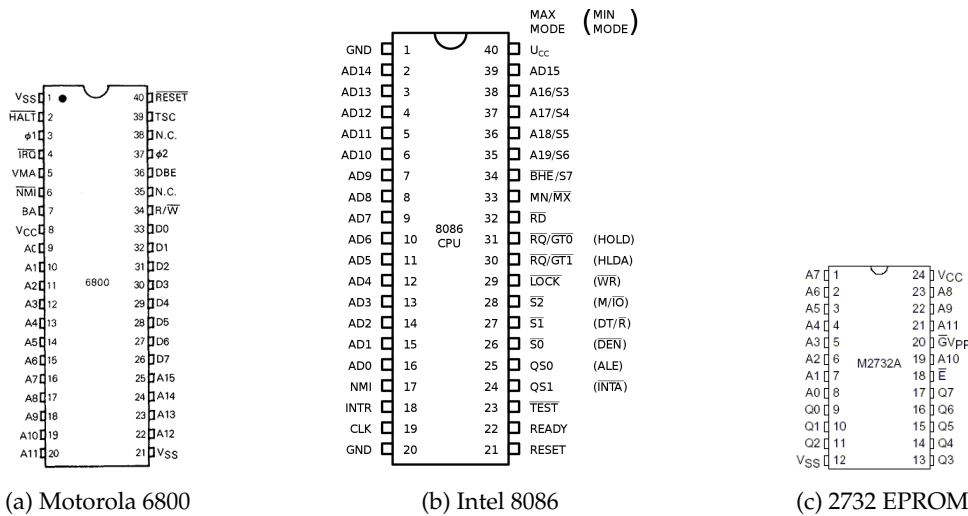
Bilgi (**information**) ise, ham verinin belirli bir amaç doğrultusunda sınıflandırılması, ilişkilendirilmesi, anlamlandırılması ve istatistiksel süreçlerden geçirilmesiyle elde edilen yapılandırılmış çıktılardır. Kısacası bilgi, *bağlamlandırılmış ve işlenmiş veridir*. Veri "Ne?" sorusuna yanıt ararken; bilgi, veriler arasındaki ilişkileri analiz ederek sistemin bütününe dair işlevsel bir örüntü ortaya koyar.

İstatistiksel yöntemlerle işlenen veriler, geleceğe yönelik tahmine dayalı (**predictive**) ve betimsel (**descriptive**) çıkarımlar yapma kabiliyeti kazanır. Örneğin; belirli bir yerdeki geçmişe dönük hava sıcaklığı verilerinin aritmetik ortalaması alındığında, bu ortalama değer artık sıradan bir veri değil; o bölgede yeni bir ölçüm yapmaya gerek kalmaksızın mevsim normallerini tahmin etmeye yarayan bir "bilgi" haline gelir. Benzer şekilde, bir sınıftaki öğrencilerin sınav notlarının ortalaması veya standart sapması, sınıftan rastgele seçilecek bir öğrencinin başarı eğilimini olasılıksal olarak öngörmemizi sağlayan semantik birer göstergedir.

1.4 En Basit Bilgisayar

En temel bilgisayar mimarisi (Von Neumann mimarisi), birbirleriyle sürekli iletişim halinde olan üç ana bileşenden oluşur:

1. **Merkezi İşlem Birimi** (**Central Processing Unit - CPU**): Bilgisayarın "beyni" konumundadır ve kısaca **işlemci** (**processor**) olarak adlandırılır. Matematiksel mantık işlemlerini yürütür ve sistemdeki tüm akışı kontrol eder.
2. **Bellek** (**memory**): İşlemcinin o an yürüteceği **emirleri** (**instruction**) ve bu emirlerin işleyeceği ham verileri geçici olarak barındıran çalışma alanıdır.
3. **Yol** (**bus**): İşlemci ile belleği birbirine bağlayan, üzerinden elektrik akımının aktığı mikroskobik bakır veya altın hatlardır.



Şekil 1.1: Motorola 6800, Intel 8086 İşlemciler ile 2732 EPROM Belleği

Hem belleğin hem de işlemcinin dış dünya ile iletişim kurmasını sağlayan fiziksel bağlantı noktalarına bacak (**pin**) adı verilir. Bu bacaklar karşılıklı olarak yollar (**bus**) vasıtasıyla birbirine bağlanır. Bir bilgisayarın çalışabilmesi için işlemci ve bellekte şu temel bacak grupları bulunur:

- ◇ **Veri (data) Bacakları:** İşlenecek olan asıl bilginin aktarıldığı kapılardır.
- ◇ **Adres (address) Bacakları:** İşlenecek verinin bellekte tam olarak nerede durduğunu işaret eden konum bilgilerinin iletildiği kapılardır.
- ◇ **Kontrol (control) Bacakları:** İşlemin yönünü belirleyen okuma (**read**) ve yazma (**write**) bacaklarının yanı sıra; işlemciyi dış dünyadaki acil durumlardan haberdar eden kesme (**interrupt**) veya işlemcinin çalışma hızını (kalp atışını) belirleyen kristalin bağlandığı saat (**clock**) bacaklarıdır.

İşlemci ve bellek, milyonlarca transistörün mikroskobik boyutlarda bir araya getirilmesiyle oluşan entegre devre (**integrated circuit**), yani halk arasındaki tabiriyle birer **yonga** (çip) şeklinde üretilirler.

BİLGİ: Elektrikten Sayılara (Bit Kavramı)

Bilgisayarlar temelde elektrikle çalışan sayısal (dijital) aletlerdir. Bilgisayar hatlarındaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. **Programcılar için gerilimin ne olduğu değil varlığı önemlidir.** İşlemci ile bellek arasındaki bir metal hatta (kabloda) belirli bir voltajda elektrik varsa bu durum mantıksal olarak **1** (Var/Doğru), elektrik yoksa **0** (Yok/Yanlış) olarak anlamlandırılır. Bu iki durum, **ikili (binary)** sayı sisteminin rakamlarıdır ve bilişimin en küçük yapı taşı olan **bit (Binary DigiT)** olarak adlandırılır.

- ◇ 1 elektrik hattı varsa: Sadece $2^1 = 2$ farklı durum ifade edilebilir (**0** veya **1**).
- ◇ 2 elektrik hattı varsa: $2^2 = 4$ farklı durum ifade edilebilir (**00**, **01**, **10**, **11**).
- ◇ n elektrik hattı yan yana gelirse: 2^n adet farklı sayı veya durum taşınabilir.

İşlemci ile bellek arasında köprü kuran bu hatlar, taşıdıkları bilginin niteliğine göre isimlendirilirler:

- ◇ **Veri Yolu (data bus):** İşlemcinin bellekten okuyacağı emirleri veya belleğe yazacağı sonuçları taşır. İşlemci bellekten veri alıp (okuma) aynı zamanda belleğe veri gönderebildiği (yazma) için veri yolları çift yönlüdür (**bidirectional**).
- ◇ **Adres Yolu (address bus):** Belleğin hangi bölgesindeki (hangi adresindeki) veriye ulaşılacağını belirleyen hatlardır. Bilgiyi her zaman işlemci üretirek belleğe "Şu adresteki veriyi getir" veya "Şu adrese bu veriyi yaz" dediği için adres yolu tek yönlüdür (**unidirectional**).

Analoji: Postacı ve Adres Kutuları

Belleği yan yana dizilmiş milyonlarca posta kutusu gibi düşünebilirsiniz. Her kutunun üzerinde benzersiz bir **adres numarası** vardır ve kutuların içinde de asıl **veriler** (mektuplar) saklanır. İşlemci bir veriye erişmek istediğinde, önce **Adres Yolu** üzerinden kutunun numarasını gönderir. Ardından kapaklar açılır ve kutunun içindeki mektup (veri), **Veri Yolu** üzerinden işlemciye akar.

Tarihsel süreçte mühendisler, yonga üzerindeki bacak sayısını azaltarak maliyeti düşürmek ve yerden tasarruf etmek için yaratıcı çözümler geliştirmişlerdir. Örneğin, efsanevi 8086 işlemcisinde veri ve adres hatları fiziksel olarak birbirinden tamamen ayrılmamıştır. AD (**Address/Data**) ile işaretli ortak bacaklar kullanılır. Bu ortak hatların o mikrosaniyede bir adres mi yoksa bir veri mi taşıdığı, işlemcinin mimariye göre **Address Latch Enable - ALE** kontrol bacağından gönderilen sinyalle belirlenir. Bu tekniğe hat çoklama (**multiplexing**) denir.

İşlemcinin tek bir seferde (aynı anda) veri yolundan geçirebildiği bit sayısına **kelime uzunluğu (word length)** adı verilir. Kelime uzunluğu, işlemcinin mimari gücünü ve aynı anda işleyebileceği maksimum veri boyutunu belirler. Örneğin 16 bitlik bir işlemci tek seferde 16 bit veri taşıyabilirken, modern 64 bitlik işlemciler tek bir saat darbesinde 64 bitlik devasa verilerle oynayabilirler.

Bilişim dünyasında standart bir kural olarak, sekiz (8) bitlik yan yana gelmiş veri öbeğine **bayt (byte)** adı verilir. Bellekteki her bir posta kutusu (adres) genellikle tam 1 baytlık veri saklayacak genişliktedir. Bu yüzden bilgisayarların bellek (RAM) miktarları ve depolama kapasiteleri tarih boyunca bayt (KB, MB, GB, TB) cinsinden ölçülmüştür.

İşlemcinin saat bacaklarına uygulanan saat frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 5 GHz frekansını aşan işlemciler bulunmaktadır.

1.5 Emir Çevrimi

Bir bilgisayar sistemine ilk elektrik verildiği (sistemin başlatıldığı) andan itibaren işlemci, işletim sistemi veya yüklenen program fark etmeksizin temel bir yaşam döngüsüne girer. İşlemci, elektrik kesilene veya sistem kapatılana kadar mikro saniyeler içinde milyarlarca kez tekrarlanan üç adımlık **Emir Alma (fetch)**, **Çözme (decode)** ve **İcra Etme (execute)** döngüsünü yürütür. Bilgisayar biliminde bu sonsuz döngüye **Emir**

Çevrimi (instruction cycle) veya *Makine Çevrimi (machine cycle)* adı verilir. Bu çevrimin hatasız yürütülmesi için CPU içinde **kaydedici (register)** adı verilen, işlemcinin en hızlı çalışan mikroskobik bellek hücreleri görev yapar. Bu süreç adım adım şu şekilde gerçekleşir:

1. **Emri Alma (fetch)**: Yazdığımız programlar bellekte sırayla dizilmiş emirler bütününden oluşur. Bu aşamanın amacı, sıradaki emri bellekten işlemcinin içine güvenle getirmektir:
 - (a) **Program Sayacı (Program Counter - PC)**: İşlemci içindeki bu özel kaydedici, bir sonraki adımda çalıştırılacak emrin bellekteki adres numarasını hafızasında tutar. Tıpkı bir kitabın hangi sayfasında kaldığımızı gösteren bir kitap ayracı gibidir.
 - (b) **Bellek Adres Kaydedicisi (Memory Address Register - MAR)**: PC içindeki adres bilgisi, adres yoluna çıkmadan hemen önce MAR'a kopyalanır. MAR, bellekten veri istenirken hedef adresin kilitlendiği geçici bir kapıdır.
 - (c) **Bellek Veri Kaydedicisi (Memory Data Register - MDR)**: Adres yolundan iletilen istek üzerine bellek, ilgili kutunun içindeki emri veri yolu üzerinden CPU'ya gönderir. Bu emir ilk olarak MDR'ye giriş yapar.
 - (d) **Mevcut Emir Kaydedicisi (Current Instruction Register - CIR)**: MDR'ye gelen ham emir, işlemek üzere CIR'e aktarılır. Bu sırada Program Sayacı (PC), bir sonraki emrin yerini şimdiden işaret etmek amacıyla otomatik olarak bir artırılır ($PC = PC + 1$). Ayrıca emir boyutu 2 veya 4 bayt ise PC, emrin bayt boyutu kadar artırılır.
2. **Çözme (decode)**: İşlemcinin içine getirilen emir henüz saf bir ikili sayı yığından (0 ve 1 dizisinden) ibarettir. Bu aşamada işlemcinin **kontrol birimi (Control Unit - CU)**, emrin ne anlama geldiğini ve hangi donanım bileşenlerini harekete geçirmesi gerektiğini analiz eder:
 - (a) CIR içindeki emir kodu iki temel parçaya ayrıştırılır: Ne yapılacağını belirten **işlem kodu** yani **Opcode (operation code)** ve bu işlemin hangi veriyle yapılacağını belirten **işlenen** yani **Operand (operand)**. Örneğin, bir toplama emrinde *Opcode* "TOPLA" talimatı iken, *Operand* toplanacak sayıların bulunduğu adreslerdir.
 - (b) Kontrol Birimi (CU), bu kodu çözer çözmez, gerekli verilerin getirilmesi ve ilgili donanım yollarının (örneğin toplama yapılacaksa matematik biriminin) hazırlanması için işlemci içi sinyalleri tetikler.
3. **İcra Etme (execute)**: Çevrimin son aşamasında, çözülen emir kodunun komutları doğrultusunda asıl iş gerçekleştirilir ve CPU bileşenleri çalıştırılır:
 - (a) Eğer emir bir matematiksel hesaplama (toplama, çıkarma) veya mantıksal bir karşılaştırma (büyüktür, küçüktür) gerektiriyorsa, CU tüm verileri **aritmetik mantık birimine (Arithmetic Logic Unit - ALU)** yönlendirir. ALU, işlemini tamamladıktan sonra çıkan sonucu geçici olarak **akümülatör (accumulator - ACC)** adı verilen özel bir kaydedicide saklar veya MDR üzerinden tekrar ana belleğe yazar.
 - (b) Matematiksel olmayan bazı emirlerde ise doğrudan işlemci içindeki genel amaçlı kaydedicilerin (**registers**) içerikleri güncellenir veya programın akışı başka bir adresleme noktasına sıçrar (**jump**) ya da dallanır (**branch**).

Bu üç adımlık döngü bittiği an, işlemci hiç duraksamadan PC'nin güncel değerine bakarak bir sonraki "Emir Alma" adımına geçer. Bilgisayarın performansını belirleyen "Saat Frekansı" (GHz cinsinden ölçülen hız), aslında işlemcinin saniyede bu döngü adımlarını kaç milyar kez tekrarlayabildiğinin bir göstergesidir.

İşlemcinin Emir Çevrimi (**Fetch-Decode-Execute**) esnasında bellekten alıp çözdüğü her bir emir (**instruction**), arka planda aslında sadece belirli bir ikili sayıdan (0 ve 1'lerden) ibarettir. Bilgisayar dilinde bu sayılara **emir kodu (instruction code)** veya **makine kodu (machine code)** adı verilir. Bir emrin en önemli parçası, işlemciye tam olarak hangi donanımsal fonksiyonu tetiklemesi gerektiğini söyleyen işlem kodudur (**operation code - opcode**). Ancak bir işlemci, karşısına çıkan her rastgele sayıyı bir emir olarak algılayamaz. Tıpkı her dilin belirli sayıda kelimeden oluşan bir sözlüğü olması gibi, her işlemci mimarisinin de fabrikada üretilirken içine fiziksel olarak kazanmış, tanıyabildiği ve çözebildiği sınırlı sayıda emir vardır.

Bir işlemcinin donanımsal olarak tanıyabileceği ve icra edebileceği bu benzersiz sayıların (emir kodlarının) oluşturduğu tüm kümeye **emir seti (instruction set)** adı verilir. Eğer işlemciye bu setin dışındaki bir makine kodu gönderilirse, işlemci ne yapacağını bilemez ve **bilinmeyen emir (illegal instruction)** hatası vererek kilitlenir.

Farklı firmalar tarafından üretilen işlemcilerin tasarımları, yetenekleri ve dolayısıyla emir setleri birbirinden tamamen farklıdır. Örneğin, mikrodenetleyici dünyasında sıkça kullanılan efsanevi **Motorola 6800** işlemcisi donanımsal olarak sadece 72 farklı emri doğrudan tanıyıp çözebilir. Kişisel bilgisayarlarımızın temelini atan mimariye sahip **Intel 8086** işlemcisi ise biraz daha karmaşık bir yapıda olup 116 farklı temel emri işleme kabiliyetine sahiptir.

Analoji: Mikserin Düğmeleri ve İşlem Kodu

Bir mutfak robotunu (mikseri) ele alalım. Bu robotun üzerinde sadece 3 tane düğme olsun: 1 numaralı düğme "Karıştır", 2 numaralı düğme "Doğra", 3 numaralı düğme ise "Durdur" anlamına gelsin. İşte bu mutfak robotunun Emir Seti sadece bu 3 düğmeden (1, 2, 3) oluşur. Eğer siz robota elektronik olarak 5 sinyali gönderirseniz, robotun içinde 5 sayısını çözecek bir mekanizma olmadığı için hiçbir işlem yapamaz. Bilgisayar işlemcisindeki **opcode** mantığı da tam olarak budur; örneğin Intel işlemci için **00000100** sayısı donanımsal olarak "Topla" anlamına gelen bir düğmedir.

Günümüzde bizler program yazarken doğrudan bu karmaşık ve ezberlemesi imkansız olan makine kodlarını (sayıları) yazmayız. Bunun yerine Python dilinde `print()` veya C dilinde `x = a + b;` gibi insan diline yakın komutlar kullanırız. Bizim yazdığımız bu kelimeler, Derleyici (**compiler**) veya Montajcı (**assembler**) adı verilen özel yazılımlar sayesinde, hedef işlemcinin (Motorola veya Intel) tam olarak anlayacağı, 0 ve 1'lerden oluşan benzersiz makine kodlarına tercüme edilir.

1.6 İşlemci ve Bellek İş Birliğinin Temeli

Bir bilgisayarın çalışma mantığını anlamak için iki temel soruya yanıt vermemiz gerekir: "Veri nerede durur?" ve "İşlem nerede yapılır?"

§1.6.1 İşlem Nerede Yapılır?

İşlemci, bilgisayarın "beyni" veya bir mutfağın "şefi" gibidir. Yazdığınız kodlardaki tüm matematiksel hesaplamalar, mantıksal karşılaştırmalar ve karar verme süreçleri burada gerçekleşir. Ancak işlemcinin kendi içinde veri saklama kapasitesi çok sınırlıdır; o sadece önüne gelen işi o an yapmakla görevlidir.

§1.6.2 Veri Nerede Durur?

Bellek yani RAM (Random Access Memory), bilgisayarın **çalışma masasıdır**. Programınız çalışırken kullandığınız değişkenler, diziler ve nesneler geçici olarak burada tutulur. İşlemcinin bir veriyi işlemesi için, o verinin önce bellekten çağırılması gerekir. Bellek hızlıdır ancak geçicidir; bilgisayar kapandığında içindeki tüm veriler silinir.

Bu ilişkiyi daha somutlaştırmak için bir mutfağı hayal edelim:

- ◊ İşlemci (CPU): Yemek yapan aşçıdır. İşlemi (doğrama, pişirme) o yapar.
- ◊ Bellek (RAM): Aşçının önündeki kesme tahtası veya tezgâhtır. O an kullanılan malzemeler (veriler) burada durur.
- ◊ Sabit Disk (HDD/SSD): Mutfağın kileri veya buzdolabı gibidir. Tüm malzemeler orada saklanır (kalıcı depolama).

Programınızı çalıştırdığınızda, verileriniz kalıcı depolamadan (disk) alınır ve belleğe (RAM) yerleşir. İşlemci ise bellekte hazır bekleyen bu verileri sırayla alır, işler ve sonucu tekrar belleğe yazar.

1.7 İşlemci Tasarlama Stratejileri

Mikroişlemci mimarisinde, işlemcinin donanımsal yetenekleri ile yazılımsal esnekliği arasındaki dengeyi kurmak amacıyla temel olarak üç farklı tasarım stratejisi izlenir:

1. **İndirgenmiş Emir Setli Bilgisayarlar - RISC (Reduced Instruction Set Computing)**: Donanımsal karmaşıklığı azaltıp, komutların olabildiğince basit ve sabit uzunlukta tutulmasını hedefleyen işlemci mimarisidir. RISC felsefesinde, her bir emir tek bir mikro işlem adımına indirgenmiştir. Az sayıda ve basit yapıdaki bu emirlerin çözülmesi (**decode**) ve icra edilmesi (**execute**) çok kısa sürer. Donanım basitleştirildiği için yük yazılım tarafına aktarılır; yani yüksek seviyeli dilde yazılmış tek bir satır kod, makine diline çevrilirken daha fazla sayıda RISC emrine bölünür ve bu durum üretilen ikili kod tabanının (**code size**) büyümesine neden olur.
2. **Karmaşık Emir Setli Bilgisayarlar - CISC (Complex Instruction Set Computing)**: Tek bir makine emriyle birden fazla alt işlemi (örneğin bellekten veri yükleme, matematiksel işlem yapma ve sonucu tekrar belleğe yazma) aynı anda gerçekleştirebilen, donanım odaklı işlemci mimarisidir. CISC işlemciler, değişken uzunluktaki yüzlerce karmaşık emri doğrudan tanıyabilir. Bu zengin emir seti, yüksek seviyeli programlama dillerindeki ifadelerin çok daha az sayıda makine koduna (daha küçük kod boyutuna) dönüştürülmesini sağlar. Ancak bu karmaşıklık, emrin çözülme ve icra edilme sürelerini uzatır; işlemci içinde daha büyük ve yavaş çalışan kontrol birimleri (CU) gerektirir.
3. **Uygulamaya Özel İşlemciler - ASIP/ASIC (Application-Specific Instruction-Set Processor)**: Genel amaçlı bilgi işlem süreçlerinin ötesinde, yalnızca belirli bir görev havuzunu (örneğin matris çarpımları, sinyal işleme veya kriptografik algoritmalar) maksimum enerji verimliliği ve hızla yürütmek için tasarlanmış

mimarilerdir. Grafik İşlem Birimleri (GPU), Yapay Zekâ ve Derin Öğrenme hızlandırıcıları (TPU/NPU) ile Dijital Sinyal İşlemcileri (DSP) bu kategorinin en çarpıcı örnekleridir.

İşlemcilerin performans analizinde en önemli parametrelerden biri, bir emrin tamamlanması için geçen Saat Periyodu Sayısıdır (**Clock cycles Per Instruction - CPI**). Standart bir emir çevrimi (**instruction cycle**) çoğunlukla tek bir saat darbesinde tamamlanmaz.

- ◊ **RISC** işlemcilerde komutlar eşit uzunlukta ve basit olduğu için, komutların ezici bir yoğunluğu tam olarak 1 saat periyodunda (1 çevrim) icra edilir. Bu durum, işlemcilerde *Boru Hattı* (**pipelining**) adı verilen, birden fazla emrin farklı aşamalarda (Fetch, Decode, Execute) aynı anda işlendiği montaj hattı teknolojisinin uygulanmasını son derece kolaylaştırır.
- ◊ **CISC** işlemcilerde ise emirlerin uzunlukları ve karmaşıklıkları değişken olduğundan, tek bir emrin tamamlanması birden fazla saat periyodu gerektirir. Kontrol birimi, karmaşık bir CISC emrini aldıktan sonra onu işlemci içinde *mikro-kod* (**microcode**) adı verilen küçük alt adımlara bölerek çalıştırır. Bu nedenle, ham saat hızı aynı olsa bile RISC işlemciler, komut başına düşen çevrim süresinin düşüklüğü sayesinde CISC işlemcilere göre teorik olarak daha yüksek bir paralel işlem verimliliği sunar.

Özellik	RISC	CISC
Emir Seti Genişliği	Sınırlı ve Az Sayıda	Geniş ve Çok Sayıda
Emir Boyutu	Sabit (Genellikle 32-bit)	Değişken (1 ila 15 bayt arası)
Bellek Erişimi	Sadece LOAD ve STORE ile	Emirlerin içinde doğrudan erişim
Saat Periyodu (CPI)	Birçok emir için 1 Çevrim	Emir başına birden fazla Çevrim
Kod Boyutu	Büyük / Uzun	Küçük / Yoğun

Tablo 1.1: RISC ve CISC Mimarilerinin Temel Karşılaştırması

Günümüzdeki Durum: Hibrit (Karma) Mimari Devrimi

Günümüzde ticari işlemci pazarında saf bir RISC veya saf bir CISC ayrımından bahsetmek neredeyse imkansız hale gelmiştir. Modern işlemciler, her iki mimarinin de en güçlü yönlerini bir araya getiren hibrit (karma) yapılar kullanır.

Örneğin, masaüstü ve sunucu dünyasını domine eden Intel ve AMD işlemciler dışarıdan bakıldığında birer CISC (x86) işlemcidir; geriye dönük uyumluluk adına x86 komut setlerini desteklerler. Ancak bu işlemcilerin çekirdeğine bir CISC emri girdiğinde, donanım düzeyindeki bir kod çözücü (Decoder) bu karmaşık emri anında **u-op** (mikro-operasyon) adı verilen küçük, sabit uzunluktaki RISC benzeri emirlere parçalar ve iç çekirdek bu emirleri RISC hızında çalıştırır.

Ticari pazardaki emir seti (**Instruction Set Architecture - ISA**) dağılımı günümüzde iki ana ekosisteme bölünmüştür:

- ◊ **Masaüstü ve Sunucu Mimarisi (x86 Ekosistemi):** Kişisel bilgisayarların ve veri merkezlerinin büyük kısmı Intel ve AMD marka işlemciler barındırır. Bu işlemciler tarihsel süreçte 16-bitlik IA-16 mimarisinden başlayarak sırasıyla 32-bitlik IA-32 (x86) ve günümüzün endüstri standardı olan 64-bitlik geriye uyumlu x86-64 (AMD64) emir setlerini ve multimedya/vektörel hesaplama eklentilerini (AVX, SSE vb.) desteklemektedir.
- ◊ **Mobil ve Gömülü Sistemler Mimarisi (ARM Ekosistemi):** Akıllı telefonlar, tabletler ve yeni nesil yüksek verimli taşınabilir bilgisayarlar (Masaüstü Apple Silicon serisi gibi) ARM (**Advanced RISC Machines**) tabanlı mimarileri kullanır. Saf bir RISC felsefesiyle doğan bu mimari, düşük güç tüketimi ve yüksek enerji verimliliği odaklıdır. Günümüzde modern mobil cihazlar 64-bit mimariye sahip ARMv8-A, ARMv9-A ve bunların güvenlik/yapay zekâ eklentilerini kullanmaktadır.

1.8 Onaltılı Sayı Sisteminin Kullanılması

İşlemci ile bellek arasında verilerin elektrik yolları (**bus**) ile taşındığı anlatılmıştı. Veri taşıyan elektrik hattı sayısı işlemci ile bellek arasında taşınabilecek verinin sınırlarını da belirler. Bir elektrik hattının taşıyacağı veri bir bit ile (0 veya 1) gösterilebilir. İki elektrik hattının taşıyacağı veri ise iki bit ile (00, 01, 10 veya 11) $2^2 = 4$ kadar farklı gösterilebilir. Benzer şekilde n adet elektrik hattının taşıyacağı veri 2^n farklı alternatifte veri olacaktır.

Bit sayısı arttıkça ikili sayının okunup yazılması da çok zor olacaktır. Günümüzde 64 bitlik hatta 1024 bitlik veri yoluna sahip bilgisayarlar bulunmaktadır. Bunun üstesinden gelmek için onaltılı (**hexadecimal**) sayı sistemi kullanılır. Çünkü her 4 elektrik hattının verisi onaltılık sayı sisteminin bir rakamına karşılık gelir.

Onaltılı	İkili	Onlu	Onaltılı	İkili	Onlu
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

Tablo 1.2: Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları

BİLGİ

Kod içerisinde onaltılık sayıların rakamlarının diğer sayı sistemindeki rakamlarla karışmaması için sonuna **H** konularak **113AH** şeklinde veya çoğunlukla başına **0x** konularak **0x1133** şeklinde kullanılır.

Örnek olarak ikili tabanda verilen 64 bitlik 0001 1010 1011 1101 0101 0100 1001 1010 0011 0010 0100 0110 1001 1101 1100 1111 sayısının onaltılı tabanda karşılığı oldukça kısa (16 hane) olarak 1ABD 549A 3246 9DCF şeklindedir.

BİLGİ

Onaltılı sayının **A, B, C, D, E** ve **F** rakamları büyük harf olacağı gibi küçük harf de olabilir. Anlam olarak bir şey değişmez.

Bilgisayar dünyasında ikili sayıları onaltılı sayı sistemiyle ifade etmek çoğunlukla bir kolaylık ve standarttır.

1.9 Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi emre (instruction), **bilgisayar programı** (**computer program**) denir. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlanması işlemine **programlama** (**programming**) denir. Programlama **emir kodları** (**instruction code**) ile yapıldığından bu programın dili, **makine dili** (**machine language**) olarak adlandırılır².

Her işlemci üreticisinin kullandığı emir seti farklı olduğundan (yani her bir emre farklı makine kodu verildiğinden) ilk dönemlerde programlama oldukça zorlu ve derin teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu. Programın okunurluğunu artırmak için her bir emre **hatırlatıcı isimler** (**mnemonic**) verilmeye başlandı. Tablo 1.3'de 6800 işlemcisinin birkaç sembolik ismi verilmiştir.

Bu sembolik isimlerle yazılan program, **montaj kodu** (**assembly code**) olarak adlandırılır. Montaj kodları metin dosyalarına kaydedilir. Bu dosyaları doğrudan makine diline çeviren özel programlara ise **Montajcı** (**assembler**) adı verilir.

Montajcı ve Derleyici

Hatırlatıcı isimlerle (**mnemonic**) yazılan montaj kodlarını doğrudan makine dilindeki emir kodlarına (**instruction code**) çeviren programlara **Montajcı** (**assembler**) denir. Yüksek seviyeli dil kodlarını makine diline dönüştüren programlar ise **Derleyici** (**compiler**) olarak adlandırılır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde yazılmış örnek bir montaj kodu verilmiştir. Örnekte, genel amaçlı CL kaydedicisine koyulan sayıya kadar olan değerlerin kontrolü yapılmaktadır³.

```
section .text ; programı oluşturan emir kodları buradan başlar
global _start
_start:
    mov dx, 0 ; dx 0 olsun. Toplamı tutacak.
    mov cl, 99 ; 99 kez tekrarlanacak.
    sum:
```

²ISO/IEC 2382:2015. ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system

³<https://github.com/armut/x86-Linux-Assembly-Examples>

Mnemonic	Emir Kodu	Açıklama
LDAA	96	LOAD M to A ($M \rightarrow A$, Direct)
LDAB	D6	LOAD M to B ($M \rightarrow B$, Direct)
STAA	97	STORE A to M ($A \rightarrow M$, Direct)
STAB	D7	STORE B to M ($B \rightarrow M$, Direct)
TAB	16	TRANSFER A to B ($A \rightarrow B$)
TBA	17	TRANSFER B to A ($B \rightarrow A$)
CLR	7F	CLEAR Memory ($00 \rightarrow M$, Extended)
CLRA	4F	CLEAR A ($00 \rightarrow A$)
CLRB	5F	CLEAR B ($00 \rightarrow B$)
ABA	1B	ADD A to B ($A + B \rightarrow A$)
SBA	10	SUBTRACT B from A ($A - B \rightarrow A$)
ADDA	9B	ADD M to A ($A + M \rightarrow A$, Direct)
ADDB	DB	ADD M to B ($B + M \rightarrow B$, Direct)
SUBA	90	SUBTRACT M from A ($A - M \rightarrow A$, Direct)
SUBB	D0	SUBTRACT M from B ($B - M \rightarrow B$, Direct)
INCA	4C	INCREMENT A ($A + 1 \rightarrow A$)
INCB	5C	INCREMENT B ($B + 1 \rightarrow B$)
DECA	4A	DECREMENT A ($A - 1 \rightarrow A$)
DECB	5A	DECREMENT B ($B - 1 \rightarrow B$)
DAA	19	DECIMAL ADJUST A (BCD Formatlaması)
ANDA	94	LOGICAL AND M with A ($A \cdot M \rightarrow A$, Direct)
ANDB	D4	LOGICAL AND M with B ($B \cdot M \rightarrow B$, Direct)
ORAA	9A	LOGICAL INCLUSIVE OR M with A ($A + M \rightarrow A$, Direct)
ORAB	DA	LOGICAL INCLUSIVE OR M with B ($B + M \rightarrow B$, Direct)
EORA	98	EXCLUSIVE OR M with A ($A \oplus M \rightarrow A$, Direct)
COMA	43	COMPLEMENT A (A 'nın 1'e tümleneni $\rightarrow A$)
PSHA	36	PUSH A onto Stack ($A \rightarrow \text{Stack}$)
PULA	32	PULL A from Stack ($\text{Stack} \rightarrow A$)
PSHB	37	PUSH B onto Stack ($B \rightarrow \text{Stack}$)
PULB	33	PULL B from Stack ($\text{Stack} \rightarrow B$)
DES	34	DECREMENT Stack Pointer ($SP - 1 \rightarrow SP$)
INS	31	INCREMENT Stack Pointer ($SP + 1 \rightarrow SP$)
JMP	7E	JUMP ($PC \leftarrow \text{Adres}$, Extended)
JSR	BD	JUMP TO SUBROUTINE (Alt programa dallanma)
RTS	39	RETURN FROM SUBROUTINE (Alt programdan dönüş)
BRA	20	BRANCH ALWAYS (Koşulsuz dallanma, Relative)
BEQ	27	BRANCH IF EQUAL ($Z=1$ ise dallan, Relative)
BNE	26	BRANCH IF NOT EQUAL ($Z=0$ ise dallan, Relative)

Tablo 1.3: 6800 Emir Seti

```

mov ax, cx ; cx, ax'e kopyalanır. Bölme işlemi için.
mov bl, 3 ; bl 3 olsun.
div bl ; ax / bl
cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
jne cont ; Kalan varsa (tam bölünmüyorsa), 'cont' etiketine atla.
mov bl, 2 ; Sonra bl 2 olsun.
div bl ; ax / bl
cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
jne cont ; Kalan varsa (tam bölünmüyorsa), 'cont' etiketine atla.
add dx, cx ; Sayı hem 2 hem 3'e tam bölünüyorsa toplama ekle.
cont:
loop sum ; Bir sonraki sayı için 'sum' etiketine dön.
exit:
mov eax, 1 ; Programdan çıkış hazırlığı.
mov ebx, 0 ; Return 0.
int 80h ; Linux çekirdeğini çağır (soft interrupt 80h).
```

1.10 Program Türleri

Geliştirdiğimiz programlar üç ana grupta incelenebilir:

- ♦ **Sistem yazılımları**, bilgisayar donanımını yöneten ve uygulama yazılımları için bir platform sunan temel katmandır; işletim sistemleri (Windows, Linux) ve sürücüler bu grubun en net örnekleridir.

- ◇ **Uygulama yazılımları**, son kullanıcının belirli görevleri (metin yazma, internette gezinme, oyun oynama) yerine getirmesi için tasarlanmış, donanımdan ziyade kullanıcı odaklı araçlardır.
- ◇ **Gömülü yazılımlar** ise genel amaçlı bilgisayarlardan ziyade; beyaz eşyalar, otomobiller veya endüstriyel robotlar gibi belirli bir donanımı kontrol etmek üzere o cihazın içine hapsedilmiş, genellikle kısıtlı kaynakla çalışan ve yüksek kararlılık gerektiren özel kod parçalarıdır.

Özellik	Sistem Yazılımları	Uygulama Yazılımları	Gömülü Yazılımlar
Ana Hedef	Donanımı yönetmek	Kullanıcıya hizmet etmek	Cihazı çalıştırmak
Kullanıcı Etkileşimi	Arka planda çalışır	Doğrudan etkileşim kurar	Genellikle görünmezdir
Donanım Bağımlılığı	Çok yüksek	Düşük (Platform bağımlı)	Tam bağımlı
Örnek	Windows 11, macOS	Microsoft Word, Chrome	Çamaşır makinesi kontrolcüsü

Tablo 1.4: Yazılım Türleri ve Temel Farkları

1.11 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Farklı kullanıcıların kendi ihtiyaçlarına göre özelleştirip kullanabildiği sistemlere genel amaçlı bilgisayarlar adı verilir. Bir bilgisayarın bu esnekliğe sahip olabilmesi için; veri girişini sağlayan klavye/fare (giriş birimleri), verilerin kullanıcıya sunulacağı monitör (çıkış birimi) ve en önemlisi, işlenen verilerin elektrik kesildiğinde dahi kalıcı olarak saklanabileceği sabit disk (HDD/SSD gibi ikincil depolama birimleri) gibi temel donanım bileşenlerine sahip olması gerekir.

Genel amaçlı bir bilgisayarın donanımsal olarak ayağa kalkması ve kullanıcının komutlarına yanıt verebilecek bir çalışma ortamı sunabilmesi, şu yazılımsal ve operasyonel aşamalar doğrultusunda gerçekleşir:

1. **İşletim Sistemi (operating system)**: Donanım kaynaklarını yöneten, uygulama programlarının çalışması için güvenli bir platform sunan ve kullanıcı ile bilgisayar arasında köprü kuran en temel sistem programları bütünüdür. Tarihsel ve güncel süreçte DOS, Windows, Unix, Linux, macOS, ChromeOS ve OS/2 bu ekosistemin en çok bilinen örnekleridir.
2. **BIOS ve Açılış Döngüsü (Basic Input Output System)**: Bilgisayara ilk elektrik verildiğinde, işlemci ana bellekte (RAM) henüz hiçbir şey olmadığı için doğrudan anakart üzerindeki kalıcı bir salt okunabilir bellek (**Read-Only Memory - ROM**) yongasına kaydedilmiş olan **BIOS** programını çalıştırmaya başlar. BIOS'un ilk görevi **POST (Power-On Self-Test)** adı verilen bir kendi kendini sınama sürecidir. Bu süreçte klavye, monitör, RAM ve disk gibi hayati donanımların varlığı ve hatasız çalışıp çalışmadığı elektronik olarak kontrol edilir. Eğer bir sorun yoksa, BIOS diskin özel bir sektöründe yer alan ön yükleyiciyi (**bootloader**) tetikler ve işletim sistemini ana belleğe (RAM) yükleyerek kontrolü ona devreder. Bu andan itibaren bilgisayar, kullanıcının isteklerini dinlemeye hazır hale gelir.
3. **Programlama ve Çalıştırma Esnekliği**: İşletim sistemi kontrolü ele aldıktan sonra kullanıcı, sistem kaynaklarını kullanarak amaca yönelik uygulamalar çalıştırabilir. Örneğin bir yazılım geliştirici, işletim sistemi üzerinde çalışan araçlar vasıtasıyla yeni bir programın kaynak kodunu derleyebilir (**compile**), hata ayıklaması yapabilir (**debug**) veya daha önceden derlenmiş çalıştırılabilir (**executable**) bir dosyayı doğrudan RAM'e yükleyerek koşturabilir.
4. **Metin Tabanlı Kullanıcı Arayüzü - CLI (Command Line Interface)**: Grafiksel bir ekranın olmadığı veya tercih edilmediği işletim sistemlerinde (geleneksel DOS, Unix ve Linux dağıtımları gibi) komutlar, klavye aracılığıyla doğrudan metin olarak yazılır. Komutların girildiği ve sistem çıktılarının metinsel olarak izlendiği bu siyah ekrana **konsol**, **terminal** veya **komut satırı** adı verilir. Grafik arayüzün getirdiği donanımsal yükü (RAM ve CPU tüketimini) tamamen ortadan kaldırdığı ve sistem yönetiminde mutlak hız/otomasyon sağladığı için CLI mimarisi, günümüzde sunucu (**server**) ve bulut bilişim sistemlerinde hala birincil olarak tercih edilmektedir.
5. **Grafik Kullanıcı Arayüzü - GUI (Graphical User Interface)**: Kullanıcıların komutları ezberlemek zorunda kalmadan; ikonlar, pencereler, menüler ve görsel bileşenler üzerinden fare tıklamaları veya dokunmatik ekran hareketleriyle bilgisayarı yönetmesini sağlayan modern arayüz mimarisidir. Windows ve macOS doğrudan kendilerine ait entegre grafik arayüzleriyle gelirken; Linux dünyasında kullanıcılar çekirdeğin (**kernel**) üzerine diledikleri masaüstü ortamını (**desktop environment**) seçip yükleyebilirler. KDE, Mate, GNOME, Cinnamon, LXQt, XFce ve Deepin gibi popüler Linux masaüstü ortamları, kullanıcılara yüksek düzeyde görsel esneklik ve kullanım kolaylığı sunar.

1.12 Düşün ve Çöz

1. Bir işlemcinin adres yolu 24-bit, veri yolu ise 16-bit genişliğindedir. Bu işlemcinin doğrudan adresleyebileceği maksimum bellek miktarını bayt (Byte) cinsinden hesaplayınız.
2. RISC ve CISC mimarileri arasındaki temel farkları göz önüne alarak, modern akıllı telefonlarda (ARM) neden ağırlıklı olarak RISC mimarisinin tercih edildiğini açıklayınız.

2. Bölüm

Temel Yazılım Kavramları

2.1 Düşük Düzey Programlama

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascalline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar (punch card)** kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

İlk Programcı

Augusta Ada Byron, **ilk programcı** olarak kabul edilir.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı **bool cebiri** üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabilirdiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 Yılında *Konrad Zuse* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde **ENIAC (Electronic Numerical Integrator and Computer)** isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. Neumann tarafından geliştirilmiş bu mimari (**Stored-program computer and Universal Turing machine** § **Stored-program computer**), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m² yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. MARK-I bilgisayarından bin kat daha hızlıydı.

Yukarıda anlatılan bilgisayarlar, bir önceki bölümde anlatılan emir kodu (**instruction code**) veya hatırlatıcı isimlerle (**mnemonic**) yazılan programlar ile programlanmaktadır.

Düşük Düzey Programlama

Emir kodu ve hatırlatıcı isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama (low level programming)** olarak adlandırılır. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır. Bu programlama türünde hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

1943-1945 Yılları arasında *Konrad Zuse*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisai-

yarı için geliştirmiştir.

Yüksek Düzey Programlama Dilleri

Emir kodlarını (**instruction code**) ve hatırlatıcı isimleri (**mnemonic**) içermeyen programlama dilleri, **yüksek düzey programlama dili** (**high level programming language**) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

2.2 Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (**high level programming language**) olarak adlandırılır. Yani emir kodu (instruction code) ve sembolik isim (mnemonic) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (**statement**) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla emirden (**instruction**) oluşur.

Talimat

Yüksek seviyeli bir dilde talimatlar, yapılması gerekeni kısa ve net olarak belirten **mantıksal sıralardır** (**logical sequence**). Talimatlar, anahtar kelimeler kullanılarak yazılır.

1954 Yılında **FORTRAN** (**FORmula TRANslation**), John Backus liderliğinde IBM 704 için geliştirildi. FORTRAN, **READ**, **WRITE**, **IF** gibi 32 farklı talimata (**statement**) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir. Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir ¹.

```

1 C AREA OF A TRIANGLE WITH A STANDARD SQUARE ROOT FUNCTION
2 C INPUT - TAPE READER UNIT 5, INTEGER INPUT
3 C OUTPUT - LINE PRINTER UNIT 6, REAL OUTPUT
4 C INPUT ERROR DISPLAY ERROR OUTPUT CODE 1 IN JOB CONTROL LISTING
5 C READ INPUT TAPE 5, 501, IA, IB, IC
6 read(*,*) IA, IB, IC
7 C 501 FORMAT (3I5)
8 C IA, IB, AND IC MAY NOT BE NEGATIVE OR ZERO
9 C FURTHERMORE, THE SUM OF TWO SIDES OF A TRIANGLE
10 C MUST BE GREATER THAN THE THIRD SIDE, SO WE CHECK FOR THAT, TOO
11 IF (IA) 777, 777, 701
12 701 IF (IB) 777, 777, 702
13 702 IF (IC) 777, 777, 703
14 703 IF (IA+IB-IC) 777, 777, 704
15 704 IF (IA+IC-IB) 777, 777, 705
16 705 IF (IB+IC-IA) 777, 777, 799
17 777 STOP 1
18 C USING HERON'S FORMULA WE CALCULATE THE
19 C AREA OF THE TRIANGLE
20 799 S = FLOAT(IA + IB + IC) / 2.0
21 AREA = SQRT( S * (S - FLOAT(IA)) * (S - FLOAT(IB)) *
22 + (S - FLOAT(IC)))
23 C WRITE OUTPUT TAPE 6, 601, IA, IB, IC, AREA
24 write(*, 601) IA, IB, IC, AREA
25 601 FORMAT (4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
26 + 13H SQUARE UNITS)
27 END

```

Örnekte görüldüğü üzere FORTRAN II dilinde her talimat metnin altıncı sütunundan başlar ve 72. sütununda biter. Altıncı sütuna kadar olan yerlere numara ile satır etiketi konulabilir (12-17, 20 ve 25. satırlar). Eğer talimat bu aralığa sığmaz ise beşinci sütuna "+" karakteri konularak (22 ve 26. satırlar) talimata devam edilir. Kodun içine açıklamalar "C" karakteri ile başlayan satırlarla konulur. Kod içinde yer alan talimatlar kısa öz ve anlaşılırdır.

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar **UNIVAC-I** oldu.

¹https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu.

1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Bu yıl ve sonrasında transistör içeren **ikinci kuşak bilgisayarlar** hayatımıza girmiştir.

2.3 Arapsacı Kod

1955-1959 Yılları arasında **FLOW-MATIC B0 (Business Language Version 0)** Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir ².

```
(0) INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1) COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2) TRANSFER A TO D .
(3) WRITE-ITEM D .
(4) JUMP TO OPERATION 8 .
(5) TRANSFER A TO C .
(6) MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7) WRITE-ITEM C .
(8) READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9) JUMP TO OPERATION 1 .
(10) READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11) JUMP TO OPERATION 1 .
(12) SET OPERATION 9 TO GO TO OPERATION 2 .
(13) JUMP TO OPERATION 2 .
(14) TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
OTHERWISE GO TO OPERATION 15 .
(15) REWIND B .
(16) CLOSE-OUT FILES C ; D .
(17) STOP . (END)
```

Örnekte görüldüğü üzere talimatlar parantez içine yazılan satır numaraları ile başlar ve nokta karakteri ile biter. Böylece FORTRAN'dan daha kolay birden fazla metin satırından oluşan talimatlar yazılabilmektedir.

Arapsacı Kod

Bu döneme kadar yayılan programlamada, bir sürü **GO TO** ya da **JUMP TO** talimatı kullanıldığından, talimatların hangi sırada çalıştığının yani **program akışının (control flow)** anlaşılması zordur. Bu nedenle programda **okunaklılık oldukça azdır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsacı kod (spaghetti code)** adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

1958 Yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir;

```
(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)
```

1959 Yılında **COBOL (Common Business-Oriented Language)** programlama dili, CODASYL komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir;

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.
```

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).
```

²<https://gist.github.com/marcgg/f9f2da31a973ba319809>


```
PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.
```

1958 Yılında daha sonra **ALGOL** (**ALGO**ritmic Language) adını alan **IAL** (**International Algebraic Language**) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile birlikte kod blokları (**code block**), iç içe fonksiyon (**nested function**), değişken kapsamı (**scope**) ve dizi (**array**) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```
1 BEGIN
2 REAL LOCAL, X, Y, Z;  -- Declaration
3 LOCAL :=
4 3.14;
5 X := 5; Y := 10; Z := X + Y;
6 PRINT(LOCAL)  -- No semicolon before END
7 END
```

Örneği incelediğimizde bir satırda birden fazla talimat yazılacağı gibi (5.satır) birden fazla metin satırında tek bir talimat yazılabilir (3 ve 4. satır). Çünkü talimatlar nerede başlarsa başlasın noktalı virgül karakteri ile biter. Bu de gelişigüzel (**free-format**) kod yazmanın önünü açan önemli bir özelliktir.

1958 Yılındaki FORTRAN diline çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, değer üreten fonksiyonlar için **FUNCTION**, **CALL** ve **RETURN** özellikleri eklenmiş ve **FORTRAN-II** adını almıştır.

Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için **PROCEDURE** ve **BEGIN-END** kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline **INTEGER**, **REAL**, **DOUBLE PRECISION**, **COMPLEX** ve **LOGICAL** veri tipleri (**data type**) ile Mantıksal **IF**, aritmetik (üç yollu) **IF** ve **DO** döngüsü talimatları (**statement**) eklenmiştir.

1964 yılında Dartmouth College'da John G. Kemeny ve Thomas E. Kurtz tarafından geliştirilen **BASIC** (**Beginner's All-purpose Symbolic Instruction Code**) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı olarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980'li yılların başlarına kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

Örneği incelediğimizde numara ile başlayan her satıra bir talimat yazılmaktadır. BASIC dilindeki satır numaralarının (10, 20, 30...) sadece sıralama değil, aynı zamanda **GOTO** talimatlarının hedefi olan etiketlerdir. Yani program gelişigüzel yazılamamaktadır.

2.4 Değişken

Değişkenler (**variable**), en temel tanımıyla cebirde kullandığımız soyut sembollerin bilgisayar bilimindeki doğrudan karşılığıdır. Matematikte değişken (*variable*), en genel tanımıyla, belirli bir evren veya tanımlı bir küme içerisinde herhangi bir öğeyi temsil eden ve süreç boyunca farklı öğeleri de temsil eden sembolik bir niceliktir. Matematiksel işlemlerde ve algoritmalarda doğrudan sabit sayıları kullanmak yerine, o an hangi değere sahip olacağı değişebilen, ancak formülün mantıksal yapısını koruyan sembolik ifadeler tercih edilir.

$$3x^2 + 2x + 10$$

$$c^2 = a^2 + b^2$$

$$c = 2\pi r$$

Yukarıda verilen cebirsel ifadelerin ilkinde matematiksel bir polinom, ikincisinde bir dik üçgenin kenar uzunlukları arasındaki geometrik ilişki (Pisagor Teoremi), üçüncüsünde ise bir çemberin çevre hesabı göster-

ilmiştir. Bu fonksiyonel ifadelerde yer alan x , a , b ve r değerleri dışarıdan bağımsız olarak değişebildiği için **bağımsız değişken**; bu girdilere bağlı olarak nihai sonucu üreten c ise **bağımlı değişkendir**. Formüllerdeki 3, 2, 10 sayıları ile matematiksel bir sabit olan π sayısı ise sürecin hiçbir aşamasında değişmeyen **sabit (constant)** değerlerdir.

Değişkenlerin Bellek Karşılığı

Değişkenler, bilgisayarlarda fiziksel bir miktar bellek bölgesini (adresini) işgal eder. Yüksek seviyeli dillerde (Python, C++, Java vb.) bir değişken tanımladığımızda, arka planda işletim sistemi ve derleyici, o değişkenin türüne göre ana bellekten (RAM) belirli sayıda bayt boyutunda bir alanı ayırır (rezerve eder). Değişkenin ismi, o bellek kutusunun üzerindeki etikettir; içindeki değer ise kutunun muhteviyatıdır.

Bilgisayar mimarisinde veriler, yapısal özelliklerine göre belleğe farklı matematiksel formatlarda kaydedilirler. Bu formatlar temel olarak ikiye ayrılır:

§2.4.1 Tam Sayıların Bellek Temsili

Tam sayılar (integer), doğrudan **ikili (binary)** sayı sistemine dönüştürülerek bellek hücrelerinde saklanır. Bilgisayarda negatif sayıları da ifade edebilmek adına, sayıyı tutan bit diziliminin en solundaki (en anlamlı) biti, **işaret biti (sign bit)** olarak ayrılır. Eğer bu bitin değeri 0 ise sayı pozitif, 1 ise negatiftir.

Bilgisayarlar negatif sayıları donanımsal olarak çıkarma işlemini kolaylaştırmak amacıyla **İkiye Tümlleyen (two's complement)** yöntemiyle kodlar. **İşaretli (signed)** n bitlik bir tam sayı için değer aralığı -2^{n-1} ile $2^{n-1} - 1$ arasındadır. Buna göre işaretli 8 bitlik (1 baytlık) bir sayı için sınır değerleri -2^7 ile $2^7 - 1$ (yani -128 ile +127) arasında olacaktır.

Eğer negatif sayılara ihtiyacımız yoksa, en soldaki bit de sayı değerine dahil edilir ve sayı **işaretsiz (unsigned)** olarak tanımlanır. İşaretsiz n bitlik bir sayının değer aralığı ise 0 ile $2^n - 1$ arasındadır (Örneğin 8 bit için 0 ile 255 arası). Tablo 2.1'de programlama dillerinde en sık kullanılan tam sayı türlerinin bellek gereksinimleri ve matematiksel sınırları özetlenmiştir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları (İşaretli / İşaretsiz)
8 Bit	1 bayt	-128 ile +127 arasında veya 0 ile 255 arasında
16 Bit	2 bayt	-32.768 ile +32.767 arasında veya 0 ile 65.535 arasında
32 Bit	4 bayt	-2.147.483.648 ile +2.147.483.647 arasında veya 0 ile 4.294.967.295 arasında
64 Bit	8 bayt	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807 arasında veya 0 ile 18.446.744.073.709.551.615 arasında

Tablo 2.1: Tam Sayı Türleri ve Donanımsal Sayı Sınırları

§2.4.2 Gerçek Sayıların Bellek Temsili

Ondalıklı ve kesirli ifadeleri içeren **gerçek sayılar (real numbers)**, bellekte tam sayılar gibi doğrudan saklanamaz. Bunun yerine bilimsel gösterime benzeyen **kayan noktalı sayı (floating point number)** standardına göre biçimlendirilirler. Bu deha ürünü kavramın temelleri, ilk olarak 1914 yılında İspanyol mühendis ve matematikçi Leonardo Torres Quevedo tarafından, Charles Babbage'ın teorik Analitik Makinesi'ne aritmetik esneklik kazandırmak amacıyla yayınlanmıştır.

Kayan noktalı sayıların çalışma mantığı, ondalık sistemdeki şu matematiksel kaydırma işlemine dayanır:

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Görüldüğü üzere, kesir noktasının sola veya sağa kaydırılması (yüzdürülmesi) ve bu hareketin bir üs (10^n) ile dengelenmesi sayesinde aynı sayı farklı biçimlerde temsil edilebilmektedir. Kesir noktası sola kaydırıldığında sayı taban çarpanı (10) ile çarpılmış, sağa kaydırıldığında ise bölünmüş olur.

Bilgisayar dünyasında (ikili tabanda) bu gösterim; sayının işaretini belirten bölüm, sayının büyüklük mertebesi belirleyen üs (**exponent**) kısmı ve sayının hassasiyetini/netliğini oluşturan kesir (**fraction/mantissa**) kısmı olmak üzere üç bileşene ayrılır. Kayan noktalı sayının bilimsel gösterimindeki virgülden sonra gelen hanelerine ise mantis (**mantissa**) adı verilir.

Günümüz bilgisayarlarında gerçek sayılar, küresel bir standart olan IEEE 754 protokolüne göre kodlanır. Tablo 2.2'de modern yazılım dillerinde sıklıkla karşılaştığımız tek ve çift hassasiyetli kayan noktalı sayıların bellek anatomisi verilmiştir³.

³IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1-84

Bit Sayısı	Bellek Miktarı	IEEE 754 İç Yapısı ve Hassasiyet Sınırları
32 Bit	4 bayt	$1,2 \times 10^{-38}$ ile $3,4 \times 10^{+38}$ arasındaki tek hassasiyetli (single precision) gerçek sayılar. En soldaki 1 bit işaret, sonraki 8 bit üs ve kalan 23 bit kesir (mantis) olarak kodlanır.
64 Bit	8 bayt	$2,3 \times 10^{-308}$ ile $1,7 \times 10^{+308}$ arasındaki çift hassasiyetli (double precision) gerçek sayılar. En soldaki 1 bit işaret, sonraki 11 bit üs ve kalan 52 bit kesir (mantis) olarak kodlanır.

Tablo 2.2: Kayan Noktalı Sayı Standartları ve Bit Dağılımları

1938 yılında *Konrad Zuse*, ilk ikili (**binary**) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır.

Kayan Noktalı Sayılar Da Bellekte İkili Tabanda Tutulur

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (binary) sayı tabanında tutulur**. Örneğin ondalık tabanda 50,25 sayısını ikili tabanda 1.1001001×2^5 olarak ifade edilir.

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi (data type)** belirler. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya **bayt** denilirken, C dilinde **char** adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

2.5 Fonksiyon

Matematikte **fonksiyon (function)**, bir kümenin bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir (relationship)**.

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 2.3: İki ile Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade (expression)** edilir.

$$f(x) = 2x \quad (2.1)$$

Burada f , fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x \quad (2.2)$$

Bu durumda bağımlı değişken y ile bağımsız değişken olan x arasındaki ilişkiyi formülüze eder. Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$f(x) = 2x + 33 \quad (2.3)$$

Bu durumda ise anlatılan değişkenler arasındaki ilişkiyi tanımlayan formüle f gibi bir isim verilir. İsmi belirlenen bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan $f(x)$ değerine eşleyen bir ifadedir. Yani $x = 1.0$ argümanını $f(1.0) = 5.0$ reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür (return)**.

$$g(a, b) = 2a + b \quad (2.4)$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile b elemanını, hesaplanan $g(a, b)$ değerine eşleyen bir ifadedir. Yani $a = 1.0$ ve $b = 1.0$ argümanlarını $g(1.0, 1.0)$ fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (**return**).

Parametre ve Argüman

Örneklerdeki x , a ve b bağımsız değişkenler olup **parametre** (**parameter**) olarak adlandırılır. $f(1.0)$ ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre** (**actual parameter**) ya da **argüman** (**argument**) olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0) \quad (2.5)$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler** (**call**).

2.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, birden fazla çağrı noktası olabilen **COROUTINE** ve **FUNCTION** kullanılıyor olsa da hala **GOTO** ve **JUMPTO** talimatları kullanılmakta ve kafa karışıklığına devam etmekteydi. 1968 Yılında *Edsger W. Dijkstra* önderliğinde **GOTO** talimatları **zararlı ilan edilmiştir**.

1970 Yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. **Yapısal programlamada** (**structural programming**) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (**if**, **for**, **do**, **repeat**) birbirinden ayrılmıştır. Pascal, kendi kendini çağıran **özyinelemeli** (**recursive**) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (**pointer**) desteklemektedir⁴. Aşağıda iki sayıyı kullanıcıdan alan ve konsola toplamı yazdıran bir program örneği verilmiştir;

```
program ToplamaIslemi;
var
sayi1, sayi2, toplam: integer; { Değişken tanımlama bölümü }
begin
{ Kullanıcıdan veri alma }
writeln('Birinci sayiyi giriniz:');
readln(sayi1);
writeln('İkinci sayiyi giriniz:');
readln(sayi2);
{ Hesaplama }
toplam := sayi1 + sayi2;
{ Sonucu ekrana yazdırma }
writeln('Girdiginiz sayıların toplamı: ', toplam);
{ Programın hemen kapanmaması için bir tuş beklemesi (isteğe bağlı) }
readln;
end.
```

Yapısal Programlamada Okunaklılık Yüksek

Yapısal programlamada, bir ana fonksiyon ve alt fonksiyonlara tanımlanarak fonksiyonların birbirini çağırmasıyla programlama yapılır. Her bir fonksiyonda verinin tutulacağı veri yapıları ile bu verileri işleyen kontrol yapıları vardır. Bu yapılar birbirinden ayrıldığından, arapsaçı programlamanın aksine, talimatların hangi sırayla icra edildiğinin anlaşılması yani kontrol akışı (**control flow**) kolay anlaşılır. Bu nedenle **yapısal programlamada okunaklılık çok yüksektir**.

2.7 Yapısal Programlama Genel Çerçevesi

Yapısal programlama dillerinde aşağıdaki gene çerçeveye uyulur;

1. Programın başladığı yeri belirten bir **ana fonksiyon** (**main function**) tanımlanır. Yapılacak işi fonksiyonlar olarak yazar ve bu ana fonksiyonda birbirini çağıran fonksiyonlar olarak yazarız.
2. Ana fonksiyon dahil her fonksiyonda ilk önce **veri yapıları** (**data structure**) tanımlanır. Veri yapısı;
 - (a) En basit veri yapısı **char**, **integer**, **float** ve **double** gibi **ilkel veri tiplerinden** (**primitive data type**) olabilen **değişken** (**variable**),
 - (b) İlkel veri tiplerinden meydana gelen **dizi** (**array**),

⁴Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

- (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (**structure**),
 - (d) Ya da dinamik olarak yani çalıştırma anında birbirine bağlanmış verilerden oluşan **bağlı liste** (**linked list**) olabilir.
3. Veri yapısı tanımlandığı her fonksiyonda bu yapıları işleyecek **kontrol yapıları** (**control structure**) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **if**, **if-else**, **switch**, **while**, **do-while**, **for**, **continue**, **break**, **goto** ve **return** **talimatlarından** (**statement**) oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten **BEGIN-END** bloğudur. C dilinde ana fonksiyon, **main()** fonksiyonudur. C programlama dili, en çok kullanılan, öğrenmesi kolay bir yapısal programlama dilidir, hızlı ve verimli programlar üretir, assembly dili desteği ile **düşük düzey programlamayı** (**low level programming**) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda örneği verilen Pascal programının C dilindeki karşılığı aşağıda verilmiştir;

```
#include <stdio.h>
int main() {
    // Değişken tanımlama bölümü
    int sayi1, sayi2, toplam;
    // Kullanıcıdan veri alma
    printf("Birinci sayiyi giriniz:\n");
    scanf("%d", &sayi1);
    printf("İkinci sayiyi giriniz:\n");
    scanf("%d", &sayi2);
    // Hesaplama
    toplam = sayi1 + sayi2;
    // Sonucu ekrana yazdırma
    printf("Girdiginiz sayilarin toplami: %d\n", toplam);
    // Programın hemen kapanmaması için bir tuş beklemesi (isteğe bağlı)
    // Not: Pascal'daki boş 'readln' yerine C'de yaygın olarak getchar() kullanılır.
    getchar(); // scanf'ten kalan Enter karakterini temizlemek için
    getchar(); // Asıl tuş bekletme işlevi için
    return 0;
}
```

2.8 Nesne Yönelimli Programlama İhtiyacı

Yapısal programlamada talimatlar (statements) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden paradigma** (**imperative programming**) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden paradigmanın bir örneği olan yapısal programlamada;

- ◊ Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. Değişim yönetimi (change management) çok zordur ve uzun zaman alır.
- ◊ Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda hataların izini sürmek (error handling) işi zorlaştırır.
- ◊ Yapısal programlamada, gösterici (pointer) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun güvenli (safe code) olarak çalışmasının incelenmesi çok zordur.
- ◊ Sürekli olarak benzer projelerde aynı kodları tekrar yazmak (duplicate code) gereklidir.
- ◊ Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi başarısız (fail) olmuştur. Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur.

Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillerindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi. Smalltalk, 1972 yılında Xerox Park şirketinde Alan Kay önderliğinde Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, Scott Wallace ve Peter Deutsch tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (object) temel yapıtaşlarıdır. Nesneler;

- ◇ Durum (state) ve davranışlara (behavior) sahiptir.
- ◇ Programlama, nesnelerin birbirlerine ileti göndermesiyle (message-passing) yapılır.
- ◇ Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani yansıma (reflection) özelliği vardır.

Emreden paradigma (imperative programming) aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya nesne yönelimli programlama (object oriented programming-OOP) paradigması adı verilir.

2.9 Nesne Yönelimli Programlama

Yapısal programlamada talimatlar (statement) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya emreden programlama paradigması (imperative programming) adı verilir. Bu paradigmaya sahip diller, yazılım yapılacak süreçle ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Emreden programlama (imperative programming) bir örneği olan yapısal programlamada talimatlar (statements) art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve bir önceki başlıkta anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir. Bu dillerde nesneler (object) programın temel yapıtaşlarıdır. Nesneler; durum (state) ve davranışlara (behavior) sahiptir. Programlama, nesnelerin birbirlerine ileti göndermesiyle (message-passing) yapılır. Emreden paradigmanın aksine programlama, nesnelerin birbirlerine ileti göndermesi ile yapılır. Bu bakış açısıyla yapılan programlamaya nesne yönelimli programlama (object oriented programming) paradigması adı verilir.

C++ programlama dili, C diline yapılan nesne yönelimli programlama eklentileri yapılarak geliştirilmiş bir programlama dilidir. Danimarkalı bilgisayar bilim adamı olan Bjarne Stroustrup, sonradan C++ olarak bilinecek olan "C with Classes" üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++, nesne yönelimli programlama kavramlarına sahip C programlama dilinin bir uzantısıdır. C++ gibi genel amaçlı (general purpose), Java ve C# gibi nesne yönelimli birçok programlama dili daha sonra geliştirilmiş ve günümüzde popüler olmuştur. Şimdiye kadar anlatılan bu dillerin dolaylı da olsa bilgisayar mimarisi ile ilişkisi vardır. Gelişmiş yazılımlar yapabilmek için bir miktar mimariden anlamak gereklidir. Ayrıca Java, C# ve C++ nesnelerle ilgili birçok kütüphaneye sahiptir. Bu kütüphaneleri kullanabilmek için birçok kavramı bir anda özümsemek gerektiğinden öğrenme eğrisi diktir.

2.10 Python Programlama Dili

Python dili ise, *Guido van Rossum* tarafından geliştirilmiş ve ilk olarak 1991'de yayınlanan, genel amaçlı (general purpose) yüksek düzey (high level) bir programlama dilidir. *Guido van Rossum*, bu dili **Monty Python** komedi grubundan esinlenerek isimlendirmiştir.

Python, dinamik bir tip sistemine (dynamic type system) ve otomatik bellek yönetimine (automatic garbage collection) sahiptir ve nesne yönelimli (object oriented), emreden (imperative), fonksiyonel programlama ve yordamlı (procedural) programlama paradigmalarını destekler.

Python programlama dili;

- ◇ Fonksiyonel ve yapısal programlama paradigmasının yanı sıra nesne yönelimli programlamayı da destekler.
- ◇ Bir betik dili olarak kullanılabilir veya büyük uygulamalar oluşturmak için bayt koduna derlenebilir.
- ◇ Yüksek seviyede dinamik veri tipleri sağlar ve denetimini destekler.
- ◇ Otomatik çöp toplamayı destekler.
- ◇ C, C++, COM, ActiveX, CORBA ve Java ile kolayca entegre edilebilir.

Geniş ve kapsamlı bir standart kütüphanesi vardır;

- ◇ Veri Biliminde, veri analizi ve görselleştirme için Numpy, Pandas ve Matplotlib gibi Python kütüphaneleri kullanılır.
- ◇ Django ve Pyramid gibi Python çerçeveleri, Web Uygulamalarının geliştirilmesini ve dağıtımını kolaylaştırır.
- ◇ Uygulama alanları arasında bilgisayarlı görme ve görüntü işleme alanları da vardır.
- ◇ Otomasyon, İş Planlama, GUI geliştirme gibi birçok görevde de tercih edilmektedir.

Bilgisayarın mimarisini bilmeden programlama yapmayı sağlayacak nesneler kullanılarak ve tasarlanarak programlama yapılır. Bu nedenle Python, şüphesiz harika bir dildir. Çünkü sağladığı işlevsellik miktarı başka hiçbir dilde mevcut değildir. Ancak Python dilindeki bellek tüketimi çok yüksek olduğundan bellek kısıtlamalarının olduğu bir yerde kötü bir seçim haline getirir.

2.11 Soyutlama Merdiveni

Yazılım dünyasında soyutlama merdiveni (**abstraction ladder**), bir programlama dilinin bilgisayar donanımına (işlemci ve bellek) ne kadar yakın veya insan diline (mantık ve söz dizimi) ne kadar benzer olduğunu gösteren bir hiyerarşidir.

Seviye	Tanım	Örnekler	Donanım Yakınlığı
Donanım	Elektrik sinyalleri, kapılar ve transistörler.	İşlemci (CPU), RAM	%100
Makine Dili Assembly	İşlemcinin doğrudan anladığı 0 ve 1'ler (binary). Emir kodunun (instruction) kısa kelimelerle temsilidir.	01101010... x86 ASM, ARM ASM	Çok Yüksek Yüksek
Düşük Düzey	Donanım üzerinde tam kontrol, manuel bellek yönetimi.	C, Rust	Orta-Yüksek
Yüksek Düzey	Nesne yönelimli yapılar, otomatik bellek yönetimi.	C#, Java, C++	Orta-Düşük
Ultra Yüksek	İnsan diline en yakın, yüksek soyutlama seviyesi.	Python, Ruby	Düşük

Tablo 2.4: Soyutlama Merdiveni

Merdivende tırmandıkça veya indikçe iki temel değişken arasında bir takas (**trade-off**) gerçekleşir.

Tabloda üst satırlara doğru yani **C, Assembly Tarafına Doğru İlerledikçe; Performans Artar**: Kod doğrudan donanıma hitap eder, aracı katmanlar azalır. **Kontrol Artar**: Belleğin her bir baytı siz yönetirsiniz. **Zorluk Artar**: Yazması ve hata ayıklaması (**debugging**) çok daha zordur.

Tabloda alt satırlara doğru yani **C#, Python Tarafına Doğru İlerledikçe; Verimlilik Artar**: Az kodla çok iş yapılır (Üretkenlik). **Güvenlik Artar**: Bellek yönetiminde çöp toplama (**garbage collector**) gibi süreçler dil tarafından otomatik yapılır. **Taşınabilirlik Artar**: Kodunuzun farklı işletim sistemlerinde ve işlemcilerle çalışması daha kolaydır.

Soyutlamayı anlamak için araba örneğini düşünelim:

- ♦ **Düşük Düzey (C/Assembly)**: Arabanın motorunu, pistonlarını ve yakıt enjeksiyonunu doğrudan yönetmektir. Arabanın nasıl çalıştığını bilmeniz gerekir.
- ♦ **Yüksek Düzey (C#/Python)**: Sadece direksiyon, gaz ve freni kullanmaktır. Motorun içindeki karmaşık detaylar sizden soyutlanmıştır. Siz "hızlan" (gaz) dersiniz, alt katmanlar bunu motorun anlayacağı dile çevirir.

2.12 Derleyici ve Yorumlayıcı

Yüksek seviyeli programlama dilleriyle yazdığımız kaynak kodlar (**source code**), insanlar tarafından kolayca okunup anlaşılabilir semantik bir yapıya sahiptir. Ancak bilgisayar mimarisinin kalbi olan işlemci, daha önce gördüğümüz gibi sadece elektrik sinyallerine karşılık gelen 0 ve 1 dizilimlerinden, yani makine kodundan (**machine code**) anlar. İnsan algısına uygun bu metinlerin, donanımın icra edebileceği deterministik emirlere dönüştürülmesini sağlayan iki temel yazılımsal dönüştürme stratejisi mevcuttur: **Derleyici (compiler)** ve **Yorumlayıcı (interpreter)**.

§2.12.1 Derleyici

Derleyici (compiler), geliştirici tarafından yazılan kaynak kodun tamamını dil bilgisi (**syntax**) kuralları açısından **tek bir kerede** analiz eden ve **hedef işlemci mimarisinin emir setine uygun**, doğrudan çalıştırılabilir ikili (**binary executable**) bir dosya (**.exe**, **.out** vb.) üreten sistem yazılımıdır. Derleme sonrasında oluşan dosya defalarca çalıştırılabilir. Derleme felsefesi, yabancı dildeki bir kitabın tamamını bir kerede hedef dile çevirip matbaada yeni bir kitap olarak basan mütercime benzer. Bu mimarinin özellikleri şu şekildedir:

- ♦ **Performans Odaklılık**: Program çalıştırılmadan önce bir kez derlendiği için, yürütme esnasında tekrar bir çeviri işlemine gerek kalmaz; bu nedenle derlenen diller (C, C++, Rust vb.) **maksimum hızda ve verimli** çalışır.
- ♦ **Fikri Mülkiyet ve Dağıtım Kolaylığı**: Yazılım dağıtılırken ham kaynak kodların paylaşılmasına gerek yoktur. Kullanıcıya yalnızca derlenmiş ikili dosya (**executable binary file**) teslim edilir.
- ♦ **Platform Bağımlılığı**: Derleyici, kodu doğrudan hedef donanımın (Örn: x86-64 Intel işlemci ve Windows işletim sistemi) emir setine göre ürettiği için, o dosya başka bir mimaride (Örn: ARM tabanlı bir akıllı telefonda) doğrudan çalışmaz.
- ♦ **Katı Hata Denetimi**: Kaynak kodda tek bir virgül hatası dahi olsa derleme işlemi başarısız olur. Tüm hatalar raporlanır ve kodun düzeltilip **baştan aşağı yeniden derlenmesi** gerekir.

Derleme işlemi sırasında aşağıdaki adımlar izlenir;

- ◊ Koda ilişkin derleyiciye argüman olarak kod dosyası verilir.
- ◊ Derlenecek kodda hata yoksa işlemci özel makine kodlarından oluşan icra edilebilir (executable) dosya oluşur. Bu nedenle bu dosyaya doğal kod (native code) adı da verilir.
- ◊ Kullanıcı arayüzünden verilen bir komutla icra edilebilir bu dosya işletim sistemi kontrolünde işlemci tarafından çalıştırılır. Çalıştırılan bu program ihtiyacımızı gördüğü sürece yeniden derlemeye gerek yoktur.

Aşağıda bir C programının derleme çalıştırma süreci verilmiştir;

1. Daha sonra metin editörü ile ilk kaynak kodumuzu oluşturmak için komut satırında aşağıdaki komut yazılır;
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. Tüm hakları saklıdır.
C:\Users\ILHANOZKAN>notepad hello.c
2. Eğer belirtilen isimde bir dosya yoksa editör yeni bir dosya oluşturmak için onay ister. Açılan metin editörü penceresine ilk C programı yazılır ve `hello.c` adıyla kaydedilir.
#include <stdio.h>
int main() {
/* İlk C Programı */
printf("Hello, World! \n"); //Konsola Hello, World! Yazılacak Talimat (Statement)
return 0; // Programdan çıkış talimatı
}
3. Yazılan C programı iki satırlık bir programdır. Yazılım dilinde satır, aslında mantıksal satır (logical sequence) anlamındadır ve C dilinde her mantıksal satır bir talimat (**statement**) olup noktalı virgül karakteri ile biter.
4. Aşağıdaki komut ile `hello.c` olarak hazırlanan kaynak kodumuz GCC derleyicisi tarafından icra edilebilir `hello.exe` dosyası olarak derlenir. (GCC derleyicisinin bilgisayarda kurulu olduğu varsayılmıştır.)
C:\Users\ILHANOZKAN>gcc hello.c -o hello.exe
5. Derlenen programı çalıştırmak için yine aynı komut satırında `hello.exe` yazarak icra edilebilir programımız işletim sistemi tarafında işlemciye hedef gösterilerek çalıştırılır.
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
6. Burada çalıştırılabilir dosya olan `hello.exe`'nin, istenildiği kadar çalıştırılabildiğine dikkat edilmelidir;
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
7. Eğer konsola bir başka şey yazdıran bir program isteseydik birinci adımdaki `hello.c` dosyasını yeniden açtıktan sonra kod üzerinde gerekli değişiklikleri kaydedip sonraki adımları tekrar yapmalıyız.

§2.12.2 Yorumlayıcı

Yorumlayıcı (interpreter), kaynak kodu baştan sona tek bir seferde çevirmek yerine, programın çalışması esnasında (**runtime**) **talimatlardan (statement) oluşan kod satırlarını** yukarıdan aşağıya doğru sırayla, satır satır okur; anlık olarak makine diline çevirir ve o anda işlemciye ileterek icra edilmesini sağlar. Yorumlama felsefesi, uluslararası bir konferansta konuşmacının sözlerini dinleyicilere eş zamanlı olarak aktaran simültane tercümana benzer. Ortada kalıcı, bağımsız çalışabilir ikili bir dosya oluşmaz. Programın her çalıştırılışında **yorumlayıcı yazılımın** arka planda aktif olması zorunludur.

Yorumlayıcı, bir programlama dilinde veya betik (**script**) dilde yazılmış talimatları, makine diline derleme olmadan doğrudan çalıştıran bir bilgisayar programıdır. Betik diller için derleme süreci yoktur. Betik diller her zaman bir ev sahibi uygulama tarafından her seferinde çalıştırılır. Örnek olarak;

- ◊ JavaScript için ev sahibi uygulama bir tarayıcıdır (**web browser**).
- ◊ PHP için ev sahibi uygulama web sunucusudur (**web server**).

- ◇ Windows'taki toplu işlem dosyaları (*.bat) için ev sahibi uygulama Windows komut istemidir (**Windows Command Prompt**).
- ◇ Linux için kabuk dosyaları (*.sh) için ev sahibi uygulama /bin/bash kabuğu (**shell**) uygulamasıdır.

Kısaca yorumlayıcı, bir dilde yazılmış kaynak kodu talimatlar (**statement**) halinde satır satır (**logical sequence**) okuyup çalıştıran bir programdır. Yorumlayıcılarda hata oluşursa, hatanın olduğu satırda durulur ve sonrasını çalıştırılmaz. Yorumlayıcılar derleyicilere göre genellikle daha yavaştır çünkü her seferinde kodu çözümlemesi gerekir. Bir yorumlayıcı genellikle program çalıştırmak için aşağıdaki stratejilerden birini kullanır:

- ◇ Kaynak kodu ayrıştırır (**parse**) ve çalıştırılacak satırları belirleyerek sırayla icra eder.
- ◇ Kaynak kodu daha kolay işleyecek bir biçime veya nesne koduna çevir ve ardından icra eder.
- ◇ Kaynak kod derleyici tarafından yorumlayıcının sanal makinesiyle eşleşen bir bayt kodu üretir. Yorumlayıcı sanal makinesi bu kodu doğrudan çalıştırır. Python dilinde bu strateji kullanılır.

§2.12.3 Python Dünyasında Durum

Kitabımızın ana odağı olan Python, geleneksel olarak yorumlanan bir dil kategorisinde değerlendirilse de, modern bilgisayar biliminde saf yorumlama performans kayıplarına yol açtığı için Python, hibrit (karma) bir mimari kullanır. Bir Python programını (bir .py dosyasını) çalıştırdığınızda arka planda şu iki aşamalı süreç işler:

1. **Bayt Koduna Derleme (Bytecode Compilation):** Python, kodunuzu çalıştırır çalıştırmaz önce onu satır satır makine diline çevirmez. Önce tüm kod bloklarını analiz ederek hızlıca Bayt Kodu (Bytecode) adı verilen, insan tarafından okunamayan ancak makine kodundan daha yüksek seviyede olan optimize edilmiş ara bir dile derler. Bu işlem sonucunda bilgisayarınızda otomatik olarak `__pycache__` klasörleri ve içinde `.pyc` uzantılı dosyalar oluşur.
2. **Python Sanal Makinesi (Python Virtual Machine):** Üretilen bu `.pyc` uzantılı bayt kodları, doğrudan fiziksel işlemciye (CPU) gönderilemez. İşte bu aşamada devreye Python'ın asıl yorumlayıcı motoru olan Python Sanal Makinesi girer. PVM, bu standart bayt kodlarını alır, satır satır okur ve o an üzerinde çalıştığı fiziksel bilgisayarın (Windows, Linux, Mac) işlemcisinin anlayacağı gerçek makine diline anlık olarak tercüme eder.

Python'ın bu hibrit ve akıllı yorumlama mimarisi, yazılımcılara çok büyük avantajlar sağlar:

- ◇ **Platform Bağımsızlığı (Cross-Platform):** Python ile yazdığınız bir kaynak kod, hiçbir değişikliğe uğramadan hem bir Windows bilgisayarda, hem bir Linux sunucuda, hem de bir macOS cihazda çalışabilir. Çünkü kod aynı bayt koduna çevrilir; gerisini o işletim sistemine özel üretilmiş olan PVM (yorumlayıcı) halleder.
- ◇ **Hızlı Prototipleme ve Kolay Hata Ayıklama (Debugging):** Derleme için uzun süreler beklenmediği için kod anında test edilebilir. Yorumlayıcı satır satır ilerlediği için, bir hata ile karşılaştığı anda programın yürütülmesini o satırda durdurur ve hatanın yerini geliştiriciye tam olarak bildirir.

Özellik	Derleyici Mimarisi (Örn: C++)	Python Yorumlayıcı Mimarisi (PVM)
Çeviri Zamanı	Program çalıştırılmadan önce.	Programın çalıştırıldığı esnada (<i>Runtime</i>).
Çıktı Dosyası	Bağımsız ikili makine kodu (.exe, .bin).	Ara seviye bayt kodu (.pyc). Kalıcı makine kodu üretilmez.
Çalışma Hızı	Çok hızlı ve donanımına yakın performans.	Ara çevrimler nedeniyle derlenen dillere göre nispeten daha yavaş.
Taşınabilirlik	Derlendiği mimariye özeldir, Başka mimariye taşınmaz.	Platform bağımsızdır; uygun PVM'in olduğu her yerde çalışır.
Hata Tespiti	Derleme sonunda tüm hatalar topluca gösterilir.	Talimatın bulunduğu kod satırında hata verip programı durdurur.

Tablo 2.5: Derleyici ve Python Yorumlayıcı Mimarilerinin Karşılaştırılması

2.13 Python Sanal Makinesinin Kurulması

Python tabanlı yazılım geliştirme sürecinin ilk adımı, dilin resmi web sitesinden ilgili işletim sistemine uygun yorumlayıcı motorunun indirilip kurulmasıdır. Bu kitapta yer alan tüm pratik uygulamalar ve metodolojiler, dilin modern standartlarını barındıran Python 3 ve üzeri sürümleri temel alınarak yapılandırılmıştır. Kurulum işleminin başarıyla tamamlandığı ve sistem yollarına (**PATH variables**) eklendiği, işletim sisteminin komut satırı arayüzünde (**Command Line Interface-CLI**) aşağıdaki komut çalıştırılarak kontrol edilebilir:

```
C:\Users\ILHANOZKAN>python3 --version
Python 3.13.4
C:\Users\ILHANOZKAN>
```

Bilgisayar bilimlerinde, dinamik ve yorumlanan dillerin komutlarını anlık olarak işlemek amacıyla kul-

lanılan iki temel yürütme ve ana bilgisayar uygulaması (**host application**) mevcuttur: Etkileşimli Kabuk (**Interactive Shell**) ve Senaryo Tabanlı Çalışma Modu (**Script Mode**).

§2.13.1 Etkileşimli Kabuk

Etkileşimli kabuk, programcının yazdığı tekil kod satırlarını anında kabul eden, yürüten ve çıktı üreten dinamik bir prototipleme ortamıdır. Bu ortam, bilgisayar mimarisinde **Okuma-Değerlendirme-Yazdırma Döngüsü** (**Read-Eval-Print Loop - REPL**) olarak adlandırılan deterministik bir döngü çerçevesinde çalışır.

REPL döngüsünün işleyiş anatomisi şu dört adımdan oluşur:

1. **Read (Okuma):** Kullanıcının klavyeden girdiği talimatı satır sonu (**new line / Enter**) karakterine kadar okur ve belleğe alır.
2. **Eval (Değerlendirme):** Alınan metni Python sözdizimi (**syntax**) açısından analiz eder, bayt koduna çevirir ve Python Sanal Makinesi'nde (**PVM**) icra eder.
3. **Print (Yazdırma):** İcra edilen komutun ürettiği nihai sonucu veya varsa hata mesajını doğrudan konsola basar.
4. **Loop (Döngü):** Bellekteki geçici durumları güncelleyerek imleci başa sarar ve kullanıcının bir sonraki komutu girmesi için hazır konumda bekler.

Bu ortamlardan birincisi, işletim sisteminin yerel konsolu üzerinden tetiklenen **python3** kabuk programıdır. Kabuk başlatıldığında, sistemin talimat almaya hazır olduğunu belirten ve üç adet büyüktür (**>>>**) karakterinden oluşan **Python komut istemi** (**Python command prompt**) ekranda belirir. **REPL döngüsü aktifken tanımlanan değişkenler ve fonksiyonlar, kabuk kapatılana kadar ana bellekte (RAM) geçici olarak muhafaza edilir.** Etkileşimli kabuk oturumunu güvenli bir şekilde sonlandırmak için **exit()** veya **quit()** fonksiyonları çağrılır; Unix tabanlı sistemlerde ve Windows Komut İstemi'nde ise alternatif olarak **CTRL + D** klavye kısayolu kabuk sürecini sonlandırır.

```
C:\Users\ilhan>python3
Python 3.12.11 (main, Jun 7 2025, 12:31:34) [GCC UCRT 15.1.0 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World!")
Hello World!
>>> print("İlhan")
İlhan
>>> toplam=0
>>> a=10
>>> b=20
>>> toplam=a+b
>>> print(toplam)
30
>>> quit()
C:\Users\ilhan>
```

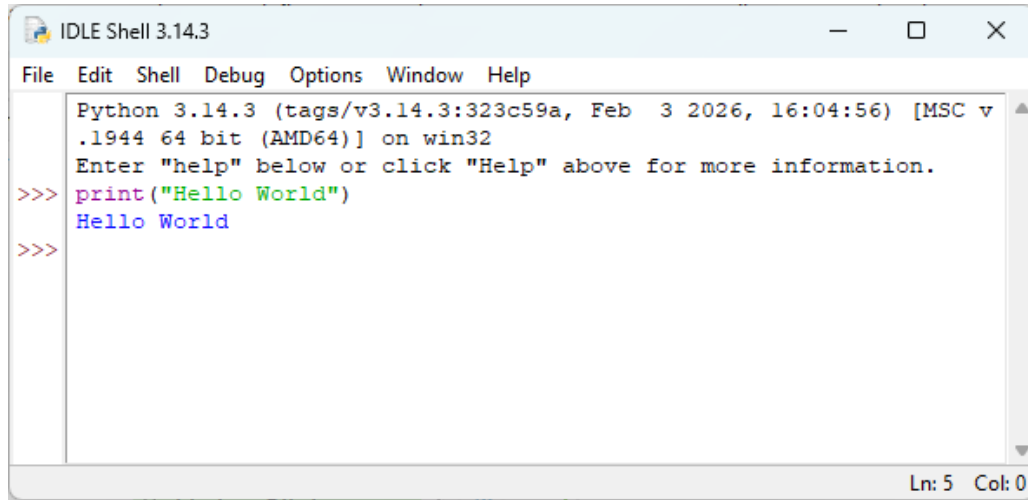
İkinci etkileşimli çalışma ortamı ise grafik kullanıcı arayüzüne (**GUI**) sahip işletim sistemlerinde çalışan ve Python dağıtımıyla yerleşik olarak gelen IDLE (**Integrated Development and Learning Environment**) kabuk programıdır (Şekil 2.1). IDLE, giriş seviyesindeki öğrencilerin kod yazma, hata ayıklama ve test süreçlerini kolaylaştırmak amacıyla tasarlanmış hafif bir Entegre Geliştirme Ortamıdır. Standart konsol kabuğu ile aynı **>>** komut istemi yapısını kullanır ancak sözdizimi renklendirmesi (**syntax highlighting**) ve otomatik kod tamamlama gibi ek konfor özelliklerine sahiptir.

§2.13.2 Betik Tabanlı Çalışma Modu

Etkileşimli kabuk (REPL), anlık testler için mükemmel olsa da, yazılan kodlar kalıcı olarak saklanamaz ve kabuk kapatıldığında bellekten silinir. Büyük ölçekli, tekrar kullanılabilir ve taşınabilir programlar geliştirmek için talimatlar bir dosya içine kaydedilir. Python kod bloklarını içeren ve **.py** uzantısına sahip bu dosya dosyalarına betik/senaryo dosyası (**script file**) adı verilir. Yukarıda tek tek yazdığımız talimatları aşağıdaki **hello.py** dosyasına yazmış olalım;

```
1 print("Hello World!")
2 print("İlhan")
3 toplam=0
4 a=10
5 b=20
6 toplam=a+b
7 print(toplam)
```

Bu dosya içine kaydedilmiş Python talimatlarının tamamını baştan sona tek bir süreç halinde çalıştır-



Şekil 2.1: IDLE Entegre Geliştirme ve Öğrenme Ortamı Kabuğu

mak için işletim sisteminin konsolundan, yorumlayıcıya dosya yolu parametre olarak geçirilerek şu komut yürütülür:

```
C:\Users\ILHANOZKAN>python3 hello.py
Hello World!
İlhan
30
C:\Users\ILHANOZKAN>
```

Bu modda Python yorumlayıcısı dosyayı açar, yukarıdan aşağıya tüm talimatları icra eder ve programın son satırına ulaştığında süreci otomatik olarak sonlandırarak kontrolü yeniden işletim sisteminin konsoluna bırakır.

2.14 Düşün ve Çöz

1. **Düşün (İki tabanında kayan noktalı sayı sınırı):** İnsanlar günlük hayatta ondalık (base-10) sistemde çalışırken, bilgisayarlar verileri ikili (base-2) sistemde saklar. Ondalık sistemdeki 0.1 sayısını ikili tabana çevirmeye çalıştığınızda nasıl bir durum ortaya çıkar? Python veya C’de `0.1 + 0.2 == 0.3` ifadesinin `False` (yanlış) dönmesinin sebebini IEEE 754 kayan noktalı sayı standardı çerçevesinde açıklayınız.
2. **Çöz (Bellek Sınırı ve Taşma / Overflow):** 8-bitlik işaretli (**signed**) bir tam sayı değişkenine maksimum değeri olan +127 atandıktan sonra bu değere +1 eklenirse, İkiye Tümlleyen (**Two’s Complement**) yönemine göre bellekteki ikili bit dizilimi nasıl değişir ve değişkenin yeni sayısal değeri ne olur?
3. **Düşün (Derleyici ve Yorumlayıcı Hibrit Mimarisi):** Pure (Saf) Yorumlayıcı kullanan diller ile saf Derleyici kullanan diller kıyaslandığında, Python’ın `.pyc` bayt kodlarına derlenip ardından Python Sanal Makinesi (**PVM**) tarafından yorumlanması hem performans hem de platform bağımsızlığı (**cross-platform**) açısından yazılımcıya ne tür avantajlar kazandırır?
4. **Çöz (Arapşacı Koddan Yapısal Programlamaya):** Aşağıdaki `GOTO` ifadeleri içeren akışı inceleyiniz:


```
10 SAYI = 1
20 IF SAYI > 10 GOTO 60
30 PRINT SAYI
40 SAYI = SAYI + 1
50 GOTO 20
60 END
```

Bu akışın amacını belirleyiniz ve aynı mantığı yapısal programlama ilkesine uygun olarak C veya Python dilinde bir `while` veya `for` döngüsü ile yeniden yazınız.

3. Bölüm

Python Diline Giriş

3.1 En Basit Python Programı

Program Hangi Talimattan İcra Edilmeye Başlar?

Python programlama dili; genel amaçlı, emreden paradigmalı (**imperative**), nesne yönelimli (**object-oriented**) ve yapısal (**structured**) programlamayı destekleyen çoklu paradigmalı (**multi-paradigm**) modern bir dildir.

C gibi yapısal dillerde programın başladığı yeri mekanik olarak belirten zorunlu bir **ana fonksiyon** (**main function**) tanımlanması gerekirken, Python'da yorumlayıcı (**interpreter**) kaynak kod dosyasını yukarıdan aşağıya doğru doğrudan okuyarak sırayla icra etmeye başlar.

```
1 print("Hello World!")
2 print("İlhan")
3 print("")
4 print("Özkan")
```

Yapısal programlama disiplinine sadık kalmak ve programın asıl giriş noktasını (**entry point**) açıkça belirlemek adına Python dünyasında da bir **main()** fonksiyonu tanımlamak ve bunu özel bir kontrol mekanizmasıyla çağırmak endüstri standardıdır. Yapısal programlama uygun Python programı şu şekilde yazılır:

```
1 def main():
2     print("Hello World!")
3     return 0
4
5 if __name__ == "__main__":
6     main()
```

Buradaki kodu şu şekilde açıklayabiliriz:

- ◊ **Birinci Satır (def main()):** Python'da bir fonksiyon tanımlamak için **def** (**define**) anahtar kelimesi kullanılır. Burada programımızın ana yürütme merkezi olacak **main()** fonksiyonu bildirilmiştir. C dilinin aksine Python dinamik tipli (**dynamically typed**) bir dil olduğu için fonksiyonun başına **int** gibi bir geri dönüş türü yazılması zorunlu değildir.
- ◊ **İkinci Satır (print("Hello World!")):** Ana fonksiyon içinde icra edilecek ilk talimat olup konsola "Hello World!" yazar.
- ◊ **Üçüncü Satır (return 0):** Fonksiyon gövdesi içinde yer alan bu talimat (**statement**), ana fonksiyonun görevini başarıyla tamamladığını ve geriye sıfır (0) değerini döndürerek sonlandığını belirtir. Bilgisayar mimarisinde ve işletim sistemlerinde, bir programın sistemden sıfır koduyla çıkması "Program Hatasız Sonlandı" (**Exit Success**) anlamına gelir. Eğer işletim sistemine programın bir hata nedeniyle kapandığı sinyali (örneğin 11 hata kodunu) göndermek istersek bu talimatı **return 11** olarak revize etmeliyiz.
- ◊ **Blok Yapısı ve Girintileme (Indentation):** C dilinde fonksiyon sınırları süslü parantezler ({}) ile çizilirken, Python'da **kod blokları** (**code block**) tamamen **girintileme** (**indentation**) yani satır başındaki boşluklar (genellikle 4 boşluk veya bir TAB karakteri) ile belirlenir. İkinci satırdaki **return** ifadesinin içeride yazılmış olması, onun **main** fonksiyon bloğuna (**function block**) ait olduğunu gösterir.
- ◊ **Beşinci ve Altıncı Satırlar (if __name__ == "__main__"):** Bu kalıp, Python'ın program giriş noktasını denetleyen en kritik yapısal mekanizmasıdır. Python yorumlayıcısı bir dosyayı doğrudan çalıştırdığında, o dosyaya gizlice **"__main__"** adını verir. Bu **if** koşulu, "Eğer bu kod dosyası başka bir programın içine kütüphane olarak çağırılmadıysa, doğrudan ana program olarak çalıştırıldıysa" anlamına gelir. Koşul sağlandığında bir alt satırdaki **main()** fonksiyonunu tetikleyerek programın yapısal akışını başlatır.

§3.1.1 Kaynak Kodumuzu Oluşturacak Karakterler

Python dilinde programcı tarafından yazılan kaynak kodlar, dilin izin verdiği belirli bir karakter kümesinden oluşur ve katı sözdizimi (**syntax**) kurallarına uyar.

Python 3.0 ve üzeri güncel sürümlerde, dünya üzerindeki tüm dillerin karakterlerini kapsayan Unicode standardı yerleşik (**native**) olarak desteklenmektedir. Python kaynak kod dosyaları (.py) için varsayılan karakter kodlama biçimi UTF-8'dir. Bu küresel destek sayesinde kodun içinde Türkçe karakterler (ş, ç, ğ, ü, ö, ı) kullanılabilir de, kodun uluslararası standartlara uygunluğu, taşınabilirliği ve farklı sistemlerde hatasız der-

lenip/yorumlanması açısından en iyi uygulama (best practice) olarak kaynak kod bileşenlerinde şu standart ASCII karakter setinin kullanılması önerilir:

1. **Alfabe**deki Büyük ve Küçük Harfler:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
a b c d e f g h i j k l m n o p q r s t u v w x y z

2. **Rakamlar** (Onluk Sayı Sistemi):

0 1 2 3 4 5 6 7 8 9

3. **Özel Karakterler** (İşleçler, Sınırlandırıcılar ve Semboller):

< > . , _ () ; \$: % [] # ? ' & { } " ^ ! * / | - \ ~ +

Bu karakterlerin her biri Python mimarisinde matematiksel işleçler (operator) (+, -, ...), açıklama satırı başlangıcı (#) veya veri yapısı sınırlandırıcısı ([], { }, ...) gibi kuralları olan belirli görevlere sahiptir.

4. **Beyaz Boşluk Karakterleri (white space)**: Ekranda doğrudan bir harf veya sembol olarak basılmayan (görünmeyen), ancak kaynak kodun yapısını, okunabilirliğini ve en önemlisi Python'daki **kod bloklarını** belirleyen hayati karakterlerdir:

- (a) **Boşluk (space)**: Kelimeleri ve operatörleri birbirinden ayırmak veya girintileme yapmak için kullanılır.
- (b) **Yeni Satır (new line - \n)**: Bir komut satırının bittiğini ve işlemcinin bir sonraki emre geçmesi gerektiğini belirten satır sonu karakteridir.
- (c) **Sekme (tab - \t)**: Genellikle 4 boşluk genişliğinde bir atlama sağlayan ve kod bloklarında hiyerarşik girintileme (**indentation**) oluştururken kullanılan karakterdir.

§3.1.2 Talimatlar ve Anahtar Kelimeler

Anahtar kelimeler (keywords), programlama dilinin tasarımında çekirdek fonksiyonları yürütmek üzere önceden tanımlanmış ve Python yorumlayıcısı (**interpreter**) için özel bir anlama sahip olan **ayrılmış kelimelerdir (reserved words)**. Programı oluşturan mantıksal cümleler yani talimatlar (**statement**), Tablo 3.1'de verilen bu kelimeler temel alınarak inşa edilirler. Bu kelimeler benzersiz birer göreve sahip olduklarından, program içinde değişken veya fonksiyon kimliği (ismi) olarak kesinlikle kullanılamazlar.

Talimat (Statement)

Yüksek seviyeli bir dilde yazılmış bir **talimat (statement)**, yorumlayıcıya belirli bir eylemi gerçekleştirmesini söyleyen, kendi içinde bütünlüğü olan mantıksal bir cümledir. Yüksek seviyeli bir dildeki tek bir talimat satırı, arka planda (işlemci seviyesinde) birkaç emir kodunu (**instruction code**) temsil edebilir.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Tablo 3.1: Anahtar Kelimeler

Anahtar kelimeler, Python komut isteminde önce **help()** ardından **keywords** girilerek öğrenilebilir.

BİLGİ: Büyük-Küçük Harf Hassasiyeti

Python, büyük ve küçük harflere karşı son derece duyarlıdır (**case-sensitive**). Python'daki anahtar kelimelerin neredeyse tamamı küçük harflerle yazılır. Ancak çok önemli üç istisna mevcuttur: Mantıksal veri türleri olan **True** ve **False** ile yokluğu temsil eden **None** kelimelerinin ilk harfleri her zaman büyük yazılmak zorundadır.

Tarihsel süreçte talimatların nerede bittiği dillerin tasarım felsefesine göre değişiklik göstermiştir. Örneğin BASIC ve FORTRAN gibi erken dönem dillerde talimatlar katı bir şekilde satır sonunda biterdi; FORTRAN'da kod yazmaya yedinci sütundan başlama zorunluluğu varken, BASIC'te her satırın başına benzersiz bir satır numarası eklenirdi. Pascal ve C dillerinde ise bu kurallar yıkılmış, talimatların bittiğini göstermek için noktalı virgül (;) karakteri seçilerek kodun **gelişigüzel biçimde (free format)** yazılmasına izin verilmiştir.

§3.1.3 Girintileme

Python mimarisi ise bu gelişigüzel kod yazımı yaklaşımlardan farklı, kendine has estetik ve yapısal bir kural benimser. Python'da kodun okunabilirliğini artırmak ve görsel karmaşayı (sembol kalabalığını) azaltmak amacıyla noktalı virgül (;) kullanımı zorunlu olmaktan çıkarılmıştır. Python dünyasında:

- ◊ **Satır Tabanlı Sonlandırma:** Her bir talimat satırı, klavyeden basılan Enter tuşuyla, yani görünmeyen bir yeni satır (**new line**) karakteriyle doğal olarak sonlanır.
- ◊ **Blok Yapısı ve Boşluklar:** C dillerindeki gelişigüzel (**free format**) yazım özgürlüğü Python'da yerini yapısal **girintileme** (**indentation**) kurallarına bırakır. Satır başındaki beyaz boşluklar rastgele bırakılmaz; hiyerarşik bir kod bloğu oluşturmak için mutlak bir girinti (hizalama) disiplini uygulanır.

Talimatları Satır Sonu Karakteri Bitirir

Python'da her talimat, cümlelerin bittiğini gösteren yeni satır (**new line**) karakteri ile sonlanır. Aynı satıra birden fazla talimat yazılmak istenirse aralarına isteğe bağlı olarak noktalı virgül (;) konulabilir; ancak bu durum kodun okunabilirliğini düşürdüğü için Python standartlarında tercih edilmez.

Python komut isteminde talimatlar, ard arda yazılır. Bir talimat tamamlandığında diğer talimat beklenir. Ancak işlem gören değişkenler unutulmaz. Aşağıda `print()` talimatlarının girintili yazıldığına dikkat ediniz.

```
1 toplam=0.0
2 a=15; b=1.0; c=20
3 toplam=toplam+a+b+c
4 if (toplam>10):
5     print("Toplam Ondan Büyük:", toplam)
6 else:
7     print( toplam)
```

§3.1.4 Açıklamalar

Yazılım geliştirme süreçlerinde kaynak kodun sadece bilgisayarlar tarafından işletilmesi değil, aynı zamanda insanlar tarafından kolayca anlaşılabilmesi ve sürdürülebilmesi (**maintainability**) de hayati bir önem taşır. Bu doğrultuda, kod bloklarının ne amaçla yazıldığını açıklayan, algoritmik mantığı detaylandıran ve Python yorumlayıcısı (**interpreter**) tarafından çalışma zamanında (**runtime**) tamamen göz ardı edilen metin alanlarına yorum satırları / açıklamalar (**comments**) adı verilir. Python dilinde açıklama satırları mimari açıdan iki farklı yöntemle oluşturulur:

Tek Satırlı Açıklamalar

Kod dosyaları içinde belirli bir satıra veya bir talimatın (**statement**) hemen sağ tarafına kısa açıklamalar (**in-line comments**) eklemek için karma (#) karakteri kullanılır. Python yorumlayıcısı, kaynak kodu yukarıdan aşağıya doğru tararken bu karakterle karşılaştığı anda, karakterin bulunduğu noktadan o satırın sonuna (**new line**) kadar olan tüm metni birer açıklama olarak kabul eder, işleme almaz ve doğrudan bir sonraki satıra geçer.

```
indis = 0      # Bu degisken dizi veya listelerde konum (indis) belirteci olacaktır.
toplam = 0.0   # Akumulator gorevi goren bu degisken, ondalikli hesaplama sonuclarini tutar.
```

Çok Satırlı Bloklar ve Dokümantasyon Dizeleri

Projenin geliştiricisi, telif hakları, tarih veya karmaşık bir algoritmanın çalışma prensibi gibi daha uzun, çok satırlı açıklamalar yapılmak istenildiğinde üç adet tek tırnak (') veya üç adet çift tırnak (""") karakteri peş peşe yazılarak bir blok başlatılır ve yine aynı sembollerle kapatılır.

Teknik olarak Python dil mimarisinde # dışında resmi bir "çok satırlı yorum" mekanizması yoktur. Üçlü tırnak içerisine yazılan ifadeler, aslında yorumlayıcı için bellekte geçici olarak oluşturulan ancak herhangi bir değişkene atanmayan **bağımsız metin sabitleridir** (**unassigned string literals**). Nitekim etkileşimli kabukta (**REPL**) bu ifadeyi çalıştırdığınızda, Python bunu göz ardı etmek yerine satır sonu karakterlerini de içeren kaçış dizileriyle (\n) birlikte bir dize nesnesi olarak ekrana basar:

```
1 >>> '''
2 ... Elektronik Yük. Müh. İlhan Özkan
3 ... Haziran 2026
4 ... Ankara
5 ... Bu metin, program mimarisine ilişkin meta-verileri içerir.
6 ... '''
7 '\nElektronik Yük. Müh. İlhan Özkan\nHaziran 2026\nAnkara\nBu metin, program mimarisine ilişkin
  → meta-verileri içerir.\n'
```

8 >>>

Ancak bu açıklama metni ileride anlatılacak bir fonksiyonun (`def`) veya bir sınıfın (`class`) hemen ilk satırında kullanılırsa, Python tarafından otomatik olarak birer **Dokümantasyon Metni** (**Docstring**) olarak anlaşılandırılır ve yazılımın otomatik kılavuzunu oluşturmak amacıyla `__doc__` özel özneliğinde (**attribute**) saklanır.

3.2 Değişken Tanımlama

Her yapısal programlama dilinde olduğu gibi Python'da da **veri yapıları** (**data structure**) ve **kontrol yapıları** (**control structure**) teorik olarak birbirinden ayrıdır. Ancak C dilinden farklı olarak Python, veriyi işleyecek kontrol yapısından önce verinin tipini ve bellek boyutunu katı bir şekilde tanımlamanızı zorunlu kılmaz. C dilinde değişkenler doğrudan bellekteki sabit boyutlu bir hücreyi (kutu) temsil ederken; Python'da her değişken, arka planda dinamik olarak oluşturulan bir nesneye ait bir referanstır (etikettir).

§3.2.1 Python'da İlkel Veri Tipleri Yanılgısı ve Dinamik Tiplendirme

C dilinde `char`, `int`, `float` gibi doğrudan işlemci emir setine hitap eden ve bellekte kaplayacağı alan baştan belli olan **ilkel veri tipleri** (**primitive data type**) vardır. Python'da ise ilkel veri tipi yoktur. Python'daki en basit tamsayı veya karakter bile arka planda birer nesnedir (**object**).

Python'da bir değişkeni tanımlamak için onun tipini belirtmek gerekmez; yorumlayıcı, değişkene atanan değere bakarak veri tipini çalışma zamanında (**runtime**) otomatik olarak belirler (**dynamic typing**).

```
# Bu program Python'da değişken kullanımını göstermek için oluşturulmuştur.
tamsayı=1 # değişmezi bir int nesnesine dönüştürülür ve değişkene referans verilir
print(tamsayı)
#1
büyüksayı=987654321987654321 #değişmez int nesnesine dönüştürülür ve değişkene referans verilir
print(büyüksayı)
#987654321987654321
ondalık=3.14 #değişmez float nesnesine dönüştürülür ve değişkene referans verilir
print(ondalık)
#3.14
avagadro=6.02214076e23
print(avagadro)
#6.02214076e+23
karakter='A' #değişmez bir str nesnesine dönüştürülür ve değişkene referans verilir
print(karakter)
#A
metin='İlhan Özkan'
print (metin)
#İlhan Özkan
doğrumudeğilmi=True
print(doğrumudeğilmi)
#True
tipibellideğil=None
print(tipibellideğil)
#None
print(type(tamsayı))
#<class 'int'>
print(type(büyüksayı))
#<class 'int'>
print(type(ondalık))
#<class 'float'>
print(type(avagadro))
#<class 'float'>
print(type(karakter))
#<class 'str'>
print(type(metin))
#<class 'str'>
print(type(doğrumudeğilmi))
#<class 'bool'>
print(type(tipibellideğil))
#<class 'NoneType'>
```

1. **Tamsayı Veri Tipi (int):** Python 3'te C dilindeki gibi `short`, `int`, `long` veya `long long` gibi ayrık tamsayı türleri bulunmaz. Hepsi tek bir `int` (**integer**) çatısı altında birleştirilmiştir. Dahası, Python'daki tamsayıların sabit bir bit sınırı (32-bit veya 64-bit gibi) yoktur. Bellek elverdiği müddetçe Python'da sonsuz büyüklükte tamsayılar üretebilirsiniz; sistem taşma (**overflow**) tehlikesi olmadan bellek boyutunu dinamik olarak büyütür.
2. **Gerçek Sayı Veri Tipi (float):** C dilindeki `float` (32-bit) ve `double` (64-bit) ayrımı Python'da yoktur. Python'daki `float` veri tipi, donanımsal olarak doğrudan C'deki `double` mimarisine (IEEE 754 standardında 64-bitlik çift hassasiyetli gerçek sayılara) karşılık gelir.

None Anahtar Kelimesi (void Karşılığı)

C dilindeki hiçbir değer ifade etmeyen ve bellek tahsis etmeyen `void` kavramının Python'daki tam karşılığı **None** nesnesidir. Bir fonksiyon geriye hiçbir şey döndürmüyorsa, Python arka planda gizlice **None** döndürür. Veri yapılarında içi boş olan veya henüz değer atanmamış değişkenleri başlatmak için sıklıkla kullanılır.

Python dilinde tüm değişkenler (variable) aslında var olan bir nesneye (object) tanımlanan bir referanstır. Atama işlemi ile var olan bir nesneye veya nesneye dönüştürülen değişmezler (literal) sayesinde değişken tanımlanabilir. Değişkenler, bir veri tipi belirtmeksizin, benzersiz bir **kimlik / isim** (**identifier**) verilerek doğrudan atama işleci (=) vasıtasıyla tanımlanır:

```
# Bu program Python'da değişken kullanımını göstermek için oluşturulmuştur.
def main():
    yas = 25      # Dinamik olarak 'int' veri tipinde bir nesneye referans oluşturuldu.
    kat = 4       # Sınır yoktur, bellek yettiğinde büyüyecektir.
    kilo = 72.5   # Dinamik olarak 'float' (64-bit double) veri tipinde referans oluşturuldu.
    return 0

if __name__ == "__main__":
    main()
```

Yukarıda tanımlaması yapılan `yas`, `kat` ve `kilo` değişkenleri `main` fonksiyon bloğu içinde yaratıldığından, bu değişkenler sadece o fonksiyonun yerel kapsamı (**local scope**) içinde geçerlidir.

Aynı Kimlikli İkinci Değişken

Python'da aynı kod bloğu içinde aynı isimde (kimlikte) ikinci bir değişken tanımlandığında hata alınmaz. Bunun yerine eski nesneye olan referans kopar ve değişken ismi yeni atanan nesneye bağlanır. Eski nesne ise bellekten çöp toplayıcı (**Garbage Collector**) tarafından otomatik olarak silinir.

§3.2.2 Değişkenin Kapsamı (Variable Scope)

Değişkenin kapsamı (**variable scope**); bir değişkenin bildirildiği, erişilebildiği ve üzerinde işlem yapılabileceği kod bölgelerini ifade eder. Python'da değişken kapsamı kuralları **LEGB** (**Local**, **Enclosing**, **Global**, **Built-in**) kuralına göre işletilir.

Bilgi: Python'da Blok Sınırı

C dilinde her süslü parantez ({}) yeni bir kapsam alanı (scope) yaratırken, Python'da `if`, `for` veya `while` gibi döngü ve kontrol blokları yeni bir kapsam alanı yaratmaz! Python'da kapsam sınırları sadece fonksiyonlar (`def`) ve sınıflar (`class`) ile çizilir. Örneğin, bir `if` bloğunun içinde tanımlanan değişkene, `if` bloğunun dışından da (aynı fonksiyon içinde kalmak şartıyla) erişilebilir.

§3.2.3 Değişken Kimliklendirme Kuralları

Python programlama dilinde değişkenlere, fonksiyonlara veya sınıflara benzersiz isimler yani **tanımlayıcılar** (**identifier**) verilirken yorumlayıcının hata vermeden kodu işleyebilmesi için dört temel sözdizimi (**syntax**) kuralına mutlak surette uyulmalıdır:

Kimliklendirme (İsimlendirme) Kuralları

1. **Kimlik olarak dilin ayrılmış anahtar kelimeleri kullanılamaz!** Örneğin `if`, `for`, `class` veya `def` gibi kelimeler doğrudan bir değişken ismi olamaz.
2. **Kimlikler asla bir rakam karakteri ile başlayamaz!** Değişken ismi bir harfle veya alt çizgi karakteriyle (`_`) başlamak zorundadır; ilk karakterden sonraki karakterlerde rakam olabilir (`sayi1` geçerli, `1sayi` geçersizdir).
3. **Karakter uzunluk sınırı yoktur!** C dilindeki gibi derleyici düzeyinde katı bir 32 karakter sınırı bulunmaz. Bir Python tanımlayıcısı bellek sınırları dahilinde teorik olarak sonsuz uzunlukta olabilir; ancak okunabilirlik açısından makul uzunlukta tutulması önerilir.
4. **Alt çizgi hariç hiçbir özel karakter ve boşluk kullanılamaz!** Hiçbir noktalama işareti veya özel karakter (`+`, `-`, `*`, `/`, `$`, `@`, ...) değişken isminde yer alamaz. Python3 yerleşik Unicode desteğine sahip olduğu için Türkçe karakterler (`yaş`, `öğrenci_boyu`) teknik olarak geçerlidir; fakat uluslararası kod taşınabilirliği açısından İngiliz alfabesindeki harflerin tercih edilmesi endüstri standardıdır.

Aşağıda Python sözdizimi kurallarına uygun dinamik değişken tanımlama örnekleri verilmiştir:

```
# Bu program geçerli değişken tanımlama örneklerini içerir.
def main():
    # Tamsayı (int) değişkenler:
    yas = 25
    tam_sayi = -456
    uzun_tam_sayi = 9223372036854775807 # Python'da tamsayılarda boyut sınırı yoktur.
    # Gerçek Sayı (float) Değişkenler:
    kilo = 72.5
    reel_sayi = 3.14159265
    # Python'da C dilindeki gibi tek satırda tür belirtip virgülle boş değişken
    # oluşturulamaz; ancak aynı satırda birden fazla değişkene çoklu atama yapılabilir:
    kat1, kat2, kat3 = 1, 2, 3
    olcek1 = olcek2 = 1.0 # Aynı değeri birden fazla değişkene bağlama (zincirleme atama)
    return 0
```

§3.2.4 Birden Fazla Değişkeni Tek Satırda Tanımlama

Birden fazla değişkeni aralarında virgül kullanarak tek bir satırda tanımlayabilirsiniz;

```
num1, num2, num3 = 10, 20, 30
print(num1, num2, num3)
#10 20 30
```

Birden fazla değişkeni aynı nesneyi referans gösterecek şekilde de tanımlayabilirsiniz;

```
num1=num2=num3=100
print(num1, num2, num3)
#100 100 100
```

§3.2.5 Kod Okunabilirliği ve Yazım Stilleri

Sözdizimi kurallarına uymak kodun sadece "çalışmasını" sağlar. Ancak yazılım mühendisliğinde kodun kalitesini, sürdürülebilirliğini ve ekipler arası okunabilirliğini artırmak için endüstriyel yazım stilleri (*naming conventions*) benimsenir. Python topluluğunun resmi kod stili rehberi olan PEP 8, isimlendirmesi C dilinden tamamen farklıdır.

C dilinde sıklıkla tercih edilen deve yazımı (`camelCase`) stili, Python dünyasında değişken ve fonksiyon isimleri için tercih edilmez. Bunun yerine, tüm harflerin küçük yazıldığı ve kelimelerin birbirine alt çizgi (`_`) ile bağlandığı yılan yazımı (`snake_case`) stili kullanılır. Bu bir zorunluluk değil, Python ekosisteminde profesyonel bir yazılımcı olmanın getirdiği küresel bir en iyi uygulama (best practice) standardıdır.

Bu program PEP 8 standartlarına (`snake_case`) uygun değişken isimlendirmelerini içerir.

```
def main():
    sayac = 0
    ogrenci_boyu = 175
    asansorun_aldugu_kat = 4
    asansor_agirligi = 650.5
    ogrenci_yasi = 19
    ilk_harf = 'A'
    # Python'da C'deki gibi 'char ogrenciAdi[50]' şeklinde statik bellek dizileri tanımlanmaz.
```

```
# Metinler dinamik 'str' veri türünde tutulur ve sınırları bellek tarafından otomatik
→ yönetilir.
ogrenci_adi = "Ahmet"
ogrenci_soyadi = "Yılmaz"
ortalama_not = 85.5
return 0
```

BİLGİ: Python'da camelCase ve PascalCase Nerelerde Kullanılır?

Python'da **snake_case** tarzı değişken ve fonksiyonlar için ayrılmıştır. **camelCase** kullanımına modern Python kodlarında neredeyse hiç rastlanmaz. İlk harfin de büyük başladığı **PascalCase** (**OgrenciBilgisi**) stili ise Python'da yalnızca **sınıf (class)** isimlerini tanımlarken katı bir kural olarak kullanılır.

3.3 Değişmezler ve Sabit Değişkenler

Yazılım geliştirme sürecinde algoritmaları inşa ederken, program boyunca değeri hiç değişmeyen veya değişmemesi gereken bazı sabit (**constant**) değerleri sıklıkla kullanırız. Bilgisayar biliminde sabitler iki ana kategoriye ayrılır: Doğrudan kodun içine yazılan **değişmezler (literal)** ve programcının mantıksal olarak sabit kalmasını istediği **sabit değişkenler (named constant)**.

§3.3.1 Değişmezler

Kaynak kodun içinde doğrudan ham haliyle yer alan, çalışma zamanında (**runtime**) kendi kendisini temsil eden sabit veri değerlerine **değişmez (literal)** adı verilir. Değişmezler bellek yönetimi açısından dinamik olarak oluşturulan nesnelerin çekirdek değerleridir. Python dilinde en sık kullanılan değişmez türleri şunlardır:

- ◊ **Sayısal Değişmezler (Numeric Literals):** Tam sayılar (örn: 5, -42) veya kayan noktalı gerçek sayılar (örn: 3.14, 0.001).
- ◊ **Metin Değişmezleri (String Literals):** Tek veya çift tırnak içinde yazılan karakter dizileri (örn: "Ankara", 'Python').
- ◊ **Mantıksal Değişmezler (Boolean Literals):** Sadece mantıksal doğruluk değerlerini ifade eden **True** ve **False**.

Değişkenlere atanan **ilk değerler (initial value)** ile matematiksel formüllerdeki **katsayılar (coefficient)**, programlama dillerinde değişmezlerin en yaygın kullanım alanlarıdır.

Başlatma (Initialization)

Bir değişken ilk kez tanımlanırken, ona atama operatörü (=) vasıtasıyla bir değişmez (**literal**) kullanarak ilk değer verilmesi işlemine **başlatma (initialization)** adı verilir. C dillerinin aksine Python'da değer atanmadan (başlatılmadan) bir değişken sadece isim olarak bildirilemez.

Diğer değişmezler ve kullanım örnekleri aşağıda verilmiştir;

- ◊ **None:** Bir değer olmadığını belirtmek için kullanılan tekil (**unique**) bir nesnedir.
- ◊ **...: Ellipsis,** daha sonra yazılacak veya kodlanacak anlamındadır. Yorumlayıcı burayı hata vermeden geçer.
- ◊ **NotImplemented:** Bir yöntemin belirli argümanları desteklemediğini belirtmek için kullanılan tekil bir nesnedir.

```
secilen_koltuk = None # Kullanıcının seçeceği koltuk numarası henüz bilinmiyor
secim_yapildi = True # Programın ilerleyen aşamalarında seçim yapılırsa değişken güncellenir
```

```
if secim_yapildi:
    secilen_koltuk = 42
```

```
def veritabanina_baglan():
    ...
    # Fonksiyon gövdesi daha sonra yazılacak, yorumlayıcı burayı hata vermeden geçer.
```

```
class KullaniciYonetimi:
    ...
    # Sınıf gövdesi daha sonra yazılacak, yorumlayıcı burayı hata vermeden geçer.
```

```
def tl_karsilastir(deger_tl, hedef_para_birimi):
```

```
# Fonksiyon sadece TL ve USD ilişkisini kurmayı biliyor
if hedef_para_birimi == "USD":
    return deger_tl > (1 * 45) # 2026 yılı sembolik kuru: 1 USD = 45 TL varsayalım
# Eğer fonksiyon bilmediği bir para birimi ile karşı karşıya kalırsa:
return NotImplemented

# --- Fonksiyonun Test Edilmesi ---
# 1. Durum: Fonksiyonun desteklediği bir senaryo (TL ve USD)
sonuc_usd = tl_karsilastir(100, "USD")
print(f"100 TL, 1 USD'den büyük mü?: {sonuc_usd}")
# 2. Durum: Fonksiyonun desteklemediği bir senaryo (TL ve EUR)
sonuc_eur = tl_karsilastir(100, "EUR")
print(f"Euro karşılaştırma sonucu: {sonuc_eur}")
# Üst Düzey Kontrol: Fonksiyonun bu işlemi destekleyip desteklemediğini denetleme
if sonuc_eur is NotImplemented:
    print("Sistem Uyarısı: Bu para birimi dönüştürme işlemi henüz kodlanmamıştır!")
```

§3.3.2 Sabit Değişken

C, C++ veya Java gibi dillerde bir değişkenin değerinin sonradan değiştirilmesini donanımsal ve derleme düzeyinde engellemek için `const` veya `final` gibi katı anahtar kelimeler kullanılır. Python dil mimarisinde ise katı bir "değiştirilemez değişken" (`const`) mekanizması veya anahtar kelimesi mevcut değildir. Python felsefesi, programcuyu sentaks kurallarıyla kısıtlamak yerine, ortak bir disipline uymaya davet eder.

Python'da bir değişkenin "sabit değişken" olduğunu ve program boyunca değerinin kesinlikle değiştirilmemesi gerektiğini belirtmek için resmi PEP 8 kılavuzuna göre TAMAMI BÜYÜK HARFLERDEN oluşan ve kelimeleri alt çizgiyle ayrılan bir isimlendirme standardı benimsenir:

```
# Bu program Python'da sabit ve değişmez kullanım örneklerini içerir.
# PEP 8 Standartlarına Göre Sabit Değişken Tanımlamaları:
PI_SAYISI = 3.14159          # Bir gerçek sayı değişmezi (literal) ile başlatma
DUNYANIN_YARI_CAPI = 6371   # Bir tamsayı değişmezi (literal) ile başlatma
ISIK_HIZI = 299792458
YAZILIM_MODU = "PROD"       # Bir metinsel değişmez (literal) ile başlatma
# Pratik Kullanım:
yaricap = 5.0                # 'yaricap' bağımsız bir değişkendir.
cevre = 2 * PI_SAYISI * yaricap # 2 bir katsayı (literal), PI_SAYISI ise sabittir.
```

Python'da Katı Sabitler Nasıl Yapılır?

Eğer çok büyük bir projede sabitlerin güvenliği insan inisiyatifine bırakılmak istenmiyorsa, Python 3.8 ile birlikte dile eklenen **Final** tip belirteci (**type hinting**) kullanılabilir.

`from typing import Final` modülü sisteme dahil edilerek `PI: Final = 3.14` şeklinde bir tanım yapıldığında, gelişmiş kod editörleri (IDE) veya `mypy` gibi statik kod analiz araçları, bu sabitin değiştirilmeye çalışıldığı satırda programcuyu sert bir şekilde uyarır ve hata raporlar.

Teknik olarak, yukarıdaki kodda programcı isterse alt satırlarda `PI_SAYISI = 5` yazarak bu değeri ezebilir ve Python yorumlayıcısı buna hiçbir hata mesajı vermez. Ancak Python topluluğunda tamamen büyük harfle yazılmış bir değişkeni değiştirmek, en temel yazılım mühendisliği etiğine ve endüstri standartlarına aykırı kabul edilir.

3.4 Etkileşimli Kabuk Kullanımında Değişkenler

§3.4.1 Değişkenlerin Bellek Adreslerini Öğrenme

Bir değişkenin bellek adresi `id()` ile öğrenilebilir. Aşağıdaki örnekten de anlaşılacağı üzere değeri tamsayı bir olan değişmezden bir nesne imal edilip a değişkeni tanımlanmıştır. Ardından b değişkeni de a değişkeninin referans gösterdiği nesneyi gösterecek şekilde tanımlanmıştır. Her iki değişkenin de gösterdiği bellek adresi aynı olduğu gözden kaçırılmamalıdır.

```
>>> a=1
>>> b=a
>>> print(id(a),id(b))
140712917549992 140712917549992
>>>
```


C/C++/C# ve Java gibi dilleri statik tip sistemine (**static type system**) sahip olduklarından bu yapılamaz. Yani bir değişken kimliklendirildikten sonra veri tipi değiştirilemez. Çünkü veri tipine bağlı kimliklendirme yapılır. Aksine Python dilinde çalışma anında değişkenin işlediği veri tipi değiştirilebilir. Bu dinamik veri tip sistemi (**dynamic type system**) programcıya kolaylık sağlar;

```
>>> numara = 100
>>> print (numara)
100
>>> print(type(numara))
<class 'int'>
>>> numara = '0n'
>>> print (numara)
0n
>>> print(type(numara))
<class 'str'>
>>>
```

§3.4.2 Tanımlı Değişkenleri Görme

Yorumlayıcıda tanımlanan değişkenler tanımlandığı satır icra edildikten sonra hemen kaybolmazlar. Python komut isteminde (command prompt) aşağıdaki komut çalıştırılarak halihazırda tanımlı değişkenleri `dir()` komutuyla görebilirsiniz.

```
>>> a=10
>>> b=20
>>> print('toplam',a+b)
toplam 30
>>> print('çarpım',a*b)
çarpım 200
>>> dir()
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
→ '__name__', '__package__', '__spec__', 'a', 'b']
>>>
```

Tanımlı değişkenler listelenirken alt çizgi karakterleri arasında yer alan ön tanımlı (**default**) değişkenler de listelenir. Bu komutun dışında `locals()` ile yerel değişkenleri, `globals()` ile evrensel değişkenleri listeleyebilir ve değerlerini görebilirsiniz. İleride detayı verilecektir.

```
>>> locals()
{'__name__': '__main__', '__doc__': None, '__package__': '_pyrepl', '__loader__': None,
→ '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>,
→ '__file__': 'C:\\Users\\ILHANOZKAN\\...\\Python313\\Lib\\pyrepl\\__main__.py',
→ '__cached__':
→ 'C:\\Users\\ILHANOZKAN\\...\\Python313\\Lib\\pyrepl\\__pycache__\\__main__.cpython-313.pyc',
→ 'a': 10, 'b': 20}
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': '_pyrepl', '__loader__': None,
→ '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>,
→ '__file__': 'C:\\Users\\ILHANOZKAN\\...\\Python313\\Lib\\pyrepl\\__main__.py',
→ '__cached__':
→ 'C:\\Users\\ILHANOZKAN\\...\\Python313\\Lib\\pyrepl\\__pycache__\\__main__.cpython-313.pyc',
→ 'a': 10, 'b': 20}
>>>
```

§3.4.3 Değişkenleri Silme

Halihazırda tanımlı değişkenleri silmek için `del` kullanılır.

```
>>> toplam=12
>>> print(toplam)
12
>>> del toplam
>>> print(toplam)
Traceback (most recent call last):
File "<pyshell#28>", line 1, in <module>
print(toplam)
NameError: name 'toplam' is not defined
```

```
>>>
```

§3.4.4 Değişken Tiplerini Test Etme

Değişkenin veri tipi ön tanımlı `type()` fonksiyonu ile öğrenilir.

```
>>> a = 12
>>> print(type(a))
<class 'int'>
>>> b = '12'
>>> print(type(b))
<class 'str'>
>>> c = (1, 3.14)
>>> print (type(c))
<class 'tuple'>
>>>
```

Koşullu ifadelerde veri tipini `isinstance()` ile test etmek mümkündür. İleride anlatılacaktır.

3.5 Veri Tipleri

§3.5.1 Temel Veri Tipleri

bool

Doğru anlamında True veya yanlış anlamında False değer alabilen veri tipidir. True tamsayı olarak 1, False ise 0 sayısına karşılık gelir

```
devammi=True
print('Devam mı', devammi) # Çıktı: Devam mı True
print(True+True) #Çıktı:2
print(True*True) #Çıktı: 1
```

int

Tamsayı değer alabilen veri tipidir. Python son sürümündeki bu veri tipi C dilindeki long veri tipine karşılık gelir.

```
a = 1
b = 300
c = -123456789
d = 398765432109876543
print(a,b,c,d) #1 300 -123456789 398765432109876543
```

Değişmezler yukarıdaki gibi onluk sistemdeki tamsayılar olabileceği gibi ikili (binary), sekizli (octal) veya on altılı (hexadecimal) sayı tabanında olabilir. İkili sayılara ilişkin sabit örneği aşağıda verilmiştir.

```
a=0b0101 # ikili sayı sabitinin başında 0b karakterleri bulunur.
print(a," type a", type(a)) #5 type a <class 'int'>
b=int("0b1000",2)
print(b," type b", type(b)) #8 type b <class 'int'>
c=bin(b)
print(c,"type c:",type(c)) #0b1000 type c: <class 'str'>
```

Sekizli sayı sisteminde bir sayıya ait değişmezin önünde sıfır ile birlikte küçük ya da büyük o harfi bulunur. Bir sayı ya da metinden sekizlik sistemde bir dizgi elde etmek istersek `oct()` fonksiyonunu kullanırız.

```
a=0o10
print(a,"type a",type(a)) #8 type a <class 'int'>
b=0020
print(b,"type b",type(b)) #16 type b <class 'int'>
c=int('10',8)
print(c,"type c",type(c)) #8 type c <class 'int'>
d=int('0o21',8)
print(d,"type d",type(d)) #17 type d <class 'int'>
e=oct(16)
print(e,"type e", type(e)) #0o20 type e <class 'str'>
```

On altılık sistemde bir değişmezin önünde ise sıfır ile birlikte x harfi bulunur. Bir sayı ya da metinden on altılık sistemde bir dizgi elde etmek istersek `hex()` fonksiyonunu kullanırız.

```

a=0x10
print(a, 'type a', type(a)) #16 type a <class 'int'>
b=0x20
print(b, 'type b', type(b)) #32 type b <class 'int'>
c=int('10',16)
print(c, "type c", type(c)) #16 type c <class 'int'>
d=int('0x21',16)
print(d, "type d", type(d)) #33 type d <class 'int'>
e=hex(10)
print(e, "type e", type(e)) #0xa type e <class 'str'>

```

float

Kayan noktalı sayıları tutan veri tipidir. Python son sürümündeki bu veri tipi C dilindeki double veri tipine karşılık gelir.

```

a = 3.14
b = 7.5e0
c = 1e-1
d = 0.5e8
print(a,b,c,d) #3.14 7.5 0.1 50000000.0

```

Herhangi bir sayı ya da metni kayan noktalı sayıya çevirmek için `float()` fonksiyonu kullanılır. Matematikte sonsuzluk soyut bir kavramdır. Fiziksel olarak, sonsuz büyüklükteki sayılar hiçbir zaman herhangi bir miktardaki bellekte saklanamaz. Ancak bilgisayar donanım yapılandırmalarının çoğu için 10'un 400. kuvveti olan çok büyük bir sayı `Inf` ile gösterilir. `float()` fonksiyonu için argüman olarak "Infinity" kullanırsanız, `Inf` döndürür. Tanımlanmamış veya temsil edilemeyen herhangi bir değeri `Nan` (**Not a Number**) temsil eder.

```

a=3.14E0
print(a, type(a)) #3.14 <class 'float'>
b=float('Infinity')
print(b, type(b)) #inf <class 'float'>
c=float('Nan')
print(c, type(c)) #nan <class 'float'>

```

complex

Karmaşık sayıları tutan veri tipidir. Sanal kısım j harfi ile biter.

```

a = 3 + 5j
b = -1.5 + 4.2j
print (a,b) #(3+5j) (-1.5+4.2j)

```

str

Metinleri (**string**) veri olarak tutan ver tipi `str` veri tipidir. Metinler aslında `byte` cinsinden karakterleri içeren bir dizidir (`array`).

```

metin='Uşağum Merhaba'
batylar= u'Uşağum'
print(metin) #Uşağum Merhaba
dizgi = 'Hello World'
print(dizgi[0]) #İlk karakteri konsola yazar. H
print(dizgi[0:5]) #İlk 5 Karakteri konsola yazar. Hello

```

Python, metin değişmezlerini (**literal**) tanımlamak için tek tırnak ('), çift tırnak (") ve üçlü tek veya çift tırnak (""") işaretlerini, dizinin başında ve sonunda aynı tür tırnak işareti kullanıldığı sürece kabul eder.

```

harf='a'
kelime="sözcük"
cümle="Türkçe güzel bir dildir."
print(cümle) #Türkçe güzel bir dildir.
paragraf="""Python dili çok esnek bir dildir.
Şöyle ki C ve C++ dillerindeki dizki karakterleri gibi tanımlama yapabilirsiniz."""
print(paragraf)
#Python dili çok esnek bir dildir.
#Şöyleki C ve C++ dillerindeki dizki karakterleri gibi tanımlama yapabilirsiniz.

```

`str` veri tipi Unicode karakterleri içerebilir, dolayısıyla tek, çift veya üçlü tırnaklı dize sözdizimi kullanılarak oluşturulan herhangi bir metin (**string**) Unicode olarak saklanır.

```
string='ÜĞİŞÇÖİüğüşçöı'
tb=string.encode('utf-8')
print(tb)
→ #b'\xc3\x9c\xc4\x9e\xc4\xb0\xc5\x9e\xc3\x87\xc3\x96I\xc3\xbc\xc4\x9fi\xc5\x9f\xc3\xa7\xc3\xb6\xc4\xb1'
str=tb.decode('utf-8')
print(str) #ÜĞİŞÇÖİüğüşçöı
```

§3.5.2 Diziler ve Koleksiyonlar

Dizi ve koleksiyonlar yinelenebilir (iterable). Yani elemanları üzerinde `for` döngüsü ile gezinilebilir. Elemanların konumunu belirten **indisler** (index) her zaman sıfırdan başlar ve her bir elemana köşeli parantez ([]) içerisinde indis kullanılarak erişilebilir. Burada kısaca değinilen bu veri yapıları bir bölüm olarak detaylı incelenecektir.

str ve bytes

Metinlerin tutulduğu `str` veri tipi, aslında `bytes` veri tipinde karakterlerden oluşan dizilerdir (array).

```
kelime="sözcük"
print(kelime[0]) #s
print(kelime[1]) #ö

# 1. Senaryo: Standart ASCII Karakterlerinden Oluşan Metin
metin1 = "Cemil"
# Metni UTF-8 standardına göre ham baytlara dönüştürüyoruz:
bayt1 = metin1.encode('utf-8')
print(f"Metin: {metin1} -> Türü: {type(metin1)}") #Metin: Cemil -> Türü: <class 'str'>
print(f"Bayt Karşılığı: {bayt1} -> Türü: {type(bayt1)}") #Bayt Karşılığı: b'Cemil' -> Türü:
→ <class 'bytes'>
# İndisleme yaparak bellekteki gerçek sayısal (onluk) değerine ulaşıyoruz:
print(f"Bellekteki Sayısal Değeri: {bayt1[0]} (İkilik: {bin(bayt1[0])})") #Bellekteki Sayısal
→ Değeri: 67 (İkilik: 0b1000011)

# 2. Senaryo: Unicode (Türkçe) Karakter İçeren Metin
# Kitabımızın varsayılan kodlaması olan UTF-8'in dinamik yapısını inceleyelim:
metin2 = "Çağla"
bayt2 = metin2.encode('utf-8')

print(f"Metin: {metin2}")
print(f"Bayt Karşılığı (UTF-8): {bayt2}") #Bayt Karşılığı (UTF-8): b'\xc3\x87a\xc4\x9fla'
# UTF-8 mimarisinde Türkçe 'Ç' harfi bellekte ardışık 2 bayt yer kaplar:
print(f"Bellekte Kapladığı Bayt Sayısı: {len(bayt2)} bayt") #Bellekte Kapladığı Bayt Sayısı: 7
→ bayt
print(f"1. Bayt Değeri: {bayt2[0]} | 2. Bayt Değeri: {bayt2[1]}") #1. Bayt Değeri: 195 | 2. Bayt
→ Değeri: 135

# 3. Senaryo: Ham Bayt Dizisinden Metne Geri Dönüş (Decoding)
# Donanımdan veya dosyadan gelen ham bayt verisi:
gelen_veri = b'\x49\x6c\x68\x61\x6e' # Onluk sistemde: [73, 108, 104, 97, 110]
cozulen_metin = gelen_veri.decode('utf-8')
print(f"Gelen Ham Bayt Dizisi: {gelen_veri}") #Gelen Ham Bayt Dizisi: b'Ilhan'
print(f"Kod Çözme (Decode) Sonucu Metin: {cozulen_metin}") #Kod Çözme (Decode) Sonucu Metin: Ilhan
```

tuple

Demet (tuple), herhangi bir veri tipinde n adet değer in bir araya gelerek oluşturduğu sıralı koleksiyondur. Elemanlar yuvarlak parantez içerisinde yazılarak ilk değer verilir. Demet koleksiyonunda elemanlar değiştirilemez (immutable)!

```
a = (1, 2, 3) # tamsayılardan oluşan demet
print(a) #(1, 2, 3)
b = (1, 'İlhan', 3.14, 'Özkan') # karışık veri tiplerinden oluşan demet
print(b) #(1, 'İlhan', 3.14, 'Özkan')
c = (1, 'ilhan', ('Özkan', 3.14)) # içinde bir başka demet olan demet
print(c) #(1, 'ilhan', ('Özkan', 3.14))
print(c[0]) #1
```

```
print(c[1]) #ilhan
print(c[2]) #('Özkan', 3.14)
d=(a,b,c) # başka demet elemanlar ile yeni demet tanımlama
print(d) #((1, 2, 3), (1, 'İlhan', 3.14, 'Özkan'), (1, 'ilhan', ('Özkan', 3.14)))
e=(100,) #tek elemanlı bir demet: öncelik için kullanılan parantez ile karışmaması için eleman
→ sonrasına bir virgül konulur
print(e) #(100,)
```

list

Listeler (**list**) herhangi bir veri tipindeki değerlerin sıralı bir koleksiyonudur. Elemanları değiştirilebilir (**mutable**). Elemanları köşeli parantez içerisinde yazılarak ilk değer verilir.

```
a = [1, 2, 3]
print(a) #[1, 2, 3]
print(a[0]) #birinci eleman #1
print(a[1]) #ikinci eleman #2
print(a[2]) #üçüncü eleman #3
#print(a[3]) #dördüncü eleman!!
#Traceback (most recent call last):
#File "<pyshell#5>", line 1, in <module>
#print(a[3])
#IndexError: list index out of range
print(a[-1]) #sondan birinci eleman #3
print(a[-2]) #sondan ikinci eleman #2
print(a[-3]) #sondan üçüncü eleman #1
b= [1, 'ilhan', (3,5.0), ['özkan',12345]]
print (b) #[1, 'ilhan', (3, 5.0), ['özkan', 12345]]
b[0]=-1 # birinci eleman değiştiriliyor
b[1]='özkan' # ikinci eleman değiştiriliyor
b[2]= 3.14 # üçüncü eleman değiştiriliyor
print(b) # [-1, 'özkan', 3.14 , ['özkan', 12345]]
c=[] #boş liste
c.append(1) #listenin sonuna 1 sayısı eklendi.
c.insert(0,100) #listenin sıfırıncı indisine 100 sayısı eklendi.
print(c) #[100, 1]
c.index(100) #100 sayısı listede hangi indiste? #0
len(c) #c listesinin kaç elemanı var? Uzunluğu nedir? #2
c.reverse() #c listesinin elemanlarını sondan başa yer değiştir. Tersini Al #[100, 1]
c[::-1] #c.reverse() ile aynı işi görür # [1, 100]
c.pop() #varsayılan olarak son elemanı listeden çıkarır. Yada belirtilen indistekini.#1
c #[100]
```

Dilim (**slice**) kavramı dizi, demet ve listenin elemanlarının bir veya birkaçını ifade etmek için kullanılır. Dilim, [**başlangıç:bitiş:adım**] şeklinde ifade edilir. Bu dilimin çıktısı, **başlangıç** indisinden **bitiş-1** indisine olan öğeleri içeren yeni bir listedir. adım, **başlangıç** ve **bitiş-1** arasında ile belirtilen aralıklar ile öğe seçimini ifade eder.

```
liste = [1, 2, 3, 4]
liste[1:] #[2, 3, 4]
liste[:3] #[1, 2, 3]
liste[::2] #[1, 3]
liste[::-1] #[4, 3, 2, 1]
liste[-1:0:-1] #[4, 3, 2]
liste[4:9] #[]
liste[1:10] #[2, 3, 4]
```

set

Küme (**set**), herhangi bir veri tipinde ve tekil (**unique**) elemanlardan oluşan sırasız bir koleksiyondur. Bir nesne, ömrü boyunca asla değişmeyen bir karma (**hash**) değerine sahipse (bir **__hash__()** yöntemine ihtiyaç duyar) ve diğer nesnelerle karşılaştırılabiliriyorsa (bir **__eq__()** yöntemine ihtiyaç duyar) karmalanabilir (**hashable**). Eşitliği karşılaştıran karmalanabilir nesnelerin, aynı karma değerine sahip olması gerekir. Karma, bir nesneye eşsiz bir sayı olarak karşılık gelen değerlerdir.

```
a = { 1, 2, 'a' }
```

```
b = { 1, 1, 2, 2 } # bir değer bir kümede yalnızca 1 kez yer alabilir.
print(b) #{1, 2}
```

dict

Sözlük (**dict**), anahtar-değer çiftlerini (**key-value pair**) tekil olarak tutan bir koleksiyondur. Anahtarlar karmalanabilir (hashable).

```
a={ 1: 'Elma', 2: 'Armut', 3: 'Muz'}
print(a) #{1: 'Elma', 2: 'Armut', 3: 'Muz'}
b= { 'a': [1,2,3], 'b': (1,1), 'c': 'Özkan'}
print(b) # {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
```

Numaralandırma

Numaralandırma (**enumeration**), benzersiz, sabit bir değere bağlı bir dizi sembolik adlara sahip üyelerden oluşan değerlerdir. Bir numaralandırma içinde, üyeler kimliğe göre karşılaştırılabilir ve Numaralandırmanın kendisi yinelenir (**iterable**). Şimdiye kadar verilen örnekler Python yorumlayıcısı ile gerçekleştirilebilir. Bundan sonra verilecek örnekler bir Python dosyasına yazılmış gibi anlaşılabilir python dosya.py şeklinde çalıştırılmalıdır. Python 3.4 ve üzeri sürümlerde numaralandırma (**enum**) modülü standart kütüphanede bulunur, ayrı kurulum gerekmez.

```
from enum import Enum
class Renkler(Enum):
    kırmızı = 1
    yeşil = 2
    mavi = 3
    sarı = 4
print(Renkler.kırmızı) #Renkler.kırmızı
print(Renkler(1)) #Renkler.kırmızı
print(Renkler['kırmızı']) #Renkler.kırmızı

for renk in Renkler: #Numaralandırmalar yinelenir.
    print(renk)
#Renkler.kırmızı
#Renkler.yeşil
#Renkler.mavi
#Renkler.sarı
```

3.6 Düşün ve Çöz

- ◊ `a = (1, 2, [3, 4])` demetinde `a[2][0] = 99` işlemi geçerli midir? Demetlerin değiştirilemez (**immutable**) yapısı ile elemanlarının değiştirilebilir (**mutable**) nesne olması arasındaki farkı tartışınız.
- ◊ `0.1 + 0.2 == 0.3` ifadesinin Python'da `False` dönmesinin nedenini IEEE 754 float standardı açısından açıklayınız.

4. Bölüm

Ardışık İşlemler

4.1 Ardışık İşlem Nedir?

Yapısal Programlama başlığı altında programın ana fonksiyondan başlayacağı ve programlamanın ise birbirini çağıran fonksiyonlarla yapıldığı anlatılmıştı. Yapısal programlamada, ana fonksiyon dahil tüm fonksiyonlarda ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen talimatlar yazılır.

Programın icrası, ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte talimatların icra sırasını yani **kontrol akışını** (**control flow**) değiştirmeyen talimatlara **ardışık işlem** (**sequential operation**) adı verilir.

Ardışık İşlemler

- Değişkenlerin kimliklendirildiği değişken tanımlamaları (identifier definition):
 - `yariCap=3`
 - `PI=3.14`
 - `daireninAlani=0.0;daireninCevresi=0.0`
- İfadelerden** (**expression**): (Sabit, değişken ve operatör içeren söz dizimlerine **ifade** adı verilir)
 - `daireninAlani=PI*yariCap*yariCap`
 - `float daireninCevresi=2.0*PI*yariCap`
- Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (**input-output operation**):
 - `print("Kapasite Oranı: %", oran)`
 - `soyadi = input("Soyadınızı Giriniz:")`

4.2 Konsoldan Girdi Okuma ve Konsola Veri Yazma

Her bilgisayar uygulamasının çalışırken kullanıcıdan girdi kabul etme olanağı olmalıdır. Bu, uygulamayı etkileşimli hale getirir. Nasıl geliştirildiğine bağlı olarak, bir uygulama kullanıcı girdisini konsola (`sys.stdin`) girilen metin, grafiksel bir düzen veya web tabanlı bir ara yüz biçiminde kabul edebilir. Bunu sağlayan `input()` fonksiyonudur. Bu fonksiyonda okunan değerın aktarılacağı bir metin (**string**) değişken bulunmalıdır;

```
adi = input()
soyadi = input("Soyadınızı Giriniz:")
telefon = input("Telefon Numaranızı Giriniz:")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İlhan
Soyadınızı Giriniz:Özkan
Telefon Numaranızı Giriniz:3121234567
```

Yukarıdaki örnekte okunan telefon numarası metin bir değişkene atanmıştır. Sayıya çevirmek gerekir. Benzer bir örnek;

```
yariCapstr = input('Yarıçapı Giriniz:')
yariCap = float(yariCapstr)
# yariCap = float(input('Yarıçapı Giriniz:')) # şeklinde daha Pythonvari (Pythonic)
→ yazılabilirdi.
alan = 3.14*yariCap*yariCap
print('Dairenin Alanı:',alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Yarıçapı Giriniz:2.5
Dairenin Alanı: 19.625
```

Konsola biçimlendirilmiş metin `print()` fonksiyonu ile yazılır. Bu fonksiyon her çağrıldığında konsola yazar ve alt satıra iner.

```
boy=180
kilo=90.5
print(boy)
# Çıktı: 180
print(boy,kilo)
# Çıktı: 180 90.5
```

Eğer birden fazla değişken var ve aralarındaki ayraç (**separator**) değiştirilmek isteniyorsa isimlendirilmiş **sep** parametresi değiştirilerek bu yapılabilir. Metin sonundaki karakteri değiştirmek için ise isimlendirilmiş **end** parametresini kullanabiliriz.

```
boy=180
kilo=90.5
print(boy,kilo,sep=' - ')
# Çıktı: 180 - 90.5
print(boy,'-',end='')
# Çıktı: 180- #bunu yazar ve alt satıra geçmez!
print(kilo)
# Çıktı: 180-90.5 #önce yazılan yerden devam eder!
```

4.3 Konsola Biçimlendirilmiş Metin Yazma

Python programlama dilinde çıktı biçimlendirme (**output formatting**), ham verilerin kullanıcıya sunulurken veya bir dosyaya kaydedilirken belirli bir görsel düzen, genişlik, hizalama ve hassasiyet süzgecinden geçirilmesi işlemidir. Yapısal bir çıktı düzeni, üretilen raporların okunabilirliğini ve yazılım mühendisliği standartlarına uygunluğunu doğrudan etkiler. Python mimarisi, geriye dönük uyumluluk barındıran geleneksel yöntemlerden modern yaklaşımlara uzanan üç ana biçimlendirme tekniği sunar.

§4.3.1 Biçim Belirleyicisi Kullanarak Biçimlendirme

C dilindeki **printf()** fonksiyon ailesinden miras kalan bu geleneksel yöntemde, metinsel kalıbın içerisine değerlerin yerleştirileceği dinamik alanlara **biçim belirleyicileri** (**format specifiers**) yerleştirilir. Geleneksel olarak kullanılan bu yöntemde; biçimlendirme metni içindeki **%** işleci (**string modulo operator**) vasıtasıyla, kendisinden sonra gelen argümanlar sırasıyla ilgili belirteçlerin yerine yerleştirilir ve oluşan nihai metin konsola yazdırılır.

Her bir belirteç mutlaka bir yüzde (**%**) karakteriyle başlar ve yerleştirilecek verinin bellekteki veri türünü (tam sayı, karakter, metin) temsil eden bir harfle sonlanır.

```
# Standart tam sayı (%d) ve kayan noktalı sayı (%f) biçimlendirme
print("Toplam Adet: %2d, Ortalama : %5.2f" % (27, 7.444))
# Çıktı: Toplam Adet: 27, Ortalama : 7.44

print("Öğrenci Sayısı : %3d, Erkek : %2d, Kadın:%2d" % (98, 25, 73))
# Çıktı: Öğrenci Sayısı : 98, Erkek : 25, Kadın:73

# Sayı sistemleri dönüşümlü çıktılar
print("%7o" % (64)) # Çıktı: 100 (Sekizlik - Octal sistem)
print("%10.3E" % (3.141527)) # Çıktı: 3.142E+00 (Bilimsel gösterim)
```

Geleneksel **%** yöntemiyle kullanılacak geçerli biçim karakterleri Tablo 4.1’de listelenmiştir.

İşleç	Adı	Akademik ve Teknik Açıklama
%d	Ondalık Tam Sayı	İşaretsiz onluk tabandaki tam sayıları konsola basar.
%c	Karakter	Tek bir karakteri veya ASCII/Unicode sayısal karşılığını harfe dönüştürerek basılır.
%f	Kayan Noktalı Sayı	Ondalıklı gerçek sayıları standart sabit noktalı biçimde yazar.
%e, %E	Bilimsel Gösterim	Sayıları üssel (eksponansiyel) gösterim formatına çevirir.
%g, %G	Genel Gösterim	Sayının büyüklüğüne göre %f veya %e formatından en kısa olanı otomatik seçer.
%x, %X	Onaltılık Sayı	Sayıları onaltılık (hexadecimal) tabanda küçük/büyük harfle basar.
%o	Sekizlik Sayı	Sayıları sekizlik (octal) tabanda konsola basar.
%s	Metin (string)	Nesnelerin metinsel karşılıklarını ekrana basar.
%%	Yüzde Sembolü	Ekrana doğrudan ham bir % karakteri yazdırmak için kullanılır.

Tablo 4.1: Geleneksel Biçim Belirleyici Karakterleri

C dilindeki alışkanlıkların aksine, Python’ın geleneksel **%** biçimlendirme motorunda ikili sistem için **%b** diye bir belirteç bulunmaz. Eğer kodda **"%b"** % 5 yazılırsa Python yorumlayıcısı **ValueError: unsupported format character 'b'** hatası fırlatarak çöker. İkili sistemde çıktı üretmek için modern **.format()** veya **f-string**

yöntemleri kullanılmalıdır.

§4.3.2 str.format() Yöntemi

Python 2.6 ile dile eklenen bu yöntem, metinlerin içerisine yer tutucu (**placeholder**) olarak süslü parantezler konumlandırarak çalışır. `.format()` fonksiyonuna gönderilen parametreler, indis numaralarına veya anahtar kelimelere (**keyword arguments**) göre bu parantezlerin yerine esnek bir şekilde enterpolasyon yapılır.

```
# 1. Konumsal (Index) Kullanım
print("Meyveler: {0}, {1}, {2}".format("Elma", "Armut", "Vişne"))
# Çıktı: Meyveler: Elma, Armut, Vişne

# İndislerin yerini değiştirerek argüman sırasını manipüle etme
print("Pi:{1:4.2f}, Adet: {0:4d}".format(12, 3.141527))
# Çıktı: Pi:3.14, Adet: 12
```

```
# 2. Adlandırılmış (Keyword) Kullanım
sablon = "Toplam: {0}, Ortalama: {1} ve Standart Sapma: {sapma}."
print(sablon.format(1000, 100, sapma=1.0))
# Çıktı: Toplam: 1000, Ortalama: 100 ve Standart Sapma: 1.0
```

Bu yöntem, raporlama şablonlarında verilerin sağa, sola veya merkeze hizalanması için muazzam hizalama belirteçleri içerir:

```
boy = 180
kilo = 90.5

# Süslü parantezlerin içine ':>Genişlik' yazılarak sağa hizalama sağlanır
print('{0:>10}'.format(boy))
# Çıktı: 180 (10 karakterlik alanda sağa yaslı)

# ': <Genişlik' ile sola hizalama sağlanır
print('{0:<10}-{1:>10}'.format(boy, kilo))
# Çıktı: 180 - 90.5
```

§4.3.3 f-string

Python 3.6 (PEP 498) standartlarıyla hayata geçirilen **f-metni** (**f-string**), modern Python dünyasında en çok kabul gören, okunabilirliği en yüksek ve çalışma zamanında (**runtime**) en hızlı yürütülen yeni nesil çıktı biçimlendirme tekniğidir. Buna değişken enterpolasyonu adı da verilir.

Bir metnin f-metni olabilmesi için açılış tırnağının hemen başına küçük veya büyük harfle **f** veya **F**{} vasıtasıyla doğrudan metnin gövdesine entegre edilebilir.

```
boy = 180
kilo = 90.5

# Doğrudan değişken isimlerini süslü paranteze gömme
print(f"boy: {boy}, kilo: {kilo}") # Çıktı: boy: 180, kilo: 90.5

# f-metni içinde aynı anda genişlik ve hassasiyet ayarı yapma
print(f"kilo: {kilo:5.2f}, boy: {boy:4d}") # Çıktı: kilo: 90.50, boy: 180
```

Ekrana tarih-saat gibi karmaşık nesneleri belirli bir standartta basmak için de f-metni biçimlendirme protokollerinden faydalanılır:

```
import datetime
bugun = datetime.datetime.today()

# %B: Ay ismi, %d: Gün hanesi, %Y: 4 haneli yıl
print(f"Tarih: {bugun:%B %d, %Y}") # Örn Çıktı: June 24, 2026
```

f-string ve Ters Bölü (\) Özgürlüğü

Eski Python sürümlerinde f-string süslü parantezlerinin içinde ters bölü (\) karakterinin veya kaçış dizilimlerinin kullanımı sentaks hatasına yol açmaktaydı. Ancak **Python 3.12 ile birlikte bu kısıtlama tamamen ortadan kaldırılmıştır**. Artık modern Python mimarisinde f-string ifadelerinin içinde satır atlama (`\n`), sözlük anahtarları veya tırnak işaretleri hiçbir kısıtlama olmaksızın özgürce yazılabilmektedir.

§4.3.4 Hangisini Kullanmalıyız?

Yazılım mühendisliği temiz kod ilkeleri (**clean code**) açısından, kodun okunabilirliğini bozması, performans açısından yavaş kalması ve tip güvenliği açıkları barındırması nedeniyle geleneksel `%` yöntemi kesinlikle kullanılmamalıdır.

`str.format()` yöntemi, şablon metinlerinin dinamik olarak dışarıdan alındığı veya raporlama kalıplarının önceden tanımlandığı soyut senaryolarda esnekliği nedeniyle tercih edilmelidir. Günlük geliştirme süreçlerinde, temizliği, doğrudan değişken bağlama gücü ve hızı nedeniyle her zaman **f-string** (**f-metni**) yapısı birincil tercih olmalıdır.

4.4 Tip Dönüşümleri

Programlama dillerinde farklı veri türlerine sahip nesnelerin aynı matematiksel veya mantıksal işleme tabi tutulması, ya da bir kaynaktan (örneğin kullanıcı klavyesinden veya bir dosyadan) metin olarak alınan verilerin sayısal hesaplamalara dahil edilmesi esnasında **veri tipi dönüşümleri** (**type conversion**) gerçekleştirilir.

Python, tip güvenliği açısından **güçlü dinamik veri tipli** (**strongly typed**) bir dil mimarisine sahiptir. Bu mimari, mantıksal olarak birbiriyle uyumsuz olan veri türleri arasında (örneğin bir metin ile bir tam sayı arasında) örtük işlemlere izin vermez. Python'da tip dönüşümleri; sistem tarafından kendiliğinden icra edilen örtük (otomatik) Dönüşüm ve programcı tarafından el ile tetiklenen Açık Dönüşüm olmak üzere ikiye ayrılır.

§4.4.1 Örtük (Otomatik) Tip Dönüşümü

Herhangi bir derleyici veya yorumlayıcının, bir işlem esnasında veri kaybı riskini en aza indirmek adına alt seviyedeki bir veri türünü otomatik olarak daha geniş veya kapsayıcı bir veri türüne dönüştürmesine örtük dönüşüm (**implicit type conversion**) denir.

Python, veri hassasiyetini korumak amacıyla tam sayı (**int**) veri türünü, kayan noktalı sayı (**float**) veri türü ile işleme soktuğunda tam sayıyı otomatik olarak **float** nesnesine dönüştürür. Benzer şekilde, Boole (**bool**) veri tipindeki **True** mantıksal sabiti arka planda matematiksel olarak **1** değerine, **False** sabiti ise **0** değerine örtük olarak dönüştürülebilme yeteneğine sahiptir.

```
# Tam sayı ve Kayan Noktalı Sayı Arasında Örtük Dönüşüm
tamsayi = 10
kayan_noktali = 15.5
```

```
# int ile float toplanıyor; Python veriyi korumak için sonucu float yapar.
sonuc = tamsayi + kayan_noktali
print(sonuc)           # Çıktı: 25.5
print(type(sonuc))     # Çıktı: <class 'float'>
```

```
# Boole Veri Türünün Örtük Dönüşümü
# True sabiti matematiksel işlemlerde 1 olarak anlamlandırılır.
mantiksal_dogru = True
sonuc = sonuc + mantiksal_dogru
print(sonuc)           # Çıktı: 26.5
```

Bilgi Kaybı ve Dönüşüm Yönü

Otomatik dönüşümler her zaman dar kapsamlı kümeden geniş kapsamlı kümeye doğru (**int** → **float**) gerçekleşir. Bunun tam tersi (**float** → **int**) otomatik olarak yapılamaz; çünkü bir kayan noktalı sayının tam sayıya indirgenmesi, virgülden sonraki kesirli kısmın tamamen silinmesi yani **veri/bilgi kaybı** anlamına gelir. Python programcının izni olmadan veri silmeye yetkili değildir.

§4.4.2 Açık Tip Dönüşümü

Programcının, dil çekirdeğinde yer alan `int()`, `float()`, `str()` gibi özel yapıcı fonksiyonları kullanarak bir nesneyi bilinçli ve zorunlu olarak başka bir veri türüne dönüştürmesi işlemine açık dönüşüm (**explicit type conversion**) veya zorlama (**casting**) denir.

Özellikle klavyeden alınan girdilerin (`input()`) varsayılan olarak metin (`str`) olarak gelmesi, açık dönüşümlerin pratik yazılım süreçlerinde sıklıkla kullanılmasını zorunlu kılar.

```
# Metinden (str) Sayısal Türlerle Açık Dönüşüm
tamsayi_metni = '321'
tamsayi_deger = int(tamsayi_metni)
print(tamsayi_deger)          # Çıktı: 321

kayan_noktali_metin = '7.5'
kayan_noktali_deger = float(kayan_noktali_metin)
print(kayan_noktali_deger)    # Çıktı: 7.5

# Kayan Noktalı Sayıdan Tam sayıya Dönüşüm (Kesirli kısım budanır)
yeni_tamsayi = int(kayan_noktali_deger)
print(yeni_tamsayi)          # Çıktı: 7 (Veri kaybı gerçekleşti)

# Sayısal Bir Nesneyi Metne (str) Dönüştürme
# UYARI: 'str' kelimesini değişken adı olarak kullanmaktan kaçınmalıyız!
metin_karsiligi = str(kayan_noktali_deger)
print('Metin:', metin_karsiligi, 'Türü:', type(metin_karsiligi))
# Çıktı: Metin: 7.5 Türü: <class 'str'>
```

§4.4.3 Taban Parametresi ile Gelişmiş Tam Sayı Dönüşümleri

`int()` fonksiyonu, yalnızca standart onluk sistemdeki metinleri değil; bilgisayar bilimlerinde sıklıkla kullanılan ikili (**binary**), sekizli (**octal**) ve onaltılık (**hexadecimal**) sayı sistemlerindeki metinsel ifadeleri de onluk tabandaki tam sayılara dönüştürebilir.

Bunun için fonksiyonun ikinci parametresine dönüştürülmek istenen metnin sayısal tabanı (**radix/base**) açıkça yazılmalıdır. Dönüştürülecek dizgi, belirtilen tabanın sayı/harf kümesi sınırlarına kesinlikle uyumlu olmak zorundadır.

```
# İkili tabandan (Base 2) Onluk tabana dönüşüm (Sadece '0' ve '1' içerebilir)
ikili_tabandan = int("1010", 2)
print(ikili_tabandan)        # Çıktı: 10

# Sekizli tabandan (Base 8) Onluk tabana dönüşüm (0-7 arasındaki rakamlar)
sekizli_tabandan = int("12", 8)
print(sekizli_tabandan)      # Çıktı: 10

# Onaltılık tabandan (Base 16) Onluk tabana dönüşüm (0-9 ve A-F karakterleri)
onaltilik_tabandan = int("A", 16)
print(onaltilik_tabandan)    # Çıktı: 10
```

Geçersiz Taban Hataları (ValueError)

Taban dönüşümü yapılırken metin içerisindeki karakterlerin o tabana ait olması şarttır. Örneğin `int("102", 2)` kodu çalıştırıldığında Python `ValueError: invalid literal for int() with base 2` hatası verir; çünkü ikili sistemde 2 rakamı bulunamaz.

4.5 İşleçler

İşleçler (**operator**) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (**operand**) adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

Atama İşleci Her Zaman Sağdaki İşleneni Soldakine Atar

En çok kullanılan işleç, atama (=) işlecidir. **Atama işleci, her zaman sağdaki işleneni soldaki değişkene atar.** Bu nedenle kodlama yapılırken $2+2=x$ veya $a+b=x$ yazılamaz!

§4.5.1 Aritmetik İşleçler

Python dilinde matematiksel hesaplamaları ve sayısal dönüşümleri gerçekleştirmek amacıyla kullanılan sembollere **aritmetik işleçler** (**arithmetic operators**) adı verilir. Bu işleçlerin üzerinde eylem gerçekleştirdiği değişken veya sabit değerlere ise işlenen (**operand**) denir. Python, matematiksel işlemlerde donanımsal taşmaları (**overflow**) otomatik engellemesinin yanı sıra, bölme işlemlerinde bilgisayar bilimi standartlarına tam uyumlu iki farklı yaklaşım sunar. Python dilindeki temel aritmetik işleçlerin yapısal ve işlevsel özellikleri Tablo 4.2'de detaylandırılmıştır.

İşleç	Adı	Açıklama
+	Toplama	İki sayısal işlenenin matematiksel toplamını verir. Sayısal olmayan yapılarda (str , list) ise iki nesneyi ardışık olarak birleştirir (concatenation).
-	Çıkarma	Soldaki işlenenden sağdaki işleneni çıkarır. Eğer tek bir işlenenin solunda simetrik olarak kullanılırsa (unary operator), o sayının matematiksel işaretini tersine çevirir.
*	Çarpma	İki sayısal değeri birbiriyle çarpır. Bir metin (str) veya liste ile bir tam sayı çarpılırsa, o nesneyi belirtilen sayı kadar ardı ardına çoğaltır (repetition).
/	Gerçek Bölme	Soldaki işleneni sağdakine böler. C dilinin aksine, bölünen sayılar tam sayı olsa bile sonuç her zaman kayan noktalı gerçek sayı (float) olarak döner (Örn: $4 / 2$ işleminin sonucu 2.0 'dir).
//	Taban Bölme	Soldaki işlenenin sağdakine bölünmesiyle elde edilen bölümün en yakın küçük tam sayıya yuvarlanmış (floor division) değerini verir. Sonucun virgülden sonraki kısmı atılır. Örneğin $7 / 2$ işlemi 3.5 sonucunu verirken, $7 // 2$ işlemi 3 sonucunu üretir. Negatif sayılarda ise alta yuvarlama kuralı gereği $-7 // 2$ işleminin sonucu -4 olacaktır. Soldaki işlenenin sağdaki işlenene kalansız bölünmesinden elde edilen kalan değeri (modulus) verir. Çift-tek sayı kontrollerinde veya döngüsel indeks yönetiminde sıklıkla kullanılır.
**	Üs Alma	Soldaki işlenenin (taban), sağdaki işlenenin (üs) kuvveti kadar değerini hesaplar (x^y). C dilindeki pow() fonksiyonunun dil çekirdeğine gömülmüş yerleşik karşılığıdır.

Tablo 4.2: Aritmetik İşleçler

Aşağıdaki örnekte aritmetik işleçlerin hem sayısal hem de sayısal olmayan veri türleri üzerindeki davranışları uygulamalı olarak gösterilmiştir:

```
# Sayısal Hesaplama Örnekleri
a = 15
b = 4
print(f"Gerçek Bölme (15 / 4) : {a / b}") # Çıktı: 3.75
print(f"Taban Bölme (15 // 4) : {a // b}") # Çıktı: 3
print(f"Modül Alımı (15 % 4) : {a % b}") # Çıktı: 3 (15'in 4'ten kalanı)
print(f"Üs Alma (2 ** 3) : {2 ** 3}") # Çıktı: 8

# Operatör Aşırı Yükleme (Nesne Birleştirme ve Çoğaltma) Örnekleri
selamlama = "Merhaba " + "Dünya!" # + işleci metinleri birleştirdi
cizgi = "-" * 20 # * işleci karakteri 20 kez çoğalttı

print(selamlama)
print(cizgi)
```

§4.5.2 İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler.

Python'da mantıksal doğruluk değerlerini temsil etmek amacıyla özelleştirilmiş, yerleşik bir **mantıksal veri tipi** (**boolean data type**) mevcuttur ve **bool** anahtar kelimesi ile ifade edilir. Bu veri tipi yalnızca iki benzersiz değişmeze (**literal**) sahip olabilir: **True** ve **False**.

Python mimarisinde iki değer arasındaki büyüklük, küçüklük veya eşitlik gibi durumları denetlemek için ilişkisel (**relational**) yani karşılaştırma (**comparison**) işleçleri kullanılır. Bu işleçler matematiksel bir hesaplama yapmak yerine, lojik bir önerme değerlendirmesi yaparak her zaman doğrudan **bool** (**True** veya **False**) türünde bir sonuç üretirler. İlişkisel işleçlerin yapısal ve işlevsel özellikleri Tablo 4.3'de detaylandırılmıştır.

İşleç	Adı	Açıklama
<code>==</code>	Eşittir	Sağdaki ve soldaki işlenenin değerce birbirine eşit olması durumunda True , aksi halde False döndürür.
<code>!=</code>	Eşit Değildir	İşlenenlerin değerleri birbirine eşit değilse True , eşitse False değerini üretir.
<code>></code>	Büyüktür	Soldaki işlenenin sayısal veya alfabetik değeri sağdakinden büyükse True döndürür.
<code><</code>	Küçüktür	Soldaki işlenenin sayısal veya alfabetik değeri sağdakinden küçükse True döndürür.
<code>>=</code>	Büyük veya Eşit	Soldaki işlenenin değeri sağdakinden büyük veya ona eşitse True döndürür.
<code><=</code>	Küçük veya Eşit	Soldaki işlenenin değeri sağdakinden küçük veya ona eşitse True döndürür.

Tablo 4.3: İlişkisel İşleçler

Aşağıda eşitlik işleçlerin ilişkin örnek verilmiştir:

```
a = 10
b = 20
```

```
# Karşılaştırma sonuçlarını doğrudan değişkenlere aktarıyoruz
kontrol1 = (a < b) # 10 < 20 önermesi doğrudur
kontrol2 = (a == b) # 10 == 20 önermesi yanlıştır
```

```
print(f"10 < 20 Sonucu : {kontrol1} | Veri Türü: {type(kontrol1)}") #10 < 20 Sonucu : True | Veri
→ Türü: <class 'bool'>
print(f"10 == 20 Sonucu: {kontrol2} | Veri Türü: {type(kontrol2)}") #10 == 20 Sonucu: False |
→ Veri Türü: <class 'bool'>
```

```
# Python'da zincirleme karşılaştırma (Chained Comparison) yapılabilir.
# C dilinde '5 < a < 15' ifadesi mantık hatası üretirken Python'da matematiksel olarak çalışır:
not_araligi = 5 < a < 15
print(f"5 < 10 < 15 Doğruluk Durumu: {not_araligi}") #5 < 10 < 15 Doğruluk Durumu: True
```

Doğruluk (truthiness)

Python, ilişkisel işleçlerin sonucunda her ne kadar katı bir şekilde **True/False** döndürse de, kontrol bloklarında (**if**) tıpkı C dilindeki benzer örtük (**implicit**) bir mantıksal dönüşüm yapar. Bilgisayar biliminde bu duruma Python ekosisteminde **doğruluk (truthiness)** adı verilir. Python mimarisinde mantıksal olarak **False** (Yanlış) kabul edilen nesneler şunlardır:

- ◊ Sayısal sıfır değerleri (0, 0.0)
- ◊ Boş metinsel ve veri yapıları ("", [], (), {})
- ◊ Yokluk değişmezi (None)

Bunların dışındaki tüm tam sayılar (negatif sayılar dahil), içi dolu metinler ve tüm nesneler örtük olarak **True** kabul edilir.

§4.5.3 Atama İşleçleri

Python programlama dilinde eşittir (=) sembolü, matematiksel bir eşitlik bildiriminden ziyade, sağ tarafta üretilen bir veriyi sol taraftaki bir değişkene bağlayan bir **atama işleci (assignment operator)** olarak tanımlanır.

Atama işleçleri her zaman sağdan sola doğru işler; yani sağdaki işlenenin (**operand**) değerini veya bellek adresini, solunda yer alan tekil bir değişkene atar. Python, temel atama işlecinin yanı sıra, aritmetik işlemlerle atama işlemini tek adımda birleştiren **birleşik atama işleçlerine (augmented assignment operators)** de sahiptir. Bu işleçler hem kodun yazım süresini kısaltır hem de yorumlayıcı seviyesinde bellek optimizasyonu sağlar.

Aşağıda atama işleçlerine örnekler gösterilmiştir;

```
# Temel Atama İşlemi (Sağdaki değer sola aktarılır)
sayi = 10

# Birleşik Atama İşlemleri
sayi += 5 # sayi = sayi + 5 -> Yeni değer: 15
print(f"Toplayarak Atama Sonucu (+= 5): {sayi}") #Toplayarak Atama Sonucu (+= 5): 15

sayi *= 2 # sayi = sayi * 2 -> Yeni değer: 30
print(f"Çarparak Atama Sonucu (*= 2) : {sayi}") #Çarparak Atama Sonucu (*= 2) : 30
```

İşleç	Adı	Açıklama
=	Temel Atama	Sağdaki işlenenin değerini/referansını doğrudan soldaki değişkene atar. Kesinlikle soldan sağa atama yapılamaz.
+=	Toplayarak Atama	Soldaki değişkenin mevcut değerine sağdaki işleneni ekler ve elde edilen yeni sonucu yine sola atama yapar ($x = x + y$).
-=	Çıkararak Atama	Soldaki değişkenin mevcut değerinden sağdaki işleneni çıkarır ve sonucu sola atama yapar ($x = x - y$).
*=	Çarparak Atama	Soldaki değişkeni sağdaki işlenen ile çarpar ve elde edilen yeni değeri sola atama yapar ($x = x * y$).
/=	Bölerek Atama	Soldaki değişkeni sağdakine böler ve sonucu her zaman float türünde sola atama yapar ($x = x / y$).
//=	Taban Bölerek Atama	Soldaki değişkeni sağdakine böler, bölümü en yakın küçük tam sayıya yuvarlayarak sola atama yapar ($x = x // y$).
%=	Modül Alarak Atama	Soldaki değişkenin sağdakine bölümünden kalan değeri hesaplar ve sola atama yapar ($x = x \% y$).
**=	Üs Alarak Atama	Soldaki değişkenin sağdaki işlenen kadar kuvvetini (üssünü) hesaplar ve sonucu sola atama yapar ($x = x ** y$).

Tablo 4.4: Atama İşleçleri

```
sayi /= 3 # sayi = sayi / 3 -> Yeni değer: 10.0 (Türe dikkat: float)
print(f"Bölerek Atama Sonucu (/= 3) : {sayi}") #Bölerek Atama Sonucu (/= 3) : 10.0

sayi **= 2 # sayi = sayi ** 2 -> Yeni değer: 100.0
print(f"Üs Alarak Atama Sonucu (**= 2): {sayi}") #Üs Alarak Atama Sonucu (**= 2): 100.0
```

Python'da ++ ve -- Neden Yoktur?

C, C++ ve Java gibi dillerde bir değişkenin değerini 1 artırmak için `x++` yapısı sıklıkla kullanılır. Ancak Python dil mimarisinde `++` veya `--` gibi tekil artırma/azaltma işlemleri **kesinlikle mevcut değildir**. Python tasarımcıları, bu sembollerin kod okunabilirliğini bozduğunu ve **yan etkilere (side-effects)** yol açtığını savunmuştur. Bu nedenle Python'da bir sayıcıyı 1 artırmak için her zaman birleşik toplayarak atama (`x += 1`) işleci tercih edilir.

§4.5.4 Mantıksal İşleçler

Birden fazla ilişkisel önermeyi birbirine bağlayarak daha karmaşık mantıksal denetimler gerçekleştirmek amacıyla **mantıksal işleçler (logical/boolean operators)** kullanılır. Bu işleçlerin her bir işleneni (**operand**) kendi başına birer boole ifadesidir veya mantıksal olarak bir doğruluk değerine (**Truthiness**) sahiptir.

Python dilinde mantıksal işleçler C tabanlı dillerdeki semboller (`&&`, `||`, `!`) yerine tamamen İngilizce kelimelerle ifade edilir ve mutlaka küçük harfle yazılmalıdır: **and**, **or** ve **not**. Bu işleçlerin davranış kuralları Tablo 4.5'de listelenmiştir.

İşleç	Adı	Açıklama
and	Mantıksal VE	Sağdaki ve solundaki işlenenlerin her ikisi de mantıksal olarak True ise sonuç True olur. İşlenenlerden en az biri False ise sonuç False olarak değerlendirilir.
or	Mantıksal VEYA	Sağdaki ve solundaki işlenenlerden en az birinin True olması durumunda True sonucunu üretir. Sadece iki işlenenin de False olduğu durumda False döner.
not	Mantıksal DEĞİL	Tek bir işlenen üzerinde işlem yapan bir işleçtir (unary operator). Sağdaki Boole ifadesinin mevcut mantıksal durumunu tersine çevirir (True ise False , False ise True yapar).

Tablo 4.5: Mantıksal İşleçleri

Kısa Devre Değerlendirmesi (Short-Circuit Evaluation)

Python yorumlayıcısı, mantıksal ifadeleri soldan sağa doğru incelerken performansı ve işlemci zamanını optimize etmek için Kısa Devre mimarisini kullanır:

- ♦ **and işleminde:** Sol taraftaki ifade **False** ise, sağdaki ifadenin ne olduğuna bakılmaksızın sonucun kesinlikle **False** olacağı anlaşılır ve Python sağ tarafı hiç çalıştırmadan döngüyü keser.
- ♦ **or işleminde:** Sol taraftaki ifade **True** ise, sonucun her halükarda **True** olacağı netleşir ve sağdaki ifade tamamen göz ardı edilir.

Mantıksal İşleçler Aslında Ne Döndürür?

Bilgisayar bilimlerinde Python'a özgü en büyüleyici dinamiklerden biri şudur: **and** ve **or** işleçleri saf Boole ifadeleri yerine normal nesnelerle (**int**, **str** vb.) kullanıldığında geriye **True** ya da **False** döndürmez; mantıksal karara son noktayı koyan nesnenin ham halini döndürür!
Örneğin: **"Ahmet" or "Mehmet"** ifadesinde **"Ahmet"** metni mantıksal olarak **True** (dolmuş) olduğu için Kısa Devre çalışır ve Python geriye doğrudan **"Ahmet"** metnini döndürür.

Aşağıdaki örnekte mantıksal işleçlerin hem standart boole değerleri hem de sayısal aralıklar üzerindeki pratik kullanımları gösterilmiştir:

```
yas = 22
ehliyet_var_mi = True
# 1. and İşleci Örneği (İki koşul da sağlanmalı)
arac_kullanabilir_mi = (yas >= 18) and ehliyet_var_mi
print(f"Araç Kullanma Durumu: {arac_kullanabilir_mi}") # Çıktı: True
# 2. or İşleci Örneği (En az biri sağlanmalı)
bilet_fiyati = 100
ogrenci_mi = True
yasli_mi = False
if ogrenci_mi or yasli_mi:
    bilet_fiyati -= 20 # İndirim uygula
print(f"Güncel Bilet Fiyatı : {bilet_fiyati} TL") # Çıktı: 80 TL
# 3. not İşleci Örneği (Durumu tersine çevirme)
sistem_acik_mi = False
if not sistem_acik_mi:
    print("Sistem Uyarısı: Sunucu şu anda kapalı!")
```

§4.5.5 Bit Düzeyi İşleçler

Yüksek seviyeli bir dil olan Python, donanıma yakın alt seviyeli optimizasyonlar, şifreleme algoritmaları, ağ protokolleri ve veri sıkıştırma işlemlerini gerçekleştirebilmek amacıyla bit düzeyi işleçleri (**bitwise operators**) bünyesinde barındırır. Bu işleçler, tamsayı (**int**) veri tipindeki nesneleri bellekteki ham halleriyle, yani birer bit dizisi (0 ve 1) olarak ele alır ve mantıksal işlemleri doğrudan işlemci yazmaçları (**registers**) seviyesinde icra eder. Python programlama dilinde tamsayılar üzerinde bit seviyesinde manipülasyon yapabilen altı farklı işleç mevcuttur. Bu işleçlerin çalışma prensipleri Tablo 4.6'de detaylandırılmıştır.

İşleç	Adı	Açıklama
&	Bit Düzeyi VE (AND)	İki işlenenin karşılıklı bitlerini karşılaştırır. Yalnızca her iki bit de 1 ise sonuç bitini 1 yapar; aksi halde 0 üretir.
	Bit Düzeyi VEYA (OR)	Karşılıklı bitlerden en az birinin 1 olması durumunda sonuç bitini 1 yapar. Sadece iki bitin de 0 olduğu durumda 0 üretir.
^	Bit Düzeyi ÖZEL VEYA (XOR)	Karşılıklı bitler birbirinden farklıysa (biri 1 diğeri 0 ise) sonuç bitini 1 yapar. Bitler aynıysa 0 üretir.
~	Bit Düzeyi DEĞİL (NOT)	Tek bir işlenen üzerinde çalışan (unary) bu işleç, tamsayının tüm bitlerini tersine çevirir (0'ları 1, 1'leri 0 yapar).
<<	Sola Kaydırma (Left Shift)	Soldaki işlenenin bit haritasını, sağdaki işlenen miktarı kadar sola öter. Her bir adım sola kaydırma, sayıyı matematiksel olarak 2 ile çarpma anlamına gelir.
>>	Sağa Kaydırma (Right Shift)	Soldaki işlenenin bit haritasını, sağdaki işlenen miktarı kadar sağa öter. Her bir adım sağa kaydırma, sayıyı matematiksel olarak 2'ye tam bölmek anlamına gelir.

Tablo 4.6: Python Programlama Dili Bit Düzeyi İşleçleri

(DEĞİL) İşleci ve İkiye Tümlleyen (Two's Complement)

Öğrencilerin bit düzeyinde en çok bocaladığı yer **~** işlecinin ürettiği sonuçtur. Python'da **~x** işleminin matematiksel sonucu her zaman **-(x + 1)** formülüyle hesaplanır.
Örneğin **~12** yazdığınızda Python bize **-13** cevabını verir. Bunun sebebi, bilgisayar mimarisinde negatif sayıların hafızada **İkiye Tümlleyen** yöntemiyle saklanması ve Python'daki tam sayıların sınır kapısının (bit genişliğinin) olmamasıdır. Yorumlayıcı en baştaki işaret bitini de tersine çevirdiği için sayı negatife evrilir.

Python'da tam sayıların bellekteki ikili (**binary**) kod karşılıklarını görebilmek ve ekrana yazdırmak için yerleşik `bin()` fonksiyonu kullanılır. Bu fonksiyon çıktı olarak metin (**string**) üretir ve sayının ikili sistemde olduğunu belirtmek için başına `0b` (**binary prefix**) ön ekini koyar. Aşağıdaki programda 12 (1100_2) ve 5 (0101_2) sayıları üzerinden tüm bit düzeyi işlemlerin pratik çıktı analizleri gerçekleştirilmiştir:

```
# Sayısal değerlerin ve bit haritalarının tanımlanması
# 12 = 8 + 4 + 0 + 0 -> 1100
# 5 = 0 + 4 + 0 + 1 -> 0101
a = 12
b = 5

print(f"a sayisi: {a} -> İkili karşılığı: {bin(a)}") #a sayisi: 12 -> İkili karşılığı: 0b1100
print(f"b sayisi: {b} -> İkili karşılığı: {bin(b)}") #b sayisi: 5 -> İkili karşılığı: 0b101

# 1. Bit Düzeyi VE (&) İşlemi
# 1100 (12)
# 0101 (5)
# & 1100 & 0101 = 0100 (Ondalık karşılığı: 4)
print(f"a & b: {a & b} -> İkili: {bin(a & b)}") #a & b: 4 -> İkili: 0b100

# 2. Bit Düzeyi VEYA (|) İşlemi
# | 1100 | 0101 = 1101 (Ondalık karşılığı: 13)
print(f"a | b: {a | b} -> İkili: {bin(a | b)}") #a | b: 13 -> İkili: 0b1101

# 3. Bit Düzeyi ÖZEL VEYA (XOR) İşlemi
# ^ 1100 ^ 0101 = 1001 (Ondalık karşılığı: 9)
print(f"a ^ b: {a ^ b} -> İkili: {bin(a ^ b)}") #a ^ b: 9 -> İkili: 0b1001

# 4. Bit Düzeyi DEĞİL (~) İşlemi -> Formula: -(x + 1)
print(f"~a: {~a} -> İkili: {bin(~a)}") #~a: -13 -> İkili: -0b1101

# 5. Bit Kaydırma İşlemleri
# 5 sayısı (0101) 2 adım sola kayarsa: 010100 -> 20 olur (5 * 2 * 2)
print(f"b << 2: {b << 2} -> İkili: {bin(b << 2)}") #b << 2: 20 -> İkili: 0b10100

# 12 sayısı (1100) 2 adım sağa kayarsa: 0011 -> 3 olur (12 / 2 / 2)
print(f"a >> 2: {a >> 2} -> İkili: {bin(a >> 2)}") #a >> 2: 3 -> İkili: 0b11
```

§4.5.6 Üyelik İşlemleri

Python programlama dilinde, bir veri ögesinin (karakter, sayı veya nesne) bir kapsayıcı yapı (metin dizisi, liste, demet veya küme) içerisinde yer alıp almadığını denetlemek amacıyla **üyelik işlemleri** (**membership operators**) kullanılır. C gibi dillerde bir dizide arama yapmak için doğrusal arama (**linear search**) döngüleri yazılması zorunluyken, Python bu süreci dilin söz dizimine yerleşik olarak eklemiştir. Üyelik işlemleri, aranan öğenin koleksiyondaki varlık durumuna göre doğrudan **bool** (**True** veya **False**) veri tipinde bir mantıksal sonuç üretir. Bu işlemlerin özellikleri Tablo 4.7'de gösterilmiştir.

İşleç	Adı	Açıklama
<code>in</code>	İçinde (Üyesidir)	Aranan öğe, hedef kapsayıcı koleksiyonun içinde mevcut ise True , aksi halde False sonucunu verir.
<code>not in</code>	İçinde Değil (Üyesi Değildir)	Aranan öğe, hedef kapsayıcı koleksiyonun içinde mevcut değil ise True , mevcutsa False sonucunu verir.

Tablo 4.7: Python Programlama Dili Üyelik İşlemleri

Aşağıdaki programda üyelik işlemlerinin karakter dizileri (**str**) ve listeler (**list**) üzerindeki pratik kullanımları aritmetik ifadelerle birlikte örneklenmiştir:

```
# 1. Metinlerde (str) Karakter/Alt metin Araması
metin = "Bugün bayram."
bul = "."

# Nokta karakteri metnin içinde var mı?
print(f"'{bul}' karakteri metinde var mı? -> {bul in metin}") # '.' karakteri metinde var mı? ->
↪ True
```

```
# Alt dize araması (Aynı anda birden fazla karakter aranabilir)
print(f"'gün' kelimesi metinde var mı? -> {'gün' in metin}") #'gün' kelimesi metinde var mı? ->
→ True

# 2. Listelerde (list) Eleman Araması
liste = [10, 20, 30, 40, 50]
a = 20
b = 3

print(f"{a} elemanı listede var mı? -> {a in liste}") #20 elemanı listede var mı? -> True
print(f"{b} elemanı listede var mı? -> {b in liste}") #3 elemanı listede var mı? -> False

# 3. Aritmetik İfade ve 'not in' (İçinde Değil) Birlikte Kullanımı
# a * b = 20 * 3 = 60. 60 değeri [10, 20, 30, 40, 50] listesinde YOKTUR.
# Dolayısıyla 'yoktur' önermesi TRUE sonuç verecektir.
sonuc = (a * b) not in liste
print(f"{a * b} elemanı listede YOK mu? -> {sonuc}") #60 elemanı listede YOK mu? -> True
```

Üyelik İşleçlerinin Arka Plan Performansı

Üyelik işleçlerinin arama hızı, kullanılan veri yapısının mimarisine doğrudan bağlıdır:

- ◊ **Listelerde (list) ve Metinlerde (str):** Python arama işlemini baştan sona tek tek kontrol ederek yapar. Veri boyutu büyüdükçe arama süresi doğrusal olarak artar ($O(n)$ karmaşıklığı).
- ◊ **Kümelerde (set) ve Sözlüklerde (dict):** Python arka planda **Karma Tablo (Hash Table)** mimarisini kullanır. Bu sayede koleksiyonda milyonlarca eleman olsa bile **in** işleci aranan elemanı anında ($O(1)$ karmaşıklığıyla) bulur.

§4.5.7 Kimlik İşleçleri

Python programlama dilinde, iki farklı değişkenin sadece değerlerinin mi eşit olduğunu, yoksa bellekte tamamen aynı nesneyi mi işaret ettiğini denetlemek amacıyla **kimlik işleçleri (identity operators)** kullanılır.

Python mimarisinde her nesnenin benzersiz bir kimlik numarası (bellek adresi) bulunur. Kimlik işleçleri nesnelerin içindeki ham veriyi değil, doğrudan bu id (kimlik) yani bellek adresi değerlerini karşılaştırır. Bu yönüyle C dilindeki gösterici (**pointer**) karşılaştırmalarına benzetilebilir. Kimlik işleçlerinin özellikleri Tablo 4.8’de gösterilmiştir.

İşleç	Adı	Açıklama
is	Aynı Nesnedir	Sağındaki ve solundaki değişkenler bellekte tamamen aynı adresi/nesneyi işaret ediyorsa True , farklı nesneleri işaret ediyorsa False sonucunu verir.
is not	Aynı Nesne Değildir	Sağındaki ve solundaki değişkenler bellekte farklı adresleri/nesneleri işaret ediyorsa True , aynı adresi işaret ediyorsa False sonucunu verir.

Tablo 4.8: Python Programlama Dili Kimlik İşleçleri

Değer ve Referans Eşitliği

Yeni başlayan programcıların en sık yaptığı mantık hatası ile **is** işleçlerini karıştırmaktır:

- ◊ **== (Değer Eşitliği):** Nesnelerin içindeki verilerin aynı olup olmadığına bakar (**Value Equality**).
- ◊ **is (Kimlik/Referans Eşitliği):** Nesnelerin bellekteki yerleşim adreslerinin aynı olup olmadığına bakar (**Reference Identity**).

İçerikleri kelimesi kelimesine aynı olan iki farklı kitap düşünün. Bu kitapların içerikleri eşittir (**== True**), ancak fiziksel olarak masada farklı yer kaplayan iki ayrı nesnedirler (**is False**).

Python’da bir nesnenin benzersiz bellek adresine ulaşmak için yerleşik **id()** fonksiyonu kullanılır. Aşağıdaki kod laboratuvarında, aynı değerlere sahip koleksiyonların bellekteki kimlik analizleri uygulamalı olarak simüle edilmiştir:

```
# liste1 ve liste2 aynı elemanlara sahip iki farklı nesne olarak oluşturulur
liste1 = [10, 20, 30, 40, 50]
liste2 = [10, 20, 30, 40, 50]
```

```
# liste3'e liste1'in referansı (bellek adresi) kopyalanır (Aliasing)
liste3 = liste1

# 1. Değer Eşitliği Kontrolü (==)
print(f"liste1 ve liste2 değerleri eşit mi?: {liste1 == liste2}") # True

# 2. Kimlik/Bellek Adresi Kontrolü (is / is not)
print(f"liste1 is liste2 (Aynı nesne mi?) : {liste1 is liste2}") # False
print(f"liste1 is liste3 (Aynı nesne mi?) : {liste1 is liste3}") # True
print(f"liste1 is not liste2 (Farklı mı?) : {liste1 is not liste2}") # True

# 3. Arka Planın İspatı (id() Fonksiyonu ile Doğrulama)
print(f"liste1'in Bellek Adresi (id) : {id(liste1)}")
print(f"liste2'nin Bellek Adresi (id) : {id(liste2)}")
print(f"liste3'ün Bellek Adresi (id) : {id(liste3)}")
```

§4.5.8 İşleç Öncelikleri

Matematikte ve cebirsel ifadelerde işlem önceliği kuralları (örneğin önce çarpma, sonra toplama) nasıl geçerliyse, Python programlama dilinde yazılan karmaşık ifadelerin değerlendirilmesinde de katı bir **işleç öncelik sırası** (**operator precedence**) yürütülür.

Bir ifade içinde farklı önceliklere sahip birden fazla işleç yer aldığında, Python yorumlayıcısı önceliği yüksek olan işleçten başlayarak matematiksel veya mantıksal çözümlemeyi tamamlar. Programcı, kodun herhangi bir bölümündeki işlem sırasını standart hiyerarşinin dışına çıkararak öncelikli hale getirmek isterse, o ifadeyi parantez **()** içerisine almalıdır. İç içe yazılmış parantezlerde öncelik her zaman en içteki parantezdedir.

Python dilindeki temel işleçlerin en yüksek öncelikten (en önce çalışacak olandan) en düşük önceliğe doğru sıralaması Tablo 4.9'de gösterilmiştir.

Öncelik	İşleçler	Açıklama
1	()	Parantez İçi İfadeler (En yüksek öncelik)
2	[index], [start:stop]	İndisleme, Dilimleme ve Nesne Çağrılar
3	**	Üs Alma İşlemi
4	+x, -x, ~x	Tekil (unary) İşleçler: Pozitif/Negatif İşareti ve Bit Ters
5	*, /, //, %	Çarpma, Gerçek Bölme, Taban Bölme ve Modül
6	+, -	Toplama ve Çıkarma
7	<<, >>	Bit Düzeyi Sola ve Sağa Kaydırma
8	&	Bit Düzeyi VE (AND)
9	^	Bit Düzeyi ÖZEL VEYA (XOR)
10		Bit Düzeyi VEYA (OR)
11	<, <=, >, >=, ==, !=	İlişkisel (Karşılaştırma) İşleçleri
12	in, not in, is, is not	Üyelik ve Kimlik Denetim İşleçleri
13	not x	Mantıksal DEĞİL
14	and	Mantıksal VE
15	or	Mantıksal VEYA (En düşük öncelik)

Tablo 4.9: Python Programlama Dili İşleç Öncelikleri

Aşağıdaki örnek programda, parantezlerin ve işleç önceliklerinin matematiksel ve mantıksal sonuçları nasıl doğrudan manipüle ettiği simüle edilmiştir:

```
# 1. Senaryo: Matematiksel Öncelik Analizi
# Tabloya göre çarpma (*) ve bölme (/), toplama (+) ve çıkarmadan (-) üstündür.
sonuc1 = 10 + 4 * 2 - 6 / 2
# İşleyiş: 10 + (4 * 2) - (6 / 2) -> 10 + 8 - 3.0 = 15.0
print(f"Standart Öncelikli Hesaplama Sonucu: {sonuc1}")

# Parantez kullanarak öncelik sırasını tamamen değiştiriyoruz:
sonuc2 = (10 + 4) * (2 - 6) / 2
# İşleyiş: (14) * (-4) / 2 -> -56 / 2 = -28.0
print(f"Parantezli Hesaplama Sonucu: {sonuc2}")

# 2. Senaryo: Mantıksal Öncelik Analizi
# Tabloya göre ilişkisel işleçler (<), mantıksal işleçlerden (and, or) üstündür.
# Ayrıca 'and' işleci 'or' işlecinden daha yüksek önceliğe sahiptir.
```



```
kontrol = True or False and 5 < 2
# İşleyiş: True or (False and (5 < 2)) -> True or (False and False) -> True or False -> True
print(f"Karmaşık Mantıksal Önerme Sonucu: {kontrol}")
```

Birleşme Yönü (Associativity) Nedir?

Eğer bir satırda **aynı öncelik sırasına sahip** birden fazla işlem yan yana gelirse ne olur? Bu durumda devreye **Birleşme Yönü** kuralı girer.

- ◊ **Soldan Sağa Birleşme:** Python'daki işlemlerin neredeyse tamamı soldan sağa doğru çözülür. Örneğin $10 - 5 + 2$ ifadesinde $-$ ve $+$ aynı önceliktedir; işlem soldan başlar: Önce $10 - 5 = 5$, ardından $5 + 2 = 7$ hesaplanır.
- ◊ **Sağdan Sola Birleşme (İstisna):** Üs alma ($**$) işlemi bu kuralın istisnasıdır; sağdan sola doğru işler. Örneğin $2**3**2$ ifadesinde önce $3**2 = 9$ hesaplanır, ardından $2**9 = 512$ sonucu üretilir.

4.6 Kayan Noktalı Sayılarda Hassasiyet ve Yaklaşıklık Problemleri

Kayan noktalı sayıların (`float`, `double`) bilgisayar mimarisinde küresel **IEEE-754 standardına** uygun olarak ikili (**binary**) tabanda saklandığı Bölüm 2.4'de incelenmişti. Geliştiriciler olarak kod yazarken kaynak koda onluk (**decimal**) tabanda yazdığımız kesirli sayılar, işlemci ve bellek katmanında zorunlu olarak ikili sisteme dönüştürülür. Bu donanımsal zorunluluk, matematiksel teorik beklentiler ile pratik yazılım çıktıları arasında "garip" ve tehlikeli uyumsuzlukların doğmasına yol açar.

Yazılım dünyasında acemi programcıların sıklıkla düştüğü ilk büyük hata, kayan noktalı sayıları döngü kontrol mekanizmalarında katı bir eşitlik şartıyla sınamaktır. Aşağıdaki C kod blokunu inceleyelim:

```
// C Programlama Dili Örneği
float total = 0.0f;
for(float a = 0.0f; a != 2.0f; a += 0.01f) {
    total += a;
}
```

Dışarıdan bakan bir göz, `a` değişkeninin `0.0`, `0.01`, `0.02` şeklinde düzenli olarak artacağını, tam `2.0` olduğunda döngünün güvenli sonlanacağını ve matematiksel olarak `199.0` teorik sonucuna ulaşacağını varsayar. Fakat bu kod çalıştırıldığında iki büyük kırılma yaşanır:

1. **Sonsuz Döngü Riski:** Program asla sonlanmaz. Çünkü `0.01` sayısı ikili sistemde tam olarak ifade edilemediğinden, ardışık toplamalar hiçbir zaman tam olarak kesintisiz bir `2.0` tam sayısına denk düşmez. `a` değişkeni `2.0` sınırını örtük bir küsurat farkıyla (örn: `2.000001`) aşar ve döngü sonsuza kadar döner.
2. **Birikimli Hata (Accumulation Error):** Eğer döngü şartı güvenli hale getirilip `a < 2.0f` şeklinde revize edilirse döngü durur; ancak elde edilen `total` değeri mikro sapmaların üst üste eklenmesinden dolayı tam olarak `199.0` değil, sapmalı bir reel sayı olur.

§4.6.1 İkili Sistemde Sonsuz Devretme Paradoksu

Bu durumun temel nedeni; onluk sayı sisteminde tam olarak ifade edilebilen bazı kesirli sayıların, ikili sayı sisteminde rasyonel karşılığının bulunmamasıdır. Tıpkı onluk sistemde $1/3$ rasyonel sayısının `0.333333...` şeklinde sonsuza kadar devretmesi gibi; onluk sistemdeki `0.1`, `0.2` ve `0.3` sayıları da ikili sistemde sonsuza kadar devreden kusurlu yapılar üretir. Bilgisayar belleği sınırlı olduğundan (64-bit), bu sonsuz sayı bir noktada budanır (**truncation**) ve sayının "yaklaşık" bir kopyası saklanır. Aşağıdaki klasik mantıksal sınama kodu, bu donanımsal gerçeği açıkça gözler önüne sermektedir:

```
#include <stdio.h>
int main() {
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;
    if(a + b == c) {
        printf("Matematiksel olarak esit: 0.3 == 0.3\n");
    } else {
        printf("IEEE-754 standardı gereği sapma oluştu!\n");
        // Ekrana basılacak gerçek değerleri görelim:
        printf("a + b = %.17f\n", a + b);
        printf("c      = %.17f\n", c);
    }
    return 0;
}
```

```

}
/* Program Çıktısı:
IEEE-754 standardı gereği sapma oluştu!
a + b = 0.29999999999999999
c      = 0.30000000000000000
*/

```

64-bitlik bir mimaride donanımın arka planda gördüğü ve sakladığı ham ikili (**binary**) bit haritalarının onluk sistemdeki tam karşılıkları şu şekildedir:

0.1 → Bellekteki Karşılığı: 0.10000000000000000555111512312578...

0.2 → Bellekteki Karşılığı: 0.20000000000000001110223024625156...

0.3 → Bellekteki Karşılığı: 0.29999999999999998889776975374843...

Görüldüğü üzere 0.1 ile 0.2 değerlerinin bellek adreslerindeki yaklaşık kopyaları toplandığında ortaya çıkan sonuç 0.30000000000000004 olurken, doğrudan tanımlanan c değişkeni 0.29999999999999998 seviyesinde kalmaktadır. Bu iki değer donanımsal bit seviyesinde birbirine eşit değildir.

§4.6.2 Python Perspektifi ve Kesin Çözüm: decimal Modülü

Aynı hassasiyet problemi Python dili için de tamamen geçerlidir. Python'da `0.1 + 0.2 == 0.3` sorgusu yaptığınızda sistem geriye `False` cevabını döndürür. Python, özellikle finans, bankacılık ve muhasebe gibi tek bir kuruşun bile kaybolmaması gereken iş kollarında bu IEEE-754 zaafiyetini aşmak için dil çekirdeğinde yerleşik `decimal` modülünü sunar.

`decimal.Decimal` sınıfı, sayıları ikili tabanda değil, tıpkı insanların yaptığı gibi onluk tabanda donanımdan bağımsız olarak simüle eder ve kusursuz hesaplama imkanı sağlar:

```

from decimal import Decimal

# Standart kayan noktalı sayı problemi
print(f"Standart Python (0.1 + 0.2): {0.1 + 0.2}")
# Çıktı: 0.30000000000000004

# Mühendislik ve Finansal Çözüm: Decimal Kullanımı
# NOT: Sayısal kayıp yaşamamak için değerler metin (str) olarak parametre verilir!
a = Decimal('0.1')
b = Decimal('0.2')
c = Decimal('0.3')

print(f"Decimal ile Güvenli Toplam : {a + b}")
print(f"Eşitlik Kontrolü (is_equal) : {a + b == c}") # True

```

4.7 Düşün ve Çöz

- Mantıksal Kısa Devre ve Nesne Dönüşü:** Python'da `x = "" or [] or 0 or "Python" or 42` ifadesi çalıştırıldığında `x` değişkeninin değeri ve veri türü ne olur? Kısa devre (**short-circuit**) mekanizmasını dikkate alarak açıklayınız.
- Bit Düzeyi İşlemler ve Matematiksel Karşılık:** `a = 18` tam sayısı veriliyor. `(a << 3) >> 1` işleminin sonucunu bit kaydırma mantığı ile hesaplayınız ve bu işlemin matematiksel olarak hangi çarpma/bölme kombinasyonuna denk geldiğini belirtiniz.
- Referans ve Değer Eşitliği:** Aşağıdaki kod bloğunun ekran çıktısını tahmin ediniz ve nedenini `id()` fonksiyonu açısından açıklayınız:

```

x = [1, 2, 3]
y = [1, 2, 3]
z = x
print(x == y, x is y, x is z)

```
- Kayan Noktalı Sayılarda Hassasiyet:** Bir muhasebe yazılımında kullanıcıdan alınan 0.1 ve 0.2 değerleri toplanarak bir borç bakiyesine eklenmektedir. Bu işlemin `float` türü yerine neden `decimal.Decimal` türü ile yapılması gerektiğini IEEE-754 standardı çerçevesinde tartışınız.

5. Bölüm

Kontrol İşlemleri

5.1 Kontrol İşlemi Nedir?

Yapısal Programlama başlığı altında programın ana fonksiyondan başlayacağı ve programlamanın ise birbirini çağıran fonksiyonlarla yapıldığı anlatılmıştı. Yapısal programlamada, ana fonksiyon dahil tüm fonksiyonlarda ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır.

Şu ana kadar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş-çıkış işlemleri talimatları içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem** (**sequential operation**) adı verilir ve Bölüm 4.1 başlığında anlatılmıştır.

Kontrol işlemleri (**control operation**) ise programın icra sırasını yani **kontrol akışını** (**control flow**) değiştiren kontrol yapılarıdır.

Kontrol İşlemleri

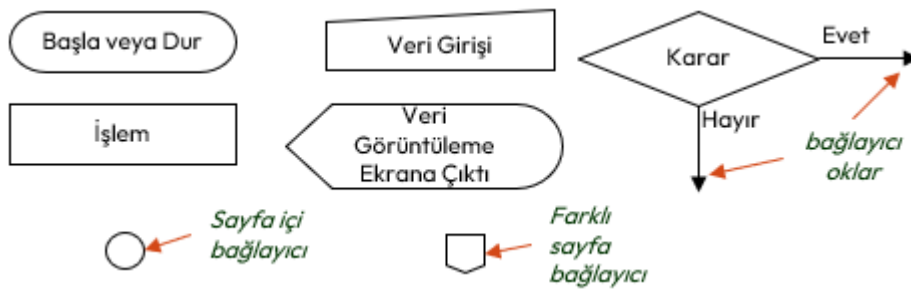
1. **Duruma göre seçimler** (**conditional choice**): **if**, **if-else** ve **match** talimatları.
2. **İlişkisel döngü** (**relational loop**): **for**, **for-else** ve **while** talimatları.
3. **Dallanmalar** (**jump**): **continue**, **break**, **pass** ve **return** talimatları.

Mantıksal ve Fiziksel Satırlar

Kontrol işlemlerinde; **kodlanan mantıksal satırlar** (**logical sequence**) ile **fiziksel olarak icra edilen satırlar** (**physical sequence**) birbirinden farklıdır.

5.2 Akış Diyagramları

Kontrol akışını (**control flow**) değiştiren kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.



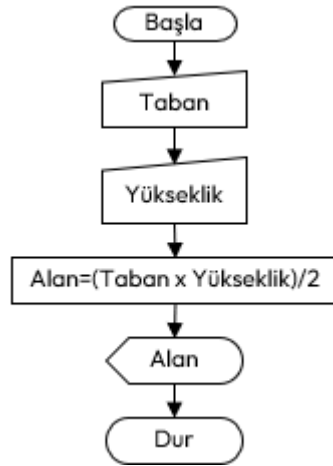
Şekil 5.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (**flowchart**), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- ◊ Akış şemalarında tek bir başlangıç simgesi olmalıdır.
- ◊ Bitiş simgesi birden çok olabilir.



Şekil 5.2: Dik Üçgenin Alan Hesabını Gösteren Akış Diyagramı

- ◊ Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- ◊ Bağlaç simgesi sayfanın dolmasından ötürü parçalanmış akış şemasının öğelerini birleştirmede kullanılır.
- ◊ Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- ◊ Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

5.3 Duruma Göre Seçimler

Duruma göre seçimler (**conditional choice**) programın akışını bir koşula göre dallandıran talimatlardır.

§5.3.1 If Talimatı

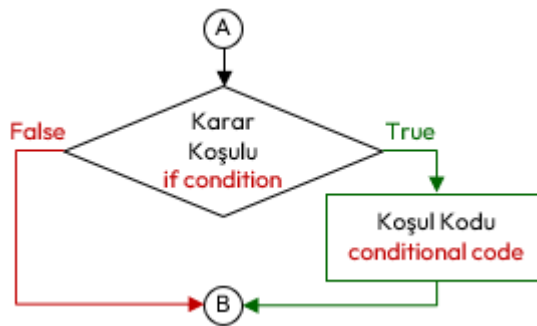
If talimatı, karar vermeye (**decision making**) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. Bu talimatta ilişkisel işlemler içeren **karar koşulu** (**condition**) ifadesi (**expression**) test edilir. Sonuç **True** ise kod bloğu olarak **girintili** yazılan **koşul kodu** (**conditional code**) icra edilir, değilse icra edilmez. Koşul kodu bloğu birden fazla talimattan oluşabilir.

If talimatının sözde kodu aşağıda verilmiştir;

if (karar-kosulu):

koşul-kodu #koşul-kodu, blok içinde yani girintili yazılır!

If talimatının akış diyagramı aşağıdaki gibidir;



Şekil 5.3: If Talimatı İcra Akışı

Aşağıdaki örnek programı inceleyelim;

```

1 yas=18
2 cinsiyet='K'
3 if yas<30:
4     print("Genç") # tek talimatlık bir kod bloğu.
5 if cinsiyet=='E':
6     print("Cinsiyet:") # iki talimatlık kod bloğu
7     print("Erkek") # Çünkü iki talimat girintili yazılmıştır
8 print("Son") #girinti sonlandığı için kod bloğu dışına çıktık.
  
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. Birinci ve ikinci satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
2. Üçüncü satırda bir `if` talimatı bulunmaktadır. **Karar koşulu** (`condition`), `yas<30` ifadesiyle test edilecektir. Test sonucu `True` olduğundan dördüncü satırda girintili yazılan kod bloğundaki **koşul kodu** (`conditional code`) talimatı **icra edilir**. Yani `print("Genç")` talimatı ile konsola "Genç" yazılır.
3. Beşinci satırda da bir `if` talimatı bulunmaktadır. Karar koşulu, `cinsiyet=='E'` ifadesiyle test edilecektir. Test sonucu `False` olduğundan altıncı ve yedinci satırda girintili yazılan kod bloğundaki koşul kodu talimatları **icra edilmez**.
4. Son olarak sekizinci satırdaki `print("Son")` ifadesi icra edilerek konsola "Son" yazılır.

If talimatında koşul kodu (`conditional code`) her zaman ardışık işlem (`sequential operation`) olmaz. Bazen `if` gibi bir başka kontrol işlemi (`control operation`) de olabilir. Aşağıda kademeli (`compound/cascade`) yani iç içe (`nested if`) kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının koşul kodudur;

```
yas=4
if (yas<30):
    if (yas<7): # Buradaki if, üsteki if (yas<30): talimatının koşul kodu içinde yer alır.
        print("Çocuk") #İkinci girintiye (blok içinde blok) dikkat edelim.
```

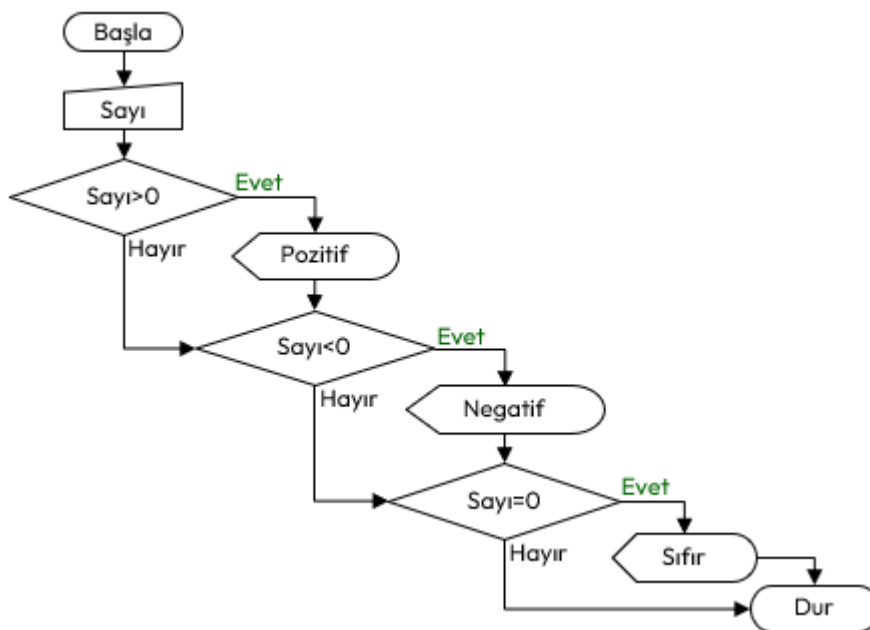
If talimatında koşul (`condition`) her zaman tek bir test ifadesinden (`expression`) oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
yas=15
cinsiyet='K'
if ((cinsiyet=='E') and (yas<18)): # hem Erkek, hem de Genç
    print("Genç Delikanlı")
if ((cinsiyet=='E') and (yas>60)): # hem Yaşlı, hem de Erkek
    print("Yaşlı Adam")
if ((cinsiyet=='K') and (yas<18)): # hem Kadın, hem de Genç
    print("Genç Kız")
if ((cinsiyet=='K') and (yas>60)): # hem Yaşlı, hem de Kadın
    print("Yaşlı Kadın")
```

Bazen programcı tarafından aşağıdaki gibi if talimatları yazabilir.

```
if (True):
    print("Bu metin konsola her zaman yazılır.")
if (False):
    print("Bu metin konsola hiçbir zaman yazılmaz!")
```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir programı göz önüne alalım. Bu talimatla programın akış diyagramı Şekil 5.4'deki gibi olacaktır.



Şekil 5.4: Bir Sayının İşaretini Bulan Akış Diyagramı

Bu akış diyagramına ilişkin program aşağıdaki şekilde yazılır;

```
sayistr = input("Sayı Giriniz: ")
```

```

sayi=int(sayistr)
# 1. Koşul: Sayının pozitiflik kontrolü yapılır.
if sayi > 0:
    print("Pozitif") # Şart doğruysa bu blok çalışır.
# 2. Koşul: Sayi < 0 testi yapılır.
if sayi < 0:
    print("Negatif") # Şart doğruysa bu blok çalışır.
# 3. Koşul: Sayi == 0 testi yapılır.
if sayi == 0:
    print("Sıfır")

```

§5.3.2 If-Else Talimatı

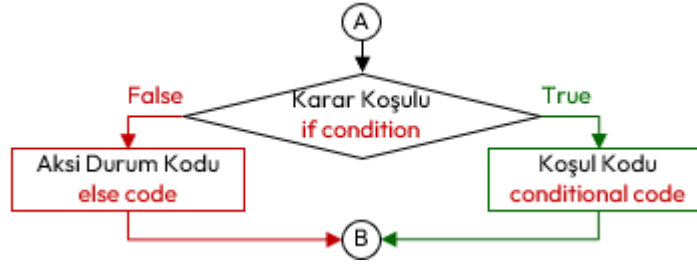
If-Else Talimatı, bir başka karar verme (**decision making**) talimatı olup bir koşula bağlı olarak programın icrası dallanır. Sözde kodu aşağıda verilmiştir;

```

if (karar-kosulu):
    koşul-kodu #koşul-kodu, blok içinde yani girintili yazılır!
else: # else kelimesi if ile aynı hizada/sütunda yer alır
    aksi-durum-kodu #aksi-durum-kodu, yine blok içinde yani girintili yazılır!

```

If-Else talimatında, **karar koşulu** (**condition**) ifadesi **True** ise **koşul kodu** (**conditional code**) icra edilir, karar koşulu **False** ise **aksi durum kodu** (**else code**) icra edilir.



Şekil 5.5: If-Else Talimatı İcra Akışı

Aşağıda verilen örnek programı inceleyelim;

```

1 yas=65
2 if (yas<50):
3     print("Durumu: ")
4     print("Genç")
5 else:
6     print("Durumu: ")
7     print("Yaşlı")
8 print("Son")

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

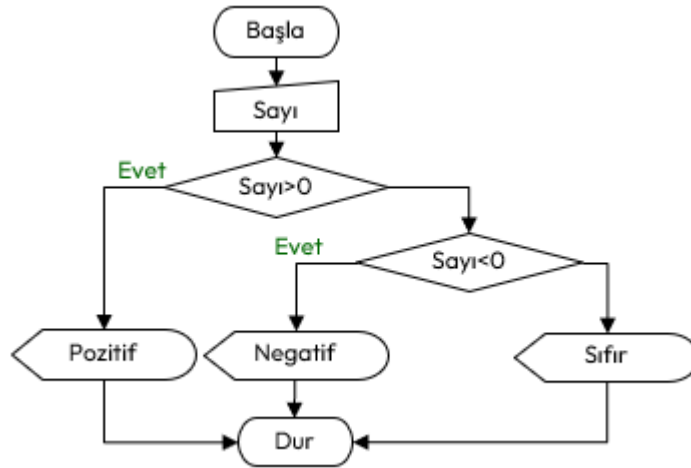
1. Birinci satırda **yas** değişki başlatılmıştır.
2. İkinci satırda bir **if** talimatı bulunmaktadır. **Koşul kodu** (**condition**), **yas<50** ifadesiyle test edilecektir. Test sonucu **False** olduğundan üçüncü ve dördüncü satırdaki kod bloğundan oluşan **koşul kodu** (**conditional code**) talimatları **icra edilmez**. Bunun yerine **else** sonrası altıncı ve yedinci satırdaki kod bloğu olan **aksi durum kodu** (**else code**) talimatları **icra edilir**. Yani konsola "Durumu: Yaşlı" yazılır.
3. Son olarak sekizinci satırdaki ifadesi icra edilerek konsola "Son" yazılır.

If talimatı başlığında bir sayının işaretini bulan programının icra akışı (Şekil 5.4) incelendiğinde ilk **if** talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılabacaktır. Benzer şekilde ilk iki test geçilirse üçüncü **if** talimatındaki teste gerek yoktur. Bu durumu aşağıdaki akış diyagramında daha net görebilir ve aynı program **If-else** talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```

1 sayi = int(input("Sayı Giriniz: "))
2 if sayi > 0: # 1. Koşul: Sayının pozitiflik kontrolü
3     print("Pozitif")
4 else:
5     if sayi < 0: # 2. Koşul: İlk şart sağlanmadıysa burası test edilir.
6         print("Negatif")
7     else: # 3. Koşul: Yukarıdaki hiçbir şart sağlanmadıysa geriye tek bir ihtimal kalır.
8         print("Sıfır")

```

Şekil 5.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `sayı` pozitif girilirse;
 - İkinci satırdaki if karar koşulunda `sayı>0` test edilip `True` çıkacak ve sadece üçüncü satırdaki koşul kodu olan `print("Pozitif")` talimatı icra edilecektir. Beşinci satırdan başlayan aksi durum kodu icra edilmeyecektir.
- ◊ Eğer `sayı` negatif girilirse;
 - İkinci satırdaki if karar koşulunda `sayı>0` test edilip `False` çıkacak ve beşinci satırdan başlayan aksi durum kodu icra edilecektir.
 - Bu blok içinde beşinci satırdaki if karar koşulu `sayı<0` test edilip `True` çıkacak ve altıncı satırdaki koşul kodu olan `print("Negatif")` talimatı icra edilecektir. Yedinci satırdan başlayan aksi durum kodu icra edilmeyecektir.
- ◊ Eğer `sayı` sıfır girilirse;
 - Altıncı satırdaki if karar koşulunda `sayı>0` test edilip `False` çıkacak ve beşinci satırdan başlayan aksi durum kodu icra edilecektir.
 - Bu blok içinde beşinci satırdaki if karar koşulu `sayı<0` test edilip `False` çıkacak ve sekizinci satırdaki aksi durum kodu olan `print("Sıfır")` icra edilecektir.

Ayrıca örnekteki kod incelendiğinde her bir `if` ve `else` sonrasında bir girinti verilmektedir. İç içe yapılan bu kontrollerde girinti sayısı artmaktadır.

If Talimatında Birden Fazla Test

Eğer aynı değişkene ilişkin birden fazla test yapılacaksa `elif` kullanılır.

Bu durumda bir sayının işaretini bulan programın nihai hali aşağıda verilmiştir.

```

sayı=int(input("Bir Tamsayı Giriniz:"))
if sayı>0: #sayıya ilişkin 1. durum test ediliyor
    print("Pozitif")
elif sayı<0: #sayıya ilişkin 2. durum test ediliyor
    print("Negatif")
else: #sayıya ilişkin geri kalan durumlar
    print("Sıfır")
  
```

Buna ilişkin bir başka örnek rakam olarak verilen notu harf notuna çeviren program verilebilir;

```

sayı=int(input("Not Giriniz (0-100):"))
if (sayı>90) and (sayı<=100): #nota ilişkin 1. durum test ediliyor
    print("AA")
elif sayı>80: #nota ilişkin 2. durum test ediliyor
    print("BA")
elif sayı>70: #nota ilişkin 3. durum test ediliyor
    print("BB")
elif sayı>60: #nota ilişkin 4. durum test ediliyor
    print("CB")
elif sayı>50: #nota ilişkin 5. durum test ediliyor
  
```

```

print("CC")
elif sayı>40: #nota ilişkin 6. durum test ediliyor
    print("DC")
elif sayı>30: #nota ilişkin 7. durum test ediliyor
    print("DD")
elif sayı>20: #nota ilişkin 8. durum test ediliyor
    print("FD")
elif sayı>0: #nota ilişkin 9. durum test ediliyor
    print("FF")
else: #sayıya ilişkin geri kalan durumlar
    print("--")

```

§5.3.3 Match Talimatı

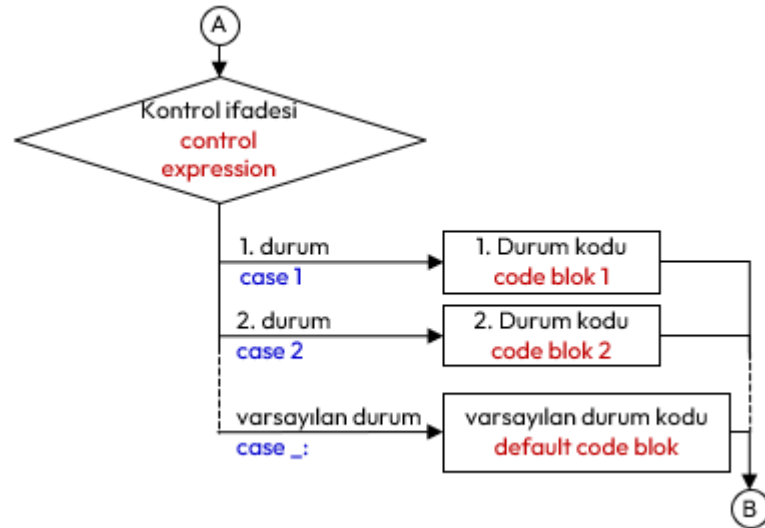
Seçme (match) talimatı, bir kontrol ifadesi (**expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Alternatifler, kontrol ifadesini seçen **desenlerdir (pattern)**. Sözde kodu aşağıda verilmiştir;

match kontrolifadesi:

```

case alternatif-1:
    alternatif-1-kodu
case alternatif-2:
    alternatif-2-kodu
# ...
case _:
    varsayılan-alternatif-kodu

```



Şekil 5.7: Match Talimatı İcra Akışı

Seçim talimatı, bir ifadeyi alır ve değerini bir veya daha fazla durum bloğu olarak verilen ardışık desenlerle karşılaştırır. İfadeye uyan desendeki durum kodu icra edilir.

Match Talimatına İlişkin Kurallar

1. Kontrol ifadesi (control expression), sonucu alternatifleri içeren ifadedir ve bir kez değerlendirilir.
2. Bloğu olmayan bir **match** talimatı yazılamaz!
3. Tüm alternatifleri içerecek şekilde alternatiflerin tümü yazılmayabilir.
4. Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için **_:** etiketli alternatif yazılabilir.
5. Bu talimat, Python 3.10 ve üzeri sürümlerde kullanılabilir. Eski sürümlerde **if-elif-else** kullanılır!

Aşağıda örnek olarak gün seçimi için yazılmış bir program örneği verilmiştir;

```

gun=input("Gün Giriniz:")
match gun:
    case "Pazartesi" | "Salı" | "Çarşamba" | "Perşembe": #birden fazla seçim içeren desen
        print(f"Girdiğiniz Gün: {gun}, Çalışma Günüdür.")
    case "Cuma": #tek gün seçimine ilişkin desen

```

```

    print("Mübarek Gün.")
    case "Cumartesi": #tek gün seçimine ilişkin desen
        print("Yarım Gün Çalışma.")
    case "Pazar": print("Tatil.")
    case _: print("Geçerli bir gün adı Girmediniz!") #gün için yukarıdaki desenlere uymayanlar

```

Seçim talimatı sadece ifadeyi herhangi bir sabitle eşleştirmez. Bir listeyi durum değeri olarak kullanılabılır.

```

liste=["İyi Çalışmalar", "İlhan"] #iki elemanlı bir liste
match liste:
    case [selamlama,isim]: # her bir eleman bir değişkenile işlenebilir.
        print(selamlama,"!",isim)

hesaplistesi=[True,2000] # iki elemanlı liste: ilk elemam haftasonu çalışmış mı?
match hesaplistesi:
    case [haftasonumu,yevmiye]:
        if haftasonumu: print("Haftasonu Yevmiyesi:",yevmiye)
        else: print("Hafta İçi Yevmiyesi:", yevmiye*0.8)

hesaplistesi=[False,2000] # iki elemanlı liste: ilk elemam haftasonu çalışmış mı?
match hesaplistesi:
    case [haftasonumu,yevmiye]:
        if haftasonumu: print("Haftasonu yevmiyesi:",yevmiye)
        else: print("Hafta İçi Yevmiyesi:", yevmiye*0.8)

```

Program çıktısı aşağıdaki gibi olacaktır.

```

İyi Çalışmalar ! İlhan
Haftasonu Yevmiyesi: 2000
Hafta İçi Yevmiyesi: 1600.0

```

Ayrıca bu talimat, ifadede yer alan listedeki değişken sayıda öge için, yıldız (*) operatörüyle bir diziye ayrıştırılabilir.

```

liste=["İlhan","Pazartesi","Çarşamba","Perşembe","Cuma","Cumartesi"] # 6 elemanlı liste
match liste:
    case [isim,*günler]: #ikiden fazla elemanlı liste girilirse ilk elemandan sonrası diziye
        ↪ ayrıştırılıyor.
        print(isim," Şu günler Çalıştı:")
        for gün in günler:
            print(gün,end=",")

```

Program çıktısı aşağıdaki gibi olacaktır.

```

İlhan  Şu günler Çalıştı:
Pazartesi,Çarşamba,Perşembe,Cuma,Cumartesi,

```

Bir başka örnek;

```

liste=["İlhan","Pazartesi"]
match liste:
    case [isim,gün]: # Eğer iki elemanlı bir liste girilirse
        print(isim,"", gün, " Günü çalıştı")
    case [isim,*günler]: # İkiden fazla elemanlı liste girilirse
        print(isim," Şu günler Çalıştı:")
        for gün in günler:
            print(gün,"")

```

Program çıktısı aşağıdaki gibi olacaktır.

```

İlhan , Pazartesi  Günü çalıştı

```

5.4 İlişkisel Döngüler

Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir. Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım.

```

print("İLHAN") #1

```

```
print("ILHAN") #2
print("ILHAN") #3
print("ILHAN") #4
print("ILHAN") #5
print("ILHAN") #6
print("ILHAN") #7
print("ILHAN") #8
print("ILHAN") #9
print("ILHAN") #10
```

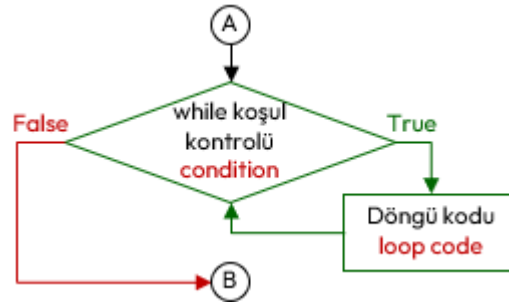
Program incelendiğinde aynı `print` talimatının tekrar tekrar yazıldığını görürüz. **Bu istenmeyen bir durumdur.**

Döngüler (loop) bir ifadeyi veya ifadeler grubunu birden çok kez yürütmemize olanak tanır. Bunu yapmak için tekrarlanacak kodun başına gitmek için eskiden `GOTO` talimatı kullanılırdı. Bu talimatın kullanılması kodun okunaklılığını azaltan bir şeydir. `GOTO` talimatı, 1968 Yılında Edsger W. Dijkstra `GOTO/JUMPTO` ifadelerini zararlı olarak ilan edilmiştir¹. `GOTO` kullanmamak için; talimatların icra sırasını yani Kontrol akışını (**control flow**) daha kolay anlamak için **ilişkisel döngü** (**relational loop**) talimatlarını kullanırız. İlişkisel döngü talimatlarının kullanıldığı yapısal programlamayı Python dili de destekler. Bunun için `for`, `for-else` ve `while` gibi **kontrol talimatları** (**control statement**) yapısal programlamaya eklenmiştir.

§5.4.1 While Talimatı

While talimatında (**while statement**), koşul (**condition**) kontrolü **True** olduğu sürece döngü kodu (**loop code**) icra edilir. Koşul herhangi bir ifade olabilir ve **True** sıfır olmayan herhangi bir değerdir. İfade **False** olmaz, program kontrolü döngü dışındaki izleyen talimatla devam eder. Sözde kodu aşağıda verilmiştir;

while koşul-ifadesi:
 döngü-kodu



Şekil 5.8: While Talimatı İcra Akışı

Bu durumda konsola 10 defa ismimizi yazdıran bir program örneğini, `while` talimatı ile aşağıdaki şekilde yazılabilir;

```
1 sayac=1
2 while sayac<=10:
3     print("ILHAN")
4     sayac+=1
```

Burada `sayac` değişkenine döngüye girmeden bir ilk değer verilmiştir. Her döngüde `sayac` değişkenine bağlı olarak koşul test edilir. Döngü içerisinde `sayac` değişkeni güncellenir. Bu tip döngülere **sayac kontrollü döngü** adı verilir.

Burada döngü bloğu iki adet mantıksal satır (**logical sequence**) içermesine rağmen 20 adet fiziksel satır (**physical sequence**) icra edildiğine dikkat edilmelidir.

§5.4.2 For Talimatı

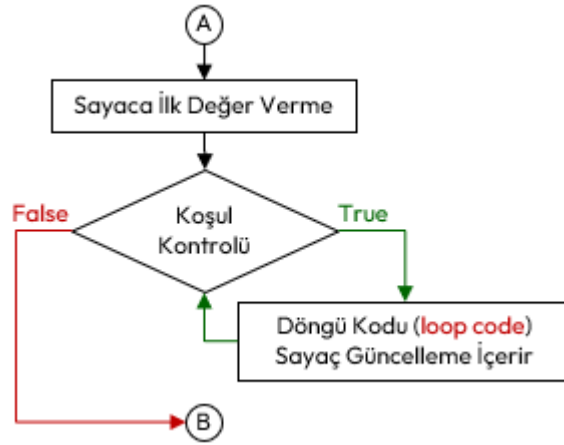
Python dilindeki `for` döngüsü, bir liste, demet (tuple) veya bir dizgi (string) gibi herhangi bir dizinin elemanları üzerinde döngü yapma yeteneği sağlar. Dizinin her elemanına aynı eylemi gerçekleştirir.

`for` yineleme-elemanı-değişkeni `in` yilenebilir-yapı:
 döngü-kodu

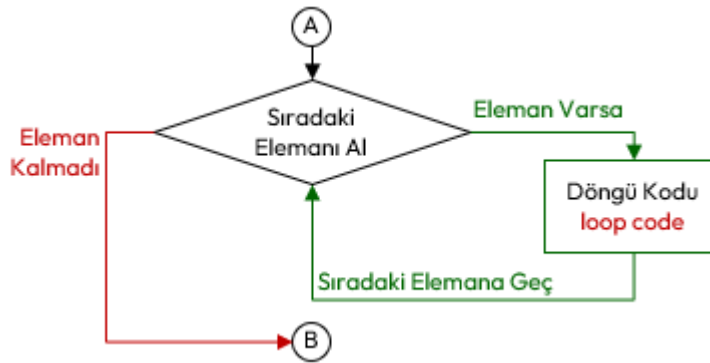
Örnek program aşağıda verilmiştir;

```
metin="Merhaba!"
```

¹<https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>



Şekil 5.9: Sayaç Kontrollü Döngü İcra Akışı



Şekil 5.10: For Talimatı İcra Akışı

```

for karakter in metin:
    print(karakter)
  
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```

M
e
r
h
a
b
a
!
  
```

Bir liste üzerinde for döngüsüne örnek;

```

satışlar=[10000,12000,10000,5000] #liste üzerinde for döngüsü
toplam=0
for satış in satışlar:
    toplam +=satış
print("Satış Toplamı",toplam)
  
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

Satış Toplamı 37000

Tek sayılardan oluşan demet (tuple) üzerine for döngüsü;

```

ondanküçükteksayılar=(1,3,5,7,9)
for teksayı in ondanküçükteksayılar:
    print(teksayı,end="-")
  
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

1-3-5-7-9-

Sözlük üzerinde for döngüsü;

```
sözlük={10:"On", 20:"Yirmi", 30:"Otuz"}
for eleman in sözlük:
    print(eleman)
for eleman in sözlük:
    print(eleman,"-", sözlük[eleman])
for eleman in sözlük.items():
    print(eleman)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
10
20
30
10 - On
20 - Yirmi
30 - Otuz
(10, 'On')
(20, 'Yirmi')
(30, 'Otuz')
```

§5.4.3 Aralık Nesneleri

Dahili `range()` fonksiyonu, bir tamsayı dizisi oluşturarak bu dizi üzerinde bir **yineleyici** (**iterable**) nesnesi döndürür. Bu aralık (**range**) nesnesi, adım parametresiyle ayrılmış, başlangıçtan bitişe kadar tam sayılar içerir. Sözde kodu aşağıda verilmiştir;

`range (başlangıç, bitiş, adım)`

Burada **başlangıç**, aralığın başlangıç değeri. İsteğe bağlı olup varsayılan değeri sıfırdır. **bitiş** değerini almayacak son elemana kadar dizi oluşturulur. **adım** kadar dizi elemanlarının arasındaki değer farkıdır ve varsayılan değer 1'dir. Aşağıda örnek program verilmiştir;

```
aralık=range(4)
for sayı in aralık:
    print(sayı,end=',') # end-ayraç (separator) anlamındadır
#Çıktı: 0,1,2,3,
aralık=range(1,5)
for sayı in aralık:
    print(sayı,end='-') # end-ayraç (separator) anlamındadır
#Çıktı: 1-2-3-4-
aralık=range(10,20,2)
for sayı in aralık:
    print(sayı,end=':') # end-ayraç (separator) anlamındadır
#Çıktı: 10:12:14:16:18:
```

Bu durumda konsola 10 defa ismimizi yazdıran bir program örneğini, **for** talimatı ile aşağıdaki şekilde yazılabilir;

```
for sayı in range(10):
    print("ILHAN")
```

§5.4.4 For-Else Talimatı

C programlama dilinde bir dizi içerisinde belirli bir elemanı ararken, elemanın bulunup bulunmadığını döngü dışından anlamak için geleneksel olarak yardımcı bir "bayrak" (**flag**) değişkeni tanımlarız (Örn: `int bulundu = 0;`). Eğer eleman bulunursa bu değişkeni 1 yapar ve döngüden çıkarız. Döngü bittiğinde ise bu bayrak değişkenini kontrol ederiz.

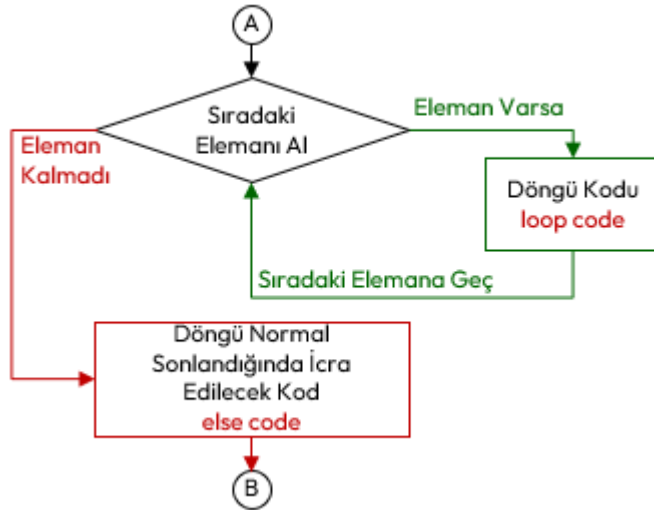
Python dil mimarisi, bu kalıplaşmış ve hata yapmaya açık yapıyı ortadan kaldırmak için **for-else** söz dizimini sunar.

for değişken in yinelenenebilir-yapı:

#tekrarlanacak talimatlar

else:

#döngü normal bir şekilde sonlandığında icra edilecek talimatlar



Şekil 5.11: For-Else Talimatı İcra Akışı

KURAL: else Ne Zaman Çalışır?

`for` döngüsünün altındaki `else` bloku, döngü doğal yollarla tamamlandığında (yani listedeki tüm elemanlar taranıp bittiğinde) çalışır. Eğer döngü akışı, gövde içerisindeki ileride göreceğimiz bir `break` ifadesiyle kesintiye uğramışsa (erken çıkış yapılmışsa), `else` bloku **kesinlikle çalıştırılmaz**.

Aşağıda örnek bir program verilmiştir;

```
for sayac in range(5):
    print(f'{sayac}. Adım...')
else:
    print('For Döngüsü Bitti.!')
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
0. Adım...
1. Adım...
2. Adım...
3. Adım...
4. Adım...
For Döngüsü Bitti.!
```

§5.4.5 İç İçe Döngüler

Döngü kodu (**loop code**) bir başka döngüyü içerebilir. İç içe döngülerde (**nested loop**), bir döngü diğerinin içinde tanımlanır. Başka bir döngüyü yani iç döngüyü çevreleyen döngüye dış döngü (**outer loop**) denir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde iç içe döngü kullanırız. Aşağıda 3 satır 10 adet yıldız basan bir örnek verilmiştir. İlk `for` döngüsünün tekrarlanan kodu ikinci `for` döngüsüdür.

```
for satir in range(1,4):
    for sutun in range(1,10):
        print('*',end='')
    print() #alt satıra geçmek için.
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

```
*****
*****
*****
```

Aşağıda elemanları, satır ve sütun indislerinin çarpımı olan bir 4x5 boyutlu bir matris ekrana yazdırılmıştır;

```
for satir in range(1,5):
    for sutun in range(1,6):
        print('{0:<4}'.format(satir*sutun), end=' ')
    print() #alt satıra geçmek için.
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

```
1  2  3  4  5
2  4  6  8  10
3  6  9  12  15
4  8  12  16  20
```

§5.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri (sentinel value)** adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
sayı=-1 # while döngüsüne girmek için -1 olarak ayarlandı. 0 olsaydı girilmezdi!
toplam=0
while sayı!=0: # sentinel value 0
    sayıstr=input('Sayı Giriniz (Bitirmek İçin 0):')
    sayı=int(sayıstr)
    toplam+=sayı
print(toplam)
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

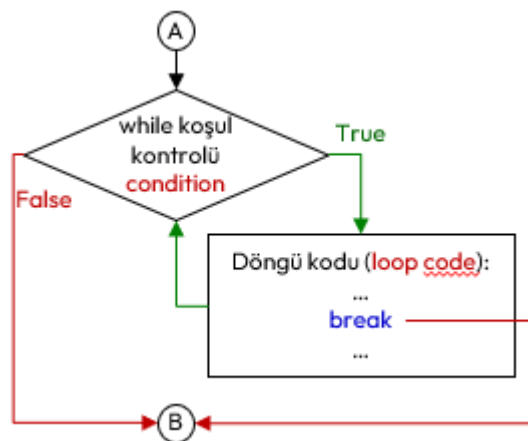
```
Sayı Giriniz (Bitirmek İçin 0):10
Sayı Giriniz (Bitirmek İçin 0):30
Sayı Giriniz (Bitirmek İçin 0):40
Sayı Giriniz (Bitirmek İçin 0):50
Sayı Giriniz (Bitirmek İçin 0):0
130
```

5.5 Dallanmalar

Dallanmalar (jump), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

§5.5.1 Break Talimatı

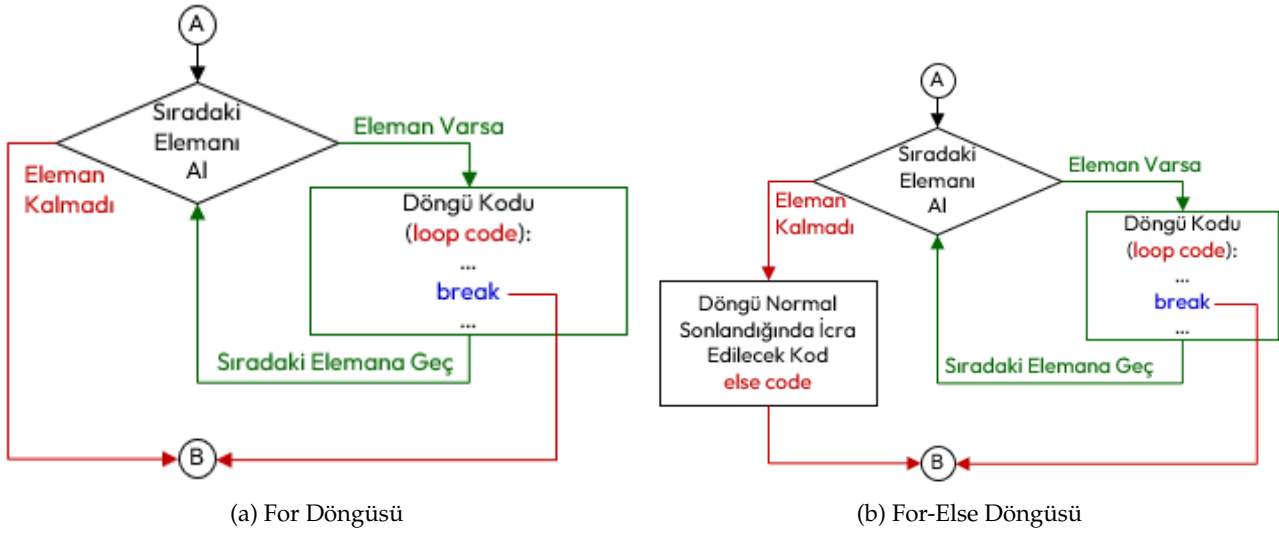
C dilindeki geleneksel **break** talimatı gibi geçerli döngüyü sonlandırarak döngü kodu sonrasındaki ilk talimatı icra etmek için kullanılır. Bu ifade, bir döngüden ani bir çıkış gerektiren bazı koşullarda kullanılır. Hem **while** hem de **for** döngülerinde kullanılabilir. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 5.13'de verilmiştir.



Şekil 5.12: While Döngüsünde Break Talimatı İcra Akışı

Aşağıda bir dizide beşe bölünen ilk elemanı bulan ve döngüyü sonlandıran program örneği verilmiştir.

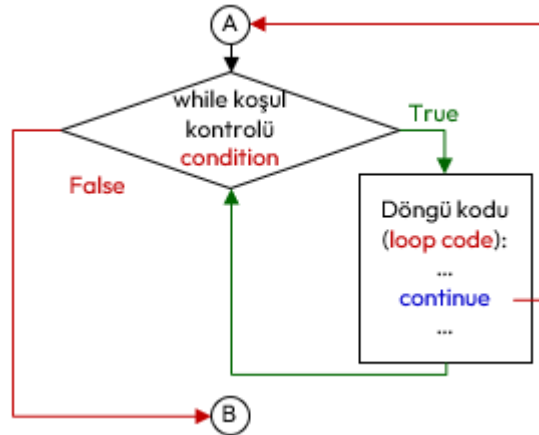
```
dizi=[1,3,6,7,15,31,27]
for eleman in dizi:
    if eleman%5==0:
        print('Dizide 5 e bölünen ilk eleman:',eleman)
        break
```



Şekil 5.13: For Döngülerinde Break Talimatı İcra Akışı

§5.5.2 Continue Talimatı

Döngülerde, döngü kodunun icrasını keserek bir sonraki eleman için döngü kodunun icrasını devam ettirmek amacıyla `continue` talimatı kullanılır. Yani geçerli döngünün başına döndürerek bir sonraki yinelemeye başlamak için kullanılır. Hem `while` hem de `for` döngülerinde kullanılabilir.



Şekil 5.14: While Döngüsünde Continue Talimatı İcra Akışı

Aşağıda bir sayının asal çarpanlarını bulan programın örneği verilmiştir.

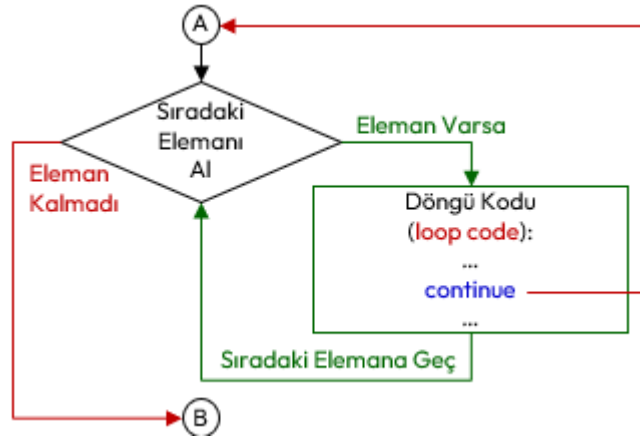
```
sayı=30
print(sayı, ' sayısının asal çarpanları:')
sayaç=2
while sayı>1:
    if sayı%sayaç==0:
        print(sayaç)
        sayı=sayı/sayaç
        continue
    sayaç+=1
```

Örneğin program akışı aşağıdaki gibi olacaktır.

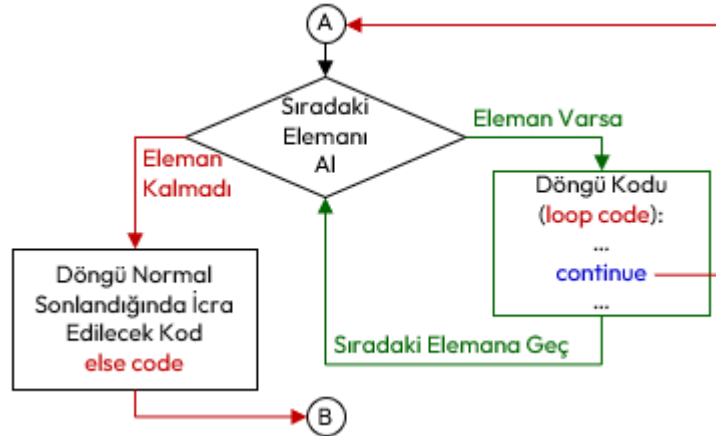
```
30 sayısının asal çarpanları:
2
3
5
```

Bir başka örnek olarak içinde 'h' harfi ayıklanmış bir metin örneği verilebilir;

```
for harf in 'Merhaba':
    if harf=='h':
        continue
```



(a) For Döngüsü



(b) For-Else Döngüsü

Şekil 5.15: For Döngülerinde Continue Talimatı İcra Akışı

```
print(harf)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
M
e
r
a
b
a
```

§5.5.3 Pass Talimatı

Bir talimatın gerekli olduğu ancak icra edilmesi istenmediği durumlarda pass talimatı kullanılır. Yani pass talimatı icra edildiğinde hiçbir şey olmayacağı anlamına gelir. Bu talimat, kod parçasının daha sonra ekleneceği yerlerde de kullanışlıdır.

```
for sayac in range(11):
    if sayac%2==0: #sayac çift ise
        print('{} sayısı çifttir.'.format(sayac))
    else:
        pass # tek olması halinde icra edilecek talimatlar sonra yazılacak.
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
0 sayısı çifttir.
2 sayısı çifttir.
4 sayısı çifttir.
6 sayısı çifttir.
8 sayısı çifttir.
10 sayısı çifttir.
```

Bir sonsuz döngü talimatı aşağıdaki şekilde yazılabilir. Bitirmek için Ctrl+C tuşuna basılmalıdır.

```
while True: pass
```

§5.5.4 Return Talimatı

Bu talimat fonksiyon başlığında incelenecektir.

5.6 Düşün ve Çöz

1. **Döngü Akışı ve else Gövdesi:** Aşağıdaki Python kod bloğu çalıştırıldığında konsola hangi çıktılar basılır? `break` ifadesinin `for-else` yapısı üzerindeki etkisini açıklayınız.

```
for sayi in range(1, 6):
    if sayi % 4 == 0:
        print("Döngü Kırıldı")
        break
    print(sayi)
else:
    print("Döngü Başarıyla Tamamlandı")
```

2. **Akış Diyagramından Koda Dönüştürme ve Gözcü Değeri:** Kullanıcı klavyeden negatif bir sayı girine kadar verilen sayıların karesini alıp konsola yazdıran ve negatif sayı girildiğinde duran bir algoritmanın;
 - a) Akış diyagramını çizin.
 - b) `while` döngüsü ve gözcü değeri (`sentinel value`) kullanarak Python kodunu yazınız.
3. **Match-Case ile Desen Eşleme (Pattern Matching):** Python 3.10 ile gelen `match-case` yapısını kullanarak, kendisine parametre olarak verilen bir listenin ilk elemanına göre işlem yapan bir program yazılmak isteniyor:
 - ◊ Liste tek elemanlı ise: "Tek elemanlı liste: [eleman]"
 - ◊ Liste iki elemanlı ise: "İki elemanlı liste, toplamı: [toplam]"
 - ◊ Diğer tüm durumlarda: "Çok elemanlı veya boş liste"

Bu mantığı sağlayan desen eşleme kodunu yazınız.

4. **Mantıksal Kısa Devre ve Kontrol İfadeleri:** `if (a != 0) and (b / a > 2):` ifadesindeki mantıksal `and` işlecinin kısa devre (`short-circuit`) özelliğinin, programın `ZeroDivisionError` (Sıfıra Bölme Hatası) almasını nasıl engellediğini açıklayınız.

6. Bölüm

Fonksiyonlar

6.1 Fonksiyon Nedir?

Yapısal programlama mantığında çözülecek problem, genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Ayrıca yazılacak programda birbiriyle ilgili fonksiyonlar bir araya toplanarak ayrı bir dosyada tutulur.

Bu bölüme kadar gördüğümüz kodlarda istenen cevabı verecek çözüm, art arda yazılmış ve kodlardan veya tek bir ana fonksiyon olan `main()` fonksiyonu içerisinde çözülmüştür. Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir ana fonksiyondan oluşmaz. Bu durumda ana problemin büyüklüğüne göre ana fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani böl ve yönet (**divide and conquer**) tekniği uygulanır. Çünkü büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur.

Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna yukarıdan aşağıya (**top down**) yaklaşım adı verilir.

Python dili de fonksiyonları destekler. Ayrıca birbiriyle ilişkili fonksiyon, değişken ve nesneler bir modül olarak tasarlanabilir. Bir Python modülü, Python tanımları ve ifadeleri içeren bir dosyadır. Bir modül, fonksiyonları, sınıfları ve değişkenleri tanımlayabilir. Birçok Python modülü vardır¹.

Python dilinde fonksiyonlar;

- ♦ Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- ♦ Parametre alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Kısaca bilgiyi fonksiyona argümanlar ile aktarırız.
- ♦ Her fonksiyonun bir kimliği vardır. Değişken kimliklendirme kuralları fonksiyonlar için de geçerlidir!
- ♦ Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- ♦ Bir fonksiyon, çağıran koda bir değer döndürebilir.
- ♦ Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

İki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan hazır fonksiyonlar ve programcının tanımlayacağı kullanıcı tanımlı fonksiyonlar (**user defined function**).

6.2 Hazır Fonksiyonlar

Programcılarının kullanmaları için, önceden yazılmış `print()` ve `input()` gibi hazır fonksiyonlar Python dilinde **dahili fonksiyon** (**built-in function**) olarak sunulur. Bu fonksiyonlar `__builtins__` modülünde bulunur. Python yorumlayıcısı, her zaman kullanılabilir olan bu modülde, bir dizi yerleşik fonksiyona ve veri tiplerine sahiptir. Bunlar `dir( __builtins__ )` talimatı ile görüntülenebilir .

```
>>> dir (__builtins__)
```

¹<https://docs.python.org/3/library/index.html>


```
[ 'ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup',
→ 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError',
→ 'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError',
→ 'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EncodingWarning',
→ 'EnvironmentError', 'Exception', 'ExceptionGroup', 'False', 'FileExistsError',
→ 'FileNotFoundError', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError',
→ 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
→ 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError',
→ 'ModuleNotFoundError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented',
→ 'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning',
→ 'PermissionError', 'ProcessLookupError', 'RecursionError', 'ReferenceError',
→ 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration',
→ 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError',
→ 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError',
→ 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError',
→ 'Warning', 'ZeroDivisionError', '__build_class__', '__debug__', '__doc__', '__import__',
→ '__loader__', '__name__', '__package__', '__spec__', 'abs', 'aiter', 'all', 'anext', 'any',
→ 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes', 'callable', 'chr', 'classmethod',
→ 'compile', 'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod',
→ 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float', 'format', 'frozenset', 'getattr',
→ 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance',
→ 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min',
→ 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr',
→ 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum',
→ 'super', 'tuple', 'type', 'vars', 'zip']
>>>
```

Bu fonksiyonların bir kısmı anlatılmıştır. Yeri geldikçe diğerleri de anlatılacaktır. Bu fonksiyonlara ilişkin yardım dokümanına `help()` komutu ile aşağıdaki şekilde ulaşılabilir.

```
>>> help()
Welcome to Python 3.12's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.12/tutorial/.
```

Enter the name of **any** module, keyword, **or** topic to get help on writing Python programs **and** using Python modules. To get a **list** of available modules, keywords, symbols, **or** topics, enter **"modules"**, **"keywords"**, **"symbols"**, **or** **"topics"**.

Each module also comes **with** a one-line summary of what it does; to **list** the modules whose name **or** summary contain a given string such **as** **"spam"**, enter **"modules spam"**.

To quit this help utility **and return** to the interpreter, enter **"q"** **or** **"quit"**.

```
help> q
```

```
>>>
```

En çok kullanılan `math` modülünde, yaygın matematik fonksiyonları ve sabitleri bulunur. Her Python programı için dahili olarak tanımlı olan `__builtins__` modülü dışındaki modülleri kodumuza dahil etmek yani aktarmak (`import`) için `import` kelimesi kullanılır.

```
# ÜÇGENİN ALANI - HERON FORMÜLÜ
import math
kenar1str=input('Üçgenin birinci kenarını Giriniz:')
kenar1=float(kenar1str)
kenar2str=input('Üçgenin ikinci kenarını Giriniz:')
kenar2=float(kenar2str)
kenar3str=input('Üçgenin üçüncü kenarını Giriniz:')
kenar3=float(kenar3str)
if (kenar1+kenar2)<=kenar3 or (kenar1+kenar3)<=kenar2 or (kenar2+kenar3)<=kenar1:
```

```

    print('Böyle bir üçgen olamaz!')
else:
    kenarlarToplamınınYarısı=(kenar1+kenar2+kenar3)/2
    alan=math.sqrt( kenarlarToplamınınYarısı * \
                    (kenarlarToplamınınYarısı-kenar1) * \
                    (kenarlarToplamınınYarısı-kenar2) * \
                    (kenarlarToplamınınYarısı-kenar3) )
print('Üçgenin Alanı:',alan)

```

Örneğin program çıktısı aşağıdaki gibi olacaktır;

```

Üçgenin birinci kenarını Giriniz:3.0
Üçgenin ikinci kenarını Giriniz:4.0
Üçgenin üçüncü kenarını Giriniz:5.0
Üçgenin Alanı: 6.0

```

Matematik modülü fonksiyon ve nesneleri aşağıdaki şekilde öğrenilebilir². Benzer şekilde aktarılan (import) edilen her modüldeki fonksiyon ve nesneler benzer şekilde listelenebilir.

```

>>> import math
>>> dir (math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',
→ 'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysign', 'cos', 'cosh',
→ 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm1', 'fabs', 'factorial', 'floor',
→ 'fma', 'fmod', 'frexp', 'fsqrt', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite',
→ 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf',
→ 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh',
→ 'sqrt', 'sumprod', 'tan', 'tanh', 'tau', 'trunc', 'ulp']

```

Aynı modüldeki sabitlerin kullanımına örnek olarak yarıçapı girilen bir dairenin çevre ve alan hesabı verilebilir.

```

import math
yarıçapstr=input('Yarıçapı Giriniz:')
yarıçap=float(yarıçapstr)
print('Dairenin Çevresi:',2*math.pi*yarıçap)
print('Dairenin Alanı:',math.pi*yarıçap*yarıçap)

```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

```

Yarıçapı Giriniz:12.0
Dairenin Çevresi: 75.39822368615503
Dairenin Alanı: 452.3893421169302

```

Yukarıdaki örnekte `math.pi` sabitini kullanmak için tüm `math` modülü aktarılmıştır (import). Eğer bir modülden sadece belirlenen bir üye aktarılacak ise `from` anahtar kelimesi kullanılır. Verilen örnek bu kapsamda aşağıdaki şekilde yazılabilir.

```

from math import pi
yarıçapstr=input('Yarıçapı Giriniz:')
yarıçap=float(yarıçapstr)
print('Dairenin Çevresi:',2*pi*yarıçap)
print('Dairenin Alanı:',pi*yarıçap*yarıçap)

```

6.3 Kullanıcı Tanımlı Fonksiyonlar

Programcılar kendi fonksiyonlarını oluşturarak tekrar tekrar çağırabileceği kodlar yazarak programlarını daha kullanışlı hale getirirler. Bu tür fonksiyonlara **kullanıcı tanımlı fonksiyonlar** (**user defined function**) denilir.

Şimdiye kadar yazdığımız kodlarda `print()`, `input()` gibi dahili fonksiyonlar ile `math` modülünü kodumuza aktararak (`import`) `sqrt()`, `log()` gibi fonksiyonları kullanabilir hale geldik. Çağırılan program, kullanılacak fonksiyonu bilmelidir.

Bir fonksiyonu çalıştıran koda, **çağırılan program** adı verilir. Çağırılan program içinde, bildirimi daha önce yapılmış bir fonksiyonu, kimliğini ve ardından parantez içindeki argüman listesini yazarak **çağırabiliriz** (**function call/invoke function**).

Python dilinde, dahili olan fonksiyonlar ya da `math` modülündeki fonksiyonlar gibi kendi ihtiyaçlarımızı

²<https://docs.python.org/3/library/math.html>

karşılamanı yeniden kullanılabilir fonksiyonlar tanımlayabilir ve modüller oluşturabiliriz.

§6.3.1 Fonksiyon Nasıl Tanımlanır?

Bir Python fonksiyonu aşağıdaki kurallar ile tanımlanır;

- ◊ Fonksiyon blokları `def` anahtar sözcüğüyle başlar, bunu fonksiyon adı ve parantezler `()` izler.
- ◊ Herhangi bir giriş parametresi bu parantezlerin içinde tanımlayabilirsiniz.
- ◊ Her fonksiyon içindeki kod bloğu iki nokta üst üste `:` ile başlar girintili (**indentation**) yazılır.
- ◊ Fonksiyon bloğunun ilk satırında fonksiyonun ne yaptığını kısaca anlatan bir dokümantasyon metni (**docstring**) yer alabilir. Bu **docstring**, modülleri, sınıfları, fonksiyonları ve yöntemleri belgelemek için kullanılan çok satırlı bir yorumdur ve fonksiyon bloğunun başında yer almalıdır.
- ◊ Fonksiyon bloğu içinde `return [expression]` ifadesi bir fonksiyondan çıkar ve isteğe bağlı olarak çağırana bir ifade geri döndürebiliriz. Eğer geri değer döndürülmeyecek ise tek başına `return` talimatı kullanılır. Argümanı olmayan bir `return` ifadesi `return None` ile aynıdır.

Fonksiyon tanımına ilişkin sözde kod aşağıda verilmiştir.

```
def fonksiyon-kimliği( parametreler ):
    "Fonksiyon Tanımı (docstring)"
    # fonksiyon gövdesi
    return [expression]
```

Aşağıda fonksiyon tanımlamalarına ve bu fonksiyonların çağırılmasına kısa bir örnek verilmiştir.

```
#Fonksiyon Tanımları
def ikiSayıTopla(x,y): # Parametrelili fonksiyon
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def sayıYaz(sayı): # Geri değer döndürmeyen fonksiyon
    print(sayı)
    return # return yazılmayabilir
def sayıOku(): # Parametresiz fonksiyon
    str=input('Bir Sayı Giriniz:')
    return float(str)
#Artık Tanımlanmış Fonksiyonları Çağırabiliriz
sayı1=sayıOku() # sayıOku fonksiyonu çağırılıyor
sayı2=sayıOku() # sayıOku fonksiyonu tekrar çağırılıyor
sonuc=ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağırılıyor
sayıYaz(sonuc) # sayıYaz fonksiyonu çağırılıyor
print(ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

Bir Sayı Giriniz:10

Bir Sayı Giriniz:20

30.0

Bu Fonksiyon Parametreleri Toplar

Dokümantasyon metni (**docstring**) için günümüzde PEP 257 standardı kullanılmaktadır. Aşağıda örneği verilmiştir;

```
def toplam(a, b) :
    """
    İki sayıyı toplar.

    Args:
    a : İlk sayı
    b : İkinci sayı

    Returns:
    İki sayının toplamı

    Examples:
    >>> toplam(2, 3)
    5
    """
```

```
return a + b
```

§6.3.2 Fonksiyonlara Parametre Geçirme

Bir fonksiyonun tanımlanması sırasında parantez içinde bildirilen parametrelerin listesi **resmi argümanlardır** (**formal argument**) ve bu argümanlar **konumsal argümanlar** (**positional argument**) olarak da bilinir. Bir fonksiyon istenildiği kadar resmi argümanla tanımlanabilir. Fonksiyonlara geçirilen parametreler tanımlı ise çağrılırken mutlaka argüman verilmelidir.

```
def negatif(x):
    "Bu fonksiyon parametresiz çağrılmaz."
    return -x;
sonuc=negatif(-1)
print(sonuc) #1
# sonuc=negatif() hata verir.
a=-1;
sonuc=negatif(a) #bir değişken de argüman olarak verilebilir
print(sonuc) #1
```

Fonksiyonlara parametre geçirirken bazı parametreler **varsayılan argümanlara** (**default argument**) sahip olabilir.

```
def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
# Artık Fonksiyonu Kullanabiliriz
sonuc=ikiSayıTopla(10) #ikinci argman 1 kabul edildi
print(sonuc) #11
```

Tüm parametreler için de ön tanımlı argüman belirlenebilir.

```
def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def ikinciSayıyıÇıkar(x=10,y=1):
    "Bu Fonksiyon Birinci Parametreden İkincisini Çıkarır"
    fark=x-y
    return fark
sonuc=ikiSayıTopla(10) #ikinciargman 1 kabul edildi
print(sonuc) #11
sonuc=ikinciSayıyıÇıkar() #birinci argüman 10, ikincisi 1 kabul edildi
print(sonuc) #9
```

Parametre ismi verilerek argümanlar fonksiyona geçirilebilir. Bu argümanlara **anahtar kelime argümanlar** (**keyword argument**) adı verilir;

```
def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
sonuc=ikiSayıTopla(x=10) #ikinciargman 1 kabul edildi
print(sonuc) #11
sonuc=ikiSayıTopla(x=10, y=20)
print(sonuc) #30
sonuc=ikiSayıTopla(y=10, x=20)
print(sonuc) #30
```

Anahtar kelime argümanlarının (keyword argument) kullanımı isteğe bağlıdır. Ancak, fonksiyonun argümanları yalnızca anahtar kelimeyle kabul etmesini zorlayabilirsiniz. Bir argümanı yalnızca anahtar kelime ile kullanılan bir argüman yapmak için, kullanıcı tanımlı fonksiyonu oluştururken argümanın önüne yıldız işareti (*) koyulur.

```
from math import pow
def kuvvet(taban,*, üs):
    sonuc = pow(taban,üs)
    return sonuc
```

```
hesap = kuvvet(3, üs=4)
print(hesap) #81.0
hesap = kuvvet(3,4) #Hata: kuvvet() takes 1 positional argument but 2 were given
```

Dahili `print()` fonksiyonu yalnızca anahtar kelime argümanlarına bir örnektir.

```
print (1,2,3,4,5, sep='-') #1-2-3-4-5
print ("Bir","İki","Üç","Dört", sep=",") #Bir,İki,Üç,Dört
```

Fonksiyon, bir veya daha fazla argümanın anahtar sözcüklerle değerlerini kabul edemeyecek şekilde tanımlanabilir. Bu tür argümanlara **yalnızca konumsal argümanlar** (**positional-only argument**) denir. Bir argümanı yalnızca konumsal yapmak için bölü (/) sembolünü kullanılır. Bu sembolden önceki tüm argümanlar yalnızca konumsal olarak ele alınacaktır.

```
from math import pow
def kuvvet(taban, üs, /):
    sonuc = pow(taban,üs) #birinci parametre taban, ikinci parametre üs kabul edilir.
    return sonuc
```

```
hesap = kuvvet(3,4)
print(hesap) #81.0
hesap =kuvvet(2,8)
print (hesap) #256.0
```

Bir başka örnek;

```
def hepsiniYaz(x,/,y,*,z): # birinci parametre her zaman ilk sırada,
    # z de anahtar kelime zorunlu
    print(x,y,z)
    return
hepsiniYaz(100,10,z=20) # 100 10 20
hepsiniYaz(100,y=10,z=20) # 100 10 20
hepsiniYaz(100,z=20,y=10) # 100 10 20
```

Değişken sayıda argüman kabul edebilen bir fonksiyon tanımlamak isteyebiliriz. Bu argümanlara **keyfi sayıda argüman** (**arbitrary argument**) adı verilir. Ayrıca, değişken sayıda argüman konumsal veya anahtar kelime argümanları olabilir.

```
def hepsinitopla(*args):
    s=0
    for x in args:
        s=s+x
    return s
```

```
sonuç = hepsinitopla(3,5,7,9)
print (sonuç) #24
sonuç = hepsinitopla(150,300,450)
print (sonuç) #900
```

Keyfi sayıda argüman yanında zorunlu argüman da tanımlanabilir;

```
def katsayıİleÇarpTopla(katsayı, *args):
    s=0
    for x in args:
        s=s+x*katsayı
    return s
sonuç = katsayıİleÇarpTopla(10,3,5,7,9) # 10 ile geri kalan argümanlar çarpılarak toplanır
print (sonuç) #240
sonuç = katsayıİleÇarpTopla(3,150,300,450) # 3 ile geri kalan argümanlar çarpılarak toplanır
print (sonuç) #2700
```

Keyfi sayıda anahtar kelime argümanı da tanımlayabiliriz;

```
def keyfianahtarkelimeargüman(**kwargs):
    for k,v in kwargs.items():
        print ("{}:{}".format(k,v))
keyfianahtarkelimeargüman(adı='İlhan', soyadı='ÖZKAN', yaş=50, boy=1.8)
```

Bu programı çalıştırdığımızda aşağıdaki çıktı elde edilir;

```
adı:İlhan
soyadı:ÖZKAN
yaş:50
boy:1.8
```

§6.3.3 Fonksiyon Açıklamaları

Python dilinde veri tipleri dinamik olarak belirlenir. Bazı fonksiyonlarda işlenecek veriler ile geri döndürülecek değerler için veri tipi belirlenmesi gerekir. Bunun için fonksiyon **açıklama (annotation)** özelliği kullanılır. Bir fonksiyon tanımında bildirilen argümanlar ve ayrıca dönüş veri tipi hakkında ek açıklayıcı meta veriler eklemenize olanak tanır.

```
def topla(x, y): #hiçbir açıklaması olmayan fonksiyon
    geçici = x+y
    return geçici
print(topla(10,20))
print(topla.__annotations__)
```

Dönüş veri tipi için de açıklama verebilirsiniz. Fonksiyon parantezinden sonra ve iki nokta üst üste sembolünden önce, eksi ve büyük karakterinden oluşan bir ok (->) koyulur. Ve dönüş tipi belirtilir.

```
def topla(x, y):
    geçici = x+y
    return geçici
print(topla(10,20)) #30
print(topla.__annotations__) #{}

def intTopla(x:int, y:int): #parametreleri int olan topla fonksiyonu
    geçici = x+y
    return geçici
print(intTopla(10,20)) #30
print(intTopla.__annotations__) #{'x': <class 'int'>, 'y': <class 'int'>}

def intToplaintDöndür(x:int, y:int) -> int: #parametreleri ve geri döndürdüğü değer
    #int olan topla fonksiyonu
    geçici = x+y
    return geçici
print(intToplaintDöndür(10,20)) #30
print(intToplaintDöndür.__annotations__)
#{'x': <class 'int'>, 'y': <class 'int'>, 'return': <class 'int'>}
```

6.4 Değişken Kapsamı

Programlama dillerinde bir değişkenin tanımlandığı yer, o değişkenin programın hangi bölgelerinden okunabileceğini ve değiştirilebileceğini belirler. Bir değişkenin erişilebilir olduğu ve yaşam döngüsünü sürdürdüğü bu etki alanına **değişken kapsamı (variable scope)** denir.

Python'da Blok Kapsamı (Block Scope) Var mıdır?

C, C++ veya Java gibi dillerin aksine Python'da **if**, **for** veya **while** gibi kod blokları yeni bir kapsam alanı oluşturmaz. Python'da yeni bir kapsam alanı **fonksiyonlar (def)**, **sınıflar (class)** ve **modüller** seviyesinde yaratılır.

Python derleyicisi bir değişken adı ile karşılaştığında onu sırasıyla LEGB kuralına göre arar:

1. Local (Yerel Kapsam)
2. Enclosing (Kapsayan/İç İç Fonksiyon Kapsamı)
3. Global (Küresel Kapsam)
4. Built-in (Yerleşik Python Kapsamı)

§6.4.1 Yerel Değişkenler

Bir fonksiyonun gövdesi içinde tanımlanan değişkenlere **yerel değişken (local variable)** denir. Yerel değişkenler yalnızca tanımlandıkları fonksiyon çalışırken bellekte var olurlar; fonksiyonun icrası tamamlandığında bellekten (**stack/heap**) otomatik olarak temizlenirler.

Bu değişkenlere fonksiyon dışından erişmeye çalışmak Python derleyicisinin **NameError** hatası fırlatmasına yol açar. Bir fonksiyon içindeki yerel değişkenlerin listesini görmek için yerleşik **locals()** fonksiyonu kul-

lanılır.

```
def hepsini_topla(*args):
    s = 0 # 's' yerel bir değişkendir (Local variable)
    for x in args: # 'x' de döngü içinde tanımlanmış yerel bir değişkendir
        s = s + x
    # Çalışma anındaki yerel sembol tablosunu yazdıralım:
    print("Yerel Değişkenler (locals):", locals())
    return s

# Fonksiyon çağırısı
sonuc = hepsini_topla(10, 20, 30, 40)
```

UYARI: Fonksiyon dışında 's' veya 'x' değişkenlerine ulaşamazsınız!
 # print(s) -> NameError: name 's' is not defined hatası verir.

Program çalıştırıldığında aşağıdaki çıktı elde edilir;

Yerel Değişkenler (locals): {'args': (10, 20, 30, 40), 's': 100, 'x': 40}

§6.4.2 Küresel Değişkenler

Herhangi bir fonksiyonun veya sınıfın dışında, ana modül seviyesinde tanımlanan değişkenlere **küresel değişken** (**global variables**) denir. Küresel değişkenler, tanımlandıkları andan itibaren programın sonlanmasına kadar bellekte kalır ve her fonksiyonun içinden okunabilirler.

Bir modüldeki tüm küresel değişkenleri ve fonksiyon tanımlarını incelemek için `globals()` fonksiyonundan yararlanılır.

```
# Modül seviyesinde tanımlanmış küresel değişkenler (Global variables)
programci_adi = 'İlhan ÖZKAN'
uyarlama = 2.3

def bir_fonksiyon():
    # Küresel değişkenler fonksiyon içinden doğrudan okunabilir.
    print("Programcı Adı :", programci_adi)
    print("Uyarlama      :", uyarlama)
    print("Fonksiyon 1 locals():", locals()) # Yerel değişkeni olmadığı için boş sözlük döner

def bir_baska_fonksiyon():
    temp = 0 # 'temp' bu fonksiyona özel yerel bir değişkendir
    print("Programcı Adı :", programci_adi)
    print("Görsel Temp   :", temp)
    print("Fonksiyon 2 locals():", locals())
```

```
print("--- Doğrudan Erişim ---")
print("Modül Seviyesi:", programci_adi)
print("\n--- Fonksiyon Çağrılarını ---")
bir_fonksiyon()
print()
bir_baska_fonksiyon()
print("\n--- Küresel Sembol Tablosu (globals) ---")
# Sözlükteki anahtar isimlerini görebilmek için sadeleştirilmiş gösterim:
g_tablo = {k: v for k, v in globals().items() if not k.startswith("__")}
print(g_tablo)
```

Program çalıştırıldığında aşağıdaki çıktı elde edilir;

```
--- Doğrudan Erişim ---
Modül Seviyesi: İlhan ÖZKAN

--- Fonksiyon Çağrılarını ---
Programcı Adı : İlhan ÖZKAN
Uyarlama      : 2.3
Fonksiyon 1 locals(): {}

Programcı Adı : İlhan ÖZKAN
```


Görsel Temp : 0

Fonksiyon 2 locals(): {'temp': 0}

--- Küresel Sembol Tablosu (globals) ---

```
{'programci_adi': 'İlhan ÖZKAN', 'uyarlama': 2.3, 'bir_fonksiyon': <function bir_fonksiyon at 0x...>,
'bir_baska_fonksiyon': <function bir_baska_fonksiyon at 0x...>, 'main': <function main at 0x...>}
```

6.5 Fonksiyonlarda Değişken Aktarımı: Değerle ve Referansla Çağırma

Geleneksel programlama dillerinde (örneğin C/C++), bir fonksiyona argüman aktarılırken iki temel yöntem kullanılır:

Değerle Çağırma (pass-by-value): Değişkenin bellekteki değerinin bir kopyası fonksiyona gönderilir. Fonksiyon içindeki değişiklikler orijinal değişkeni etkilemez.

Referansla Çağırma (pass-by-reference): Değişkenin bellekteki adresi fonksiyona gönderilir. Fonksiyon içindeki değişiklikler doğrudan orijinal değişkeni değiştirir.

§6.5.1 Nesne Referansıla Çağırma

Python bu iki geleneksel yöntemden farklı bir bellek yaklaşımı sergiler. Python'da her şey bir nesnedir ve değişkenler yalnızca nesnelere işaret eden birer etikettir (referanstır). Bu nedenle Python'daki aktarım mekanizması literatürde "Nesne Referansıla Çağırma" (**pass-by-object-reference**) olarak adlandırılır.

Bir değişkenin fonksiyon içinde orijinal değerini koruyup koruyamayacağı, aktarılan veri türünün **değiştirilebilirlik (mutability)** özelliğine bağlıdır:

- ◊ Değiştirilemez (**Immutable**) Türler (**int, float, str, tuple**): Değerle çağırma gibi davranır. Fonksiyon içinde değişken güncellenmeye çalışıldığında Python bellekte yeni bir nesne oluşturur; orijinal değişken asla etkilenmez.
- ◊ Değiştirilebilir (**Mutable**) Türler (**list, dict, set**): Referansla çağırma gibi davranır. Fonksiyon içinde nesne üzerinde yapılan değişiklikler (örneğin listeye eleman eklemek), orijinal nesneye doğrudan yansır.

§6.5.2 Değiştirilemez Türlerde Aktarım

Aşağıdaki örnekte bir tamsayı (**int**) fonksiyona aktarılmakta ve fonksiyon içinde değeri değiştirilmektedir. **id()** fonksiyonu kullanılarak nesnelerin bellek adresleri takip edilmiştir:

```
def sayiyi_artir(x):
    print(f" [Fonksiyon İçi - Giriş] x adresi: {id(x)}, Değer: {x}")
    x = x + 10 # Immutable nesne olduğu için Python bellekte YENİ bir int oluşturur.
    print(f" [Fonksiyon İçi - Çıkış] x adresi: {id(x)}, Değer: {x}")
```

```
sayi = 5
print(f"[Ana Program - Başlangıç] sayi adresi: {id(sayi)}, Değer: {sayi}")
sayiyi_artir(sayi)
# Orijinal değişkenin etkilenmediğini ve bellek adresinin değişmediğini görüyoruz:
print(f"[Ana Program - Bitiş] sayi adresi: {id(sayi)}, Değer: {sayi}")
```

Programın çıktısı:

```
[Ana Program - Başlangıç] sayi adresi: 140705123456000, Değer: 5
[Fonksiyon İçi - Giriş] x adresi: 140705123456000, Değer: 5
[Fonksiyon İçi - Çıkış] x adresi: 140705123456320, Değer: 15
[Ana Program - Bitiş] sayi adresi: 140705123456000, Değer: 5
```

§6.5.3 Değiştirilebilir Veri Tiplerinde Aktarım

Aşağıdaki örnekte ise bir liste (**list**) fonksiyona aktarılmakta ve fonksiyon içinde yeni bir eleman eklenmektedir:

```
def eleman_ekle(liste):
    print(f" [Fonksiyon İçi - Giriş] liste adresi: {id(liste)}")
    liste.append(99) # Orijinal liste nesnesi bellek seviyesinde doğrudan güncellenir.
    print(f" [Fonksiyon İçi - Çıkış] liste adresi: {id(liste)}")
```

```
sayilar = [1, 2, 3]
```

```
print(f"[Ana Program - Başlangıç] sayılar adresi: {id(sayilar)}, İçerik: {sayilar}")
eleman_ekle(sayilar)
# Orijinal listenin doğrudan değiştiğini görüyoruz:
print(f"[Ana Program - Bitiş] sayılar adresi: {id(sayilar)}, İçerik: {sayilar}")
```

Program çıktısı;

```
[Ana Program - Başlangıç] sayılar adresi: 2198123456780, İçerik: [1, 2, 3]
[Fonksiyon İçi - Giriş] liste adresi: 2198123456780
[Fonksiyon İçi - Çıkış] liste adresi: 2198123456780
[Ana Program - Bitiş] sayılar adresi: 2198123456780, İçerik: [1, 2, 3, 99]
```

KRİTİK YAZILIM TUZAĞI: Yan Etki (Side Effect) Oluşturma

Değiştirilebilir (**mutable**) nesneleri fonksiyonlara parametre olarak gönderirken çok dikkatli olunmalıdır. Eğer fonksiyonun orijinal veriyi bozması istenmiyorsa, fonksiyon çağrılırken veri kümesinin bir kopyası (**shallow/deep copy**) gönderilmelidir:

`eleman_ekle(sayilar.copy())` veya `eleman_ekle(sayilar[:])`

Bu sayede fonksiyon orijinal liste üzerinde değil, onun kopyası üzerinde işlem yapar.

Veri Türü	Kategori	Fonksiyon İçi Değişiklik Davranışı	Orijinal Değişken Etkilenir mi?
<code>int, float, complex</code>	Immutable (Değiştirilemez)	Yeni bir nesne oluşturulur	Hayır (Güvenli)
<code>str, tuple, bytes</code>	Immutable (Değiştirilemez)	Yeni bir nesne oluşturulur	Hayır (Güvenli)
<code>list</code>	Mutable (Değiştirilebilir)	Mevcut nesne değiştirilebilir	Evet (Kalıcı Etki)
<code>dict, set</code>	Mutable (Değiştirilebilir)	Mevcut nesne değiştirilebilir	Evet (Kalıcı Etki)

Tablo 6.1: Fonksiyonlara Geçirilen Parametreler

6.6 Özyinelemeli Fonksiyonlar

Özyineleme (**recursion**), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (**iteration**), döngüler (**loop**) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Özyineleme ve Temel Durum

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz **temel durum** (**base case**)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **0 sayısının faktöriyeli 1 kabul edilir**. Bu aslında faktöriyel problemi için **temel durumdur** (**base case**) ve **boş ürün** (**empty product**) kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
def faktöriyel(n):
    if n>=1:
        carpim=1
        for i in range(2,n+1):
            carpim=carpim*i
        return carpim
    else:
        return 1
```

```
faktöriyel(5) #120
```

```
faktöriyel(7) #5040
```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktöriyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n - 1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli** (**recursive**) tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n - 1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametrelili olarak** aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n - 1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! = 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (**recursive**) olarak ise faktöriyel fonksiyonu aşağıdaki şekilde kodlanabilir;

```
def faktöriyel(n):
    if n==1:
        return 1 #base case
    else:
        return n*faktöriyel(n-1)
```

faktöriyel(5) #120

faktöriyel(7) #5040

Bir başka örnek olarak, girilen taban ve üs ile kuvvet hesaplama fonksiyonu verilebilir. Bu fonksiyonun tanımı aşağıdaki şekilde yazılabilir;

$$f(\text{taban}, \text{us}) = \begin{cases} 0 & \text{eğer } \text{taban} = 0 \\ 1 & \text{eğer } \text{us} = 0 \\ \text{taban} \cdot f(\text{taban}, \text{us} - 1) & \text{eğer } \text{taban} > 0 \text{ ve } \text{us} > 0 \end{cases}$$

Buna ilişkin kod aşağıda verilmiştir;

```
def kuvvet(taban,üs):
    if taban==0:
        return 0 #base case
    if üs==0:
        return 1 #base case
    return taban*kuvvet(taban,üs-1)
```

kuvvet(2,3) #8

kuvvet(3,2) #9

6.7 Kendi Modülümüzü Oluşturma

Bir modül, tanımları ve ifadeleri içeren içe aktarılabilir bir Python dosyadır. Modüller, bir **.py** dosyası oluşturarak oluşturulabilir. Yukarıdaki örneğimizde fonksiyonları **Fonksiyonlar.py** dosyasında yazalım;

```
#Fonksiyonlar.py
"""Bu Modül Okuma Yazma ve Toplama Fonksiyonu Bulunur.""" # Modül Dokümantasyon Metni
def ikiSayıTopla(x,y): # Parametrelili fonksiyon
    """Bu Fonksiyon Parametreleri Toplar""" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def sayıYaz(sayı): # Geri değer döndürmeyen fonksiyon
    print(sayı)
    return #yazılmayabilir
def sayıOku(): # Parametresiz fonksiyon
    str=input('Bir Sayı Giriniz:')
    return float(str)
```

Bir modüldeki fonksiyonlar, modülün içe aktarılmasıyla (import) kullanılabilir. Bu durumda fonksiyonları çağırın kodumuz aşağıdaki şekilde olacak ve aynı çıktıyı verecektir;

```
import Fonksiyonlar
```

```
sayı1=Fonksiyonlar.sayı0ku() # sayı0ku fonksiyonu çağrılıyor
sayı2=Fonksiyonlar.sayı0ku() # sayı0ku fonksiyonu tekrar çağrılıyor
sonuc=Fonksiyonlar.ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağrılıyor
Fonksiyonlar.sayıYaz(sonuc) # sayıYaz fonksiyonu çağrılıyor
print(Fonksiyonlar.ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Oluşturduğunuz modüller, bunları içeri aktardığınız (**import**) dosya ile aynı klasörde olmalıdır. Ancak, önceden dahil edilmiş modüllerle birlikte Python **lib** klasörüne de koyabilirsiniz, ama mümkünse bundan kaçınılmalıdır.

Ayrıca modüllere takma isim de verilebilir;

```
import Fonksiyonlar as Fn
sayı1=Fn.sayı0ku() # sayı0ku fonksiyonu çağrılıyor
sayı2=Fn.sayı0ku() # sayı0ku fonksiyonu tekrar çağrılıyor
sonuc=Fn.ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağrılıyor
Fn.sayıYaz(sonuc) # sayıYaz fonksiyonu çağrılıyor
print(Fn.ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Modüle ilişkin dokümantasyon metnini öğrenmek için aşağıdaki kod yazılabilir;

```
import Fonksiyonlar as Fn
print( Fn.__doc__ )
```

Örneğin program çıktısı aşağıdaki gibi olacaktır.

Bu Modül Okuma Yazma ve Toplama Fonksiyonu Bulunur.

6.8 Paket Yönetimi

Python ilinde modül (module), .py uzantılı sınıflar, fonksiyonlar gibi nesneler içeren bir Python dosyasıdır. Python dilinde paketler (package), bir veya daha fazla modül dosyası içeren bir klasördür ve **__init__.py** adlı özel bir dosya içerir. Bu dosya boş olabilir ancak paket listesini içerebilir. Örnek olarak hesap adı bir Python paketi aşağıda verilmiştir;

```
D:/ → Disk
├─ PythonKlasoru/ → Klasör (Directory)
│   └─ hesap/ → Paketin Adını Belirleyen Klasör
│       ├── __init__.py → Paket Listesini İçeren Özel Dosya
│       ├── polinom.py → Paket İçindeki Bir Modül
│       ├── alan.py → Paket İçindeki Bir Başka Modül
│       ├── alanOrnek.py → Paket dışındaki betik dosyası
│       └─ polinomOrnek.py → Paket dışındaki bir başka betik dosyası
```

Polinom modülü içeriği aşağıdaki gibidir;

```
# polinom.py
"""Bu Modül aşağıda terim katsayıları verilen polinom değerini hesaplar."""
```

```
def polinomDegeri(x,*katsayilar):
    """İlk Parametre x, ikinci parametre sabit terim katsayısı ..."""
    deger=0.0;
    indis=0
    for katsayi in katsayilar:
        deger+=katsayi*x**indis
        indis+=1
    return deger
```

Alan hesabı modülü içeriği aşağıdaki gibidir;

```
# alan.py
"""Bu Modül çeşitli geometrik şekillerin alanını hesaplar."""

from math import sqrt
def kareninAlani(x):
    """bir kenarı bilinen karenin alanı"""
    return x*x
def dikdörtgeninAlani(x,y):
    """eni ve boyu bilinen dikdörtgenin alanı"""
    return x*y
```

```
def ucgeninAlani(x,y,z):
    """Üç kenarı bilinen üçgenin alanı (heron formula)"""
    s= (x+y+z)/2
    return sqrt(s*(s-x)*(s-y)*(s-z))

Özel dosyanın (__init__.py) içeriği aşağıdaki gibidir;

from .polinom import polinomDegeri
from .alan import kareninAlani,dikdörtgeninAlani,ucgeninAlani

Paket içerisindeki alan hesabını kullanan örnek kod aşağıdaki gibidir;

# alanOrnek.py
from hesap.alan import ucgeninAlani
print (" 5,6,7 üçgenin alanı:", ucgeninAlani(5,6,7))
# 5,6,7 üçgenin alanı: 14.696938456699069
```

Paket içerisindeki polinom hesabını kullanan örnek kod aşağıdaki gibidir;

```
# polinomOrnek.py
from hesap.polinom import polinomDegeri
print ("x=1 için Polinom Değeri :", polinomDegeri(1,5.0,1.0)) # x=1, 5x**0+1x**1
#x=1 için Polinom Değeri : 6.0
```

Eğer bu paketi sistem genelinde kullanmak için kurmak (setup) istersek hesap klasörü ile aynı klasörde olacak şekilde `setup.py` dosyası hazırlarız.Bu dosyanın içeriği aşağıdaki gibi olabilir;

```
#setup.py
from setuptools import setup
setup(name='hesap',
      version='0.1',
      description='Alan ve polinom hesabını içerir',
      url='#',
      author='İlhan Özkan',
      author_email='hoydabre@gmail.com',
      license='MIT',
      packages=['hesap'],
      zip_safe=False)
```

Yukarıdaki gibi önceden hazırlanmış Python paketlerini yüklemek için **PIP (Package Installer for Python)** aracı kullanılır. Bir paketi yüklemek için komut satırında aşağıdaki komut çalıştırılır;

```
C:\Users\ILHANOZKAN>pip install <paket adı>
// Ya da
C:\Users\ILHANOZKAN>python -m pip install <paket adı>
```

Hazırladığımız hesap paketlerini paket yükleme aracı kullanılarak kurulumu aşağıdaki şekilde yapılır;

```
D:\PythonKlasoru> pip install .
Processing d:\PythonKlasoru
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: hesap
Building wheel for hesap (pyproject.toml) ... done
Created wheel for hesap: filename=hesap-0.1-py3-none-any.whl size=2016
sha256=a6827110636b8bb072e3431f1ad7664ba56081bbaec495933802a065138c0b41
Stored in directory: C:\Users\...\AppData\Local\Temp\pip-ephem-wheel-cache-fzr6qquh\
wheels\98\62\e3\ed2d7957ca1514c88218e667de8a81a5632b3b9b01640ecca1
Successfully built hesap
Installing collected packages: hesap
Successfully installed hesap-0.1
```

```
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
D:\PythonKlasoru>
```

Aynı paketi kaldırmak istersek aşağıdaki komut çalıştırılabilir;

```
D:\PythonKlasoru> pip uninstall hesap
```

```
Found existing installation: hesap 0.1
Uninstalling hesap-0.1:
Would remove:
c:\users\...\appdata\local\programs\python\python313\lib\site-packages\hesap-0.1.dist-info\*
c:\users\...\appdata\local\programs\python\python313\lib\site-packages\hesap\*
Proceed (Y/n)?
Successfully uninstalled hesap-0.1
D:\PythonKlasoru>
```

6.9 Lambda İfadeleri

Lambda ifadeleri (**lambda expression**) anonim fonksiyonlardır, yani bir adları yoktur. Bildiğimiz gibi, `def` anahtar kelimesi Python diline normal bir fonksiyonu tanımlamak için kullanılır. Benzer şekilde, **lambda** anahtar kelimesi de anonim bir fonksiyonu tanımlamak için kullanılır. Lambda ifadelerinin söz dizimi aşağıdaki şekildedir;

lambda argümanlar : ifade

Aşağıda metin (string) olarak argüman alan üç farklı lambda fonksiyonu tanımlanmıştır;

```
str = "Python Esnek Bir Dildir!"
küçüğeCevir = lambda s: s.lower()
print(küçüğeCevir(str)) #PYTHON ESNEK BİR DILDIR!
büyüğeCevir = lambda s: s.upper()
print(büyüğeCevir(str)) #python esnek bir dildir!
birlestir = lambda birinci,ikinci: birinci+" "+ikinci #iki parametreli lamda
print(birlestir("İlhan","Özkan")) #İlhan Özkan
```

Bir lambda ifadesi `if` içerebilir. Buradaki kullanım `if` talimatından farklı olduğuna dikkat ediniz.

```
durum = lambda x: "Pozitif" if x > 0 else "Negatif" if x < 0 else "Sıfır"
print(durum(1)) #Pozitif
print(durum(-1)) #Negatif
print(durum(0)) #Sıfır
tekniciftmi= lambda x: "Çift" if x % 2 == 0 else "Tek"
print(tekniciftmi(2)) #Çift
print(tekniciftmi(1)) #Tek
```

Lambda ifadelerini liste (**list**) başlığında anlatılan **liste kavrama** (**list comprehension**) ile birleştirmek mümkündür. Böylece dönüşümleri verilere özlü bir şekilde uygulayabiliriz.

```
liste = [1, 2, 3, 4, 5, 6]
onkat = [lambda arg=x: arg * 10 for x in liste]
for i in onkat:
    print(i(),end=' ')
# 10 20 30 40 50 60
print()
```

Bir lambda fonksiyonunda demet (**tuple**) kullanılarak birden fazla değer geri döndürülebilir;

```
hesaplama = lambda x, y: (x + y, x * y)
sonuclar = hesaplama(5, 6)
print(sonuclar) #(11,30)
```

Python dilindeki `filter()` fonksiyonu, argüman olarak bir fonksiyon ve bir liste alır. Bu fonksiyon listedeki uygun elemanlar için `True` döndürerek filtreleme sağlar.

```
liste = [1, 2, 3, 4, 5, 6]
ciftler = filter(lambda x: x % 2 == 0, liste) # İlk parametre lambda fonksiyonudur
print(type(ciftler)) #<class 'filter'>
print(list(ciftler)) #[2, 4, 6]
```

Python dilindeki `map()` fonksiyonu, bir fonksiyon ve bir listeyi argüman olarak alır. Fonksiyon, bir lambda fonksiyonuyla çağrılır ve her bir öğe için bu fonksiyon, lambda ile değiştirilmiş öğeleri içeren yeni bir liste döndürülür.

```
liste = [1, 2, 3, 4, 5, 6]
yuzkat = map(lambda x: x *100, liste)
print(type(yuzkat)) #<class 'map'>
print(yuzkat) #[100, 200, 300, 400, 500, 600]
```

Python dilindeki `reduce()` fonksiyonu, bir fonksiyon ve bir listeyi argüman olarak alır. Fonksiyon, bir yinelemeli fonksiyonla çağrılır ve yeni bir indirgenmiş tek sonuç döndürülür. Bu, yinelemeli fonksiyon çiftleri üzerinde tekrarlayan bir işlem gerçekleştirir. `reduce()` fonksiyonu `functools` modülüne aittir.

```
from functools import reduce
liste = [1, 2, 3, 4, 5, 6]
elemanlaritopla = reduce(lambda x,y: x+y, liste) #lamda fonksiyonu ardışık iki üyeyi toplar
print(type(elemanlaritopla)) #<class 'int'> #sonuç tek bir int değeridir.
print(elemanlaritopla) #21
```

Hem `lambda` hem de `def` kelimeleri, fonksiyonları tanımlamak için kullanılabilir, ancak farklı amaçlara hizmet ederler. `def`, standart yeniden kullanılabilir (`reusable`) fonksiyonlar oluşturmak için kullanılırken, `lambda` fonksiyonları çoğunlukla geçici olarak ihtiyaç duyulan kısa ve öz işlemleri gerçekleştiren anonim fonksiyonlar için kullanılır.

6.10 Düşün ve Çöz

1. **Değişken Kapsamı ve `global` Anahtar Kelimesi:** Aşağıdaki Python kod bloğu çalıştırıldığında konsola hangi çıktılar basılır? `global` anahtar kelimesinin fonksiyon içindeki etkisini LEGB kuralına göre açıklayınız.

```
x = 5

def harici_islem():
    x = 10
def dahili_islem():
    global x
    x = 20
```

```
dahili_islem()
print("Harici x:", x)
```

```
harici_islem()
print("Kuresel x:", x)
```

2. **Mutable Nesneler ve Varsayılan Parametre Tuzağı:** Python'da fonksiyon tanımlarında varsayılan argüman olarak değiştirilebilir (`mutable`) bir nesne (örneğin boş bir liste `[]`) kullanıldığında ne tür bir yazılım tuzağı (`bug`) ortaya çıkar? Aşağıdaki kodun çıktılarını sebep-sonuç ilişkisiyle açıklayınız.

```
def liste_ekle(eleman, hedef_liste=[]):
    hedef_liste.append(eleman)
    return hedef_liste
```

```
print(liste_ekle(1))
print(liste_ekle(2))
print(liste_ekle(3))
```

3. **[Saf Fonksiyon Mantiğı - Pure Functions]**

Bir fonksiyonun "Saf Fonksiyon" (`Pure Function`) olarak adlandırılması için hangi iki temel şartı sağlaması gerekir? Aşağıdaki `f1` ve `f2` fonksiyonlarından hangisi saf fonksiyondur? Nedenini açıklayınız.

```
toplam = 0
def f1(x):
    global toplam
    toplam += x
    return toplam
```

```
def f2(x, y):
    return x + y
```

4. **[Varsayılan Parametre Tuzağı - Mutable Default Arguments]**

Aşağıdaki Python kodunda fonksiyon üç kez üst üste çağırılmıştır. Kodun çıktısını yazınız ve oluşan çıktının nedenini bellek yönetimi açısından açıklayınız.

```
def liste_ekle(eleman, liste=[]):
    liste.append(eleman)
    return liste
```

```
print(liste_ekle(10))
print(liste_ekle(20))
```



```
print(liste_ekle(30))
```

5. **[Kapsam Etki Alanı - Scope & LEGB Rules]**

Python'daki **LEGB** (*Local, Enclosing, Global, Built-in*) kuralını açıklayınız. Aşağıdaki kod bloğu derlendiğinde/çalıştırıldığında ekrana ne basılır?

```
x = "Global"
```

```
def dis_fonksiyon():
```

```
    x = "Enclosing"
```

```
def ic_fonksiyon():
```

```
    x = "Local"
```

```
    print(x)
```

```
ic_fonksiyon()
```

```
dis_fonksiyon()
```

6. **[Esnek Argümanlar - *args ve **kwargs]**

*args ve **kwargs parametrelerinin temel kullanım amaçları nedir? Fonksiyon içerisinde veri tipi olarak sırasıyla hangi yapılara dönüştürülürler? Gönderilen tüm sayısal değerlerin ortalamasını alan ve opsiyonel olarak `yuvarla=True` gönderilirse sonucu tam sayıya yuvarlayan bir `ortalama_hesapla(*args, **kwargs)` fonksiyonu yazınız.

7. **[Özyinelemeli Düşünme]**

Bir özyinelemeli fonksiyonda Taban Durum (*Base Case*) ne anlama gelir? Taban durumu tanımlanmamış bir özyinelemeli fonksiyon çağrıldığında Python nasıl bir hata (*Exception*) fırlatır ve bunun bellek mimarisindeki karşılığı nedir?

8. **[Özyinelemeli Mantık - Metin Tersine Çevirme]**

Döngü (`for` veya `while`) ve hazır dize dilimleme (`[::-1]`) kullanmadan, verilen bir metni özyinelemeli olarak tersine çeviren `metni_ters_cevirmeli(metin)` fonksiyonunu yazınız.

9. **Özyineleme (recursion) ve Temel Durum:** Bir dizideki tüm sayıların toplamını özyinelemeli (*recursive*) olarak hesaplayan bir Python fonksiyonu yazınız. Yazdığınız fonksiyondaki **temel durum** (*base case*) ve **özyinelemeli adım** (*recursive step*) bölümlerini açıkça belirtiniz.

10. **[Lambda ve Yüksek Seviyeli Fonksiyonlar - Filtering]**

Sadece `lambda` ve `filter()` yapılarını kullanarak, verilen bir sayı dizisindeki asal sayıları ayırt edip liste olarak döndüren Python kodunu yazınız (İpucu: Yardımcı bir asal kontrol fonksiyonu tanımlayabilirsiniz).

11. **Yüksek Dereceli Fonksiyonlar ve Lambda:** tam sayılardan oluşan bir listede; önce tek sayıları filtreleyen, ardından filtrelenen bu sayıların küplerini alan ve son olarak elde edilen sayıların toplamını hesaplayan tek satırlık bir Python ifadesini `filter()`, `map()` ve `reduce()` (veya `sum()`) fonksiyonlarını `lambda` ifadeleriyle birleştirerek yazınız.

12. **[Birleştirici Fonksiyon - Type Hinting]**

Tür ipuçlarını (*Type Hints*) kullanarak, iki liste alan ve bu listelerin kesişim kümesini (ortak elemanları) yeni bir liste olarak döndüren `kesisim_bul(l1: list[int], l2: list[int]) -> list[int]` fonksiyonunu yazınız.

13. **[Gömülü Fonksiyonlar - Zip ve Map]**

Ellerinde öğrencilerin isimleri, vize notları ve final notları bulunan 3 ayrı liste vardır:

```
isimler = ["Ahmet", "Ayşe", "Mehmet"]
```

```
vizeler = [60, 80, 45]
```

```
finaller = [70, 90, 50]
```

`zip()` ve `map()` fonksiyonlarını kullanarak, her öğrencinin başarı notunu (`%40 Vize + %60 Final`) hesaplayan ve sonuçları `{"Ahmet": 66.0, ...}` şeklinde bir sözlük olarak döndüren tek satırlık bir ifade/fonksiyon yazınız.

14. **[İç İç Fonksiyonlar ve Closure (Kapanış)]**

Bir fonksiyonun *Closure* oluşturabilmesi için hangi 3 şart gereklidir? Belirli bir üst üs değerini (n) sabitleyip girilen sayının n 'inci kuvvetini hesaplayan bir `us_alici(n)` kapanış fonksiyonu yazınız.

15. **[Dekoratör (Decorator) Mimarisi]**

Herhangi bir fonksiyonun çalışması için geçen süreyi saniye cinsinden ölçen ve konsola `"[LOG] fonksiyon_adi çalışması saniye sn sürdü."` yazdıran `@zaman_olcer` dekoratörünü yazınız.

16. **[Sonsuz Rekürsyon ve Önbellekleme - Memoization]**

Fibonacci dizisinin n 'inci elemanını bulan standart rekürsif fonksiyon $O(2^n)$ zaman karmaşıklığına sahiptir. Bu fonksiyonun performansını artırmak amacıyla, önceden hesaplanan değerleri bellekte tutan (Memoization) bir dekoratör yazınız veya `functools.lru_cache` kullanımını açıklayarak koda uygulayınız.

17. **[Jeneratörler ve Yield - Bellek Dostu Veri İşleme]**

`return` ile `yield` arasındaki temel fark nedir? Belleğe tek seferde yüklenemeyecek kadar devasa bir metin dosyasındaki satırları tek tek okuyup, sadece belirli bir kelimeyi içeren satırları döndüren bir `dosya_süzgeci(dosya_yolu, aranan_kelime)` jeneratör fonksiyonu yazınız.

18. **[Fonksiyonel Programlama - Reduce ve Map]**

Bir listedeki sayı grubunun varyansını hesaplamak istiyoruz. Varyans formülü:

$$\sigma^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N} \quad (6.1)$$

Hiçbir açık `for` veya `while` döngüsü kullanmadan; sadece `map()`, `sum()` ve `len()` gömülü fonksiyonlarıyla verilen bir listenin varyansını hesaplayan `varyans_hesapla(veri)` fonksiyonunu yazınız.

19. **[Kapsamlı Mühendislik Projesi - Hata Yönetimli Fonksiyon Pipeline'ı]**

Metin tabanlı bir veri temizleme boru hattı (*data pipeline*) tasarlamak istiyorsunuz. Bir metin listesini alıp sırasıyla:

- a) Baştaki ve sondaki boşlukları temizleyen,
- b) Tüm karakterleri küçük harfe çeviren,
- c) Rakamları ve noktalama işaretlerini temizleyen

üç ayrı küçük fonksiyon yazınız. Ardından bu fonksiyonları bir liste içinde parametre olarak sırayla veriye uygulayan ve hatalı veri tiplerinde hatayı yakalayıp loglayan `pipeline_calistir(veri_listesi, fonksiyon_listesi)` ana fonksiyonunu oluşturunuz.

7. Bölüm

Nesne Yönelimli Programlama

7.1 Nesne Yönelimli Paradigma

Python, sadece popüler bir betik (**script**) dili olarak değil, aynı zamanda **nesne yönelimli programlama** (**object-oriented programming**) paradigmasını da destekler. Bu paradigmada nesneler (**object**) programın temel yapı taşlarıdır. Nesneler; durum (**state**) ve davranışlara (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine ileti göndermesiyle (**message-passing**) yapılır. Emredici (**imperative**) paradigmanın aksine bu paradigmada programlama, nesnelerin birbirlerine ileti göndermesi ile yapılır.

Nesne yönelimli programlama paradigması aşağıdaki kavramlar üzerine kuruludur;

- ◇ **Sınıf (class)**: Nasıl ki dünyamızda evler bir mimari plan (**blueprint**) üzerinden inşa ediliyor, nesneler (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu durumların (**state**) ve nesnelerin göstereceği davranışlarının (**behavior**) tanımlandığı bu plana sınıf (**class**) adı verilir.
- ◇ **Nesne (object)**: Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan birçok nesne imal edilebilir. Sınıfta uygun olarak imal edilen her nesne (**object**) sınıfın bir örneğidir (**instance**). İmal edilen her bir nesnenin kendine özgü verisi bulunur. Durumlar (**state**), nesnelere ilişkin verilerin tutulduğu alanlardır (**field**).
- ◇ **Soyutlama (abstraction)**: Bir nesnenin veya bileşenin tasarımına ilişkin detay yerine ihtiyacımızı gördüğü kadar kullanılması işlemidir. Yani nesne ve bileşenleri soyutlama (**abstraction**) yaparak kullanmalıyız.
- ◇ **Sarmalama (encapsulation)**: Nesneye ilişkin durum ve davranışların nesne kontrolünde olmasıdır. Nesnenin verisine ve davranışlarına dışarıdan müdahale kontrollü biçimde olmalıdır.
- ◇ **Miras Alma (inheritance)**: Bir sınıfın başka sınıflardan nitelik ve davranışları devralmasıdır.
- ◇ **Çok Biçimlilik (polymorphism)**: Çalıştırma ortamında bulunan birden çok nesneye aynı mesajın verilip karşılığında her bir nesnenin farklı davranış göstermesi işlemidir.
- ◇

7.2 Sınıf ve Nesne

Kısacası sınıflar, verileri tanımlar ve bu verileri işlemek için yöntemler sağlar. Bu yöntemler sınıfın verilerini işleyecek şekilde icra edilirler. Bu nedenle, sınıflar somut uygulama ayrıntılarını sınıf verilerinden ayırarak soyutlama sağlar.

Python dilinde nesne verilerini tutan durumlar (**state**), nitelikler (**attribute**) olarak adlandırılır. Nesnenin bu durumlarını işleyecek fonksiyonlar da yöntem (**method**) olarak adlandırılır. Sınıflar da nitelikler ve yöntemlerden oluşur. Nitelikler değişkenler olarak, yöntemler ise fonksiyonlar olarak tanımlanır. Genel şablon şu şekildedir:

```
class SınıfKimliği(object):
    """Sınıfı tanımlayan docstring"""

    # Sınıf Durumları/Nitelikleri
    sınıf_değişkeni = değişken_ilk_değeri

    def __init__(self): # Nesne Yapıcı
        self.örnek_değişkeni = imal_edilecek_nesnelerdeki_örnek_değişkeni_ilk_değeri # Nesne
        ↪ Durumları/Nitelikleri

    # Davranışlar/Yöntemler
    def davranış_kimliği(self, parametre):
        """Yöntemi Tanımlayan docstring"""
        geri_dönüş_değeri = self.örnek_değişkeni + parametre
        return geri_dönüş_değeri
```

Bir nesne, belirli bir sınıfın örneğidir. Bir mimari plandan nasıl ki birden fazla ev imal edilebilirse, bir sınıftan da istenildiği kadar örnek (**instance**) alınabilir; yani nesne (**object**) imal edilebilir.

Python dilinde sınıf üyelerine varsayılan olarak halka açık (**public**) olarak erişilir. Yani C++/Java/C# dillerindeki **public** anahtar kelimesinin davranışını taklit eder. Örnek ve sınıf niteliklerine dışarıdan erişim ve

değişiklik her zaman mümkündür.

Durumlar nesnelere ve sınıflara ait verilerdir. Tüm sınıf içerisinde ve imal edilecek nesneler tarafından ortak kullanılacak nitelikler önce tanımlanır ve bu nitelikler **sınıf nitelikleri** (**class attribute**) olarak adlandırılır. Bu niteliklere sınıf dışından erişilebilir ve değiştirilebilir. Bu nitelikler varsayılan nesne verileri için ve **tekil nesne** (**singleton**) oluşturulması halinde yararlıdır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0 # sınıf niteliği (class attribute) tüm nesneler için ortak

kişi = Kişi() # Kişi sınıfından bir örnek (instance) oluşturuldu.
Kişi.kişiSayısı += 1
print(type(kişi)) # <class '__main__.Kişi'>
print(kişi.__doc__) # Kişi Sınıfı
print(Kişi.kişiSayısı) # 1

başkakişi = Kişi()
Kişi.kişiSayısı += 1
print(Kişi.kişiSayısı) # 2
```

Sınıf tanımında yer alan her yöntem (**method**), imal edilecek ilgili nesneyi birinci parametre olarak geçirir. Bunun için **self** kelimesi kullanılır. Bir ev planını değiştirmeden farklı renklerde ev imal edilebilir. Benzer şekilde bir sınıftan da farklı veriler içeren yani farklı durumlarda (**state**) nesneler imal edilebilir. Bunu sağlayan sınıftaki **özel yöntem** (**special method**) olan **__init__()** adlı nesne **yapıcısıdır** (**constructor**). Yapıcı içerisinde tanımlanan nitelikler sadece imal edilen nesneye aittir. Bu niteliklere **örnek nitelikleri** (**instance attribute**) adı verilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0 # sınıf niteliği
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı = 'ilhan' # örnek niteliği (instance attribute)
        self.yaşı = 50 # örnek niteliği (instance attribute)
        Kişi.kişiSayısı += 1 # her nesne imalatında 1 artar

kişi = Kişi()
print(type(kişi)) # <class '__main__.Kişi'>
print(kişi.__doc__) # Kişi Sınıfı
print(kişi.__init__.__doc__) # Kişi Sınıfı Yapıcısı
print(Kişi.kişiSayısı) # 1 - Sınıf niteliğine erişim
print(kişi.adı) # ilhan - Örneğin niteliğine erişim
print(kişi.yaşı) # 50 - Örneğin niteliğine erişim

başkakişi = Kişi()
print(Kişi.kişiSayısı) # 2
print(başkakişi.adı) # ilhan
print(başkakişi.yaşı) # 50
```

Örnek Niteliğinde Yapılan Değişiklikler

Örnek niteliğinde (**instance attribute**) yapılan değişiklikler yalnızca imal edilen nesneleri etkiler!

Her bir örnek değişkeni bellekten silineceği zaman ise sınıfın **__del__()** isimli özel yöntemi çağrılır. Bu yöntem **yıkıcı** (**destructor**) adı verilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı = 'ilhan'
        self.yaşı = 50
        Kişi.kişiSayısı += 1
```

```
def __del__(self):
    """Kişi Sınıfı Yıkıcısı."""
    Kişi.kişiSayısı -= 1 # her nesne silindiğinde 1 eksilir

kişi = Kişi()
print(Kişi.kişiSayısı) # 1
başkakişi = Kişi()
print(Kişi.kişiSayısı) # 2
del başkakişi
print(Kişi.kişiSayısı) # 1
```

Özel yöntemlerden en çok kullanılan diğer iki yöntem `__repr__` ve `__str__` yöntemleridir. `__str__` imal edilen nesnenin metin olarak karşılığını veren yöntemdir. `__str__` kullanıcılar için kullanıcı dostu çıktı oluştururken, `__repr__` ideal olarak nesneyi yeniden imal edebilmek için ayrıntılı, geliştirici odaklı metin oluşturur.

```
class Araba:
    def __init__(self, marka, model):
        self.marka = marka
        self.model = model
    def __str__(self):
        # Kullanıcı için okunabilir
        return f"{self.marka} {self.model}"
    def __repr__(self):
        # Geliştirici için detaylı
        return f"Araba('{self.marka}', '{self.model}')
```

Bir de her sınıfta ön tanımlı olarak `__doc__`, `__name__` ve `__module__` nitelikleri bulunur.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        """Kişi Sınıfı Yapıcısı Parametre Almaz!"""
        Kişi.kişiSayısı += 1
        self.adı = 'ilhan'
        self.yaşı = 50
    def __del__(self):
        Kişi.kişiSayısı -= 1
```

```
kişi = Kişi()
başkakişi = Kişi()
print(Kişi.__doc__) # Kişi Sınıfı
print(Kişi.__name__) # Kişi
print(Kişi.__module__) # __main__
```

Sınıflardan imal edilen nesnelerin kendilerine özgü davranışları (**behavior**) olabilir. Aşağıda bir `Kişi` sınıfı ve bu sınıftan imal edilen `kişi` ve `başkakişi` nesnelerine `adınıSöyle()` davranışını göstermesi için mesaj gönderilmesi örneğine yer verilmiştir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        """Kişi Sınıfı Yapıcısı."""
        Kişi.kişiSayısı += 1
        self.adı = 'İlhan'
        self.yaşı = 50
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı -= 1
    def adınıSöyle(self):
        """Kişi Sınıfından İmal Edilecek Nesnelerin Adını Konsola Yazdırır"""
        print('Adım:{}'.format(self.adı))
    def yaşınıGetir(self):
        """Kişi Sınıfından İmal Edilecek Nesnelerin Yaşını Döndürür"""
        return self.yaşı
```

```

kişi = Kişi()
print('Kişi Sayısı', Kişi.kişiSayısı) # Kişi Sayısı 1
kişi.adınıSöyle()                     # Adım:İlhan
kişi.adı = 'Ali'                      # Örneğin niteliği değiştiriliyor
kişi.adınıSöyle()                     # Adım:Ali

```

```

başkakişi = Kişi()
başkakişi.adınıSöyle()                # Adım:İlhan

```

Örnekte imal edilen tüm nesnelerin `adı` niteliği `'İlhan'`, `yaşı` niteliği ise `50` olacaktır. Bu durumda yapıcı parametrelili tanımlanarak her seferinde başka nitelikler yani durumlarla nesne imal edilebilir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self, adı='İlhan', yaşı=50):
        """Kişi Sınıfı Yapıcısı"""
        Kişi.kişiSayısı += 1
        self.adı = adı
        self.yaşı = yaşı
    def __del__(self):
        Kişi.kişiSayısı -= 1
    def adınıSöyle(self):
        print('Adım:{}'.format(self.adı))
    def yaşınıGetir(self):
        return self.yaşı

kişi = Kişi()
print('Kişi Sayısı', Kişi.kişiSayısı) # Kişi Sayısı 1
kişi.adınıSöyle()                     # Adım:İlhan

başkakişi = Kişi('Mehmet')
print('Kişi Sayısı', Kişi.kişiSayısı) # Kişi Sayısı 2
başkakişi.adınıSöyle()                # Adım:Mehmet
print(başkakişi.yaşınıGetir())        # 50

birbaşkakişi = Kişi('Abdullah', 40)
print('Kişi Sayısı', Kişi.kişiSayısı) # Kişi Sayısı 3
birbaşkakişi.adınıSöyle()             # Adım:Abdullah
print(birbaşkakişi.yaşınıGetir())     # 40

```

Bir başka özel yöntem ise `__str__()` yöntemidir. Aslında Java/C# dillerinde olan `toString()` yöntemi işlevini yerine getirir. Yani imal edilen nesnenin metin olarak karşılığının ne olduğunu belirleyen yöntemdir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self, adı='İlhan', yaşı=50):
        Kişi.kişiSayısı += 1
        self.adı = adı
        self.yaşı = yaşı
    def __del__(self):
        Kişi.kişiSayısı -= 1
    def adınıSöyle(self):
        print('Adım:{}'.format(self.adı))
    def yaşınıGetir(self):
        return self.yaşı
    def __str__(self):
        return 'Tür:İnsan,Adı:{},Yaşı:{}'.format(self.adı, self.yaşı)

kişi = Kişi('ilhan', 50)
kişi.adınıSöyle() # Adım:ilhan
metin = kişi.__str__()

```

```
print(metin)          # Tür:İnsan,Adı:ilhan,Yaşı:50
print(kişi)           # Tür:İnsan,Adı:ilhan,Yaşı:50 #dolaylı olarak __str__() çağrılır
```

```
başkakişi = Kişi('Mehmet', 45)
print(başkakişi)     # Tür:İnsan,Adı:Mehmet,Yaşı:45
```

Bunların dışında sınıfımızda `__fonksiyon__(self)` şeklinde yöntemler de tanımlayabiliriz. Bütün bu tür yöntemlere **sihirli yöntemler** (**magic method**) denir ve Python dilindeki sınıfların önemli bir parçasını oluşturur.

7.3 Yöntemler

Yöntemler (**method**), bir sınıfın nesnesine aittir ve nesne davranışlarını gösterecek belirli işlemleri gerçekleştirmek için kullanılır. Python metotlarını üç farklı kategoriye ayırabiliriz: sınıf yöntemi, örnek yöntemi ve statik yöntem.

Sınıf yöntemi (**class method**), sınıf nitelikleri gibi sınıfın örneğine değil, sınıfa bağlı bir metottur. Sınıfın bir örneği yerine, sınıfın kendisi üzerinde çağrılabilir. Sınıf yönteminin aksine, **örnek yöntemi** (**instance method**) bir nesnenin örneklerine yani imal edilmiş nesnelere erişebilir. Ayrıca, tüm nesneler için ortak olduğu için sınıf değişkenine de erişebilir.

Sınıf yöntemi yazmanın iki yolu vardır. Birincisi dahili `classmethod()` fonksiyonu ile örnek yöntemi sınıf yöntemine çevrilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        self.adı = 'ilhan'
        self.yaşı = 50
        Kişi.kişiSayısı += 1
    def __del__(self):
        Kişi.kişiSayısı -= 1
    def KişiSayısınıGöster(self):
        print('Kişi Sayısı:', Kişi.kişiSayısı)
    NesneSayısı = classmethod(KişiSayısınıGöster)
```

```
kişi = Kişi()
Kişi.NesneSayısı() # 1
başkakişi = Kişi()
Kişi.NesneSayısı() # 2
del başkakişi
Kişi.NesneSayısı() # 1
```

İkincisinde ise sınıf yöntemi başında `@classmethod` dekoratörü kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        self.adı = 'ilhan'
        self.yaşı = 50
        Kişi.kişiSayısı += 1
    def __del__(self):
        Kişi.kişiSayısı -= 1
    @classmethod
    def KişiSayısınıGöster(cls): # cls geleneksel olarak kullanılır.
        print('Kişi Sayısı:', cls.kişiSayısı)
```

```
kişi = Kişi()
Kişi.KişiSayısınıGöster() # 1
başkakişi = Kişi()
Kişi.KişiSayısınıGöster() # 2
del başkakişi
Kişi.KişiSayısınıGöster() # 1
```

Statik yöntemler (**static method**), çağrılması için herhangi bir sınıfın örneğine veya nesneye ihtiyaç duy-

mayan yöntemlerdir. Sınıf yöntemine çok benzer ancak farkı, statik yöntemin `self` nesnesine veya `cls` sınıfına referans gibi zorunlu bir argümana sahip olmamasıdır. Statik bir yöntem:

- ◊ Sıklıkla tekrar kullanılan görevleri gerçekleştirmek için yardımcı bir fonksiyon olarak kullanılabilir.
- ◊ Sınıf adını kullanarak çağırırız. Bu sayede sınıftan imal edilmiş nesne örneklerine olan bağımlılık ortadan kalkar.
- ◊ Sınıf durumundan bağımsız olarak davranışı değişmeden kaldığı için her zaman öngörülebilirdir.
- ◊ İleride göreceğimiz geçersiz kılmayı (**overriding**) önlemek için kullanılır.

Statik yöntemi yazmanın iki yolu vardır. Birincisi dahili `staticmethod()` fonksiyonu ile örnek yöntemini statik yöntemine çevirmektir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        self.adı = 'ilhan'
        self.yaşı = 50
        Kişi.kişiSayısı += 1
    def __del__(self):
        Kişi.kişiSayısı -= 1
    def KişiSayısınıGöster(): # parametrede self YOK!
        print('Kişi Sayısı:', Kişi.kişiSayısı)
    NesneSayısı = staticmethod(KişiSayısınıGöster)
```

```
Kişi.NesneSayısı() # 0
kişi = Kişi()
Kişi.NesneSayısı() # 1
başkakişi = Kişi()
Kişi.NesneSayısı() # 2
del başkakişi
Kişi.NesneSayısı() # 1
```

İkincisinde ise sınıf yöntemi başında `@staticmethod` dekoratörü kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı = 0
    def __init__(self):
        self.adı = 'ilhan'
        self.yaşı = 50
        Kişi.kişiSayısı += 1
    def __del__(self):
        Kişi.kişiSayısı -= 1
    @staticmethod
    def KişiSayısınıGöster(): # parametrede self YOK!
        print('Kişi Sayısı:', Kişi.kişiSayısı)
```

```
Kişi.KişiSayısınıGöster() # 0
kişi = Kişi()
Kişi.KişiSayısınıGöster() # 1
başkakişi = Kişi()
Kişi.KişiSayısınıGöster() # 2
del başkakişi
Kişi.KişiSayısınıGöster() # 1
```

7.4 Erişim Belirleyiciler

Erişim belirleyiciler (**access modifier**), sınıf üyelerine yani değişkenlere ve yöntemlere sınıf dışından erişimi kısıtlamak için kullanılır. Halka açık (**public**), korumalı (**protected**) ve mahrem (**private**) olmak üzere üç tür erişim belirleyici vardır.

Şimdiye kadar örneklerde tanımlanan sınıflarda nitelikler ve yöntemler varsayılan olarak halka açık olarak belirlenmiştir. Yani sınıf üyelerine sınıf ve nesne dışından erişilebilirdir.

Bazı durumlarda imal edilecek nesnelerin bazı üyelerine dışarıdan erişim kapatılmak istenir. Bu tür üyeler **mahrem** (**private**) üyeler olarak adlandırılır. Bir üyeyi mahrem yapmak için üye kimliği önünde iki altçizgi

karakteri (__) kullanılır. Bazı üyeler de mevcut sınıf ve bu sınıftan miras alan sınıflar dışında erişime kapatılır. Bu tür üyeler **korumalı** (**protected**) üye olarak adlandırılır. Korumalı bir üyenin kimliği önünde tek alt çizgi (_) karakteri kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self, adı, sır, maaş):
        """Kişi Sınıfı Yapıcısı"""
        self.adı = adı          # halka açık (public) nitelik
        self.__sır = sır        # mahrem (private) nitelik
        self._maaş = maaş       # korumalı (protected) nitelik
    def kişiBilgileri(self):
        print('Ad:{},Sır:{},Maaş:{}'.format(self.adı, self.__sır, self._maaş))
```

```
kişi = Kişi('İlhan', 'Ben bişey biliyorum', 20000)
print(kişi.adı)          # İlhan
# print(kişi.__sır)      # AttributeError: 'Kişi' object has no attribute '__sır'
print(kişi._maaş)        # 20000
kişi.kişiBilgileri()     # Ad:İlhan,Sır:Ben bişey biliyorum,Maaş:20000
```

Burada imal edilen **kişi** nesnesinin **kişiBilgileri()** yöntemi erişim belirleyicisi ne olursa olsun kişinin tüm verilerine eriştiği gözden kaçırılmamalıdır.

7.5 Soyutlama

Soyutlama (**abstraction**), nesne yönelimli programlamanın önemli ilkelerinden biridir. Bu, bir nesneyle ilgili yalnızca önemli verilerin dış dünyaya açıldığı ve diğer tüm detayların gizlendiği bir programlama yaklaşımını ifade eder. Bu yaklaşım, karmaşıklığı azaltmaya ve uygulama geliştirme verimliliğini artırmaya yardımcı olur.

İki tür soyutlama vardır. Biri, nesnenin gizlendiği verilerle ilgili **veri soyutlamasıdır** (**data abstraction**); bu, gizli veriye nesne içinden erişilebilen bir veri yapısı aracılığıyla ulaşılır. Diğer tür ise süreç ya da **kontrol soyutlamasıdır** (**control abstraction**). Bu ise, nesnenin gerçekleştirdiği sürecin temel uygulama detaylarının gizlenmesini ifade eder.

Aşağıda verilen **Kişi** sınıfını göz önüne alalım. Bu sınıftan nesne imal edildikten sonra yaşı -1, 160, 2000 gibi anormal değerler vermemize kimse engel olamaz.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self, adı, yaşı):
        """Kişi Sınıfı Yapıcısı"""
        self.adı = adı
        self.yaşı = yaşı
```

Bu gibi problemlerin üstesinden veri soyutlaması (**data abstraction**) ile gelebiliriz. Yani **yaşı** alanını mahrem (**private**) yapıp bu alanın değerini kontrollü bir şekilde belirleyen **set** yöntemi yazabiliriz. Benzer şekilde bu alan mahrem olduğundan **get** yöntemi yazıp nesne verisini dışarıya aktarabiliriz.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self, adı='İlhan', yaşı=50):
        """Kişi Sınıfı Yapıcısı"""
        self.adı = adı
        self.__yaşı = yaşı
    def setYaş(self, yaş):
        if yaş < 0:
            print("Hata:Yaş değeri sıfırın altında olamaz!")
        elif yaş > 120:
            print("Hata:Yaş değeri en fazla 120 olabilir!")
        else:
            self.__yaşı = yaş
    def getYaş(self):
        return self.__yaşı
```

```
ali = Kişi('Ali', 40)
ali.setYaş(200)          # Hata:Yaş değeri en fazla 120 olabilir!
```

```
print(ali.getYaş()) # 40
```

Veri soyutlamasına benzer şekilde imal edilen nesnenin bazı davranışlarına dışarıdan erişim kontrollü bir şekilde yapılmalıdır. Bu işleme ise kontrol soyutlaması (**control abstraction**) adı verilir. Aşağıda **HesapMakinesi** sınıfı ve bu sınıftan imal edilen nesnelerin **ekraniYenile** davranışı için kontrol soyutlaması yapılmıştır.

```
class HesapMakinesi(object):
    """Hesap Makinesi Sınıfı"""
    def __init__(self):
        """Hesap Makinesi Sınıfı Yapıcısı"""
        self.__sonuç = 0
    def __ekraniYenile(self): # mahrem davranış
        print('İşlem:', self.__sonuç)
    def setHesapMakinesi(self, değer):
        self.__sonuç = değer
        self.__ekraniYenile()
    def makineyiSıfırla(self):
        self.__sonuç = 0
        self.__ekraniYenile()
    def faktöriyelHesapla(self):
        temp = 1
        for x in range(1, self.__sonuç + 1):
            temp = temp * x
        self.__sonuç = temp
        self.__ekraniYenile()
```

```
makine = HesapMakinesi()
makine.setHesapMakinesi(10) # İşlem: 10
makine.makineyiSıfırla()    # İşlem: 0
makine.setHesapMakinesi(5)  # İşlem: 5
makine.faktöriyelHesapla()  # İşlem: 120
# makine.__ekraniYenile() yazılamaz çünkü mahrem bir davranıştır.
```

7.6 Sarmalama

Bir sınıf üzerinde **veri soyutlaması** (**data abstraction**) ile veriler üzerine, **kontrol soyutlaması** (**control abstraction**) ile de davranışlar üzerine kılıf çekilir. Bir sınıf üzerinde her iki soyutlamanın yapılmasına **sarmalama** (**encapsulation**) adı verilir. Soyutlamaların gerekli olup olmadığına programcı karar verir.

Sarmalama işlemi **erişim değiştiricilerle** (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış olur. Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden **soyut** (**abstract**) olarak kullanmak mümkün olmaktadır.

Özetle **sarmalama** (**encapsulation**) tekniğini uygulamak, kullanımı kolay yapılar elde etmemizi sağlar:

- ♦ n adet veri ve n adet davranış sarmalama işlemine tabi tutulursa **sınıf**,
- ♦ n adet sınıf sarmalama işlemine tabi tutulursa **bileşen**,
- ♦ n adet bileşen sarmalama işlemine tabi tutulursa **kütüphane**,
- ♦ n adet kütüphane sarmalama işlemine tabi tutulursa **çerçeve yapılar** (**framework**) oluşur.

7.7 Kalıtım

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek alarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir.

```
class Öğrenci(object):
    def __init__(self):
        self.yaş = 20
        self.cinsiyet = 'E'
        self.ogrenciNo = 0
    def yaşınıSöyle(self):
        print('Yaşım:', self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:', self.cinsiyet)
    def dersÇalış(self):
        print('Ders Çalışıyorum')
```

```

class Öğretmen(object):
    def __init__(self):
        self.yaş = 50
        self.cinsiyet = 'E'
        self.verdiğiDersKodu = 'BIL101'
    def yaşınıSöyle(self):
        print('Yaşım:', self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:', self.cinsiyet)
    def dersVer(self):
        print('Ders Veriyorum')

class Müdür(object):
    def __init__(self):
        self.yaş = 55
        self.cinsiyet = 'K'
        self.müdürOlduğuBölümKodu = 'Elektronik'
    def yaşınıSöyle(self):
        print('Yaşım:', self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:', self.cinsiyet)
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')

```

Bu sınıfların her birinde `yaş` ve `cinsiyet` gibi ortak özellikler veya `yaşınıSöyle()` ve `cinsiyetiniSöyle()` gibi ortak davranışlar bulunur. Hâlbuki bu gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir.

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına **kalıtım** (**inheritance**) adı verilir. Ortak özellikler **Kişi** sınıfına toplanarak kod yeniden düzenlenebilir:

```

class Kişi(object):
    def __init__(self):
        self.yaş = 20
        self.cinsiyet = 'E'
    def yaşınıSöyle(self):
        print('Yaşım:', self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:', self.cinsiyet)

class Öğrenci(Kişi):
    def __init__(self):
        super().__init__() # miras alınan sınıfın yapıcısı çağrılıyor.
        self.ogrenciNo = 0
    def dersÇalış(self):
        print('Ders Çalışıyorum')

class Öğretmen(Kişi):
    def __init__(self):
        super().__init__()
        self.verdiğiDersKodu = 'BIL101'
    def dersVer(self):
        print('Ders Veriyorum')

class Müdür(Kişi):
    def __init__(self):
        super().__init__()
        self.müdürOlduğuBölümKodu = 'Elektronik'
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')

```

Kalıtımın iki önemli özelliği vardır: **Paylaşılan ortak kod** (**shared code**) vardır ve paylaşılan kodda yapılan değişiklik **anında alt sınıflara yansır** (**snapp change**).

Türemiş sınıfta yapıcıda `super()` kullanımı:

```
# Kötü kullanım:
class Kopek(Hayvan):
    def __init__(self, isim):
        Hayvan.__init__(self, isim)
```

```
# İyi kullanım (Pythonic yol):
```

```
class Kopek(Hayvan):
    def __init__(self, isim):
        super().__init__(isim)
```

Python dilinde aşağıdaki kalıtım türleri mevcuttur:

- ♦ **Tek Kalıtım:** Bir alt sınıf, tek bir üst sınıftan kalıtım alır.
- ♦ **Çoklu Kalıtım:** Bir alt sınıf, birden fazla üst sınıftan kalıtım alır.
- ♦ **Çok Düzeyli Kalıtım:** Bir sınıf, başka bir sınıftan türetilen bir sınıftan türetilir.
- ♦ **Hiyerarşik Kalıtım:** Birden fazla sınıf, tek bir üst sınıftan kalıtım alır.
- ♦ **Hibrit Kalıtım:** Birden fazla kalıtım türünün birleşimidir.

Yöntem çözümleme sırası (method resolution order), sınıf hiyerarşisinde bir yöntemin arandığı sıradır.

```
class A:
    def metod(self):
        print("A")
```

```
class B(A):
    def metod(self):
        print("B")
```

```
class C(A):
    def metod(self):
        print("C")
```

```
class D(B, C):
    pass
```

```
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
```

7.8 Aşırı Yükleme

Aynı kimliğe sahip ancak farklı parametrelere işlem yapan birden fazla yöntemin tanımlanmasına **yöntemlerin aşırı yüklenmesi (method overloading)** adı verilir. Python, varsayılan olarak yöntem aşırı yükleme özelliğini desteklemez; en son yapılan tanım öncekileri geçersiz kılar.

```
class Hesaplayıcı:
    def toplama(self, a=None, b=None, c=None):
        x = 0
        if a is not None and b is not None and c is not None:
            x = a + b + c
        elif a is not None and b is not None and c is None:
            x = a + b
        return x
```

```
nesne = Hesaplayıcı()
print(nesne.topla(10, 20, 30)) # 60
print(nesne.topla(10, 20))     # 30
```

Harici kütüphane olan `multipledispatch` kullanılarak da aşırı yükleme simüle edilebilir:

```
from multipledispatch import dispatch
```

```
class Hesaplayıcı:
    @dispatch(int, int)
    def toplama(self, x, y):
        return x + y
    @dispatch(int, int, int)
    def toplama(self, a, b, c):
```

```
return a + b + c
```

```
nesne = Hesaplayıcı()
print(nesne.topla(10, 20, 30)) # 60
print(nesne.topla(10, 20))      # 30
```

7.9 Geçersiz Kılma

Bir yöntemi **geçersiz kılma** (**overriding**), bir alt sınıfta, üst sınıfındaki bir yöntemle aynı adı taşıyan bir yöntem tanımlamayı ifade eder.

```
class Taban:
    def yöntem(self):
        print('Taban Sınıf Nesnesinin Yöntemi Çağrıldı.')
```

```
class Türemiş(Taban):

    def yöntem(self):
        print('Türemiş Sınıf Nesnesinin Yöntemi Çağrıldı.')
```

```
tabannesne = Taban()
tabannesne.yöntem() # Taban Sınıf Nesnesinin Yöntemi Çağrıldı.
türemişnesne = Türemiş()
türemişnesne.yöntem() # Türemiş Sınıf Nesnesinin Yöntemi Çağrıldı.
```

7.10 Çok Biçimlilik

Çok biçimlilik (**polymorphism**), birden çok nesnenin olduğu bir ortamda, verilen aynı iletiye (**message**) karşı, nesnelerin farklı davranış (**behavior**) göstermeleridir.

Ördek tiplemesi (**duck typing**) örneği:

```
class Ordek:
    def seslen(self):
        return "Vak, Vak!"
```

```
class BaskaKus:
    def seslen(self):
        return "Civ Civ!"
```

```
def sesCikar(ordek):
    print(ordek.seslen())
```

```
duck = Ordek()
anotherBird = BaskaKus()
sesCikar(duck) # Vak, Vak!
sesCikar(anotherBird) # Civ, Civ!
```

Geçersiz kılma ile çok biçimlilik kullanımı:

```
class Kişi(object):
    def raporYaz(self):
        print('Kişi Raporu: ...')
```

```
class Öğrenci(Kişi):
    def raporYaz(self):
        print('Öğrenci Raporu: ...')
```

```
class Öğretmen(Kişi):
    def raporYaz(self):
        print('Öğretmen Raporu: ...')
```

```
class Müdür(Kişi):
    def raporYaz(self):
        print('Müdür Raporu: ...')
```

```
kişiler = [Öğrenci(), Öğretmen(), Müdür()]
for kişi in kişiler:
    kişi.raporYaz()
```

7.11 Soyut Sınıf

Bir sınıfın örneği/nesnesi oluşturulamıyorsa, bu sınıfa **soyut sınıf** (**abstract class**) denir. Python dilinde soyut sınıf oluşturmak için **abc** modülünün **ABC** sınıfı türetilir ve en az bir tane **@abstractmethod** tanımlanır.

```
from abc import ABC, abstractmethod
```

```
class Kişi(ABC):
    def __init__(self):
        self.yaş = 20
        self.cinsiyet = 'E'
    @abstractmethod
    def raporYaz(self):
        print('Soyut Yöntem')

class Öğrenci(Kişi):
    def __init__(self):
        super().__init__()
        self.ogrenciNo = 0
    def raporYaz(self):
        super().raporYaz()
        print('Öğrenci Raporu: ...')
```

```
öğrenci = Öğrenci()
öğrenci.raporYaz()
```

7.12 Ara Yüzler / Roller

Python dilinde **interface** anahtar kelimesi bulunmadığından ara yüzler resmi (**formal**) olarak soyut sınıflar veya gayriresmi (**informal**) olarak gövdesi **pass** olan sınıflar aracılığıyla uygulanır.

Resmi ara yüz tanımı:

```
from abc import ABC, abstractmethod
```

```
class IBaba(ABC):
    @abstractmethod
    def cocugaBak(self): pass

class IMuzisyen(ABC):
    @abstractmethod
    def gitarCal(self): pass

class Kisi(IBaba, IMuzisyen):
    def cocugaBak(self):
        print('Çocuğa Bakıyorum...')
    def gitarCal(self):
        print('Gitar Çalıyorum...')
```

```
kisi = Kisi()
kisi.cocugaBak()
kisi.gitarCal()
```

7.13 İç Sınıf

Bir sınıfın içinde tanımlanan sınıfa **iç sınıf** (**inner class**) denir.

```
class Sirket: # outer class
    def __init__(self):
        self.Bolum1 = self.Muhasebe()
        self.Bolum2 = self.Pazarlama()
    def SirketAdiniYaz(self):
```



```

    print("Şirket Adı: Python A.Ş")
class Muhasebe: # inner class
    def BolumunAdiniYaz(self):
        print("Bölüm Adı: Muhasebe")
class Pazarlama: # inner class
    def BolumunAdiniYaz(self):
        print("Bölüm Adı: Pazarlama")

```

```

disNesne = Sirket()
disNesne.SirketAdiniYaz()
icNesne1 = disNesne.Bolum1
icNesne1.BolumunAdiniYaz()

```

7.14 Tekil Nesne Sınıfı

Tekil nesne sınıfı (**singleton class**), ilgili sınıftan yalnızca bir nesne türetilmesini garanti eder.

`__new__` yöntemi ile Tekil Deseni:

```

class SingletonClass:
    __tekilNesne = None
    def __new__(cls):
        if cls.__tekilNesne is None:
            cls.__tekilNesne = super(SingletonClass, cls).__new__(cls)
        return cls.__tekilNesne

```

```

nesne1 = SingletonClass()
nesne2 = SingletonClass()
if nesne2 is nesne1:
    print("İki Nesne AYNIDIR!")

```

7.15 Yansıma

Yansıma (**reflection/introspection**), bir nesnenin tipini, niteliklerini ve davranışlarını çalışma zamanında inceleyebilme yeteneğidir.

- ◊ `type(nesne)`: Nesnenin sınıf tipini döndürür.
- ◊ `isinstance(nesne, sınıf)`: Nesnenin belirtilen sınıftan olup olmadığını doğrular.
- ◊ `issubclass(altSınıf, üstSınıf)`: Sınıf kalıtım ilişkisini sorgular.
- ◊ `callable(nesne)`: Nesnenin çağrılabilir olup olmadığını kontrol eder.
- ◊ `getattr()`, `setattr()`, `hasattr()`: Dinamik nitelik yönetimi sağlar.
- ◊ `dir(nesne)`: Nesnenin erişilebilir tüm nitelik ve metotlarını listeler.

§7.15.1 type() fonksiyonu

Bir nesnenin hangi sınıfa ait olduğunu söyler.

```

print (type(100)) #<class 'int'>
print (type(7.5)) #<class 'float'>
print (type(5+2j)) #<class 'complex'>
print (type("İlhan")) #<class 'str'>
print (type([10,20,30])) #<class 'list'>
print (type((10,20,30))) #<class 'tuple'>
print (type({'1':'bir', 2:'iki'})) #<class 'dict'>

```

```

class Sinif:
    pass

```

```

nesne=Sinif()
print(type(nesne)) #<class '__main__.Sinif'>

```

§7.15.2 isinstance() fonksiyonu

Bir nesnenin verilen sınıfın bir örneği olup olmadığını yani verilen sınıftan imal edilip edilmediğini belirler. Aşağıdaki kod örneğinde bu fonksiyon True değerini geri döndürür.

```

print (isinstance(100, int))

```

```
print (isinstance(7.5, float))
print (isinstance(5+2j, complex))
print (isinstance("İlhan", str))
print (isinstance([10,20,30], list))
print (isinstance((10,20,30), tuple))
print (isinstance({'1':'bir', 2:'iki'}, dict))
```

```
class Sinif:
    pass
```

```
nesne=Sinif()
print(isinstance(nesne, Sinif))
```

§7.15.3 issubclass() fonksiyonu

Bu fonksiyon bir sınıfın başka bir sınıfın alt sınıfı olup olmadığını kontrol eder. Sınıflar için kullanılır. Tüm Python sınıfları **object** sınıfından alt sınıflara ayrılır. Dolayısıyla, aşağıdaki `print()` ifadelerinin çıktısı hepsi için **True** olacaktır.

```
class Sinif:
    pass
```

```
class AltSinif(Sinif):
    pass
```

```
print (issubclass(int, object))
print (issubclass(str, object))
print (issubclass(Sinif, object))
print (issubclass(AltSinif, object))
print (issubclass(AltSinif, Sinif))
```

§7.15.4 callable() fonksiyonu

Bir nesne, belirli bir davranışı çağırıyorsa **çağrılabilir**dir (**callable**). Belirli bir davranışı gerçekleştiren bir Python fonksiyonu ise çağrılabilir bir nesnedir. Herhangi bir fonksiyon, dahili (**built-in**), **kullanıcı tanımlı** (**user-defined**) veya yöntem çağrılabilir. **int**, **str** ve benzeri gibi dahili veri tiplerine sahip nesneler çağrılmaz. Bir sınıf örneği, `__call__()` yöntemine sahipse çağrılabilir.

```
def yontem():
    pass
```

```
print (callable("Hello")) #False
print (callable(12)) #False
print (callable(min)) #True
print (callable([1,2])) #False
print (callable(list.clear([1,2]))) #False
print (callable(yontem)) #True
```

```
class Sinif:
    def __init__(self):
        pass
    def __call__(self):
        print ("Bu Sınıftan İmal Edilen Nesneler Çağrılabilir!")
```

```
nesne = Sinif()
print ("callable?", callable(nesne)) #True
```

§7.15.5 getattr() fonksiyonu

Bu fonksiyon, bir nesnenin adlandırılmış niteliğinin değerini almak için kullanılır.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50
```

```
nesne = Kisi()
print (getattr(nesne, "Adi")) #İlhan
print (getattr(nesne, "Soyadi")) #Özkan
print (getattr(nesne, "Yasi")) #50
```

§7.15.6 setattr() fonksiyonu

Bu fonksiyon, bir nesneye yeni bir **nitelik** (**attribute**) ekler ve ona bir değer atar. Ayrıca mevcut bir niteliğin değerini de değiştirebilir.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"

nesne = Kisi()
print (getattr(nesne, "Adi")) #İlhan

setattr(nesne, "Adi", "Mehmet")
setattr(nesne, "Soyadi", "Özkan")
setattr(nesne, "Yasi", 50)
print (nesne.Adi) #Mehmet
print (nesne.Soyadi) #Özkan
print (nesne.Yasi) #50
```

§7.15.7 hasattr() fonksiyonu

Bu fonksiyon, nesnenin verilen niteliğe sahip olup olmadığını belirler. Sahipse **True** değerini, aksi durumda **False** değerini döndürür.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50

nesne = Kisi()
print (hasattr(nesne, "Adi")) #True
print (hasattr(nesne, "Cinsiyeti")) #False
print (hasattr(nesne, "Kilosu")) #False
```

§7.15.8 dir() fonksiyonu

Bu yerleşik fonksiyon argüman olmadan çağrılırsa, geçerli kapsamdaki adları döndürür. Herhangi bir nesne argüman olarak kullanıldığında, belirtilen nesnenin ve bu nesneden erişilebilen niteliklerin bir listesini döndürür.

```
#print("dir(int):",dir(int))
#print("dir(list):",dir(list))
```

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50
    def AdivaSoyadiEkranayaYaz(self):
        print("Adı:",self.Adi)
        print("Soyadı:", self.Soyadi)
    def YasiEkranayaYaz(self):
        print("Yaşı:",self.Yasi)

print("dir(Kisi):",dir(Kisi))
```

Kod çalıştırıldığında aşağıdaki çıktı elde edilir.

```
dir(Kisi): ['AdivaSoyadiEkranayaYaz', 'YasiEkranayaYaz', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__',
```

7.16 Dekoratorler

Dekoratorler, fonksiyonların veya yöntemlerin kodlarını değiştirmeden davranışlarını genişletmenin veya değiştirmenin esnek bir yoludur.

Yazılım mühendisliğinde açık/kapalı (**open/closed**) prensibi, bir nesnenin geliştirilmeye açık ama değiştirilmeye kapalı olması gerektiğini savunur. Python'da bir fonksiyonun veya sınıfın davranışını, onun kaynak koduna dokunmadan değiştirmemizi veya genişletmemizi sağlayan yapıya **dekoratör** (**decorator**) denir. Dekoratorler, aslında başka bir fonksiyonu parametre olarak alan ve yeni bir fonksiyon döndüren **yüksek seviyeli fonksiyonlardır** (**higher-order functions**).

Fonksiyonların Nesne Olarak Kullanılması

Dekoratör mantığını anlamak için Python'da fonksiyonların da birer **birinci sınıf nesne** (**first-class object**) olduğunu hatırlamak gerekir. Her birinci sınıf nesne gibi fonksiyonlar da; değişkenlere atanabilir, başka fonksiyonlara parametre olarak gönderilebilir veya bir fonksiyonun içinden geri döndürülebilir.

Dekoratorler @ işleci ile tanımlanırlar. Bir fonksiyonu dekore etmek için `@dekorator_adi` ifadesi kullanılır. Bu, kodun okunabilirliğini artıran bir **söz dizimsel kolaylık** (**syntactic sugar**).

Aşağıdaki örnekte, bir fonksiyonun ne kadar sürede çalıştığını ölçen bir mühendislik aracı tasarlayalım:

```
import time
from functools import wraps

def sure_olcer(fonksiyon):
    @wraps(fonksiyon)
    def sarici(*args, **kwargs):
        baslangic = time.time()
        sonuc = fonksiyon(*args, **kwargs)
        bitis = time.time()
        print(f"{fonksiyon.__name__} çalışma süresi: {bitis - baslangic} sn.")
        return sonuc
    return sarici

@sure_olcer
def veri_isle(n):
    return sum(range(n))

veri_isle(10000000)
```

Bir dekoratörün her türlü fonksiyona uyum sağlayabilmesi için (farklı sayıda parametre alan fonksiyonlar gibi), sarmalayıcı (**wrapper**) fonksiyonun esnek bir yapıda olması gerekir. Bu noktada daha önce işlediğimiz `*args` ve `**kwargs` yapıları devreye girer.

Bir fonksiyonu dekore ettiğinizde, o fonksiyonun teknik kimliği (ismi, dokümantasyon dizini vb.) dekoratörün içindeki sarmalayıcı fonksiyonla yer değiştirir. Bu durum, hata ayıklama (**debugging**) süreçlerinde kafa karışıklığına yol açabilir. Bu mühendislik zafiyetini önlemek için `functools.wraps` kullanılır.

```
from functools import wraps
def log_kaydi(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        print(f"{f.__name__} fonksiyonu çağrıldı.")
        return f(*args, **kwargs)
    return wrapper
```

Dekoratorleri Niçin Kullanırız?

- ♦ **Yetkilendirme** (**authorization**): Kullanıcının bir fonksiyonu çalıştırma yetkisi olup olmadığını kontrol etmek.
- ♦ **İz Kaydı Oluşturma** (**logging**): Fonksiyon çağrılarını ve çıktılarını merkezi bir dosyada kayıt altına almak.
- ♦ **Önbelleğe Alma** (**caching**): Ağır hesaplamaların sonuçlarını belleğe alarak performansı artırmak (`@lru_cache`).
- ♦ **Hata Yönetimi**: Fonksiyonun belirli hata durumlarında otomatik olarak yeniden denenmesini (**retry**).

sağlamak.

Dekoratörler, kodun içine **enine kesen ilgiler** (**cross-cutting concerns**) eklemek için mükemmeldir. Ana iş mantığınızı (**business logic**) kirlletmeden, yardımcı işlemleri bu yapılarla yönetebilirsiniz.

7.17 Düşün ve Çöz

1. **Gizleme (Name Mangling) Mekanizması:** Python'da bir niteliğin önüne iki alt çizgi (__) getirildiğinde (örneğin `self.__sir`) bu nitelik tamamen erişilemez mi olur? Python'ın arka planda uyguladığı İsim Değiştirme (**name mangling**) mekanizmasını açıklayarak, bu niteliğe sınıf dışından dahi olsa nasıl erişilebileceğini kod örneğiyle gösteriniz.
2. **Elmas Problemi (Diamond Problem) ve MRO:** Aşağıdaki kalıtım hiyerarşisinde **D** sınıfından bir nesne oluşturulup `nesne.yaz()` çağrıldığında ekrana ne basılır? Python'ın kullandığı C3 Lineerleştirme algoritması doğrultusunda **D.__mro__** çıktısını adım adım analiz ediniz.

```
class A:
    def yaz(self): print("A")

class B(A):
    def yaz(self): print("B")

class C(A):
    def yaz(self): print("C")

class D(B, C):
    pass
```

3. **Sınıf Değişkeni vs. Örnek Değişkeni Tuzağı:** Bir sınıfa ait sınıf değişkeni (**class attribute**) değiştirilebilir (**mutable**) bir tipte (örneğin bir liste) tanımlandığında, imal edilen farklı nesnelerin bu değişkene eleman eklemesi nasıl bir yan etki doğurur? Kod örneği vererek açıklayınız.
4. **Dekoratör Tasarımı (Parametrelili Dekoratör):** Bir fonksiyonun çıktısını istenen sayı kadar (örneğin *N* defa) ekrana yazdıran ve parametre kabul edebilen bir `@tekrar_et(n=3)` dekoratör fonksiyonunu Python'da `functools.wraps` kullanarak kodlayınız.

8. Bölüm

Veri Yapıları

Python dilinde kullanılan veri yapılarının her biri nesnedir. Nesneler davranışları ve özellikleri olan varlıklardır. Dolayısıyla bu dildeki veri yapılarının her birinin kendine has yöntemleri (**method**) bulunur. C/C++, C# ve Java gibi dillerde bulunan **int**, **float** gibi temel veri tiplerinin aksine Python dilinde tüm değişkenler (**variable**) aslında var olan bir nesneye (**object**) tanımlanan bir referanstır.

Python dilinde dahili veri tipleri olan **bool**, **int**, **float**, **complex** ve **str** gibi veri yapılarında değişken tanımlama (**definition**) sırasında nesne yapıcılar (**constructor**) örtük olarak kullanılır. Aslında **list**, **tuple**, **set** ve **dict** veri yapılarında ise sırasıyla **[]**, **()**, **{}**, **{}** karakterleri ile örtük olarak bu veri yapılarının yapıcılar çağrılır.

8.1 Dizi (array)

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle (**variable**) karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Elemanların sırası **indis** (**index**) olarak adlandırılır. Kümenin aksine aynı elemanlar dizide farklı konumlarda birkaç kez bulunabilir. Elemanlara terim adı da verilir.

Örneğin Elemanları 1'den 10'a kadar sayıların karesinden oluşan bir dizi; $a_k = k^2$ ($k = 1 \dots 10$) veya $a_1, a_2, a_3, \dots, a_{10}$ veya 1, 4, 9, ..., 100 olarak gösterilir. Örneğin;

- ♦ K,İ,T,A,P Dizisi ile P,A,T,İ,K dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları farklıdır.
- ♦ 1,3,5,1,2,1,7 Dizisinde birden farklı indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler yani uzunluğu belirli diziler kullanılır.

Python dilindeki diziler (**array**), C, C++ ve C# ve Java gibi geleneksel programlama dillerindeki dizilere benzer. Aynı veri tipindeki üyelerden oluşan homojen bir veri yapısıdır. Listeler (**list**), dizilere benzer ancak elemanları farklı veri tiplerinden oluşabilir. Yani bir liste, homojen ya da heterojen elemanların bir koleksiyonu olabilir.

Dizi içindeki bireysel elemanlara **indisler** (**index**) aracılığıyla erişilebilir. Bir dizi aşağıdaki şekilde tanımlanır;

```
diziKimliği = array(tipKodu, [Başlatıcılar])
```

Dizi İndisleri

Python dizilerinde indisler her zaman sıfırdan başlar.

Python dilinde dizileri kullanmak için standart **array** modülünü aktarmanız (**import**) gerekir.

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
print(tamsayıDizisi) #diziyi konsola yazma
print(tamsayıDizisi[0]) #ilk elemanı konsola yazma
print(tamsayıDizisi[1]) #ikinci elemanı konsola yazma
#Tüm elemanları yineleyici ile konsola basma:
for i in tamsayıDizisi:
    print(i)
diziuzunluğu=len(tamsayıDizisi)
print(diziuzunluğu) #5
```

Diziyi oluşturan ilk parametre elemanların veri tipini belirler.

Dizilerin sonuna bir ekleme **append** ile veya indis belirterek istenilen yere **insert** ile yeni eleman eklenebilir.

Parametre	Eleman Veri Tipi
b	1 bayt boyutunda işaretli tam sayı
B	1 bayt boyutunda işaretli tam sayı
u	2 bayt boyutunda Unicode karakter
h	2 bayt boyutunda işaretli tam sayı
H	2 bayt boyutunda işaretli tam sayı
i	2/4 bayt boyutunda işaretli tam sayı
I	2 bayt boyutunda işaretli tam sayı
l	4 bayt boyutunda işaretli tam sayı
L	4 bayt boyutunda işaretli tam sayı
f	4 bayt boyutunda kayan noktalı sayı
d	8 bayt boyutunda kayan noktalı sayı

Tablo 8.1: Dizi Eleman Veri Tiplerini Belirleyen Parametreler

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
tamsayıDizisi.append(100) # dizi sonuna 100 sayısını ekleme
print(tamsayıDizisi) # array('i', [1, 2, 3, 1, 3, 100])
tamsayıDizisi.insert(0,200) # ilk eleman olarak 200 ekleme
print(tamsayıDizisi) # array('i', [200, 1, 2, 3, 1, 3, 100])
```

Dizilerin sonundaki eleman `pop` ile silinir veya indis belirterek istenilen konumdaki eleman `remove` ile yeni eleman silinebilir.

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
tamsayıDizisi.pop() # dizi sonundaki eleman silinir
print(tamsayıDizisi) # array('i', [1, 2, 3, 1])
tamsayıDizisi.remove(2) # 2 değerine sahip eleman silinir
print(tamsayıDizisi) # array('i', [1, 3, 1])
```

Bir dizi aynı tipten elemanları olan bir başka dizi ile `extend` yöntemi ile birleştirilebilir. Ayrıca bir listeden de dizi sonuna `fromlist` yöntemi ile eleman eklenebilir.

```
from array import *
dizi = array('i', [1,2,3,4,5])
eklenecekdizi = array('i', [7,8,9,10])
dizi.extend(eklenecekdizi)
print(dizi) #array('i', [1, 2, 3, 4, 5, 7, 8, 9, 10])
liste=[11,12,13]
dizi.fromlist(liste)
print(dizi) #array('i', [1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13])
```

Argüman olarak verilen değer ile eşleşen ilk indisi `index` döndürür.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.index(4)) #3
print(dizi.index(1)) #0
```

Dizideki eleman konumları tersine çevrilmek istenirse `reverse` yöntemi kullanılır.

```
from array import *
dizi = array('i', [1,2,3,4,1])
dizi.reverse()
print(dizi) #array('i', [1, 4, 3, 2, 1])
```

Bellekteki dizi tamponunun (buffer) başlangıç adresini ve dizideki öge sayısını `buffer_info` yöntemi verir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.buffer_info()) #(140514698296784, 5)
```

Bir öğenin bir dizide kaç kez görüldüğünü `count` yöntemi döndürecektir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.count(1)) #2
```

Bir metin (`string`) `encode` yöntemiyle her bir karakteri sayıya kodlanır. Ardından bayt dizisine `frombytes`

yöntemi ile eklenebilir. Tersine bir bayt dizisinden `decode` ile karakterlere dönüştürülecek sayılara dönüştürülebilir.

```
from array import array
baytdizizi = array('B') # 'b' yerine 'B' (unsigned char)
baytdizizi.frombytes('İlhan'.encode('utf-8'))
print(baytdizizi)
metin = baytdizizi.tobytes()
print(metin.decode('utf-8')) # İlhan
```

Dizilerde bir kısım hazır yani dahili (**built-in**) fonksiyonlar kullanılır;

```
from array import *
dizi = array('i', [1,2,3,4,1])
print('Eleman Sayısı:', len(dizi)) #5
print('En Küçük Eleman:', min(dizi)) #1
print('En Büyük Eleman:', max(dizi)) #4
```

Bu duruma örnek `len` fonksiyonunu içeren olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani sapma (**deviation**) ve karesi alınmış sapmalar yani varyans (**variance**) ile standart sapmayı yazdıran program örnek verilebilir.

```
from math import sqrt
satışlar=[100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0]
toplama=0.0
for satış in satışlar:
    toplama+=satış
ortalama=toplam/len(satışlar)
print('Ortalama', ortalama)
varyanstoplama=0.0
for satış in satışlar:
    sapma=satış-ortalama
    print('Sapma', sapma)
    varyanstoplama+=sapma*sapma
standartsapma=sqrt(varyanstoplama/len(satışlar))
print('Standart Sapma', standartsapma)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Ortalama 571.0
Sapma -471.0
Sapma 279.0
Sapma -71.0
Sapma 29.0
Sapma 179.0
Sapma 79.0
Sapma -121.0
Sapma 229.0
Sapma 329.0
Sapma -461.0
Standart Sapma 145.7810001337623
```

Bir dizide indis negatif kullanılabilir. Bu durumda son elemandan geriye doğru elemana erişilir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi[-2]) #4 #sondan ikinci eleman
print(dizi[-4]) #2 #sondan dördüncü eleman
```

Dizilerde **dilimleme** (**slicing**), belirli bir diziden alt dizi oluşturmanın bir yoludur. Bu işlemde, dizinin bir kısmını veya parçasını çıkarırız. Python'da dilimleme işlemini gerçekleştirmek için dilim işlecini `[:]` kullanırız.

```
from array import *
dizi = array('i', [1,2,3,4,1])
altdizi = dizi[0:2] #ilk elemandan itibaren 2. indise kadar elemanı alarak alt dizi oluştur
print(altdizi) #array('i', [1, 2])
altdizi2 =dizi[1:3] #ikinci elemandan itibaren 3. indise kadar elemanı alarak alt dizi oluştur
print(altdizi2) #array('i', [2, 3])
```

Dilimlemede negatif indisler de kullanılabilir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
altdizi = dizi[-3:-1] # sondan 3. elemandan sondan 1. elamana kadar alt dizi oluřtur
print(altdizi) # array('i', [3, 4])
altdizi2 =dizi[-4:-2] # sondan 4. elemandan sondan 2. elamana kadar alt dizi oluřtur
print(altdizi2) # array('i', [2, 3])
```

8.2 Liste (list)

Listeler (**list**) dizilerden farklı olarak homojen veri iermezler. Yani ařağıdaki gibi elemanları her tipten nesne olabilir ve aynı yöntemlere sahiptir.

```
liste=[1, 'a', 3.14, 1+2j, [4,5,6], (7,8,9)]
print(liste) #[1, 'a', 3.14, (1+2j), [4, 5, 6], (7, 8, 9)]
```

C/C++, C# ve Java dillerinin aksine iki boyutlu diziler yani matrisler (**matrix**), liste ieren listeler řeklinde tanımlanabilir.

```
matris=[[1,2,3],[4,5,6],[7,8,9]]
print(matris[0]) #[1, 2, 3] # birinci satır
print(matris[1]) #[4, 5, 6] # ikinci satır
print(matris[2]) #[7, 8, 9] # üçüncü satır
print(matris[0][1]) #2 # birinci satır ikinci sütun elemanı
print(matris[1][1]) #5 # ikinci satır ikinci sütun elemanı
print(matris[2][1]) #8 # üçüncü satır ikinci sütun elemanı
matris[2][2]=100
print(matris) #[[1, 2, 3], [4, 5, 6], [7, 8, 100]]
```

Benzer řekilde üç boyutlu ya da çok boyutlu diziler tanımlanabilir.

```
üçboyutlu=[[11,12,13],[14,15,16],[17,18,19]],\
            [[21,22,23],[24,25,26],[27,28,29]],\
            [[31,32,33],[34,35,36],[37,38,39]]]
print(üçboyutlu[0]) # [[11, 12, 13], [14, 15, 16], [17, 18, 19]] # ilk matris
print(üçboyutlu[1]) # [[21, 22, 23], [24, 25, 26], [27, 28, 29]] # ikinci matris
print(üçboyutlu[2][0]) # [31, 32, 33] # üçüncü matrisin ilk satırı
print(üçboyutlu[1][0][1]) # 22 # ikinci matrisin ilk satırının ikinci sütun elemanı
```

Dizilerde olan ve dizi ve liste nesnelerinin kendine has yöntemleri vardır;

Yöntem	Açıklama
<code>append()</code>	Listenin sonuna bir eleman ekler
<code>clear()</code>	Tüm elemanları siler
<code>copy()</code>	Listenin bir kopyasını döndürür
<code>count()</code>	Belirtilen değerde listenin sahip olduėu eleman sayısını döndürür
<code>extend()</code>	Listeye bir başka liste ekleyerek genişletir
<code>index()</code>	Verilen bir değerin listede bulunan ilk indisini döner
<code>insert()</code>	Belirtilen indis konumuna yeni bir eleman ekler
<code>pop()</code>	Belirtilen indis konumundan bir elemanı siler
<code>remove()</code>	Belirtilen değerdeki ilk elemanı siler
<code>reverse()</code>	Listeyi ters çevirir
<code>sort()</code>	Değerlere göre listeyi sıralar

Tablo 8.2: Dizi ve Liste Nesnesi Yöntemleri

Liste yöntemleri, bir listenin son giren elemanı ilk çıkar (**Last In First Out-LIFO**) mantığı yani yığın (**stack**) mantığında kullanmayı da sağlar;

```
yığın=[1,2,3]
yığın.append(4) # sonuna 4 eklendi
yığın.append(5) # sonuna 5 eklendi
çıkan=yığın.pop()
print(çıkan) #5
```

Bir listedeki yinelenen değerleri kaldırmak, listeyi bir kümeye (yani, farklı nesnelerin sırasız bir koleksiyonu) dönüřtürerek yapılabilir.

```
meyveler = ["armut", "elma", "erik", "elma", "armut"]
```

```
tekilmeyveler=list(set(meyveler))
print(tekilmeyveler) #['erik', 'armut', 'elma']
```

Liste ve dizilerde elemanlar üzerinde yinelemeler (iteration) aşağıdaki şekilde yapılabilir;

```
liste=[10,20,30,40,50]
for (indis,eleman) in enumerate(liste):
    print('{}.indisteki eleman={}'.format(indis,eleman))
for i in range(0,len(liste)):
    print(liste[i])
```

Bu kod çalıştığında vereceği çıktı aşağıda verilmiştir;

```
0.indisteki eleman=10
1.indisteki eleman=20
2.indisteki eleman=30
3.indisteki eleman=40
4.indisteki eleman=50
10
20
30
40
50
```

Listede bir elemanın varlığı **in** işleci (**operator**) ile test edildiği daha önce anlatılmıştı. Bir yineleme (**iterable**) içerisinde tüm değerlerin **True** olarak değerlendirip değerlendirmedeğini belirlemek için dahili **all()** fonksiyonunu kullanabilirsiniz.

```
liste=[1,2,3,0] #en az bir elemanı 0
all(liste) #False
liste=['a','b',0] #en az bir elemano 0
all(liste) #False
liste=['a','b','c'] #hiç 0 değerli elemanı yok
all(liste) #True
```

Benzer şekilde, dahili **any()** fonksiyonu bir yinelemede bir veya daha fazla değer **True** olarak değerlendirip değerlendirmedeğini belirler.

```
liste=['a','b','c']
any(liste) #True
liste=[0,0,0] #Tüm değerler 0
any(liste) #False
```

Bir listeyi tersine döndüren yani **reversed()** dahili fonksiyonunu kullanarak tersine bir yineleyici elde ederiz.

```
sayılar=[1,2,3,4]
for eleman in reversed(sayılar):
    print(eleman, end='-') #4-3-2-1-
terstensayılar=sayılar[::-1] #yileleyici değil de tersine bir dizi için ::-1 kullanılır
print(terstensayılar) # [4, 3, 2, 1]
```

Bir **liste kavrama (list comprehension)**, bir ifade (**expression**) ile liste üzerinde yineleme ile her bir elemana uygulayarak yeni bir liste oluşturur. Sözde kodu aşağıda verilmiştir;

```
[ <expression> for <element> in <iterable> ]
#yada
[ <expression> for <element> in <iterable> if <condition> ]
```

Aşağıda bir listedeki sayıların karelerinden yeni bir liste oluşturan örnek verilmiştir;

```
liste=[1,2,3,4,5]
kareler=[x*x for x in liste]
print(kareler) #[1, 4, 9, 16, 25]
kelimeler=['ilhan','Özkan','trabzon']
büyükkelimeler=[s.upper() for s in kelimeler]
print(büyükkelimeler) #['ILHAN', 'ÖZKAN', 'TRABZON']
onakadarçiftsayılar=[x for x in range(10) if x % 2 == 0]
print(onakadarçiftsayılar) # [0, 2, 4, 6, 8]
yıldızlı=[x if x in 'aeiou' else '*' for x in 'Şifre']
print(yıldızlı) # ['*', 'i', '*', '*', 'e']
```

İki liste **+** işleci ile birleştirilebilir.

```
liste1=[1,2,3]
liste2=['a','b','c']
birleşikliste=liste1+liste2
print(birleşikliste) #[1, 2, 3, 'a', 'b', 'c']
```

Dahili **sıkıştırma (zip)** fonksiyonu, iki listenin karşılıklı elemanlarından oluşan bir **demet (tuple)** üretir. Eleman sayısı az olan kadar demet üretilir.

```
liste1=[1,2,3] #3 elemanlı
liste2=['a','b','c'] #üç elemanlı
liste3=[10.5, 20.6] #iki elemanlı
for x in zip(liste1,liste2):
    print(x)
for x in zip(liste1,liste3):
    print(x)
```

Programın çıktısı aşağıdaki gibi olacaktır;

```
(1, 'a')
(2, 'b')
(3, 'c')
(1, 10.5)
(2, 20.6)
```

Bir listenin belirli sayıda elemanı olması isteniyorsa aşağıdaki gibi tanımlanabilir.

```
liste1 = [None] * 10 #veri tipi belirlenmemiş 10 elemanlı liste
print(liste1) #[None, None, None, None, None, None, None, None, None, None]
liste2= ['a']*5 #veri tipi str belirlenmiş 5 elemanlı liste
print(liste2) #['a', 'a', 'a', 'a', 'a']
```

8.3 Metin (str)

Python dilinde metinleri işlemek için **metin (string)** veri yapısı **str** ile tanımlanır. Metinler, Unicode karakterlerinin sıralı bir dizisidir (**array**). C++ dilindeki **string** veri yapısı gibidir. Yani tüm metin nesnelerine ilişkin yöntemler, orijinal metni bozmaz ve yeni bir **str** nesnesi döndürür.

Aşağıdaki örnekte metin nesnesine ilişkin yöntemlerin birkaçı yer almaktadır.

```
metin=' ilhan özkan '
print(metin.lower()) # ilhan özkan
print(metin.upper()) # ILHAN ÖZKAN
print(metin.title()) # İlhan Özkan
print(metin.strip()) #ilhan özkan
print(metin.find('kan')) #9
print(len(metin)) #13 #Dahili fonksiyonlar da metinlerle kullanılabilir.
print(metin.islower()) #True
print(metin.isprintable()) #True
```

metinler üzerinde toplama (**+**) işleci sağ ve solundaki metni birleştirir. Çarpma (*****) işleci ise işlenenlerden (**operand**) biri tamsayı olmak kaydıyla metni belirtilen tamsayı kadar çoğaltır;

```
metin1='İlhan'
metin2='Özkan'
print(metin1,metin2) #İlhan Özkan
print(metin1+metin2) #İlhanÖzkan
print(metin1*2) #İlhanİlhan
print(metin1*4) #İlhanİlhanİlhanİlhan
print(2*metin1) #İlhanİlhan
```

Birleştirilecek metin değişmezleri (**literal**) yan yana yazılarak da metinler oluşturulabilir;

```
metin='İlhan' " " "Özkan" ',' ""Merhaba!""
print(metin) #İlhan Özkan,Merhaba!
```

metinlere tek veya çift tırnak arasında yazılan değişmezler ile başlatılır. Metnin içerisinde geçecek tek ve çift tırnak karakterleri için değişmez içinde C dilindeki gibi ters bölü karakteri ile belirtilir.

```
metin='İlhan\'ın soyadı "Özkan" dır'
print(metin) #İlhan'ın soyadı "Özkan" dır
```

Yöntem	Açıklama
<code>capitalize()</code>	İlk harfi büyüterek döndürür
<code>casefold()</code>	Tüm harfleri küçüğe çevirerek döndürür
<code>center()</code>	Metni belirtilen karakter sayısına göre ortasına yerleştirir
<code>count()</code>	Belirtilen metnin metin içerisinde kaç defa geçtiğini döndürür
<code>encode()</code>	Metnin kodlanmış (encoded) halini döndürür.
<code>endswith()</code>	Belirtilen metin ile bitiyorsa metni döndürür.
<code>expandtabs()</code>	Belirtilen sekme miktarına göre metin içerisindeki sekmeleri (\t) genişleterek döndürür.
<code>find()</code>	Belirtilen metni metin içerisinde arar.
<code>format()</code>	metin metnini biçimlendirme metni olarak kullanır. Biçimlendirilmiş metni döndürür.
<code>format_map()</code>	metni bir sözlükten alınan değerler ile biçimlendirir
<code>index()</code>	metinde belirtilen bir değeri arar ve bulunduğu konumu döndürür
<code>isalnum()</code>	metindeki tüm karakterler alfasayısal (alphanumeric) ise True değerini döndürür
<code>isalpha()</code>	metindeki tüm karakterler alfabetik ise True değerini döndürür
<code>isascii()</code>	metindeki tüm karakterler ASCII karakterleriyse True değerini döndürür
<code>isdecimal()</code>	metindeki tüm karakterler ondalık sayıysa True değerini döndürür
<code>isdigit()</code>	metindeki tüm karakterler rakamsa True değerini döndürür
<code>isidentifier()</code>	metin bir değişken kimliği (identifier) ise True değerini döndürür
<code>islower()</code>	metindeki tüm karakterler küçük harf ise True değerini döndürür
<code>isnumeric()</code>	metindeki tüm karakterler sayısal ise True değerini döndürür
<code>isprintable()</code>	metindeki tüm karakterler yazıcıda yazdırılabilirse True değerini döndürür
<code>isspace()</code>	metindeki tüm karakterler beyaz boşluksa (whitespace) True değerini döndürür
<code>istitle()</code>	metin bir başlığın kurallarına (Her Sözcüğün İlk Harfi Büyük) uyuyorsa True değerini döndürür
<code>isupper()</code>	metindeki tüm karakterler büyük harf ise True değerini döndürür
<code>join()</code>	Bir yineleyicinin (iterable) öğelerini bir metne dönüştürür
<code>ljust()</code>	metnin sola yaslanmış halini döndürür
<code>lower()</code>	Bir metni küçük harfe dönüştürür
<code>lstrip()</code>	metnin soldan kırılmış (boşluğu alınmış) halini döndürür
<code>maketrans()</code>	Çevirilerde kullanılacak bir çeviri tablosu döndürür
<code>partition()</code>	Dizenin üç parçaya bölündüğü bir demet (tuple) döndürür
<code>replace()</code>	Belirtilen bir değerin bir başka değerle değiştirildiği bir metin döndürür
<code>rfind()</code>	metinde belirtilen bir değer aranır ve bulunduğu son konumu döndürür
<code>rindex()</code>	metinde belirtilen bir değer aranır ve bulunduğu son konumu döndürür
<code>rjust()</code>	metnin sağa yaslanmış bir sürümünü döndürür
<code>rpartition()</code>	metnin üç parçaya bölündüğü bir demet (tuple) döndürür
<code>rsplit()</code>	meti sondan başa doğru belirtilen ayırıcı ile böler ve bir liste döndürür
<code>rstrip()</code>	metnin sağdan kırılmış (boşlukları alınmış) şeklini döndürür
<code>split()</code>	metni belirtilen ayırıcı ile böler ve bir liste döndürür
<code>splitlines()</code>	metni satır sonlarında böler ve bir liste döndürür
<code>startswith()</code>	metin belirtilen değerle başlıyorsa True değerini döndürür
<code>strip()</code>	Dizenin kırılmış (boşlukları alınmış) şeklini döndürür
<code>swapcase()</code>	Harfleri değiştirir, küçük harf büyük harfe, büyük harf küçük harfe dönüşür
<code>title()</code>	Her kelimenin ilk karakterini büyük harfe dönüşmüş halini döndürür
<code>translate()</code>	metnin çevrilmiş halini döndürür
<code>upper()</code>	Bir metni büyük harfe dönüştürür
<code>zfill()</code>	metnin başını belirtilen sayıda 0 değerle doldurur

Tablo 8.3: Metin Yöntemleri

```
metin="İlhan\'ın soyadı \'Özkan\' dır"
print(metin) #İlhan'ın soyadı "Özkan" dır
```

Ters bölü ile belirtilen bu karakterlere **kaçış dizisi** (**escape sequence**) karakterleri adı verilir.

Karakter	Açıklama
\'	Tek Tırnak
\"	Çift Tırnak
\\	Ters Bölü
\n	Yeni satır (new line): İmleç (cursor), bir alt satırda satır başına döner.
\b	İmleç, bir karakter geri gider. Yani geri (backspace) tuşu ← davranışı sergiler.
\r	Satır başı (carriage return): İmleç, bulunduğu satırın başına döner.
\t	Sekme (Konsolda 8 ve katları sütuna ilerlenir): İmleç, → tuşu davranışı sergiler.
\f	Yeni sayfa (form feed): Yazıcıda yeni sayfaya geç anlamında kullanılır.
\ooo	Sekizlik (octal) Sayı Sisteminde Karakter Kodu
\xhh	Onaltılık (hexadecimal) Sayı Sisteminde Karakter Kodu

Tablo 8.4: Kaçış Dizisi Karakterleri

Aşağıda birkaç örnek verilmiştir;

```
metin = "\110\145\154\154\157"
print(metin)
metin = "\x48\x65\x6c\x6c\x6f"
print(metin)
metin="İlhan\nÖzkan" # \n ile konsolda satır başı yapılır
print(metin)
metin="İlhan\tÖzkan" #\t ile konsolda 8 ve katları olan sütuna ilerlenir.
print(metin)
metin="İlhan\bÖzkan" #\b ile konsolda son konumdan bir karakter geri ilerlenir.
print(metin)
metin="Merhaba\rİlhan" #\r ile konsolda mevcut satırın başına ilerlenir.
print(metin)
```

Örneğin çalıştırıldığında çıktısı aşağıdaki gibi olacaktır;

```
Hello
Hello
İlhan
Özkan
İlhan  Özkan
İlhaÖzkan
İlhanba
```

8.4 Demet (tuple)

Demet (tuple), değiştirilemez bir değerler listesidir ve aşağıdaki şekilde tanımlanabilirler.

```
demet1=() #elemanı olmayan bir demet (tuple)
demet2='a','b','c'
demet3='a',
demet4=('a') #bu ifade bir str oluşturur.
print(type(demet4)) #<class 'str'>
demet5=('a',)
demet6=tuple(range(5)) #yapıcı kullanarak aralık nesnesinden demet yapma
print(demet6) #(0, 1, 2, 3, 4)
demet7=tuple("Demet") #yapıcı kullanarak str nesnesinden demet yapma
print(demet7) #('D', 'e', 'm', 'e', 't')
```

Demeti oluşturan nesneler **değiştirilemez (immutable)**. Ancak yeni elemanlar ile genişletilebilir veya iki demet birleştirilebilir.

```
demet=(1,2,3,4)
#demet[2]=100 #TypeError: 'tuple' object does not support item assignment
baskaDemet=('a','b','c')
yeniDemet=demet+baskaDemet
print(yeniDemet) #(1, 2, 3, 4, 'a', 'b', 'c')
yeniDemet+=(1+4j,"İlhan")
```

```
print(yeniDemet) #(1, 2, 3, 4, 'a', 'b', 'c', (1+4j), 'İlhan')
```

Demetlere ilişkin yöntemler aşağıda verilmiştir.

Yöntem	Açıklama
<code>count()</code>	Belirtilen değerin demet içinde kaç kez bulunduğunu döndürür.
<code>index()</code>	Belirtilen değeri demet içinde arar ve bulunduğu yeri döndürür.

Tablo 8.5: Demet Yöntemleri

Bir demetin yalnızca bazı öğelerine ihtiyaç duyuluyorsa, altçizgi sembolü (`_`) geçici bir değişken adı olarak kullanılabilir ve **yer tutucu** (**placeholder**) olarak görev yapar.

```
demet = 1, 2, 3, 4
_, x, y, _ = demet # ikinci eleman x , üçüncü eleman y
if x == 2:
    print(x+y) #5
```

Benzer şekilde bir demetin bitişik birkaç öğesine ihtiyaç duyuluyorsa, yıldız sembolü (`*`) ile geçici bir değişken adı olarak kullanılabilir ve buna **paket açma** (**unpacking**) adı verilir.

```
demet = (1, 2, 3, 4)
a,*b,c = demet # ikinci ve üçüncü elemanı içeren bir liste b değişkenini oluşturur
print(b) # [2,3]
```

8.5 Küme (set)

Küme (**set**), herhangi bir veri tipinde ve **tekil** (**unique**) elemanlardan oluşan sırasız bir koleksiyondur. Bir nesne, ömrü boyunca asla değişmeyen bir **karma** (**hash**) değerine sahipse (ki bu nesne bir `hash()` yöntemine ihtiyaç duyar) ve diğer nesnelerle karşılaştırılabilirse (bir `eq()` yöntemine ihtiyaç duyar) karmalanabilir (**hashable**). Eşitliği karşılaştıran karmalanabilir nesnelerin, aynı karma değerine sahip olması gerekir. Karma, bir nesneye eşsiz bir sayı olarak karşılık gelen değerlerdir.

```
a = {1, 2, 'a'}
print(a)
# Çıktı: {1, 2, 'a'}
b = {1, 1, 2, 2} # bir değer bir kümede yalnızca 1 kez yer alabilir
print(b)
# Çıktı: {1, 2}
c = set() # boş küme oluşturma
print(c)
# Çıktı: set()
d = set([1, 2, 3, 3, 4]) # listeden küme oluşturma
print(d)
# Çıktı: {1, 2, 3, 4}
```

Kümeler sırasız olduğundan indeks kullanılarak erişilemez. Ancak **for** döngüsü ile elemanlar üzerinde gezinilebilir.

```
renkler = {'kırmızı', 'yeşil', 'mavi'}
for renk in renkler:
    print(renk)
# Çıktı: mavi
# Çıktı: yeşil
# Çıktı: kırmızı
# in işleci ile eleman kontrolü
'kırmızı' in renkler
# Çıktı: True
'sarı' in renkler
# Çıktı: False
```

Küme elemanlarına aşağıdaki şekilde ekleme ve çıkarma yapılabilir;

```
meyveler = {'elma', 'armut'}
print(meyveler)
# Çıktı: {'elma', 'armut'}
# add() - tek eleman ekleme
meyveler.add('muz')
print(meyveler)
```



```
# Çıktı: {'elma', 'armut', 'muz'}
# update() - birden fazla eleman ekleme
meyveler.update(['portakal', 'çilek'])
print(meyveler)
# Çıktı: {'elma', 'armut', 'muz', 'portakal', 'çilek'}
# remove() - eleman çıkarma (yoksa hata verir)
meyveler.remove('armut')
print(meyveler)
# Çıktı: {'elma', 'muz', 'portakal', 'çilek'}
# discard() - eleman çıkarma (yoksa hata vermez)
meyveler.discard('kayısı') # hata vermez
meyveler.discard('muz')
print(meyveler)
# Çıktı: {'elma', 'portakal', 'çilek'}
# pop() - rastgele bir eleman çıkarır
eleman = meyveler.pop()
print(f"Çıkarılan: {eleman}, Kalan: {meyveler}")
# Çıktı: Çıkarılan: elma, Kalan: {'portakal', 'çilek'}
# clear() - tüm elemanları siler
meyveler.clear()
print(meyveler)
# Çıktı: set()
```

Kümeler matematikten bildiğimiz küme işlemlerini destekler. Aşağıda örnekleri verilmiştir;

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
# Birleşim (union) - |
print(A | B)
# Çıktı: {1, 2, 3, 4, 5, 6, 7, 8}
print(A.union(B))
# Çıktı: {1, 2, 3, 4, 5, 6, 7, 8}
# Kesişim (intersection) - &
print(A & B)
# Çıktı: {4, 5}
print(A.intersection(B))
# Çıktı: {4, 5}
# Fark (difference) - -
print(A - B) # A'da olup B'de olmayanlar
# Çıktı: {1, 2, 3}
print(A.difference(B))
# Çıktı: {1, 2, 3}
print(B - A) # B'de olup A'da olmayanlar
# Çıktı: {6, 7, 8}
# Simetrik fark (symmetric_difference) - ^
print(A ^ B) # Ya A'da ya B'de ama ikisinde birden olmayanlar
# Çıktı: {1, 2, 3, 6, 7, 8}
print(A.symmetric_difference(B))
# Çıktı: {1, 2, 3, 6, 7, 8}
```

Alt küme işlemleri da aşağıdaki şekilde yapılır;

```
A = {1, 2, 3}
B = {1, 2, 3, 4, 5}
C = {1, 2}
# issubset() - alt küme mi?
print(A.issubset(B)) # A, B'nin alt kümesi mi?
# Çıktı: True
print(C.issubset(A)) # C, A'nın alt kümesi mi?
# Çıktı: True
# issuperset() - üst küme mi?
print(B.issuperset(A)) # B, A'nın üst kümesi mi?
# Çıktı: True
print(A.issuperset(C)) # A, C'nin üst kümesi mi?
```

```
# Çıktı: True
# isdisjoint() - kesişimleri boş mu?
D = {6, 7, 8}
print(A.isdisjoint(D)) # A ve D'nin ortak elemanı var mı?
# Çıktı: True
print(A.isdisjoint(B))
# Çıktı: False
```

Küme eleman sayısı ve küme kopyalama;

```
sayilar = {1, 2, 3, 4, 5}
# len() - eleman sayısı
print(len(sayilar))
# Çıktı: 5
# copy() - kümeyi kopyala
yeni_sayilar = sayilar.copy()
yeni_sayilar.add(6)
print(sayilar)
# Çıktı: {1, 2, 3, 4, 5}
print(yeni_sayilar)
# Çıktı: {1, 2, 3, 4, 5, 6}
```

Donmuş kümeler (frozenset) de tanımlanabilir. Donmuş kümeler, küme gibidir ancak elemanları **değiştirilemez (immutable)**. Bu sayede sözlüklerde anahtar veya başka bir kümenin elemanı olarak kullanılabilir.

```
donuk_kume = frozenset([1, 2, 3, 4])
print(donuk_kume)
# Çıktı: frozenset({1, 2, 3, 4})
# Değiştirilemez
# donuk_kume.add(5) # AttributeError hatası verir
# Küme işlemleri çalışır
diger_kume = frozenset([3, 4, 5, 6])
print(donuk_kume | diger_kume)
# Çıktı: frozenset({1, 2, 3, 4, 5, 6})
# Sözlükte anahtar olarak kullanılabilir
sozluk = {donuk_kume: "değer"}
print(sozluk)
# Çıktı: {frozenset({1, 2, 3, 4}): 'değer'}
```

Kümeye tekrarlanan elemanları kaldırma aşağıdaki şekilde yapılabilir;

```
liste = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
tekil_liste = list(set(liste))
print(tekil_liste)
# Çıktı: [1, 2, 3, 4]
```

İki farklı kümenin ortak elemanlarını bulma aşağıdaki şekilde yapılabilir;

```
ogrenci1_dersler = {'matematik', 'fizik', 'kimya', 'biyoloji'}
ogrenci2_dersler = {'fizik', 'kimya', 'tarih', 'edebiyat'}
ortak_dersler = ogrenci1_dersler & ogrenci2_dersler
print(ortak_dersler)
# Çıktı: {'fizik', 'kimya'}
```

İki kümenin farkını (birinde olup diğerinde olmayan elemanları) bulma aşağıdaki şekilde yapılabilir;

```
tum_dersler = ogrenci1_dersler | ogrenci2_dersler
print(tum_dersler)
# Çıktı: {'matematik', 'fizik', 'kimya', 'biyoloji', 'tarih', 'edebiyat'}
sadece_ogrenci1 = ogrenci1_dersler - ogrenci2_dersler
print(sadece_ogrenci1)
# Çıktı: {'matematik', 'biyoloji'}
```

8.6 Sözlük (dict)

Sözlük (dict), anahtar-değer çiftlerini (**key-value pair**) tekil olarak tutan bir koleksiyondur. Anahtarlar karmalanabilir (**hashable**).

```
a = {1: 'Elma', 2: 'Armut', 3: 'Muz'}
```

```
print(a)
# Çıktı: {1: 'Elma', 2: 'Armut', 3: 'Muz'}
b = {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
print(b)
# Çıktı: {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
# Boş sözlük oluşturma
c = {}
print(c)
# Çıktı: {}
d = dict()
print(d)
# Çıktı: {}
```

Sözlük elemanlarına erişim aşağıdaki şekilde yapılabilir;

```
ogrenci = {'isim': 'Ali', 'yas': 20, 'bolum': 'Bilgisayar'}
# Köşeli parantez ile erişim
print(ogrenci['isim'])
# Çıktı: Ali
print(ogrenci['yas'])
# Çıktı: 20
# get() metodu ile erişim
print(ogrenci.get('bolum'))
# Çıktı: Bilgisayar
print(ogrenci.get('not')) # Olmayan anahtar, None döner
# Çıktı: None
print(ogrenci.get('not', 0)) # Varsayılan değer belirtme
# Çıktı: 0
# Köşeli parantez ile olmayan anahtar KeyError verir
# print(ogrenci['not']) # KeyError: 'not'
```

Sözlüğe eleman ekleme ve güncelleme aşağıdaki şekilde yapılabilir;

```
kitap = {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan'}
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan'}
# Yeni anahtar-değer ekleme
kitap['yil'] = 2025
kitap['sayfa'] = 500
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yil': 2025, 'sayfa': 500}
# Mevcut değeri güncelleme
kitap['sayfa'] = 550
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yil': 2025, 'sayfa': 550}
# update() ile birden fazla ekleme/güncelleme
kitap.update({'dil': 'Türkçe', 'fiyat': 150})
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yil': 2025, 'sayfa': 550,
→ 'dil': 'Türkçe', 'fiyat': 150}
```

Sözlükten eleman silme aşağıdaki şekilde yapılabilir;

```
urun = {'isim': 'Laptop', 'fiyat': 15000, 'stok': 5, 'kategori': 'Elektronik'}
# del ile silme
del urun['stok']
print(urun)
# Çıktı: {'isim': 'Laptop', 'fiyat': 15000, 'kategori': 'Elektronik'}
# pop() - anahtarı siler ve değeri döner
kategori = urun.pop('kategori')
print(f"Silinen: {kategori}, Kalan: {urun}")
# Çıktı: Silinen: Elektronik, Kalan: {'isim': 'Laptop', 'fiyat': 15000}
# popitem() - son eklenen anahtar-değer çiftini siler (Python 3.7+)
son = urun.popitem()
print(f"Silinen çift: {son}, Kalan: {urun}")
# Çıktı: Silinen çift: ('fiyat', 15000), Kalan: {'isim': 'Laptop'}
```

```
# clear() - tüm elemanları siler
urun.clear()
print(urun)
# Çıktı: {}
```

Sözlükte anahtar, değer ve anahtar-değer çiftleri üzerinde işlemler aşağıdaki şekilde yapılabilir;

```
notlar = {'Ali': 85, 'Ayşe': 92, 'Mehmet': 78, 'Fatma': 95}
# keys() - tüm anahtarları döner
print(notlar.keys())
# Çıktı: dict_keys(['Ali', 'Ayşe', 'Mehmet', 'Fatma'])
# values() - tüm değerleri döner
print(notlar.values())
# Çıktı: dict_values([85, 92, 78, 95])
# items() - tüm anahtar-değer çiftlerini döner
print(notlar.items())
# Çıktı: dict_items([('Ali', 85), ('Ayşe', 92), ('Mehmet', 78), ('Fatma', 95)])
# Döngü ile gezinme
for isim in notlar.keys():
    print(isim)
# Çıktı: Ali
# Çıktı: Ayşe
# Çıktı: Mehmet
# Çıktı: Fatma
for not_degeri in notlar.values():
    print(not_degeri)
# Çıktı: 85
# Çıktı: 92
# Çıktı: 78
# Çıktı: 95
for isim, not_degeri in notlar.items():
    print(f"{isim}: {not_degeri}")
# Çıktı: Ali: 85
# Çıktı: Ayşe: 92
# Çıktı: Mehmet: 78
# Çıktı: Fatma: 95
araba = {'marka': 'Toyota', 'model': 'Corolla', 'yil': 2023}
# in operatörü - anahtar var mı?
print('marka' in araba)
# Çıktı: True
print('renk' in araba)
# Çıktı: False
# not in operatörü
print('fiyat' not in araba)
# Çıktı: True
# len() - eleman sayısı
print(len(araba))
# Çıktı: 3
```

İç içe sözlükler aşağıdaki gibi işlenir;

```
okullar = {
    'lise1': {
        'isim': 'Atatürk Lisesi',
        'ogrenci_sayisi': 500,
        'sehir': 'Ankara'
    },
    'lise2': {
        'isim': 'Fen Lisesi',
        'ogrenci_sayisi': 300,
        'sehir': 'İstanbul'
    }
}
# İç içe sözlüğe erişim
```

```
print(okullar['lise1']['isim'])
# Çıktı: Atatürk Lisesi
print(okullar['lise2']['ogrenci_sayisi'])
# Çıktı: 300
# Tüm okullarda gezinme
for okul_id, okul_bilgi in okullar.items():
    print(f"\n{okul_id}:")
    for anahtar, deger in okul_bilgi.items():
        print(f"  {anahtar}: {deger}")
# lise1:
#   isim: Atatürk Lisesi
#   ogrenci_sayisi: 500
#   sehir: Ankara
# lise2:
#   isim: Fen Lisesi
#   ogrenci_sayisi: 300
#   sehir: İstanbul
```

Sözlüğe varsayılan değer `setdefault()` yöntemi ile eklenir;

```
sayac = {}
# setdefault() - anahtar yoksa ekler ve değeri döner
print(sayac.setdefault('a', 0))
# Çıktı: 0
print(sayac)
# Çıktı: {'a': 0}
# Anahtar varsa mevcut değeri döner
sayac['a'] = 5
print(sayac.setdefault('a', 0))
# Çıktı: 5
print(sayac)
# Çıktı: {'a': 5}
# Kelime sayma örneği
metin = "merhaba dünya merhaba python python python"
kelime_sayaci = {}
for kelime in metin.split():
    kelime_sayaci[kelime] = kelime_sayaci.get(kelime, 0) + 1
print(kelime_sayaci)
# Çıktı: {'merhaba': 2, 'dünya': 1, 'python': 3}
```

Sözlüğü kopyalama aşağıdaki şekilde yapılabilir;

```
orijinal = {'a': 1, 'b': 2, 'c': 3}
# Sığ kopyalama (shallow copy)
kopya1 = orijinal.copy()
kopya1['a'] = 100
print(orijinal)
# Çıktı: {'a': 1, 'b': 2, 'c': 3}
print(kopya1)
# Çıktı: {'a': 100, 'b': 2, 'c': 3}
# dict() ile kopyalama
kopya2 = dict(orijinal)
print(kopya2)
# Çıktı: {'a': 1, 'b': 2, 'c': 3}
```

Listelerde olduğu gibi sözlüklerde de **kavrama** (**comprehension**) vardır. Kavrama ile, bir ifadenin sözlük üzerinde yineleme ile her bir elemana uygulanarak yeni bir sözlük oluşturulur.

```
# Basit sözlük comprehension
kareler = {x: x**2 for x in range(1, 6)}
print(kareler)
# Çıktı: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
# Koşullu sözlük comprehension
cift_kareler = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(cift_kareler)
# Çıktı: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

```
# İki listeden sözlük oluşturma
isimler = ['Ali', 'Ayşe', 'Mehmet']
yaslar = [25, 30, 28]
isim_yas = {isim: yas for isim, yas in zip(isimler, yaslar)}
print(isim_yas)
# Çıktı: {'Ali': 25, 'Ayşe': 30, 'Mehmet': 28}
# Sözlüğü ters çevirme
orijinal = {'a': 1, 'b': 2, 'c': 3}
ters = {deger: anahtar for anahtar, deger in orijinal.items()}
print(ters)
# Çıktı: {1: 'a', 2: 'b', 3: 'c'}
```

Aşağıda bir öğrenci sistemine ilişkin örnek verilmiştir;

```
ogrenci = {
    'numara': '12345',
    'ad': 'Ali',
    'soyad': 'Veli',
    'notlar': {
        'matematik': 85,
        'fizik': 78,
        'kimya': 92
    }
}
ortalama = sum(ogrenci['notlar'].values()) / len(ogrenci['notlar'])
print(f"{ogrenci['ad']} {ogrenci['soyad']}'nin ortalaması: {ortalama:.2f}")
# Çıktı: Ali Veli'nin ortalaması: 85.00
```

Sözlükle ilgili aşağıda bir başka telefon defteri örneği verilmiştir;

```
rehber = {
    'Ali': '555-1234',
    'Ayşe': '555-5678',
    'Mehmet': '555-9012'
}
# Arama
isim = 'Ayşe'
if isim in rehber:
    print(f"{isim}'nin telefonu: {rehber[isim]}")
else:
    print(f"{isim} rehberde yok")
# Çıktı: Ayşe'nin telefonu: 555-5678
```

8.7 Düşün ve Çöz

- Veri Tipi Değiştirilemezliği (Immutability) ve Karma (Hash):** Python'da bir **set** (küme) veya **dict** (sözlük) anahtarı olarak neden **list** kullanılamazken **tuple** kullanılabilir? **hashable** (karmalanabilir) kavramı çerçevesinde açıklayınız ve aşağıdaki kodun neden hata vereceğini belirtiniz:
 sozluk = {[1, 2]: "değer"}
- Derin Kopya (Deep Copy) vs. Sığ Kopya (Shallow Copy):** İç içe elemanlar içeren bir listenin (**liste = [[1, 2], [3, 4]]**) **copy()** metodu ile kopyası alındığında, kopya liste üzerindeki iç elemanların değiştirilmesi orijinal listeyi nasıl etkiler? İki yöntem arasındaki farkı açıklayarak kod üzerinde gösteriniz.
- Liste Comprehension ile Veri Dönüşümü:** Elimizde aşağıdaki gibi kelimelerden oluşan bir liste bulunmaktadır:
 kelimeler = ["elma", "ananas", "muz", "avokado", "çilek", "armut"]
 Sadece 'a' veya 'A' harfi ile başlayan kelimelerin uzunluklarını içeren ve kelimenin kendisini anahtar, uzunluğunu değer yapan bir sözlüğü (**dict comprehension**) tek satırda oluşturunuz.
- Performans Karşılaştırması (list vs set):** 1.000.000 elemanlı bir koleksiyonda bir elemanın varlığını kontrol etmek (**x in koleksiyon**) istediğimizde, bu işlemin **list** ve **set** veri yapıları arasındaki zaman karmaşıklığı ($O(n)$ ve $O(1)$) farkını açıklayınız.

9. Bölüm

İstisna Yönetimi

Programlama dünyasında hatalar kaçınılmazdır. Ancak bir yazılımcı için önemli olan hatanın oluşması değil, bu hatanın yönetilebilir olmasıdır. Programın çalışma anında (**runtime**) karşılaştığı beklenmedik durumlara **istisna** (**exception**) denir. İstisna yönetimi, programın bu hatalar karşısında tamamen çökmesini engellemek ve kullanıcıya anlamlı geri bildirimler vererek sistemin güvenli bir duruma (**safe state**) geçmesini sağlamak için kullanılır.

Kitabımızın başında belirttiğimiz "nelerin yapılacağını değil, işin nasıl yapılacağını" bilme ilkesi burada devreye girer. **İstisna yönetimi** (**exception handling**), sadece hataları gizlemek için değil, sistemin sınırlarını belirlemek için kullanılır.

9.1 try-except Bloğu

Python'da istisnalar **try-except** mekanizması ile kontrol edilir. **try** bloğu, hata potansiyeli taşıyan, riskli kodların yazıldığı bölümdür. **except** bloğu ise eğer **try** bloğu içinde bir hata oluşursa, programın akışı anında bu bloğa sıçrar. Söz dizimi aşağıda belirtilmiştir;

```
try:
    # Riskli kodlar bu blokta yazılır
    sayi = int(input("Bir sayı giriniz: "))
    sonuc = 100 / sayi
except ValueError:
    # Veri tipi uyumsuzluğunda bu blok çalışır
    print("Hata: Lütfen geçerli bir tam sayı giriniz!")
except ZeroDivisionError:
    # Sıfıra bölünme hatasında bu blok çalışır
    print("Hata: Bir sayıyı sıfıra bölemezsiniz!")
```

Bazı durumlarda **else** ve **finally** bloğu da kullanılır.

else Bloğu: Eğer **try** bloğunda hiçbir hata oluşmazsa çalışır. İşlemin başarıyla tamamlandığını teyit etmek için kullanılır.

finally Bloğu: Hata oluşsa da oluşmasa da mutlaka çalışır. Özellikle dosya kapatma, veri tabanı bağlantısı kesme veya donanım kaynaklarını serbest bırakma gibi "temizlik" işlemleri için hayati önem taşır.

Python dilinde diğer nesne yönelimli dillerde olduğu gibi tüm istisnalar aslında birer nesnedir (**object**) ve **BaseException** sınıfından türetilmiştir. Hataları yakalarken "özelden genele" doğru bir sıra izlenmelidir.

```
try:
    dosya = open("veriler.txt", "r")
    # ... işlemler ...
except FileNotFoundError:
    print("Dosya bulunamadı!")
except Exception as e:
    # Tüm hataları yakalayan genel sınıf
    print(f"Beklenmedik bir hata oluştu: {e}")
finally:
    print("Sistem kaynakları kontrol ediliyor...")
    # finally içinde dosya.close() çağrılırsa bu sefer UnboundLocalError fırlatılır.
```

Neden try-except Kullanmalıyız?

- ◊ **Süreklilik:** Programın küçük bir hata yüzünden tamamen durmasını engeller.
- ◊ **Güvenlik:** Kullanıcıya sistemin iç yapısını ele veren (**traceback**) karmaşık hata mesajları yerine, filtrelenmiş ve güvenli mesajlar gösterilmesini sağlar.
- ◊ **Hata Ayıklama (debug):** Hatanın nerede ve neden oluştuğuna dair iz kaydı (**log**) tutulmasını kolaylaştırır.

Python'da her hata aslında bize sistemin neresinde bir mantık veya kaynak sorunu olduğunu söyler. Aşağıdaki tablo, geliştirme sürecinde en sık karşılaşılabilecek istisnaları ve bunların bilgisayar mimarisindeki

karşılıklarını özetlemektedir;

İstisna (exception)	Ne Anlama Gelir?	Mühendisçe Yaklaşım
<code>SyntaxError</code>	Yazım hatası. Python'un dil kurallarına aykırı kod yazılmıştır.	Kod daha derleme/yorumlanma aşamasına geçememiştir.
<code>IndentationError</code>	Girintileme hatası. Blok yapısı bozulmuştur.	Python'un kapsam (scope) belirleme kuralı ihlal edilmiştir.
<code>NameError</code>	Tanımlanmamış bir değişken veya fonksiyon adı kullanılmıştır.	Bellekte (RAM) karşılığı olmayan bir isme referans verilmiştir.
<code>TypeError</code>	Bir veri tipi, desteklemediği bir işleme tabi tutulmuştur.	Nesne ara yüzü (interface) ile yapılan işlem uyumsuzdur.
<code>ValueError</code>	Veri tipi doğru olsa da içeriği (değeri) işlem için uygun değildir.	Fonksiyonun kabul edilebilir parametre sınırları dışına çıkmıştır.
<code>ZeroDivisionError</code>	Bir sayı sifıra bölünmeye çalışılmıştır.	Matematiksel bir tanımsızlık ve işlemci seviyesinde bir hata durumudur.
<code>IndexError</code>	Bir listenin veya dizinin sınırları dışında bir indise erişilmeye çalışılmıştır.	Bellekte tahsis edilen alanın (buffer) dışına taşma girişimidir.
<code>KeyError</code>	Sözlükte (dict) bulunmayan bir anahtar (key) çağrılmıştır.	Hash tablosunda aranan referans bulunamamıştır.
<code>FileNotFoundError</code>	Erişilmek istenen dosya belirtilen yolda mevcut değildir.	Dosya sistemi (File System) üzerinde bir girdi/çıkı (input/output-I/O) hatasıdır.
<code>AttributeError</code>	Bir nesneye, sahip olmadığı bir özellik veya metot ile erişilmeye çalışılmıştır.	Sınıf hiyerarşisi veya nesne yapısı yanlış kurgulanmıştır.

Tablo 9.1: İstisna Türleri

Python bir hata verdiğinde, terminalde geriye doğru izlenen (traceback) bir çıktı üretir. Son fonksiyon çağrılarları için bu bir yığın izidir (stack trace). En alt satır hatanın türünü ve nedenini söyler. Üstteki satırlar ise hatanın kodun hangi katmanlarından geçerek (fonksiyon çağrı yığını) buraya ulaştığını gösterir. Hata ayıklarken her zaman en alttan başlayıp yukarı doğru "yığın boyunca" takip yapmalısınız.

9.2 İstisnai Duruma Düşme

Şu ana kadar istisnaların Python yorumlayıcısı tarafından otomatik olarak oluşturulup sanal bir istisna havuzuna atılmasını (sıfıra bölme, hatalı tip vb.) ve bizim bunları `except` ile yakalamamızı işledik. Ancak profesyonel bir yazılım mimarisinde, bazen kod teknik olarak doğru olsa da mantıksal olarak hatalı bir durum oluşabilir. Bu durumda mühendis, hatayı kendisi tetiklemeli yani hataya ait istisna nesnesini kendisi imal edip istisna havuzuna atmalıdır (`raise`). İşte bu işleme **istisnai duruma düşme** (raising an exception) adı verilir.

Kendi istisna nesnelerimizi aşağıdaki durumlarda kullanabiliriz;

- ♦ **İş Kurallarını Denetlemek:** Örneğin, bir bankacılık uygulamasında bakiyenin altına düşen bir çekim işlemi teknik bir hata değildir ama mantıksal bir hatadır.
- ♦ **Hata Zinciri Oluşturmak:** Alt katmanlarda oluşan bir hatayı yakalayıp, daha üst katmana daha anlamlı bir mesajla iletmek için.
- ♦ **Zorunlu Alan Kontrolü:** Fonksiyonun çalışması için kritik olan verilerin eksikliği durumunda süreci durdurmak için.

Kullanıcı tanımlı bir istisna nesnesi `raise` talimatı ile verilir. Söz dizimi aşağıda verilmiştir;

```
raise <İstisnaTürü>("Hata Mesajı")
```

Buradaki istisna türü Tablo 9.1'de verilen istisna sınıflarından biri olabilir. Örnek olarak; bir mühendislik projesinde, bir sıcaklık sensöründen gelen verinin fiziksel sınırlar (-50°C ile $+150^{\circ}\text{C}$) dışında olması durumunda sistemin durdurulması gerektiğini varsayalım. Buna ilişkin kod aşağıda verilmiştir;

```
def sıcaklik_kontrol_et(deger):
    if deger < -50 or deger > 150:
        # Teknik bir hata yok ama fiziksel bir imkansızlık/hata var
        raise ValueError(f"Geçersiz sıcaklık verisi: {deger}. Sensör arızalı olabilir!")
    print(f"Sıcaklık normal: {deger}")

try:
    sıcaklik_kontrol_et(200)
```

```
except ValueError as e:
    print(f"Sistem Durduruldu: {e}")

Bazen bir hatayı yakalayıp, üzerine kendi açıklamamızı ekleyerek yeni bir istisna nesnesi imal ederiz.
Buna hata zincirleme (exception chaining) denir.

try:
    # Veritabanına bağlanma denemesi
    baglanti_kur()
except ConnectionError as orijinal_hata:
    # Hatayı yakalayıp daha üst seviye bir mesajla fırlatıyoruz
    raise RuntimeError("Veritabanı erişimi başarısız, sistem başlatılamıyor.") from orijinal_hata
```

9.3 Kendi İstisna Sınıflarımızı Oluşturma

Büyük projelerde Python'un standart hata isimleri (**ValueError**, **TypeError** vb.) yetersiz kalabilir. Bu durumda, kitabımızın kalıtım (**inheritance**) prensibini kullanarak kendi hata/istisna sınıflarımızı (**custom exception**) oluşturabiliriz.

```
class SensorArizasi(Exception):
    """Sensör donanım hataları için özel istisna sınıfı"""
    pass

# Kullanımı:
raise SensorArizasi("Donanımdan sinyal alınamıyor!")

class YasHatasiError(ValueError):
    """Geçersiz yaş değeri hatası"""
    def __init__(self, yas, mesaj="Yaş 0-150 arası olmalı"):
        self.yas = yas
        self.mesaj = f"{mesaj}. Girilen: {yas}"
        super().__init__(self.mesaj)
```

9.4 Düşün ve Çöz

1. **Düşün (Sessiz Hata Tehlikesi):** Aşağıdaki kod bloğunda yapılan mimari hata nedir? Bütün hataları boş bir **except:** bloğu ile yakalayıp yutmak (**bare except / swallowed exception**) yazılımın güvenilirliği ve hata ayıklama (**debug**) süreçleri açısından ne tür tehlikeler doğurur?


```
try:
    veriyi_isle()
except:
    pass # Hata göz ardı ediliyor
```
2. **Çöz (Özel İstisna Sınıfı):** Bir ATM yazılımı geliştirdiğinizi varsayın. Bakiyeden fazla para çekilmek istendiğinde **YetersizBakiyeError**, günlük para çekme limiti aşıldığında ise **LimitAsildiError** adında iki farklı özel istisna fırlatan bir **BankaHesabi** sınıfı tasarlayınız. Bu sınıfta **para_cek(miktar)** metodunu yazarak uygun durumlar için ilgili istisnaları **raise** ediniz.
3. **Düşün (else vs. try Kapsamı):** **try** bloğunun mümkün olduğunca dar tutulması ("dar kapsamlı try") neden iyi bir mühendislik pratiğidir? Hata riski taşımayan ancak işlemler başarılı olduğunda çalışması gereken kodları **try** bloğu içine yazmak yerine **else** bloğuna yazmanın avantajını açıklayınız.
4. **Çöz (Hata Zincirleme - Exception Chaining):** Kullanıcıdan alınan bir dosya yolundaki verileri okuyup tam sayıya dönüştüren bir program yazınız. Dosya bulunamadığında veya içerik tam sayıya dönüştürülemediğinde, bu alt seviye hataları yakalayıp kendi tanımlayacağınız **VeriOkumaHatasi** adlı üst seviye bir istisna sınıfı ile **from** anahtar kelimesini kullanarak zincirleyip (**chaining**) fırlatınız.

10. Bölüm

Python Standart Kütüphanesi

Gerçek dünya kullanım senaryolarında bu kütüphane olmazsa olmazdır. Standart kütüphanedeki (**standard library**) çok kullanılan modüller; `datetime`, `os`, `sys`, `json` ve `math` modülleridir.

10.1 Zaman ve Tarih Yönetimi

Mühendislik uygulamalarında, verinin "ne" olduğu kadar "ne zaman" üretildiği de kritiktir. Bir sensör verisinin zaman damgası (**timestamp**), bir işlemin süresi veya bir sistemin log kayıtları için zaman yönetimi hataya yer bırakmamalıdır. Python, bu karmaşıklığı yönetmek için `datetime` modülünü sunar.

§10.1.1 Zamanın Temsili ve Temel Sınıflar

`datetime` modülü, zamanı farklı katmanlarda ele alan dört ana sınıftan oluşur:

- ◇ **date**: Sadece yıl, ay ve gün bilgilerini tutar (Takvim verisi).
- ◇ **time**: Sadece saat, dakika, saniye ve mikrosaniye bilgilerini tutar (saat verisi).
- ◇ **datetime**: Hem tarih hem de saat bilgilerini bir arada tutar.
- ◇ **timedelta**: İki zaman noktası arasındaki farkı (süreyi) temsil eder.

§10.1.2 Sistem Zamanına Erişim

Bir mühendis için en temel işlem, sistemin mevcut zamanını okumaktır.

```
from datetime import datetime
```

```
simdi = datetime.now()
print(f"Sistem Zamanı: {simdi}")
print(f"Yıl: {simdi.year}, Ay: {simdi.month}, Gün: {simdi.day}")
```

§10.1.3 Zaman Aritmetiği

Zaman üzerinde toplama ve çıkarma işlemleri yapmak için `timedelta` sınıfı kullanılır. Bu sınıf, özellikle periyodik işlemler (örneğin; "bundan 30 gün sonra ne olacak?") için kullanılır.

```
from datetime import datetime, timedelta
```

```
bugun = datetime.now()
bir_hafta_sonra = bugun + timedelta(days=7)
print(f"Bir hafta sonraki tarih: {bir_hafta_sonra.date()}")
```

Zaman aritmetiğinde doğrudan tam sayı toplamak yerine `timedelta` kullanmak, artık yıllar (**leap years**) ve ayların gün sayıları gibi karmaşık takvim hesaplamalarını Python'un otomatik yapmasını sağlar.

§10.1.4 Biçimlendirme

Zaman verisi genellikle dış dünyadan metin (**string**) olarak gelir veya dış dünyaya metin olarak sunulur. İki fonksiyon biçimlendirme için kullanılır:

- ◇ **strftime(string format time)**: Zaman objesini metne dönüştürür.
- ◇ **strptime(string parse time)**: Metin halindeki zamanı objeye dönüştürür.

```
# Obje -> Metin
print(bugun.strftime("%d/%m/%Y %H:%M:%S")) # 16/02/2026 14:30:00
```

```
# Metin -> Obje
tarih_metni = "2026-05-19"
tarih_objesi = datetime.strptime(tarih_metni, "%Y-%m-%d")
```

§10.1.5 Zaman Dilimleri

Mühendislikte en çok yapılan hatalardan biri, zaman dilimini (**timezone**) göz ardı etmektir.

- ◇ **Saf (naive)**: Zaman dilimi bilgisi içermeyen, sadece yerel saati gösteren objelerdir.
- ◇ **Farkında (aware)**: UTC (Evrensel Zaman) referans alınarak zaman dilimi bilgisi eklenmiş objelerdir.

Küresel sistemlerde her zaman UTC kullanılması bir mühendislik standardıdır. Örnek olarak bir algoritmanın çalışma hızını ölçmek için `datetime` farklarından yararlanabiliriz:

```
baslangic = datetime.now()
# ... Algoritma veya işlem ...
toplam = sum(range(1000000))
bitis = datetime.now()
gecen_sure = bitis - baslangic
print(f"İşlem süresi: {gecen_sure.total_seconds()} saniye.")
```

10.2 Metin İşleme ve Düzenli İfadeler

Yazılım dünyasında ham metin verisi (**raw text**) genellikle düzensizdir. Bir mühendis için binlerce satırlık bir hata iz kaydından (**log**) sadece IP adreslerini ayıklamak veya bir veri setindeki tarih formatlarını standart hale getirmek manuel olarak imkansızdır. Düzenli ifadeler (**regex**), metinler içerisinde belirli desenleri tanımlamak ve bu desenlere uyan verileri bulmak, değiştirmek veya doğrulamak için kullanılan özelleşmiş bir dildir.

Python'da bu işlemler için standart kütüphanede yer alan **re** modülü kullanılır.

§10.2.1 Temel Karakterler ve Belirleyiciler

Regex, kendine has bir sembol dili kullanır. Belirleyiciler (**meta character**) aşağıda verilmiştir:

Desen (pattern)	Açıklama
.	Herhangi bir karakteri temsil eder.
^	Satır başı karakterini temsil eder.
\$	Satır sonu karakterini temsil eder.
*	Bir karakterin sıfır veya daha fazla tekrarını temsil eder.
+	Bir karakterin bir veya daha fazla tekrarını temsil eder.
?	Bir karakterin sıfır veya bir kez tekrarını temsil eder.
{n}	Bir karakterin tam <i>n</i> kez tekrarını temsil eder.
[abc]	<i>a</i> , <i>b</i> veya <i>c</i> karakterlerinden birini temsil eder.
\d	Herhangi bir rakamı (0-9) temsil eder.
\w	Herhangi bir alfanümerik karakteri (harf, rakam, alt çizgi) temsil eder.
\s	Boşluk karakterini temsil eder.

§10.2.2 re Modülü Temel Fonksiyonları

Python'da en sık kullanılan **Regex** yöntemleri şunlardır:

- ◇ **re.search()**: Metin içinde deseni arar ve ilk bulduğu konumu (match object) döndürür.
- ◇ **re.findall()**: Desene uyan tüm kısımları bir liste olarak döndürür.
- ◇ **re.sub()**: Desene uyan kısımları başka bir metinle değiştirir.
- ◇ **re.compile()**: Bir deseni önceden derleyerek (pre-compile) performans artışı sağlar.

Gruplandırma ve Veri Ayıklama

Parantez **()** kullanarak desenin belirli kısımlarını gruplandırabilir ve sadece o gruplara erişebiliriz.

E-posta doğrulama ve ayıklama örneği verilmiştir:

```
import re

metin = (
    "İletişim için ilhan.ozkan@kurum.com veya test_kullanici@gmail.com kullanın."
)
# E-posta deseni: [karakterler]@[alan_adi].[uzanti]
desen = r"([\w.-]+)@([\w.-]+)"
eslesmeler = re.findall(desen, metin)

for kullanıcı, alan in eslesmeler:
    print(f"Kullanıcı: {kullanıcı}, Alan Adı: {alan}")
```

Açgözlü Eşleşme

Regex varsayılan olarak açgözlüdür (**greedy**), yani bulabildiği en uzun eşleşmeyi alır. Mühendislik uygulamalarında (örneğin HTML etiketlerini ayıklarken) bazen en kısa eşleşmeye ihtiyaç duyarız. Bunun için **?** operatörü kullanılır.

Regex ifadeleri güçlüdür ancak karmaşıklıkla okunabilirliği ve bakım maliyeti (**maintenance cost**) artırır. Bir mühendis olarak, çok karmaşık bir regex yazmak yerine, işlemi mantıksal adımlara bölmek veya açıklayıcı yorumlar eklemek (**verbose modu**) kodun kalitesini artıracaktır.

Derlenmiş Desenler ve Performans

Eğer aynı desen binlerce satırlık bir veri üzerinde döngü içinde kullanılacaksa, deseni bir kez derlemek işlemci zamanından tasarruf sağlar.

```
# Deseni bir kez derle
ip_deseni = re.compile(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}")

# Döngü içinde hızlıca kullan
with open("access.log", "r") as f:
    for satir in f:
        if ip_deseni.search(satir):
            # ... işlem ...
            pass
```

10.3 Dosya Yönetimi ve Bağlam Yöneticileri

Bir yazılımın dış dünya ile veri alışverişi yapmasının en temel yolu dosya sistemidir. Mühendislik uygulamalarında iz kayıtları, konfigürasyon dosyaları veya sensör verilerinin kalıcı hale getirilmesi için dosya işlemleri hayati önem taşır. Ancak dosyalar, işletim sistemi tarafından sınırlı sayıda sunulan kaynaklardır (**resource**). Açılan her dosyanın işi bittiğinde kapatılması, dosya ile işlemlerin en kısa sürede bitirilmesi, sistem kaynaklarının tükenmemesi için bir zorunluluktur.

§10.3.1 Dosya Açma ve Erişim Modları

Python'da bir dosyaya erişmek için `open()` fonksiyonu kullanılır. Bu fonksiyon, işletim sistemi seviyesindeki dosya tanıtıcısı (**file descriptor**) nesnesine bir referans oluşturur.

```
# Temel kullanım (Önerilmeyen yöntem)
dosya = open("veriler.txt", "w")
dosya.write("Mühendislik Verisi")
dosya.close() # Kapatma unutulursa kaynak sızıntısı oluşur!
```

Erişim Modları:

- ◊ **'r'**: Okuma (**read**) modu. Dosya yoksa istisna oluşur.
- ◊ **'w'**: Yazma (**write**) modu. Dosya varsa içeriği siler, yoksa oluşturur.
- ◊ **'a'**: Ekleme (**append**) modu. Dosyanın sonuna veri ekler.
- ◊ **'b'**: Verilerin bilgisayar belleğindeki gibi tutulduğu ikili (**binary**) mod. Resim, ses veya derlenmiş dosya işlemleri için kullanılır.
- ◊ **'r+'**: Hem okuma hem de yazma modu.
- ◊ **'rb'**: İkili dosyayı okuma modu.
- ◊ **'wb'**: İkili dosyayı yazma modu.

§10.3.2 Bağlam Yöneticileri

Mühendislikte "hata toleransı" esastır. Eğer dosya açıkken bir hata oluşursa, klasik yöntemle `close()` komutu hiçbir zaman çalışmayabilir. Python bu riski ortadan kaldırmak için bağlam yöneticisi (**context manager**) yapısını sunar. Bu yapı sayesinde dosya `with` bloğu kullanıldığında, işlem bittikten sonra veya bir hata oluşsa dahi dosya otomatik olarak kapatılır.

```
with open("deney_sonuclari.csv", "r") as dosya:
    icerik = dosya.read()
    # İşlem bittiğinde sistem dosyayı otomatik kapatır.
```

`with` kullanımı, C++ dilindeki Kaynak Edinimi Başlatma İşlemdir (**Resource Acquisition Is Initialization - RAII**) prensibine benzer. Kaynak kullanımı, kodun kapsamı (**scope**) ile sınırlıdır.

§10.3.3 Dosya Okuma Teknikleri

Büyük veri setleriyle (**big data**) çalışırken tüm dosyayı bir kerede belleğe (RAM) yüklemek sistemi kilitleyebilir. Mühendisçe yaklaşım, veriyi satırlar halinde işlemektir. Genelde gözle okunabilir metin dosyalarını işlerken kullanılır.

```
with open("buyuk_veri_seti.txt", "r") as dosya:
    for satir in dosya:
```

```
# Her adımda sadece bir satır bellekte tutulur
print(satir.strip())
```

§10.3.4 Yapılandırılmış Veri Saklama

Standart metin dosyaları yerine, verileri yapılandırılmış (**structured**) bir formatta saklamak veri bütünlüğü için önemlidir. Verileri gözle okunabilir şekilde barındıran ve en çok kullanılan formatlar XML (**Extensible Markup Language**) ve JSON (**JavaScript Object Notation**) formatıdır. Python dilinin **json** modülü ile nesneleri doğrudan dosyaya kaydedebiliriz.

```
import json

veriler = {"sensor_id": 101, "sicaklik": 24.5}
with open("config.json", "w") as f:
    json.dump(veriler, f) # Veriyi JSON formatında dosyaya yazar
```

§10.3.5 Dosya ve Dizin Kontrolleri

Bir dosyayı açmadan önce varlığını kontrol etmek, programın çökmesini engeller. Bu durumu **os** nesnesi ile kontrol edebiliriz.

```
import os

if os.path.exists("rapor.txt"):
    print("Dosya zaten mevcut.")
else:
    print("Yeni dosya oluşturuluyor...")
```

10.4 Sistem Parametreleri ve Etkileşimi

Python'un **sys** modülü, yorumlayıcının () çalışma anındaki (**runtime**) ortamına ve sistem kaynaklarına doğrudan erişim sağlar. **os** modülü işletim sistemiyle ilgilenirken, **sys** modülü daha çok Python'un kendisi ve bilgisayar donanımı arasındaki düşük seviyeli parametrelerle ilgilenir. Bir mühendis için bu modül, bellek yönetimi ve sistem entegrasyonu açısından vazgeçilmezdir.

sys modülü, Python'un soyutlama (**abstraction**) seviyesinden bir adım aşağı inerek donanıma yaklaşmamızı sağlar. Özellikle bellek optimizasyonu ve terminal tabanlı araçlar geliştirirken bu modülün sunduğu parametreler yazılımın performansını doğrudan etkiler.

§10.4.1 Komut Satırı Argümanları

Yazılan bir betiğin (**script**), terminal üzerinden dışarıdan parametreler alarak çalışması gerekebilir. **sys.argv**, komut satırından girilen argümanları bir liste olarak saklar.

- ◊ **sys.argv[0]**: Çalıştırılan dosyanın adıdır.
- ◊ **sys.argv[1:]**: Dışarıdan girilen parametrelerdir.

```
import sys

# terminal: python hesapla.py 10 20
print(f"Çalışan dosya: {sys.argv[0]}")
if len(sys.argv) > 2:
    sayi1 = int(sys.argv[1])
    sayi2 = int(sys.argv[2])
print(f"Toplam: {sayi1 + sayi2}")
```

§10.4.2 Bellek Yönetimi

Bir mühendis için yazdığı kodun ne kadar bellek (RAM) tükettiği hayati önem taşır. **sys.getsizeof()** fonksiyonu, bir Python nesnesinin bellekte kapladığı alanı bayt (**byte**) cinsinden döndürür.

```
import sys

liste = [i for i in range(1000)]
sozluk = {i: i for i in range(1000)}
print(f"Listenin bellek boyutu: {sys.getsizeof(liste)} byte")
print(f"Sözlüğün bellek boyutu: {sys.getsizeof(sozluk)} byte")
```


§10.4.3 Standart Giriş, Çıkış ve Hata Akışları

Bilgisayar mimarisinde I/O (Girdi/Çıktı) işlemleri üç ana kanal üzerinden yürür: `stdin` (standart giriş), `stdout` (standart çıkış) ve `stderr` (standart hata).

- ◊ `sys.stdout`: `print()` fonksiyonunun arka planda kullandığı kanaldır.
- ◊ `sys.stderr`: Hata mesajlarını sisteme iletmek için kullanılır. Mühendislik uygulamalarında hataları normal çıktılarından ayırmak için bu kanalın kullanılması standart bir pratiktir.

§10.4.4 Dosya İşlemlerinde Özel Hata Türleri

En çok karşılaşılan hata türleri aşağıda verilmiştir:

```
try:
    with open("dosya.txt", "r") as f:
        icerik = f.read()
except FileNotFoundError:
    print("Dosya bulunamadı!")
except PermissionError:
    print("Dosya erişim izni yok!")
except IOError as e:
    print(f"Dosya hatası: {e}")
```

§10.4.5 Dosyaların Ön Tanımlı Olarak Aranacağı Dizinler

Python bir modülü içe aktarırken (`import`), belirli dizinleri kontrol eder. `sys.path`, bu dizinlerin listesini tutar. Eğer kendi yazdığınız kütüphaneleri Python'un bulmasını istiyorsanız, bu listeye müdahale edebilirsiniz.

```
import sys

# Python'un kütüphane arama yollarına yeni bir dizin eklemek
sys.path.append("C:/KendiKutuphanelerim")
```

§10.4.6 Programdan Çıkış

Bir hata durumunda veya görev tamamlandığında programın akışını güvenli bir şekilde sonlandırmak ve işletim sistemine bir çıkış kodu (`exit code`) döndürmek için kullanılır.

- ◊ `sys.exit(0)`: Başarılı çıkış.
- ◊ `sys.exit(1)`: Hatalı çıkış (Sistem mühendisliğinde hata takip mekanizmaları için kritiktir).

§10.4.7 Yorumlayıcı ve Platform Bilgileri

Yazılımın hangi platformda (Windows, Linux, macOS) ve hangi Python sürümünde çalıştığını denetlemek, kodun taşınabilirliği (`portability`) için önemlidir.

```
print(f"İşletim Sistemi Platformu: {sys.platform}")
print(f"Python Sürümü: {sys.version}")
```

10.5 Matematiksel Hesaplamalar

Bilgisayar mimarisinde matematiksel işlemler işlemcinin aritmetik işlem birimi (ALU) tarafından yürütülür. Python'un `math` modülü, C diliyle yazılmış olan standart matematik kütüphanesine doğrudan erişim sağlayarak, karmaşık hesaplamaların hem hızlı hem de standartlara uygun (IEEE 754) yapılmasını sağlar.

Mühendislik hesaplamalarında kayan noktalı sayıların (`floating point`) işlemci seviyesindeki davranışı bazen istenmeyen yuvarlama hatalarına neden olabilir.

- ◊ `math` Modülü: C dilinin standart matematik kütüphanesi üzerine inşa edilmiştir. Trigonometri, logaritma ve sabitler (π , e) gibi donanım destekli hızlı hesaplama araçlarını sunar.
- ◊ `decimal` Modülü: Finansal veya yüksek hassasiyetli mühendislik hesaplamaları için `float` veri tipinin zafiyetlerini giderir. $0.1 + 0.2$ 'nin tam olarak 0.3 etmesini sağlayan deterministik bir hesaplama yapısı sunar.

§10.5.1 Matematiksel Sabitler

Mühendislik formüllerinde kullanılan evrensel sabitler, modül içerisinde yüksek hassasiyetle tanımlanmıştır.

- ◊ `math.pi`: π sayısı (3.141592...).
- ◊ `math.e`: Euler sayısı (2.718281...).

- ◇ `math.tau`: 2π değerini temsil eden sabit.

§10.5.2 Sayı Yuvarlama ve Sınır Değerler

Sayısal analizlerde, kayan noktalı bir sayının (`float`) tam sayıya nasıl dönüştürüleceği işlemin sonucunu etkiler.

- ◇ `math.ceil(x)`: Sayıyı yukarıya, en yakın tam sayıya yuvarlar.
- ◇ `math.floor(x)`: Sayıyı aşağıya, en yakın tam sayıya yuvarlar.
- ◇ `math.trunc(x)`: Sayının ondalık kısmını atarak sadece tam sayı kısmını bırakır.

§10.5.3 Üs, Logaritmik ve Güç Fonksiyonları

Donanım tasarımı veya veri analitiği gibi alanlarda kullanılan temel fonksiyonlardır.

- ◇ `math.pow(x, y)`: x sayısının y . kuvvetini alır.
- ◇ `math.sqrt(x)`: Sayının karekökünü hesaplar.
- ◇ `math.log(x, base)`: Belirtilen tabanda logaritma hesaplar.

§10.5.4 Trigonometrik İşlemler

Trigonometrik fonksiyonları radyan cinsinden hesaplar. Mühendislikte sıkça kullanılan derece-radyan dönüşümleri için `math` modülü yardımcı fonksiyonlar sunar.

- ◇ `math.sin()`, `math.cos()`, `math.tan()`: Temel trigonometrik fonksiyonlar.
- ◇ `math.radians(x)`: Dereceyi radyana çevirir.
- ◇ `math.degrees(x)`: Radyanı dereceye çevirir.

```
import math
```

```
derece = 90
radyan = math.radians(derece)
print(f"90 derecenin sinüsü: {math.sin(radyan)}") # Çıktı: 1.0
```

§10.5.5 Özel Durumlar

Bilgisayar bilimlerinde bazı matematiksel sonuçlar standart sayılarla ifade edilemez.

- ◇ `math.inf`: Sonsuz (`infinity`) değerini temsil eder.
- ◇ `math.nan`: Sayı olmayan (`not a number-NaN`) değerini temsil eder. Hatalı veri girişlerini kontrol etmek için kritiktir.

`math.isclose()` fonksiyonu, iki kayan noktalı sayının birbirine "yeterince yakın" olup olmadığını kontrol etmek için kullanılır. `if a == b` kontrolü `float` sayılarda yuvarlama hataları nedeniyle risklidir; bu yüzden toleranslı bir karşılaştırma her zaman daha güvenlidir.

Pisagor teoremini kullanarak bir dik üçgenin hipotenüsünü hesaplayan program örneği aşağıda verilmiştir:

```
import math
```

```
def hipotenus_hesapla(a, b):
    # math.hypot doğrudan karelerin toplamının kökünü alır
    return math.hypot(a, b)
```

```
kenar1 = 3
kenar2 = 4
print(f"Hipotenüs: {hipotenus_hesapla(kenar1, kenar2)}") # Çıktı: 5.0
```

10.6 Bilimsel Hesaplama

NumPy (Numerical Python), Python'da bilimsel hesaplama için temel kütüphanedir. Çok boyutlu diziler (`array`) ve bu diziler üzerinde yapılan matematiksel işlemler için optimize edilmiş fonksiyonlar sağlar.

NumPy, veri bilimi, makine öğrenmesi ve bilimsel hesaplama projelerinin temelini oluşturur. Python'un standart listelerine göre çok daha hızlı ve bellek verimlidir. Bu nedenle çok önemlidir.

`pip install numpy` ile kurulumu yapılır.

```
import time
import numpy as np
```

```
# Python listesi ile işlem
```

```
python_liste = list(range(1000000))
baslangic = time.time()
python_sonuc = [x * 2 for x in python_liste]
python_sure = time.time() - baslangic

# NumPy array ile işlem
numpy_array = np.arange(1000000)
baslangic = time.time()
numpy_sonuc = numpy_array * 2
numpy_sure = time.time() - baslangic

print(f"Python listesi: {python_sure:.4f} saniye")
# Çıktı: Python listesi: 0.0523 saniye
print(f"NumPy array: {numpy_sure:.6f} saniye")
# Çıktı: NumPy array: 0.001234 saniye
print(f"Hız artışı: {python_sure/numpy_sure:.1f}x")
# Çıktı: Hız artışı: 42.4x
# NumPy çok daha hızlıdır!
```

§10.6.1 NumPy Dizileri

NumPy'nin temel veri yapısı `ndarray` (**n-dimensional array**) yani çok boyutlu dizidir.

```
# Python listesinden dizi oluşturma
liste = [1, 2, 3, 4, 5]
arr = np.array(liste)
print(arr)
# Çıktı: [1 2 3 4 5]
print(type(arr))
# Çıktı: <class 'numpy.ndarray'>

# İki boyutlu dizi (matris)
matris = np.array([[1, 2, 3], [4, 5, 6]])
print(matris)
# Çıktı: [[1 2 3]
# Çıktı: [4 5 6]]

# Üç boyutlu dizi
ucboyutlu = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(ucboyutlu.shape)
# Çıktı: (2, 2, 2)
```

§10.6.2 Özel Dizi Oluşturma Fonksiyonları

```
# Sıfırlarla dolu array
sifirlar = np.zeros((3, 4))
print(sifirlar)

# Birlerle dolu array
birler = np.ones((2, 3))
print(birler)

# Belirli bir değerle dolu array
dolu = np.full((2, 2), 7)
print(dolu)

# Birim matris (Identity matrix)
birim = np.eye(3)
print(birim)

# Belirli aralıkta sayılar
aralik = np.arange(0, 10, 2)
print(aralik)
# Çıktı: [0 2 4 6 8]
```

```
# Eşit aralıklı sayılar
esit_aralik = np.linspace(0, 1, 5)
print(esit_aralik)
# Çıktı: [0.  0.25 0.5  0.75 1.  ]

# Rastgele sayılar (0-1 arası)
rastgele = np.random.rand(3, 3)
print(rastgele)

# Normal dağılım (Ortalama 0, standart sapma 1)
normal = np.random.randn(3, 3)
print(normal)
```

§10.6.3 Dizi Özellikleri

```
# Boyut (dimensions)
print(f"Boyut (ndim): {arr.ndim}")

# Şekil (shape) - her boyuttaki eleman sayısı
print(f"Şekil (shape): {arr.shape}")

# Toplam eleman sayısı
print(f"Boyut (size): {arr.size}")

# Veri tipi
print(f"Veri tipi (dtype): {arr.dtype}")

# Her elemanın boyutu (bytes)
print(f"Her elemanın boyutu: {arr.itemsize} bytes")
print(f"Toplam bellek: {arr.nbytes} bytes")
```

§10.6.4 Dizi İndisleri ve Dilimleme

```
# Tek eleman erişimi
print(arr[0]) # Çıktı: 10
print(arr[-1]) # Çıktı: 50

# Dilimleme (slicing)
print(arr[1:4]) # Çıktı: [20 30 40]
print(arr[:2]) # Çıktı: [10 30 50]

# İki boyutlu array
matris = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Tek eleman erişimi
print(matris[0, 0]) # Çıktı: 1
print(matris[1, 2]) # Çıktı: 6

# Satır erişimi
print(matris[1]) # Çıktı: [4 5 6]

# Sütun erişimi
print(matris[:, 1]) # Çıktı: [2 5 8]

# Alt matris
print(matris[0:2, 1:3])
# Çıktı: [[2 3]
# Çıktı: [5 6]]

# Boolean indeksleme
arr = np.array([1, 2, 3, 4, 5])
maske = arr > 3
```

```
print(arr[maske]) # Çıktı: [4 5]

# Koşullu indeksleme
print(arr[arr % 2 == 0]) # Çıktı: [2 4]
```

§10.6.5 Dizi Biçimlendirme

```
arr = np.arange(12)
print(arr)
# Çıktı: [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Yeniden şekillendirme (reshape)
matris = arr.reshape(3, 4)
print(matris)

# Otomatik boyut hesaplama (-1 kullanarak)
matris2 = arr.reshape(4, -1) # 4 satır, otomatik sütun
print(matris2)

# Düzleştirme (flatten)
duz = matris.flatten()
print(duz)

# Transpozisyon (satır ve sütunları yer değiştirme)
transpoz = matris.T
print(transpoz)
```

§10.6.6 Matematiksel İşlemler

Temel Aritmetik

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

# Toplama
print(a + b) # Çıktı: [11 22 33 44]

# Çıkarma
print(b - a) # Çıktı: [ 9 18 27 36]

# Çarpma (element-wise)
print(a * b) # Çıktı: [ 10  40  90 160]

# Bölme
print(b / a) # Çıktı: [10. 10. 10. 10.]

# Üs alma
print(a**2) # Çıktı: [ 1  4  9 16]

# Skaler işlemler
print(a + 10) # Çıktı: [11 12 13 14]
print(a * 2) # Çıktı: [2 4 6 8]
```

İstatistiksel Fonksiyonlar

```
# Toplam
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(f"Toplam: {np.sum(arr)}")
print(f"Satır toplamaları (axis=1): {np.sum(arr, axis=1)}")
print(f"Sütun toplamaları (axis=0): {np.sum(arr, axis=0)}")

# Ortalama
print(f"Ortalama: {np.mean(arr)}")

# Standart sapma
```

```

print(f"Standart sapma: {np.std(arr):.2f}")

# Minimum ve maksimum
print(f"Minimum: {np.min(arr)}")
print(f"Maksimum: {np.max(arr)}")

# Minimum ve maksimumun indeksi
print(f"Min indeks: {np.argmin(arr)}")
print(f"Max indeks: {np.argmax(arr)}")

# Medyan
print(f"Medyan: {np.median(arr)}")

# Yüzdelik dilimler (percentiles)
print(f"25. yüzdelik: {np.percentile(arr, 25)}")
print(f"75. yüzdelik: {np.percentile(arr, 75)}")

```

Matematiksel Fonksiyonlar

```

# Trigonometrik fonksiyonlar
print(f"Sinüs: {np.sin(arr)}")
print(f"Kosinüs: {np.cos(arr)}")

# Üstel ve logaritma
arr2 = np.array([1, 2, 3, 4, 5])
print(f"e^x: {np.exp(arr2)}")
print(f"log(x): {np.log(arr2)}")
print(f"log10(x): {np.log10(arr2)}")

# Karekök
print(f"sqrt(x): {np.sqrt(arr2)}")

# Mutlak değer
arr3 = np.array([-1, -2, 3, -4, 5])
print(f"abs(x): {np.abs(arr3)}")

# Yuvarlama
arr4 = np.array([1.234, 2.567, 3.891])
print(f"Round: {np.round(arr4, 2)}")
print(f"Floor: {np.floor(arr4)}")
print(f"Ceil: {np.ceil(arr4)}")

```

Lineer Cebir İşlemleri

```

# Matris çarpımı
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Element-wise çarpım
print(f"Element-wise çarpım:\n{A * B}")

# Matris çarpımı (dot product)
print(f"Matris çarpımı:\n{np.dot(A, B)}")

# @ operatörü ile matris çarpımı
print(f"A @ B:\n{A @ B}")

# Determinant
det = np.linalg.det(A)
print(f"Determinant: {det}")

# Ters matris (inverse)
A_inv = np.linalg.inv(A)

```

```
print(f"Ters matris:\n{A_inv}")

# Özdeğerler ve özvektörler
ozdegerler, ozvektorler = np.linalg.eig(A)
print(f"Özdeğerler: {ozdegerler}")
print(f"Özvektörler:\n{ozvektorler}")
```

§10.6.7 Yayın (Broadcasting)

NumPy modülünde, farklı şekillerdeki dizileri otomatik olarak uyumlu hale getirmek için yayın (**broad-casting**) yapısı.

```
# Skaler ile array
arr = np.array([1, 2, 3])
print(arr + 10) # Çıktı: [11 12 13]

# Farklı şekillerde array'ler
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([10, 20, 30]) # (3,)
print(a + b)

# Sütun vektörü ile matris
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([[10], [20]]) # (2, 1)
print(a + b)
```

§10.6.8 Dizi Birleştirme ve Ayırma

```
# Dizi birleştirme
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Dikey birleştirme (vstack)
dikey = np.vstack((a, b))
print(dikey)

# Yatay birleştirme (hstack)
yatay = np.hstack((a, b))
print(yatay)

# Concatenate (axis belirtilerek)
birlesmis = np.concatenate((a, b))
print(birlesmis)

# Dizi ayırma
arr = np.array([1, 2, 3, 4, 5, 6])

# Eşit parçalara ayırma
parcalar = np.split(arr, 3)
print(parcalar)

# Belirli indekslerde ayırma
parcalar2 = np.split(arr, [2, 4])
print(parcalar2)
```

§10.6.9 Dizi Kopyalama ve View

```
# View (görünüm) - aynı veriyi gösterir
arr = np.array([1, 2, 3, 4, 5])
view = arr.view()
view[0] = 999
print(arr) # Çıktı: [999  2  3  4  5] # Orijinal array değişti!

# Copy (kopya) - bağımsız yeni array
arr = np.array([1, 2, 3, 4, 5])
```

```
kopya = arr.copy()
kopya[0] = 999
print(arr) # Çıktı: [1 2 3 4 5] # Orijinal array değişmedi
print(kopya) # Çıktı: [999 2 3 4 5]
```

§10.6.10 Pratik Örnekler

Veri Normalizasyonu Örneği

```
# Min-Max normalizasyon (0-1 arası)
veri = np.array([10, 20, 30, 40, 50])
min_val = np.min(veri)
max_val = np.max(veri)
normalize_edilmis = (veri - min_val) / (max_val - min_val)
print(normalize_edilmis) # Çıktı: [0. 0.25 0.5 0.75 1. ]

# Z-score normalizasyon (standartlaştırma)
ortalama = np.mean(veri)
std = np.std(veri)
standartlastirilmis = (veri - ortalama) / std
print(standartlastirilmis) # Çıktı: [-1.41 -0.71 0. 0.71 1.41]
```

Mesafe Hesaplama Örneği

```
# İki nokta arasındaki Öklid mesafesi
nokta1 = np.array([1, 2, 3])
nokta2 = np.array([4, 5, 6])

mesafe = np.sqrt(np.sum((nokta1 - nokta2) ** 2))
print(f"Öklid mesafesi: {mesafe:.2f}") # Çıktı: 5.20

# Alternatif: linalg.norm kullanarak
mesafe2 = np.linalg.norm(nokta1 - nokta2)
print(f"Mesafe (linalg.norm): {mesafe2:.2f}") # Çıktı: 5.20
```

Görüntü İşleme

```
# Gri tonlamalı görüntü (8-bit, 0-255 arası)
goruntu = np.random.randint(0, 256, (5, 5), dtype=np.uint8)
print("Orijinal görüntü:")
print(goruntu)

# Parlaklık artırma
parlak_goruntu = np.clip(goruntu + 50, 0, 255).astype(np.uint8)
print("\nParlak görüntü:")
print(parlak_goruntu)

# Kontrast artırma
kontrast_goruntu = np.clip(goruntu * 1.5, 0, 255).astype(np.uint8)
print("\nYüksek kontrastlı görüntü:")
print(kontrast_goruntu)

# Negatif görüntü
negatif_goruntu = 255 - goruntu
print("\nNegatif görüntü:")
print(negatif_goruntu)
```

Polinomial Regresyon

```
# Veri noktaları
x = np.array([1, 2, 3, 4, 5])
y = np.array([2.1, 3.9, 6.2, 8.1, 9.8])

# 2. derece polinom fit etme
katsayilar = np.polyfit(x, y, 2)
```



```
print(f"Katsayılar (a, b, c): {katsayilar}")

# Tahmin yapma
x_yeni = np.linspace(1, 5, 50)
y_tahmin = np.polyval(katsayilar, x_yeni)

# Hata hesaplama
y_fit = np.polyval(katsayilar, x)
mse = np.mean((y - y_fit) ** 2)
print(f"Ortalama Kare Hata (MSE): {mse:.4f}")
```

§10.6.11 Performans İpuçları

1. **Vektörleştirme (vectorization) Kullanın:** Elemanlar üzerinde açık döngü (`for`) kullanmak yerine vektörleştirilmiş işlemleri tercih edin.
2. **In-place İşlemler Kullanın:** Bellek tasarrufu sağlamak için `arr = arr * 2` yerine `arr *= 2` tercih edin.
3. **Uygun Veri Tipi Seçin:** `float64` yerine `float32` kullanmak bellek kullanımını yarı yarıya düşürür.
4. **Broadcasting Kullanın:** Manuel döngüler yerine boyut genişletmelerini NumPy'a bırakın.

§10.6.12 En İyi NumPy Uygulamaları

- ♦ **Doğru Veri Tipi:** Bellek tasarrufu gereken yerlerde veri tiplerini (`np.int32`, `np.float32`, `np.complex64`) açıkça tanımlayın.
- ♦ **Shape Kontrolü:** Fonksiyonlarda boyut uyumsuzluklarının önüne geçmek için boyut denetimi yapın.
- ♦ **Bellek Yönetimi:** Büyük boyutlu dizileri işiniz bittiğinde `del` komutu ile bellekten temizleyin.
- ♦ **Kod Okunabilirliği:** `axis` parametrelerini belirtirken açık metinler veya açıklayıcı isimler tercih edin.

§10.6.13 Bilimsel Hesaplama Özeti

Veri bilimi, makine öğrenmesi, görüntü işleme, bilimsel hesaplama, finans ve mühendislik simülasyonlarında yaygın olarak kullanılan NumPy'nin temel özellikleri:

- ♦ **Hız:** Python listelerinden 10–100 kat daha hızlıdır.
- ♦ **Çok Boyutlu Diziler:** Güçlü `ndarray` veri yapısı.
- ♦ **Matematiksel İşlemler:** Vektörleştirilmiş eleman bazlı işlemler.
- ♦ **Yayın (Broadcasting):** Farklı boyutlardaki dizileri otomatik uyumlu hale getirme.
- ♦ **Lineer Cebir:** Matris işlemleri, özdeğerler, determinant.
- ♦ **İstatistik:** Ortalama, standart sapma, medyan vb.
- ♦ **Rastgele Sayılar:** Çeşitli olasılık dağılımlarından rastgele sayı üretimi.

10.7 Olasılık ve İstatistiksel Analiz Araçları

Mühendislikte sistem davranışlarını simüle etmek veya toplanan verileri analiz etmek için matematiksel modellerden yararlanırsınız. Python, bu ihtiyaçlar için `random` ve `statistics` modüllerini sunar.

§10.7.1 Rastlantısallık ve Simülasyon

Bilgisayar bilimlerinde gerçek rastlantısallık zordur; bu yüzden sözde rastlantısal (pseudo-random) sayılar kullanılır. `random` modülü, bu sayıları üretmek için deterministik algoritmalar kullanır.

- ♦ `random.random()`: 0.0 ile 1.0 arasında rastgele bir `float` sayı üretir.
- ♦ `random.randint(a, b)`: Belirtilen aralıkta (a ve b dahil) bir tam sayı üretir.
- ♦ `random.choice(liste)`: Bir liste içinden rastgele bir eleman seçer.
- ♦ `random.seed(x)`: Rastlantısal sayı üreticisini sabitler. Simülasyonların tekrarlanabilirliği (reproducibility) için mühendislikte hayati önem taşır.

§10.7.2 Temel Veri Analizi

Deneyisel verilerin merkezi eğilimlerini ve yayılımını ölçmek için kullanılır.

- ♦ `statistics.mean()`: Verilerin aritmetik ortalamasını hesaplar.
- ♦ `statistics.median()`: Verilerin orta değerini bulur (aykırı değerlerin etkisini azaltmak için kullanılır).
- ♦ `statistics.stdev()`: Standart sapmayı hesaplar. Verinin ne kadar güvenilir veya değişken olduğunu ölçen temel mühendislik metriğidir.

10.8 Yüksek Performanslı Döngü Yönetimi

Büyük veri setleri üzerinde iterasyon yaparken bellek verimliliği (**memory efficiency**) en öncelikli konudur. `itertools` modülü, karmaşık döngüleri ve kombinasyonel işlemleri bellek dostu yineleyiciler (**iterator**) ile

yönetmemizi sağlar.

§10.8.1 Sonsuz ve Koşullu Yineleyiciler

- ◇ `itertools.count(start, step)`: Belirli bir sayıdan başlayarak sonsuza kadar artan bir sayı dizisi üretir. Sayaç mekanizmaları için idealdir.
- ◇ `itertools.cycle(iterable)`: Bir veri grubunu sonsuz bir döngüye sokar.

§10.8.2 Kombinasyonel Araçlar

Karmaşık olasılık hesaplamalarında ve sistem konfigürasyonlarını test ederken kullanılır:

- ◇ `itertools.product()`: Kartezyen çarpım oluşturur.
- ◇ `itertools.permutations()`: Elemanların tüm sıralı dizilişlerini hesaplar.
- ◇ `itertools.combinations()`: Elemanların sırasız gruplarını (kombinasyonlarını) oluşturur.

`itertools` fonksiyonları veriyi belleğe bir anda yüklemeyi, ihtiyaç duyuldukça üretir (**lazy evaluation**). Bu sayede milyonlarca kombinasyon içeren işlemler bile sistemi kilitlemeden yürütülebilir.

10.9 İleri Seviye Veri Yapıları

Standart liste ve sözlüklerin performans olarak yetersiz kaldığı algoritmalarda, özelleşmiş koleksiyonlar (**collection**) kullanılır.

- ◇ **namedtuple**: Veriyi hem indis hem de anahtar kelime ile erişim sağlayan, bellek dostu değiştirilemez (**immutable**) bir yapıdır.
- ◇ **deque** (**double-ended queue**): Her iki uçtan da $O(1)$ karmaşıklıkla veri ekleme-çıkarma yapabilen optimize edilmiş bir kuyruk yapısıdır.

10.10 JSON Veri Değişim Formatı

Günümüzde yazılımlar tek başlarına değil, bir ağ üzerinden diğer sistemlerle konuşarak çalışırlar. Farklı dillerle (Python, C++, C#) yazılmış sistemlerin birbirini anlayabilmesi için ortak bir veri diline ihtiyaç vardır. JSON (**JavaScript Object Notation**), bu ihtiyacı karşılayan, dilden bağımsız, metin tabanlı ve insan tarafından okunabilir en popüler veri değişim formatıdır.

§10.10.1 JSON Yapısı ve Python Nesneleriyle İlişkisi

JSON yapısı bir mühendis için yabancı değildir; çünkü Python'daki sözlük (**dict**) ve liste (**list**) yapılarının birebir metin karşılığıdır:

- ◇ **Nesneler**: Süslü parantez `{}` içinde tutulan anahtar-değer çiftleridir (Python'daki **dict**).
- ◇ **Diziler**: Köşeli parantez `[]` içinde tutulan sıralı veri gruplarıdır (Python'daki **list**).

§10.10.2 Serileştirme

Python nesnelerini (bellekteki canlı verileri) bir dosyaya yazılacak veya ağdan gönderilecek bir metin formatına dönüştürme işlemine **serileştirme** (**serialization**) denir. Bunun tersi işleme ise **deserileştirme** (**deserialization**) adı verilir.

- ◇ `json.dumps()`: Python nesnesini bir JSON metnine dönüştürür.
- ◇ `json.loads()`: JSON formatındaki bir metni Python nesnesine dönüştürür.

§10.10.3 Dosya Tabanlı İşlemler

Eğer veri bir string değişkeni yerine doğrudan bir dosyaya yazılacaksa, **"s"** takısı olmayan fonksiyonlar kullanılır (`json.dump()` ve `json.load()`).

§10.10.4 JSON ile Veri Bütünlüğü

JSON kullanmak, verinin tip güvenliğini sağlar. Örneğin, Python'daki bir **None** değeri JSON'da **null** olarak, **True** değeri ise **true** olarak saklanır. Bu sayede veri, başka bir dile (örneğin C#'a) aktarıldığında o dilin standartlarına göre doğru şekilde yorumlanır.

JSON verisi metin tabanlı olduğu için **utf-8** karakter kodlamasıyla saklanır. Büyük veri setlerinde JSON dosyaları çok fazla yer kaplayabilir; bu durumlarda veriyi sıkıştırmak veya **pickle** gibi ikili (**binary**) formatları kullanmak performans açısından değerlendirilmelidir.

§10.10.5 Örnek Uygulama

Aşağıda sistem bilgisi alma ve veri saklama üzerine bir örnek verilmiştir:

```
import json
```

```
import os
import sys

def sistem_ozeti_hazirla():
    # Sistemden bilgi toplama
    bilgi = {
        "isletim_sistemi": os.name,
        "python_surumu": sys.version.split()[0],
        "mevcut_dizin": os.getcwd(),
    }

    # Veriyi JSON formatına dönüştürme (Serileştirme)
    json_cikti = json.dumps(bilgi, indent=4)
    print("Sistem Analiz Raporu:")
    print(json_cikti)

sistem_ozeti_hazirla()
```

Bu programın çıktısı aşağıdaki gibi olacaktır:

Sistem Analiz Raporu:

```
{
    "isletim_sistemi": "nt",
    "python_surumu": "3.13.4",
    "mevcut_dizin": "D:\\Python\\PSamples"
}
```

10.11 Düşün ve Çöz

1. **Düşün (Bellek Verimliliği ve Laziness):** 1.000.000 elemanlı iki dizinin kartezyen çarpımını hesaplarken standart bir Python döngüsü veya liste kavrama (**list comprehension**) kullanmak yerine **itertools.product** tercih etmenin RAM kullanımı ve işlemci yükü açısından ne gibi avantajları vardır? Tembel değerlendirme (**Lazy evaluation**) kavramı çerçevesinde açıklayınız.
2. **Çöz (Regex ile Log Dosyası Analizi):** Bir web sunucusuna ait aşağıdaki formatta iz kayıtları (**log**) bulunmaktadır:

```
192.168.1.45 - - [16/Feb/2026:14:32:10] "GET /index.html HTTP/1.1" 200 4523
10.0.0.12 - - [16/Feb/2026:14:33:15] "POST /login HTTP/1.1" 404 128
```

re ve **json** modüllerini kullanarak, metin içerisindeki IP adreslerini ve HTTP durum kodlarını (200, 404 vb.) ayıklayıp bir sözlük formatına dönüştüren ve sonucu bir **log_ozeti.json** dosyasına kaydeden betiği yazınız.
3. **Çöz (NumPy ile Vektörleştirilmiş Mesafe Matrisi):** Elimizde $N \times 2$ boyutunda N adet 2D koordinat noktasını temsil eden bir NumPy dizisi bulunmaktadır. İçi içe **for** döngüsü kullanmadan, sadece NumPy'ın yaygın (**broadcasting**) özelliğini kullanarak tüm noktaların birbirine olan Öklid mesafelerini içeren $N \times N$ boyutunda bir mesafe matrisi oluşturunuz.
4. **Düşün (Sistem Güvenliği ve Argüman Yönetimi):** Komut satırından parametre alırken **sys.argv** kullanmak yerine daha profesyonel kütüphaneler (örneğin **argparse**) tercih edilmesinin nedenlerini düşününüz. Tip dönüşümleri ve hatalı parametre girişlerinde **sys.argv** kullanmanın ne gibi zorlukları vardır?

11. Bölüm

Test ve Hata Ayıklama

Yazılım geliştirmede test yazmak, kodun doğru çalıştığından emin olmak için kritik öneme sahiptir. Python’da test yazmak için `unittest` (standart kütüphane) ve `pytest` (üçüncü taraf, daha popüler) gibi araçlar kullanılır.

Test, kodunuzun beklediğiniz gibi çalıştığını doğrulayan küçük programlardır. Test yazmanın faydaları aşağıda sıralanmıştır:

- ◊ Hataları erken tespit eder
- ◊ Kod değişikliklerinde güven sağlar (**refactoring**)
- ◊ Kodun nasıl kullanılacağını gösterir (dokümantasyon)
- ◊ Daha iyi tasarım yapmaya zorlar
- ◊ Hata ayıklama süresini azaltır

§11.0.1 Yazılım Doğrulamada Birim Testi

Yazılım mühendisliğinde bir fonksiyonun veya sınıfın beklenen işi yapıp yapmadığını otomatik olarak kontrol eden küçük test kodlarına **birim testi (unit test)** denir. Bir mühendis için test yazmak, tasarımın doğruluğunu ispatlamaktır. Kodun bir noktasında yapılan değişikliğin, sistemin başka bir yerini bozup bozmadığını (**regression**) anlamının tek yolu budur.

Python, bu süreci yönetmek için standart kütüphanesinde `unittest` modülünü sunar.

§11.0.2 unittest Modülü Temel Yapısı

Bir test dosyası oluştururken `unittest.TestCase` sınıfından kalıtım alan sınıflar yazılır. Test yöntemlerinin ismi mutlaka `test_` ifadesi ile başlamalıdır.

Örnek Senaryo olarak matematiksel bir fonksiyonun test edilmesini ele alalım:

```
# hesaplama.py
def toplama(a, b):
    return a + b

Şimdi bu fonksiyonu test eden sınıfımızı yazalım:

import unittest
from hesaplama import toplama

class TestMatematik(unittest.TestCase):
    def test_pozitif_toplama(self):
        # assertEquals: sonucun beklenen değere eşit olup olmadığını kontrol eder
        self.assertEqual(toplama(10, 5), 15)
    def test_negatif_toplama(self):
        self.assertEqual(toplama(-1, -1), -2)

if __name__ == '__main__':
    unittest.main()
```

İddia Yöntemleri

`unittest` içinde durumları kontrol etmek için çeşitli iddia yöntemleri (**assertion**) bulunur:

- ◊ `self.assertEqual(a, b)`: $a == b$ olduğunu kontrol eder.
- ◊ `self.assertTrue(x)`: x değerinin `True` olduğunu kontrol eder.
- ◊ `self.assertRaises(İstisna)`: Kodun belirli bir istisnai duruma düşüp düşmediğini denetler.

Test Hazırlığı ve Temizliği

Her testten önce bir veri tabanı bağlantısı kurmak veya bir sensör nesnesi oluşturmak gerekebilir:

- ◊ `setUp()`: Her test metodundan önce otomatik çalışır.
- ◊ `tearDown()`: Her test metodundan sonra otomatik çalışır (temizlik işlemleri için).

```
class TestSensor(unittest.TestCase):
    def setUp(self):
```

```

    # Test öncesi sensör nesnesini hazırla
    self.sensor = Sensor(id=101)
def test_okuma_araligi(self):
    deger = self.sensor.oku()
    self.assertTrue(0 <= deger <= 100)

```

§11.0.3 pytest Modülü

Bu bölümde `pytest`'e odaklanacağız çünkü daha basit ve güçlüdür. Kurulum yapmak için `pip install pytest` komutu çalıştırılır.

`pytest` ile test dosyaları `test_` ile başlar veya `_test` ile biter. Test fonksiyonları da `test_` ile başlamalıdır. Hesaplamalar yapan bir `hesap.py` dosyasını modül olarak hazırlayalım:

```

# hesap.py
def topla(a: int, b: int) -> int:
    """İki sayıyı toplar."""
    return a + b
def carp(a: int, b: int) -> int:
    """İki sayıyı çarpar."""
    return a * b
def bol(a: float, b: float) -> float:
    """İki sayıyı böler."""
    if b == 0:
        raise ValueError("Sıfıra bölme hatası!")
    return a / b

```

Hesaplama dosyasının testini yapacak bir dosyayı aşağıdaki gibi hazırlayabiliriz:

```

# test_hesap.py
from hesaplama import topla, carp, bol
import pytest

def test_topla():
    """toplam fonksiyonunu test eder."""
    assert topla(2, 3) == 5
    assert topla(-1, 1) == 0
    assert topla(0, 0) == 0

def test_carp():
    """carp fonksiyonunu test eder."""
    assert carp(2, 3) == 6
    assert carp(-2, 3) == -6
    assert carp(0, 5) == 0

def test_bol():
    """bol fonksiyonunu test eder."""
    assert bol(6, 2) == 3.0
    assert bol(5, 2) == 2.5

def test_bol_sifira_bolme():
    """Sıfıra bölme hatasını test eder."""
    with pytest.raises(ValueError):
        bol(10, 0)

```

Testleri çalıştırmak için `pytest test_hesap.py` komutu çalıştırılır. Çalıştırma sonucu aşağıdaki çıktı elde edilir:

```

D:\Python\PSamples> pytest test_hesap.py
===== test session starts =====
platform win32 -- Python 3.13.4, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\Python\PSamples
collected 4 items
test_hesap.py ....
===== 4 passed in 0.04s =====
D:\Python\PSamples>

```

§11.0.4 assert Fonksiyonu

Bu fonksiyon yani `assert` (iddia etmek/doğrulamak) fonksiyonu, yazılımın hata toleransı ve savunmacı programlama (**defensive programming**) prensipleri çerçevesinde kullanılır.

Yazılım geliştirme sürecinde, belirli bir noktada bir koşulun mutlaka doğru (`True`) olması gerektiğini varsayarsınız. `assert` fonksiyonu, bu varsayımların doğruluğunu çalışma anında (**runtime**) kontrol eden bir güvenlik mekanizmasıdır. Eğer belirtilen koşul yanlış (`False`) ise, program akışı anında kesilir ve bir `AssertionError` istisna nesnesi yaratılır. Söz dizimi aşağıda verilmiştir:

```
assert <koşul>, <hata_mesajı>
```

Bir mühendis için `assert`, kodun sadece düzgün çalışmasını değil, aynı zamanda beklenmedik durumlara karşı kendini imha etmesini (**fail-fast**) sağlar. Hatalı bir veriyle işleme devam edip sistemin geri dönülemez bir zarara uğraması yerine, hatanın kaynağında durdurulması tercih edilir. Aşağıdaki durumlarda kullanılabilir:

- ◊ **Girdi Doğrulama:** Bir fonksiyona gelen parametrelerin, mantıksal sınırlar içerisinde olup olmadığını denetler.
- ◊ **Değişmezlerin (**invariant**) Kontrolü:** Döngülerin başında veya sonunda bir değerin asla değişmemesi gereken durumları denetler.
- ◊ **Hata Ayıklama (**debug**):** Kodun iç mantığını test etmek için kullanılır.

Bir personelin maaş artış oranını hesaplayan bir fonksiyonda, oran negatif olamaz. Bu, yazılımcının bir varsayımdır. Aşağıda buna ilişkin kod örneği verilmiştir:

```
def maas_artir(mevcut_maas, artis_orani):
    # Artış oranının negatif olamayacağını garanti altına alıyoruz
    assert artis_orani >= 0, "Artış oranı negatif olamaz!"
    yeni_maas = mevcut_maas * (1 + artis_orani)
    return yeni_maas
```

```
print(maas_artir(10000, 0.20))# Doğru kullanım
print(maas_artir(10000, -0.10))# Hatalı kullanım (Program burada AssertionError verecektir)
```

Yazılımcının yaptığı mantıksal hataları veya asla gerçekleşmemesi gereken durumları (programcı hataları) tespit etmek için `assert` fonksiyonu kullanılır. İstisna yönetimi için kullanılan `try-except` bloğu ise kullanıcı hataları (hatalı dosya yolu, yanlış şifre vb.) veya dış etkenler gibi beklenen hataları yönetmek için kullanılır.

Python yorumlayıcısı `-O` (optimize) bayrağı ile çalıştırıldığında tüm `assert` ifadeleri görmezden gelinir. Bu nedenle, iş kritik (**business-critical**) kontroller için `assert` yerine `if` blokları ve `raise` ifadeleri tercih edilmelidir. `assert` daha çok geliştirme ve test aşamasındaki iç denetim mekanizmasıdır.

`pytest` modülü Python'un dahili `assert` talimatını kullanır:

```
# Eşitlik kontrolü
assert 2 + 2 == 4

# Eşitsizlik kontrolü
assert 3 != 4

# Büyüklük kontrolü
assert 5 > 3
assert 3 < 5

# True/False kontrolü
assert True
assert not False

# Liste içeriği
assert [1, 2, 3] == [1, 2, 3]
assert 1 in [1, 2, 3]

# String içeriği
assert "python" in "python programlama"
assert "merhaba".startswith("mer")
```

§11.0.5 Birden Fazla Girdiyle Test

Aynı testin farklı girdilerle çalıştırılması için parametrelendirme (**parametrize**) kullanılır:

```
# test_hesaplama.py
import pytest

@pytest.mark.parametrize("a, b, beklenen", [
    (1, 2, 3),
    (5, 5, 10),
    (-1, 1, 0),
    (0, 0, 0),
    (100, 200, 300)
])

def test_topla_parametrize(a, b, beklenen):
    from hesaplama import topla
    assert topla(a, b) == beklenen

    Bu testin çıktısı aşağıda verilmiştir:

$ pytest test_hesaplama.py::test_topla_parametrize -v
===== test session starts =====
test_hesaplama.py::test_topla_parametrize[1-2-3] PASSED      [ 20%]
test_hesaplama.py::test_topla_parametrize[5-5-10] PASSED     [ 40%]
test_hesaplama.py::test_topla_parametrize[-1-1-0] PASSED     [ 60%]
test_hesaplama.py::test_topla_parametrize[0-0-0] PASSED      [ 80%]
test_hesaplama.py::test_topla_parametrize[100-200-300] PASSED [100%]
===== 5 passed in 0.05s =====
```

§11.0.6 Test Demirbaşları

Testler için ortak bir veri ya da kaynak havuzu oluşturulabilir. Bu verilere **demirbaş** (**fixture**) adı verilir:

```
import pytest

@pytest.fixture
def ornek_liste():
    """Test için örnek liste hazırlar."""
    return [1, 2, 3, 4, 5]

@pytest.fixture
def ornek_sozluk():
    """Test için örnek sözlük hazırlar."""
    return {"isim": "Ali", "yas": 25, "sehir": "Ankara"}

def test_liste_uzunlugu(ornek_liste):
    assert len(ornek_liste) == 5

def test_liste_toplam(ornek_liste):
    assert sum(ornek_liste) == 15

def test_sozluk_anahtarlar(ornek_sozluk):
    assert "isim" in ornek_sozluk
    assert "yas" in ornek_sozluk

def test_sozluk_degerleri(ornek_sozluk):
    assert ornek_sozluk["isim"] == "Ali"
    assert ornek_sozluk["yas"] == 25

    Demirbaşların (fixture) ne zaman oluşturulup yok edileceğini belirlenebilir. Buna demirbaş kapsamı (fixture scope) adı verilir.

    # Her test için yeni oluştur (varsayılan)
    @pytest.fixture(scope="function")
    def her_test_icin():
        print("\nHer test için oluşturuldu")
        return "veri"

    # Tüm modül için bir kez oluştur
```



```
@pytest.fixture(scope="module")
def modul_icin_bir_kez():
    print("\nModül için bir kez oluşturuldu")
    veri = {"baglanti": "veritabani_baglantisi"}
    yield veri # yield kullanımı
    print("\nModül sonu - temizlik yapıldı")

# Tüm test oturumu için bir kez
@pytest.fixture(scope="session")
def oturum_icin_bir_kez():
    print("\nOturum başlangıcında oluşturuldu")
    return "global_veri"
```

§11.0.7 Sahte Nesneler

Dış bağımlılıkları test etmek için **sahte** (**mock**) nesneler kullanılır:

```
# hava_durumu.py
import requests

def sehir_sicaklik(sehir: str) -> float:
    """Bir şehrin sıcaklığını API'den alır."""
    url = f"https://api.weather.com/{sehir}"
    yanit = requests.get(url)
    veri = yanit.json()
    return veri["sicaklik"]

# test_hava_durumu.py
from hava_durumu import sehir_sicaklik
from unittest.mock import Mock, patch

def test_sehir_sicaklik():
    """API çağrısını mock'luyoruz."""
    with patch('hava_durumu.requests.get') as mock_get:
        # Mock yanıt oluştur
        mock_yanıt = Mock()
        mock_yanıt.json.return_value = {"sicaklik": 25.5}
        mock_get.return_value = mock_yanıt

# Testi çalıştır
sicaklik = sehir_sicaklik("Ankara")

# Kontroller
assert sicaklik == 25.5
mock_get.assert_called_once_with("https://api.weather.com/Ankara")
```

pytest-mock Modülü

pytest-mock modülü daha kolay sahte (**mock**) nesne kullanımı sağlar. `pip install pytest-mock` komutu ile kurulum yapılır.

```
def test_sehir_sicaklik_mock(mock):
    """pytest-mock ile daha basit kullanım."""
    # Mock oluştur
    mock_get = mock.patch('hava_durumu.requests.get')
    mock_get.return_value.json.return_value = {"sicaklik": 30.0}
    # Test
    sicaklik = sehir_sicaklik("İstanbul")
    # Kontrol
    assert sicaklik == 30.0
```

§11.0.8 Veri Tabanı Testi

Aşağıda demirbaş (**fixture**) yöntemi ile bir veri tabanı testi örneği verilmiştir:

```
import sqlite3
import pytest
```

```

@pytest.fixture
def veritabani():
    """Test veritabanı oluşturur."""
    baglanti = sqlite3.connect(":memory:") # Bellekte veritabanı
    cursor = baglanti.cursor()

    # Tablo oluştur
    cursor.execute("""
    CREATE TABLE kullanicilar (
    id INTEGER PRIMARY KEY,
    isim TEXT NOT NULL,
    yas INTEGER
    )
    """)

    # Örnek veri ekle
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Ali', 25)")
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Ayşe', 30)")
    baglanti.commit()

    yield baglanti # Test için veritabanını ver

    baglanti.close() # Test sonrası temizle

def test_kullanici_sayisi(veritabani):
    cursor = veritabani.cursor()
    cursor.execute("SELECT COUNT(*) FROM kullanicilar")
    sayi = cursor.fetchone()[0]
    assert sayi == 2

def test_kullanici_ekle(veritabani):
    cursor = veritabani.cursor()
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Mehmet', 28)")
    veritabani.commit()

    cursor.execute("SELECT COUNT(*) FROM kullanicilar")
    sayi = cursor.fetchone()[0]
    assert sayi == 3

```

§11.0.9 Test Kapsama Oranı

Kodunuzun ne kadarının test edildiğini görmek için test kapsama oranı (**coverage**) kullanılır. `pip install pytest-cov` komutu ile kurulum yapılır.

```

$ pytest --cov=hesaplama test_hesap.py
===== test session starts =====
collected 4 items
test_hesap.py .... [100%]
----- coverage: platform win32, python 3.10.0 -----
Name          Stmts  Miss  Cover
-----
hesap.py       8      0  100%
-----
TOTAL          8      0  100%
===== 4 passed in 0.10s =====

```

HTML raporu oluşturmak için `pytest -cov=hesaplama -cov-report=html test_hesap.py` komutu kullanılabilir.

Test Kapsama Oranı Hedefleri:

- ◇ %80 ile %100 arası: **Mükemmel**
- ◇ %60 ile %80 arası: **İyi**
- ◇ %40 ile %60 arası: **Kabul edilebilir**

◇ %0 ile %40 arası: **Yetersiz**

§11.0.10 Testleri Gruplandırma

Testleri işaretleyerek gruplandırabiliriz:

```
import pytest

@pytest.mark.hizli
def test_hizli_islem():
    assert 1 + 1 == 2

@pytest.mark.yavas
def test_yavas_islem():
    import time
    time.sleep(2)
    assert True

@pytest.mark.veritabani
def test_veritabani_baglantisi():
    # Veritabanı testi
    pass

@pytest.mark.network
def test_api_cagrisi():
    # API testi
    pass
```

Bu testleri çalıştırmak için aşağıdaki komutlar çalıştırılabilir:

```
$ pytest -m hizli # Sadece hızlı testler
$ pytest -m "not yavas" # Yavaş testler hariç
$ pytest -m "veritabani or network" # Veritabanı veya network testleri
```

§11.0.11 Paylaşımlı Demirbaşlar

Aşağıdaki dosyada tanımlanan demirbaşlar (**fixture**) tüm test dosyalarında kullanılabilir.

```
# conftest.py
import pytest

@pytest.fixture
def uygulama_ayarlari():
    """Tüm testlerde kullanılabilir."""
    return {
        "veritabani": "test.db",
        "debug": True,
        "port": 5000
    }

@pytest.fixture(autouse=True)
def test_baslangic_bitisi():
    """Her testten önce ve sonra otomatik çalışır."""
    print("\n--- Test başlıyor ---")
    yield
    print("\n--- Test bitti ---")
```

§11.0.12 Zaman Uyumsuz Fonksiyon Testleri

Zaman uyumsuz (**asynchronous**) fonksiyonları test etmek için **pytest-asyncio** kullanılır. `pip install pytest-asyncio` komutu ile kurulum yapılır.

```
# async_islemler.py
import asyncio

async def asenkron_topla(a: int, b: int) -> int:
    await asyncio.sleep(0.1) # Simüle edilmiş asenkron işlem
    return a + b
```

```
async def api_cagir(url: str) -> dict:
    await asyncio.sleep(0.5)
    return {"durum": "basarili", "veri": "test"}
```

Bu işlemlerin bulunduğu dosyayı test etmek için aşağıdaki kod yazılabilir:

```
# test_async_islemler.py
import pytest
from async_islemler import asenkron_topla, api_cagir

@pytest.mark.asyncio
async def test_asenkron_topla():
    sonuc = await asenkron_topla(2, 3)
    assert sonuc == 5

@pytest.mark.asyncio
async def test_api_cagir():
    sonuc = await api_cagir("https://api.example.com")
    assert sonuc["durum"] == "basarili"
```

§11.0.13 Test Odaklı Yazılım Geliştirme (TDD)

Test odaklı yazılım geliştirme (**Test Driven Development-TDD**), önce test yazıp sonra kodu yazma metodolojisi. Bu metodolojide ilk önce başarısız bir test yazılır (**red**), sonrasında testi geçecek minimum kod yazılır (**green**), en sonunda kod iyileştirilir (**refactor**).

```
# 1. Önce test yaz (başarısız olacak)
def test_kullanici_olustur():
    kullanıcı = Kullanici("Ali", "ali@example.com")
    assert kullanıcı.isim == "Ali"
    assert kullanıcı.email == "ali@example.com"

# 2. Kodu yaz (testi geçirecek)
class Kullanici:
    def __init__(self, isim: str, email: str):
        self.isim = isim
        self.email = email

# 3. Kodu iyileştir
class Kullanici:
    def __init__(self, isim: str, email: str):
        if not isim:
            raise ValueError("İsim boş olamaz")
        if "@" not in email:
            raise ValueError("Geçersiz email")
        self.isim = isim
        self.email = email
```

§11.0.14 En İyi Uygulamalar

- ◊ Her fonksiyon için en az bir test yazılmalı.
- ◊ Test isimleri açıklayıcı olmalı (**test_topla_iki_pozitif_sayi**).
- ◊ Testler birbirinden bağımsız olmalı ve hızlı çalışmalı.
- ◊ Sahte nesne (**mock**) kullanarak dış bağımlılıklar izole edilmeli.
- ◊ Test kapsama oranı (**coverage**) %100'e yakın olmalı (kritik kod için).
- ◊ Demirbaşlar (**fixture**) tekrar kullanılabilir yapılmalı.
- ◊ Testler her sürüm sırasında düzenli çalıştırılmalı (CI/CD).
- ◊ Test kodu da temiz tutulmalı.

§11.0.15 Test İçeren Örnek Proje Yapısı

Test içeren örnek bir proje klasör yapısı aşağıdaki gibi olmalıdır:

```
proje/
├── src/
│   └── __init__.py
```

```

├── hesaplama.py
├── kullanıcı.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_hesaplama.py
│   └── test_kullanici.py
├── pytest.ini
└── requirements.txt

```

Bu projede yer alan `pytest.ini` dosyası aşağıdaki gibi olmalıdır:

[pytest]

```

testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
markers =
    hızlı: Hızlı testler
    yavaş: Yavaş testler
    veritabani: Veritabanı testleri
    network: Network gerektiren testler

```

11.1 Düşün ve Çöz

- Düşün (Test Yan Etkileri ve Isolation):** Bir veritabanı veya dosya sistemi üzerinde işlem yapan testler çalıştırdıktan sonra sistemde kalıcı veri bırakıyorsa bu duruma **test kirliliği** (**test pollution**) denir. Testlerin birbirinden bağımsız (**isolated**) çalışmasını sağlamak adına **pytest**'in **yield** içeren demirbaş (**fixture**) mekanizması nasıl bir çözüm sunar? Oturum (**session**) ve fonksiyon (**function**) kapsamaları üzerinden tartışınız.
- Çöz (TDD ve Boundary Value Analysis):** Test Odaklı Geliştirme (TDD) yaklaşımını kullanarak bir metnin E-posta adresi standartlarına uygun olup olmadığını denetleyen bir `email_gecerli_mi(email: str) -> bool` fonksiyonu yazmak istiyorsunuz.
 - ◊ Kodu yazmadan önce; boş string, '@' içermeyen string, birden fazla '@' içeren string ve geçerli e-posta durumlarını kapsayan parametrelendirilmiş (`@pytest.mark.parametrize`) test fonksiyonunu hazırlayınız.
 - ◊ Yazdığınız testleri geçecek minimum kodu yazarak TDD döngüsünü (**Red-Green-Refactor**) tamamlayınız.
- Çöz (Dış Servis İzolasyonu ve Mocking):** Döviz kuru API'sine istek atıp Türk Lirası karşılığını hesaplayan bir fonksiyonu test etmek istiyorsunuz. İnternet bağlantısı kesildiğinde veya API sunucusu çöktüğünde testlerinizin başarısız olmasını engellemek için `unittest.mock` veya `pytest-mock` kullanarak API yanıtını sahte nesneyle (**mock**) değiştiren test kodunu yazınız.
- Düşün (Assert ve Üretim Ortamı):** Python uygulaması komut satırında `python -O main.py` (optimize modu) şeklinde çalıştırıldığında `assert` ifadelerine ne olur? Kullanıcıdan alınan verilerin kontrolünde neden sadece `assert` talimatına güvenilmemesi gerektiğini güvenlik ve hata yönetimi açısından açıklayınız.

12. Bölüm

Zaman Uyumsuz Programlama

Zaman uyumsuz (**asynchronous**) programlama, bir programın birden fazla işlemi aynı anda (eşzamanlı olarak) yapabilmesini sağlar. Python'da asenkron programlama için **asyncio** kütüphanesi kullanılır. Bu özellik Python 3.4'te tanıtılmış, Python 3.5'te **async/await** talimatları da eklenmiştir.

12.1 Zaman Uyumlu ve Uyumsuz Programlama

Zaman Uyumlu (**synchronous**) programlamada, her işlem bir önceki işlemin bitmesini bekler:

```
import time

def gorev1():
    print("Görev 1 başladı")
    time.sleep(2) # 2 saniye bekle
    print("Görev 1 bitti")

def gorev2():
    print("Görev 2 başladı")
    time.sleep(1) # 1 saniye bekle
    print("Görev 2 bitti")

baslangic = time.time()
gorev1()
# Çıktı: Görev 1 başladı
# Çıktı: Görev 1 bitti

gorev2()
# Çıktı: Görev 2 başladı
# Çıktı: Görev 2 bitti

print(f"Toplam süre: {time.time() - baslangic:.2f} saniye")
# Çıktı: Toplam süre: 3.00 saniye
```

Zaman Uyumsuz (**asynchronous**) programlamada ise görevler beklerken diğer görevler çalışabilir:

```
import asyncio
import time

async def gorev1():
    print("Görev 1 başladı")
    await asyncio.sleep(2) # 2 saniye bekle (asenكرون)
    print("Görev 1 bitti")

async def gorev2():
    print("Görev 2 başladı")
    await asyncio.sleep(1) # 1 saniye bekle (asenكرون)
    print("Görev 2 bitti")

async def main():
    baslangic = time.time()
    await asyncio.gather(gorev1(), gorev2())
    print(f"Toplam süre: {time.time() - baslangic:.2f} saniye")

asyncio.run(main())
# Çıktı: Görev 1 başladı
# Çıktı: Görev 2 başladı
```

```
# Çıktı: Görev 2 bitti
# Çıktı: Görev 1 bitti
# Çıktı: Toplam süre: 2.00 saniye
```

Görüldüğü gibi asenkron programlamada toplam süre 2 saniye oldu çünkü görevler paralel çalıştı.

12.2 async ve await Anahtar Kelimeleri

Bu kelimeler zaman uyumsuz programlama için olmazsa olmazlardır:

- ◊ **async**: Bir fonksiyonu zaman uyumsuz (**coroutine**) yapar.
- ◊ **await**: Asenkron bir işlemin tamamlanmasını bekler, beklerken diğer görevlerin çalışmasına izin verir.

```
import asyncio
```

```
async def selam_ver(isim: str, bekleme: int):
    print(f"{isim} için bekleniyor...")
    await asyncio.sleep(bekleme)
    print(f"Merhaba {isim}!")
    return f"{isim} selamlandı"
```

```
async def main():
    sonuc = await selam_ver("Ali", 2)
    print(sonuc)
```

```
asyncio.run(main())
# Çıktı: Ali için bekleniyor...
# Çıktı: Merhaba Ali!
# Çıktı: Ali selamlandı
```

12.3 Birden Fazla Görevi Birlikte Çalıştırma

Bu kütüphanedeki **gather()** fonksiyonu ile birden fazla asenkron görevi aynı anda başlatabilir ve hepsinin bitmesini bekleyebiliriz:

```
import asyncio
```

```
async def indir(dosya: str, sure: int):
    print(f"{dosya} indiriliyor...")
    await asyncio.sleep(sure)
    print(f"{dosya} indirildi!")
    return f"{dosya} hazır"
```

```
async def main():
    sonuclar = await asyncio.gather(
        indir("resim1.jpg", 2),
        indir("resim2.jpg", 3),
        indir("resim3.jpg", 1)
    )
    print("Tüm indirmeler tamamlandı:")
    for sonuc in sonuclar:
        print(f" - {sonuc}")
```

```
asyncio.run(main())
```

12.4 Görev Oluşturma

Zaman uyumsuz bir görev, **create_task()** ile bir **coroutine**'i **Task** nesnesine dönüştürüp arka planda çalıştırabiliriz:

```
import asyncio
```

```
async def sayac(isim: str, sayi: int):
    for i in range(sayi):
        print(f"{isim}: {i+1}")
        await asyncio.sleep(1)
```



```

    return f"{isim} tamamlandı"

async def main():
    # Görevleri oluştur (hemen başlatılır)
    gorev1 = asyncio.create_task(sayac("A", 3))
    gorev2 = asyncio.create_task(sayac("B", 4))

    # Her iki görevin tamamlanmasını bekle
    sonuc1 = await gorev1
    sonuc2 = await gorev2
    print(sonuc1)
    print(sonuc2)

asyncio.run(main())

```

12.5 Asenkron Dosya İşlemleri

Normal dosya işlemleri zaman uyumludur (**synchronous**) ve programı bloke eder. **aiofiles** kütüphanesi ile zaman uyumsuz (**asynchronous**) dosya işlemleri yapabiliriz:

```

import asyncio
import aiofiles

async def dosya_yaz(dosya_adi: str, icerik: str):
    async with aiofiles.open(dosya_adi, 'w', encoding='utf-8') as f:
        await f.write(icerik)
    print(f"{dosya_adi} yazıldı")

async def dosya_oku(dosya_adi: str) -> str:
    async with aiofiles.open(dosya_adi, 'r', encoding='utf-8') as f:
        icerik = await f.read()
    print(f"{dosya_adi} okundu")
    return icerik

async def main():
    # Paralel yazma
    await asyncio.gather(
        dosya_yaz("dosya1.txt", "İçerik 1"),
        dosya_yaz("dosya2.txt", "İçerik 2"),
        dosya_yaz("dosya3.txt", "İçerik 3")
    )

    # Paralel okuma
    sonuclar = await asyncio.gather(
        dosya_oku("dosya1.txt"),
        dosya_oku("dosya2.txt"),
        dosya_oku("dosya3.txt")
    )

    for i, icerik in enumerate(sonuclar, 1):
        print(f"Dosya {i}: {icerik}")

asyncio.run(main())

```

12.6 Asenkron HTTP İstekleri

Web istekleri genellikle yavaştır. **aiohttp** ile birden fazla isteği aynı anda yapabiliriz:

```

import asyncio
import aiohttp

async def sayfa_getir(url: str):
    async with aiohttp.ClientSession() as oturum:
        async with oturum.get(url) as yanit:

```

```

    icerik = await yanit.text()
    print(f"{url}: {len(icerik)} karakter")
    return len(icerik)

```

```

async def main():
    urllar = [
        "https://python.org",
        "https://github.com",
        "https://stackoverflow.com"
    ]

    sonuclar = await asyncio.gather(*[sayfa_getir(url) for url in urllar])
    print(f"Toplam: {sum(sonuclar)} karakter")

asyncio.run(main())

```

12.7 Asenkron Bağlam Yöneticileri

Asenkron bağlam yöneticileri (**context manager**) **async with** kullanılarak tanımlanır:

```

import asyncio

class VeriTabani:
    async def __aenter__(self):
        print("Veritabanına bağlanılıyor...")
        await asyncio.sleep(1)
        print("Bağlantı kuruldu")
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        print("Bağlantı kapatılıyor...")
        await asyncio.sleep(0.5)
        print("Bağlantı kapatıldı")

    async def sorgu_calistir(self, sorgu: str):
        print(f"Sorgu çalıştırılıyor: {sorgu}")
        await asyncio.sleep(0.5)
        return f"Sonuç: {sorgu}"

async def main():
    async with VeriTabani() as vt:
        sonuc = await vt.sorgu_calistir("SELECT * FROM users")
    print(sonuc)

asyncio.run(main())

```

12.8 Asenkron Yineleyiciler

Asenkron yineleyiciler (**iterator**) **async for** ile kullanılır:

```

import asyncio

class AsenkronSayac:
    def __init__(self, baslangic: int, bitis: int):
        self.mevcut = baslangic
        self.bitis = bitis

    def __aiter__(self):
        return self

    async def __anext__(self):
        if self.mevcut >= self.bitis:
            raise StopAsyncIteration
        await asyncio.sleep(0.5) # Her sayıda 0.5 saniye bekle
        return self.mevcut

```

```

        self.mevcut += 1
        return self.mevcut - 1

async def main():
    async for sayi in AsenkronSayac(1, 6):
        print(f"Sayı: {sayi}")

asyncio.run(main())

```

12.9 Zaman Aşımı

Bir görevin belirli bir sürede tamamlanmasını zorlamak için `wait_for()` kullanılır:

```

import asyncio

async def uzun_islem():
    print("Uzun işlem başladı")
    await asyncio.sleep(5)
    print("Uzun işlem bitti")
    return "Tamamlandı"

async def main():
    try:
        sonuc = await asyncio.wait_for(uzun_islem(), timeout=2.0)
        print(sonuc)
    except asyncio.TimeoutError:
        print("İşlem zaman aşımına uğradı!")

asyncio.run(main())

```

12.10 Asenkron Kuyruk

Üretici-tüketici deseni (`producer-consumer pattern`) için asenkron kuyruk kullanılır:

```

import asyncio

async def uretici(kuyruk: asyncio.Queue, urun_sayisi: int):
    for i in range(urun_sayisi):
        urun = f"Ürün-{i+1}"
        await kuyruk.put(urun)
        print(f"Üretildi: {urun}")
        await asyncio.sleep(0.5)

    # Tüketicilere bittiği sinyali
    await kuyruk.put(None)

async def tuketici(kuyruk: asyncio.Queue, isim: str):
    while True:
        urun = await kuyruk.get()
        if urun is None:
            await kuyruk.put(None) # Sinyali diğer tüketicilere aktar
            kuyruk.task_done()
            break

        print(f"{isim} işledi: {urun}")
        await asyncio.sleep(1)
        kuyruk.task_done()

async def main():
    kuyruk = asyncio.Queue()

    uretici_gorev = asyncio.create_task(uretici(kuyruk, 5))
    tuketici_gorevleri = [
        asyncio.create_task(tuketici(kuyruk, "Tüketici-1")),

```

```

    asyncio.create_task(tuketici(kuyruk, "Tüketici-2"))
]

await uretici_gorev
await asyncio.gather(*tuketici_gorevleri)

# asyncio.run(main())

```

12.11 Hata Yönetimi

Asenkron fonksiyonlarda da normal `try-except` yapısı kullanılır:

```

import asyncio

async def hata_olustur():
    await asyncio.sleep(1)
    raise ValueError("Bir hata olustu!")

async def guvenli_calistir():
    try:
        await hata_olustur()
    except ValueError as e:
        print(f"Hata yakalandi: {e}")
    finally:
        print("Temizlik islemleri yapildi")

asyncio.run(guvenli_calistir())

```

12.12 İlk Tamamlanan Görevi İşleme

Birden fazla görevden ilk tamamlananı işlemek için `as_completed()` fonksiyonu kullanılır:

```

import asyncio

async def gorev(isim: str, sure: int):
    await asyncio.sleep(sure)
    return f"{isim} tamamlandi ({sure}s)"

async def main():
    gorevler = [
        gorev("Hızlı", 1),
        gorev("Orta", 2),
        gorev("Yavaş", 3)
    ]

    for tamamlanan in asyncio.as_completed(gorevler):
        sonuc = await tamamlanan
        print(sonuc)

asyncio.run(main())

```

12.13 Özet ve En İyi Uygulamalar

- ◊ Girdi/çıkış (I/O) yoğun işlemler için `asyncio` idealdir.
- ◊ İşlemci yoğun kullanılacağı işlemler (**CPU-bound**) için `multiprocessing` tercih edilir.
- ◊ `await` kullanmadan `async` fonksiyonlar çağrılmamalıdır.
- ◊ Bloke eden (**blocking**) fonksiyonlar asenkron bloklarda kullanılmamalıdır.
- ◊ Engelleyici I/O veya CPU ağır işlerin `asyncio.to_thread()` (Python 3.9+) fonksiyonu ile ayrı bir thread'e aktarılabilir.
- ◊ `time.sleep()` yerine `await asyncio.sleep()` kullanılır.
- ◊ Uzun süren işlemler için `asyncio.wait_for()` ile zaman aşımı (**timeout**) eklenmelidir.
- ◊ Birden fazla görev için `gather()` veya `create_task()` kullanılır.
- ◊ Zaman uyumsuz işlemler için asenkron kütüphaneler tercih edilmelidir (`aiohttp`, `aiofiles` vb.).
- ◊ Olay döngüsü (**event loop**), `asyncio.run()` ile başlatılır.

- ◊ Hata yönetimi için `try-except` kullanılır.
- ◊ Bağlam Yöneticisi (`context manager`) için `async with` kullanılır.
- ◊ Yineleyiciler (`iterator`) için `async for` kullanılır.

12.14 Düşün ve Çöz

1. **Düşün (I/O-Bound vs. CPU-Bound ve Event Loop Bloğu):** Asenkron bir fonksiyon (`coroutine`) içerisinde `await asyncio.sleep(2)` yerine bloklayıcı `time.sleep(2)` veya 10 milyon elemanlı bir diziyi sıralayan yoğun bir döngü çalıştırılırsa Olay Döngüsü (`Event Loop`) nasıl etkilenir? Aynı anda çalışan diğer asenkron görevlerin durumunu açıklayınız.
2. **Çöz (Asenkron Web Kazıma / Scraper):** Verilen 5 farklı web sayfasının adresine `aiohttp` kütüphanesini kullanarak aynı anda (eşzamanlı) istek atan bir program yazınız.
 - ◊ Sayfalardan herhangi biri 3 saniye içinde yanıt vermezse `asyncio.wait_for()` ile zaman aşımına (`timeout`) uğramalı ve hata yönetimiyle programın çökmesi engellenmelidir.
 - ◊ Tamamlanan isteklerin durum kodlarını (`HTTP Status Code`) geliş sırasına göre (`asyncio.as_completed`) ekrana yazdırınız.
3. **Çöz (Üretici-Tüketici Pompası ve Sınırlı Kuyruk):** Maksimum boyutu 3 olan bir sınırlı kuyruk tanımlayınız; `asyncio.Queue(maxsize=3)`. 10 adet veriyi hızlıca üreten bir üretici ve bu verileri daha yavaş işleyen 2 ayrı tüketici hazırlayınız. Kuyruk dolduğunda üreticinin nasıl askıya alındığını (`backpressure`) konsol çıktılarıyla gözlemleyerek doğrulayınız.
4. **Düşün (Multithreading vs. Asyncio):** Girdi/Çıktı (I/O) yoğun bir problemde geleneksel çoklu iş parçacığı (`Multithreading`) kullanımı yerine tek bir iş parçacığında çalışan `asyncio` mimarisini tercih etmenin bellek tüketimi, yarış durumu (`race condition`) ve bağlam değiştirme (`context switching`) maliyetleri açısından avantajları nelerdir?

13. Bölüm

Veri Tipi İpuçları

Python dinamik tipli bir dildir. Yani bir değişkenin veri tipi çalışma zamanında (**runtime**) belirlenir. Ancak büyük projelerde bu durum hatalara sebep olabilir. İşte bu noktada **veri tipi ipuçları** (**type hints**) devreye girer. Veri tipi ipuçları Python 3.5 ile tanıtılmış ve sonraki sürümlerde geliştirilmiştir.

13.1 Veri Tipi İpuçlarının Temelleri

Veri tipi ipuçları, değişkenlere, fonksiyon parametrelerine ve dönüş değerlerine beklenen veri tipini belirtmemizi sağlar. Bu sadece bir ipucudur; Python bu tipleri çalışma zamanında kontrol etmez. Ancak **mypy** gibi araçlar bu ipuçlarını kullanarak kod analizini yapar.

```
def toplan(a: int, b: int) -> int:
    return a + b
```

```
sonuc: int = toplan(5, 3)
print(sonuc)
# Çıktı: 8
```

Yukarıdaki örnekte **a** ve **b** parametrelerinin **int** tipinde olması gerektiği, fonksiyonun **int** döndüreceği ve **sonuc** değişkeninin **int** olduğu belirtilmiştir.

13.2 Temel Veri Tipleri

```
isim: str = "İlhan"
yas: int = 35
boy: float = 1.75
aktif: bool = True
sayilar: list = [1, 2, 3]
koordinat: tuple = (10, 20)
ayarlar: dict = {"dil": "tr"}
```

13.3 Gelişmiş Veri Tipi İpuçları

Daha karmaşık veri tipi ipuçları için **typing** modülü kullanılır:

```
from typing import List, Dict, Optional, Union, Tuple
```

```
# Liste, elemanları int
sayilar: List[int] = [1, 2, 3, 4, 5]
```

```
# Sözlük, anahtarları str, değerleri int
puanlar: Dict[str, int] = {"Ali": 85, "Ayşe": 92}
```

```
# None olabilir
def bul_ogrenci(id: int) -> Optional[str]:
    if id == 1:
        return "Ahmet"
    return None
```

```
# Birden fazla tip kabul eder (Python 3.10+ için | kullanılabilir)
def isle(veri: Union[int, str]) -> str:
    return str(veri)
```

```
# Python 3.10+ ile daha basit sözdizimi
def isle_yeni(veri: int | str) -> str:
    return str(veri)
```

13.4 Python 3.10+ Modern Özellikler

Eşleşme Desenleri

Python 3.10 ile gelen en önemli özelliklerden biri eşleşme desenleridir (pattern matching). `match-case` ifadesi ile karmaşık koşullu ifadeleri daha okunabilir hale getirebiliriz:

```
def http_durumu(kod: int) -> str:
    match kod:
        case 200:
            return "Başarılı"
        case 404:
            return "Bulunamadı"
        case 500:
            return "Sunucu Hatası"
        case _:
            return "Bilinmeyen Durum"
```

```
print(http_durumu(200))
# Çıktı: Başarılı
```

Daha karmaşık eşleşmeler de yapılabilir:

```
def komut_isle(komut: list) -> str:
    match komut:
        case ["git", yön]:
            return f"{yön} yönüne gidiliyor"
        case ["al", nesne, miktar]:
            return f"{miktar} adet {nesne} alınıyor"
        case ["dur"]:
            return "Duruldu"
        case _:
            return "Bilinmeyen komut"
```

```
print(komut_isle(["git", "kuzey"]))
# Çıktı: kuzey yönüne gidiliyor
print(komut_isle(["al", "elma", 5]))
# Çıktı: 5 adet elma alınıyor
```

Union İşleci

Python 3.10'dan önce birden fazla tipi belirtmek için `Union` kullanıyorduk. Artık daha basit `|` işlecini kullanabiliriz:

```
# Eski yöntem (hala geçerli)
from typing import Union

def islem(sayi: Union[int, float]) -> Union[int, float]:
    return sayi * 2

# Yeni yöntem (Python 3.10+)
def islem_yeni(sayi: int | float) -> int | float:
    return sayi * 2

# None için de kullanılabilir
def bul(anahtar: str) -> str | None:
    sozluk = {"a": "değer"}
    return sozluk.get(anahtar)
```

13.5 Statik Tip Kontrolü

Python kodundaki veri tipi ipuçlarını kontrol eden bir araç `mypy`'dir. Kodunuzu çalıştırmadan önce tip hatalarını bulmanızı sağlar. Kurulumu `pip install mypy` komutu ile yapılır.

Örnek bir hatalı kod:

```
# ornek.py
def toplan(a: int, b: int) -> int:
```



```

    return a + b
# sonuc: int = toplam(5, "10") # Hata: str kullanıldı
    mypy ile kontrol:
$ mypy ornek.py
ornek.py:4: error: Argument 2 to "toplam" has incompatible type "str"; expected "int"

```

13.6 Generic Tipler ve TypeVar

Generic tipler, fonksiyonların farklı tiplerle çalışmasını sağlar:

```

from typing import TypeVar, List

T = TypeVar('T') # Generic tip değişkeni

def ilk_eleman(liste: List[T]) -> T | None:
    return liste[0] if liste else None

sayilar = [1, 2, 3]
print(ilk_eleman(sayilar)) # mypy int olarak anlar
# Çıktı: 1

kelimeler = ["merhaba", "dünya"]
print(ilk_eleman(kelimeler)) # mypy str olarak anlar
# Çıktı: merhaba

```

13.7 Literal ve Final

Literal, bir değişkenin sadece belirli değerleri alabileceğini belirtir:

```

from typing import Literal

def yon_degistir(yon: Literal["kuzey", "güney", "doğu", "batı"]) -> str:
    return f"{yon} yönüne dönülüyor"

print(yon_degistir("kuzey")) # Geçerli
# Çıktı: kuzey yönüne dönülüyor
# print(yon_degistir("yukarı")) # mypy hata verir

```

Final, bir değişkenin değiştirilmemesi gerektiğini belirtir:

```

from typing import Final

MAX_BAGLANTI: Final[int] = 100
# MAX_BAGLANTI = 200 # mypy hata verir

```

13.8 Protocol ve Structural Subtyping

Protocol, bir sınıfın hangi yöntemlere sahip olması gerektiğini belirtir. Kalıtım gerektirmez:

```

from typing import Protocol

class Cizilebilir(Protocol):
    def ciz(self) -> None:
        pass

class Daire:
    def ciz(self) -> None:
        print("Daire çiziliyor")

class Kare:
    def ciz(self) -> None:
        print("Kare çiziliyor")

def sekil_ciz(sekil: Cizilebilir) -> None:
    sekil.ciz()

```

```

sekil_ciz(Daire()) # Geçerli - ciz() metodu var
# Çıktı: Daire çiziliyor
sekil_ciz(Kare()) # Geçerli - ciz() metodu var
# Çıktı: Kare çiziliyor

```

13.9 TypedDict

`TypedDict`, sözlüklerin (`dict`) yapısını tanımlamak için kullanılır:

```

from typing import TypedDict

class Ogrenci(TypedDict):
    isim: str
    yas: int
    sinif: str

ogrenci: Ogrenci = {
    "isim": "Ahmet",
    "yas": 15,
    "sinif": "9-A"
}

def ogrenci_bilgisi(o: Ogrenci) -> str:
    return f"{o['isim']}, {o['yas']} yaşında, {o['sinif']} sınıfında"

print(ogrenci_bilgisi(ogrenci))
# Çıktı: Ahmet, 15 yaşında, 9-A sınıfında

```

13.10 Callable

Fonksiyonları parametre olarak alan fonksiyonlar için `Callable` kullanılır:

```

from typing import Callable

def uygula(func: Callable[[int, int], int], a: int, b: int) -> int:
    return func(a, b)

def topla(x: int, y: int) -> int:
    return x + y

def carp(x: int, y: int) -> int:
    return x * y

print(uygula(topla, 5, 3))
# Çıktı: 8
print(uygula(carp, 5, 3))
# Çıktı: 15

```

13.11 Özet ve En İyi Uygulamalar

- ♦ Veri tipi ipuçları opsiyoneldir ancak büyük projelerde çok yararlıdır.
- ♦ `mypy` ile kodunuzu düzenli olarak kontrol edin.
- ♦ Python 3.10+ kullanıyorsanız `|` operatörünü tercih edin.
- ♦ Generic tipler kod tekrarını önler.
- ♦ Protocol, duck typing'i tip güvenli hale getirir.
- ♦ Tüm fonksiyonlarda dönüş tipini belirtin.
- ♦ Public API'lerde veri tipi ipuçları kullanmak özellikle önemlidir.
- ♦ IDE'ler (PyCharm, VS Code) veri tipi ipuçlarını kullanarak daha iyi otomatik tamamlama sağlar.
- ♦ Karmaşık veri yapılarında `TypedDict` kullanın.
- ♦ Değiştirilmemesi gereken değerler için `Final` kullanın.

13.12 Düşün ve Çöz

1. **Düşün (Nominal vs. Structural Subtyping):** Geleneksel nesne yönelimli programlamadaki kalıtım tabanlı tip denetimi (**Nominal Subtyping**) ile Python'ın **Protocol** yapısı üzerinden sağladığı örtük tip

denetimi (**Structural Subtyping**) arasındaki temel farklar nelerdir? C++'taki **virtual** sınıflar veya soyut taban sınıfları (**ABC**) ile kıyaslayarak açıklayınız.

2. **Çöz (Generic Yapılar ve TypeVar):** Son giren ilk çıkar (**Last-In-First-Out-LIFO**) prensibiyle çalışan generic bir **Yigit[T]** (**Stack**) sınıfı tasarlayınız. Sınıf içerisinde eleman ekleme (**it: T**), eleman çıkarma (**pop() -> T**) ve yığının boş olup olmadığını kontrol eden (**bos_mu() -> bool**) metotları yazıp tüm tip ipuçlarını ekleyiniz. **mypy** ile tip denetimini doğrulayınız.
3. **Çöz (Match-Case ve Karmaşık Veri Tipleri):** Bir e-ticaret sistemindeki sipariş durumlarını temsil eden ve sözlüklerden oluşan bir yapıyı işleyen fonksiyon yazınız. **match-case** kullanarak aşağıdaki senaryoları ayırıştırınız:
 - ◊ Odeme bekleniyor durumu: {"durum": "beklemede", "tutar": float}
 - ◊ Kargoya verildi durumu: {"durum": "kargoda", "takip_no": str}
 - ◊ İptal durumu ve nedeni: {"durum": "iptal", "neden": str}
4. **Düşün (Çalışma Zamanı Performansı ve Type Hints):** Python yorumlayıcısı, fonksiyon parametrelerine eklediğimiz **a: int** gibi veri tipi ipuçlarını çalışma zamanında (**runtime**) veri tipi zorlaması (**type enforcement**) amacıyla kullanır mı? Tip ipuçlarının uygulamamızın çalışma hızına (performansına) doğrudan bir etkisi var mıdır? **__annotations__** niteliği üzerinden tartışınız.

14. Bölüm

Veri Bilimi, Yapay Zekâ ve Makine Öğrenmesi

Bu bölümde, Python'ın modern veri işleme ve yapay zekâ ekosistemindeki gücünü, temel matematiksel prensipler ve pratik uygulamalar eşliğinde inceleyeceğiz. Amaç, hazır algoritmaları çalıştırmaktan ziyade; verinin hazırlanması, modellenmesi ve değerlendirilmesi süreçlerinin arka planındaki mantığı kavramaktır.

Bu bölüm sizi bu alanlara derinlemesine öğretmek değil, var olduklarını ve Python'la nasıl göründüklerini göstermek içindir

14.1 Veri Analizi ve Manipülasyonu (Pandas & Matplotlib)

Veri bilimi sürecinin ilk ve en kritik adımı veriyi temizlemek, dönüştürmek ve analiz etmektir. Python ekosisteminde bu sürecin kalbinde **Pandas** ve görselleştirme için **Matplotlib/Seaborn** yer alır.

§14.1.1 Pandas Veri Yapıları: Series ve DataFrame

Pandas, NumPy matris altyapısını kullanarak etiketli ve heterojen verileri yönetmeyi sağlar.

- ♦ **Series:** Tek boyutlu, indekslenmiş veri dizisidir.
- ♦ **DataFrame:** Satır ve sütun etiketlerine sahip iki boyutlu tablo yapısıdır.

Pandas ile Veri Filtreleme ve Gruplama

```
import pandas as pd
import numpy as np

# Örnek bir veri seti oluşturma
data = {
    'Geliştirici': ['Ahmet', 'Ayşe', 'Mehmet', 'Canan', 'Murat'],
    'Deneyim_Yili': [2, 5, 8, 1, 4],
    'Maaş': [45000, 75000, 110000, 38000, 62000],
    'Uzmanlık': ['Python', 'Python', 'Java', 'C++', 'Python']
}

df = pd.DataFrame(data)

# Veriyi Filtreleme ve Gruplama (Aggregation)
python_uzmanlari = df[df['Uzmanlik'] == 'Python']
ort_maas = df.groupby('Uzmanlik')['Maaş'].mean()

print("Python Uzmanları:\n", python_uzmanlari)
print("\nUzmanlığa Göre Ortalama Maaş:\n", ort_maas)
```

§14.1.2 Eksik Veri ve Aykırı Değer Yönetimi

Gerçek dünya verisi genellikle gürültülü ve eksiktir.

- ♦ **Eksik Veri Tespiti ve Doldurulması:** `isna()`, `dropna()`, `fillna()` yöntemleri kullanılır.
- ♦ **Aykırı Değer Analizi:** Z-Score veya IQR (*Interquartile Range*) yöntemleriyle tespit edilerek törpülenir veya veri setinden çıkarılır.

Eksik Verilerin Ortalama ile Doldurulması;

```
# Eksik veri simülasyonu ve ortalama ile doldurma
df.loc[2, 'Maaş'] = np.nan
df['Maaş_Temiz'] = df['Maaş'].fillna(df['Maaş'].mean())
```

§14.1.3 Veri Görselleştirme (Matplotlib & Seaborn)

Grafikler, verideki eğilimleri (*trends*) ve korelasyonları anlamamanın en hızlı yoludur. Seaborn ve Matplotlib ile Görselleştirme;

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Görselleştirme Stil Ayarı
sns.set_theme(style="whitegrid")

plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='Deneyim_Yili', y='Maaş_Temiz', hue='Uzmanlik', s=100)
plt.title("Deneyim Yılına Göre Maaş Dağılımı")
plt.xlabel("Deneyim (Yıl)")
plt.ylabel("Maaş (TL)")
plt.savefig("deneyim_maas_grafik.png") # Grafiği kaydetme
plt.show()
```

14.2 Makine Öğrenmesine Giriş (Scikit-Learn)

Makine öğrenmesi, açıkça programlanmadan veriden öğrenen ve tahmin yapan algoritmalar bütünüdür. Üç ana kategoriye ayrılır:

1. **Denetimli Öğrenme (Supervised Learning):** Girdi ve hedef çıktı (etiket) verisi mevcuttur (Sınıflandırma, Regresyon).
2. **Denetimsiz Öğrenme (Unsupervised Learning):** Etiket yoktur, veri içi kalıplar ve kümelemeler bulunur (K-Means, PCA).
3. **Pekiştirmeli Öğrenme (Reinforcement Learning):** Ödül-ceza mekanizmasıyla ajan öğrenmesidir.

§14.2.1 Regresyon Problemleri: Doğrusal Regresyon

Sürekli bir değeri tahmin etmek için kullanılır. Matematiksel olarak model şu şekilde ifade edilir:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad (14.1)$$

Scikit-Learn ile Doğrusal Regresyon;

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Sentetik Veri Oluşturma
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]]) # Deneyim
y = np.array([30, 35, 45, 50, 65, 70, 85, 90]) # Maaş (Bin TL)

# Veriyi Eğitim ve Test Setine Bölme
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Model Eğitimi
model = LinearRegression()
model.fit(X_train, y_train)

# Tahmin ve Değerlendirme
y_pred = model.predict(X_test)
print(f"R2 Skoru (Başarı Oranı): {r2_score(y_test, y_pred):.2f}")
print(f"Model Katsayısı (Eğim - w): {model.coef_[0]:.2f}")
```

§14.2.2 Sınıflandırma Problemleri ve Performans Metrikleri

Kategorik çıktıları tahmin etmede kullanılır. Model değerlendirmesinde karmaşıklık matrisi (*Confusion Matrix*) tercih edilir. K-En Yakın Komşu (KNN) Sınıflandırıcısı;

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Iris Veri Setini Yükleme
iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(iris.data, iris.target, test_size=0.3, random_state=42)

# K-En Yakın Komşu (KNN) Modeli
knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(X_tr, y_tr)

# Performans Metrikleri
predictions = knn.predict(X_te)
print("Karmaşıklık Matrisi:\n", confusion_matrix(y_te, predictions))
print("\nSınıflandırma Raporu:\n", classification_report(y_te, predictions))
```

14.3 Derin Öğrenme ve Yapay Sinir Ağlarına (ANN) Giriş

Derin Öğrenme, insan beynindeki nöron yapısından esinlenen ve yüksek boyutlu verilerden otomatik öznetelik çıkarabilen çok katmanlı yapay sinir ağları mimarisidir.

§14.3.1 Bir Nöronun Anatomisi ve İleri Besleme

Bir nöron, girdileri ağırlıklarla (w) çarpar, bir sapma (b) ekler ve sonucu doğrusal olmayan bir Aktivasyon Fonksiyonuna (σ) tabi tutar:

$$z = \sum_{i=1}^n (w_i \cdot x_i) + b \quad (14.2)$$

$$a = \sigma(z) \quad (14.3)$$

Yaygın aktivasyon fonksiyonları: **ReLU** ($f(x) = \max(0, x)$) ve **Sigmoid** ($f(x) = \frac{1}{1+e^{-x}}$).

§14.3.2 PyTorch ile İlk Yapay Sinir Ağı Modeli

PyTorch ile Sinir Ağı Mimarisi ve Eğitim Döngüsü;

```
import torch
import torch.nn as nn
import torch.optim as optim

# 1. Sinir Ağı Mimarisi Tanımlama
class BasitAgi(nn.Module):
    def __init__(self, girdi_boyutu, gizli_boyut, cikti_boyutu):
        super(BasitAgi, self).__init__()
        self.fc1 = nn.Linear(girdi_boyutu, gizli_boyut) # Tam Bağlantılı Katman 1
        self.relu = nn.ReLU() # Aktivasyon Fonksiyonu
        self.fc2 = nn.Linear(gizli_boyut, cikti_boyutu) # Tam Bağlantılı Katman 2

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

# 2. Model, Kayıp Fonksiyonu ve Optimizatör Tanımı
model = BasitAgi(girdi_boyutu=4, gizli_boyut=8, cikti_boyutu=3)
criterion = nn.CrossEntropyLoss() # Kayıp Fonksiyonu (Loss)
optimizer = optim.Adam(model.parameters(), lr=0.01) # Optimizasyon (Adam)

# 3. Sentetik Veri ile Eğitim Döngüsü (Training Loop)
ornek_girdi = torch.randn(10, 4) # 10 örnek, 4 öznetelik
ornek_etiket = torch.tensor([0, 1, 2, 1, 0, 2, 1, 0, 0, 1])

for epoch in range(50):
    # İleri Besleme (Forward Pass)
    cikilar = model(ornek_girdi)
    loss = criterion(cikilar, ornek_etiket)

    # Geri Yayılım (Backpropagation) ve Ağırlık Güncelleme
    optimizer.zero_grad() # Gradyanları sıfırla
    loss.backward() # Gradyanları hesapla
    optimizer.step() # Ağırlıkları güncelle
```

```
if (epoch+1) % 10 == 0:
    print(f"Epoch [{epoch+1}/50], Kayıp (Loss): {loss.item():.4f}")
```

14.4 Doğal Dil İşleme (NLP) ve Modern Dil Modelleri

Doğal Dil İşleme, insan dilinin bilgisayarlar tarafından anlaşılması, işlenmesi ve üretilmesiyle ilgilenir.

§14.4.1 Metin Ön İşleme ve Vektör Temsilleri

- ◊ **Tokenization:** Cümlelerin kelimelere veya alt kelimelere ayrılması.
- ◊ **TF-IDF:** Kelimenin dokümandaki önemini istatistiksel olarak hesaplar.
- ◊ **Word Embeddings:** Kelimeleri anlamsal yakınlıklarına göre vektör uzayında konumlandırır:

$$\text{Vektör(Kral)} - \text{Vektör(Erkek)} + \text{Vektör(Kadın)} \approx \text{Vektör(Kraliçe)} \quad (14.4)$$

§14.4.2 Transformer Mimarisi ve LLM Entegrasyonu

Modern dil modellerinin temelinde *Attention* (Dikkat) mekanizması yatar. Hugging Face kütüphanesi ile hazır bir duygu analizi hattı: Hugging Face Pipeline Kullanımı

```
from transformers import pipeline

# Duygu Analizi (Sentiment Analysis) hattı
duygu_analizi = pipeline("sentiment-analysis")

metin = "Bu kitabın anlatım dili ve kod örneklerinin düzeni gerçekten mükemmel!"
sonuc = duygu_analizi(metin)

print("Analiz Sonucu:", sonuc)
```

14.5 Bölüm Özeti ve Egzersizler

§14.5.1 Bölüm Özeti

- ◊ **Pandas**, yapısal verileri işlemek ve analiz etmek için temel araçtır.
- ◊ **Makine Öğrenmesi**, verideki ilişkileri matematiksel fonksiyonlara dönüştürür.
- ◊ **Derin Öğrenme**, karmaşık veri tiplerinde (görüntü, metin) **PyTorch** esnekliğiyle öne çıkar.
- ◊ **NLP**, metin verisini anlamsal vektör uzaylarına taşıyarak modern dil modellerine zemin hazırlar.

§14.5.2 Düşün ve Çöz: Algoritmik ve Teorik Sorular

- [Veri İşleme ve Pandas]**
Bir e-ticaret platformunun satış verilerini içeren bir CSV dosyasında **Fiyat** sütununda bazı eksik değerler (**NaN**) mevcuttur.
(a) Bu eksik değerleri doğrudan silmek (**dropna**) ile kategori ortalamasıyla doldurmak (**fillna**) arasındaki avantaj/dezavantaj farklarını tartışınız.
(b) Pandas kullanarak **Kategori** bazında **Fiyat** ortalamasını hesaplayıp, her eksik fiyatı **kendi kategorisinin ortalamasıyla** dolduran Python kodunu yazınız.
- [Makine Öğrenmesi - Overfitting Tespiti]**
Geliştirdiğiniz bir makine öğrenmesi modeli **Eğitim Setinde %99** başarı gösterirken, **Test Setinde %62** başarı gösteriyor.
(a) Bu durum makine öğrenmesinde hangi problem olarak adlandırılır?
(b) Bu sorunu çözmek için yazılım ve model seviyesinde alınabilecek 3 farklı önlemi açıklayınız.
- [Derin Öğrenme - Matematiksel Mantık]**
İki girdili ($x_1 = 0.5, x_2 = 0.8$) tek bir nörona sahipsiniz. Ağırlıklar $w_1 = 0.4, w_2 = -0.2$ ve sapma (bias) $b = 0.1$ olarak verilmiştir. Nöronun aktivasyon fonksiyonu **Sigmoid** ($\sigma(z) = \frac{1}{1+e^{-z}}$) olduğuna göre, bu nöronun üreteceği net çıktıyı (a) adım adım hesaplayınız.
- [NLP - Kodlama Projesi]**
Verilen bir metin listesindeki cümleleri küçük harfe çeviren, noktalama işaretlerini temizleyen ve ardından kelime frekanslarını bir Python **dict** yapısı olarak döndüren **metin_analiz_motoru(metin_listesi)** fonksiyonunu saf Python ile yazınız.

15. Bölüm

Web Geliştirme

FastAPI, modern, hızlı (yüksek performanslı) ve Python 3.6+ ile web API'leri oluşturmak için tasarlanmış bir web çerçevesidir (**framework**). Otomatik API dokümantasyonu, **veri tipi ipuçları** (**type hints**) desteği ve yüksek performansı ile popülerdir.

FastAPI'nin Özellikleri;

- ◊ **Hızlı**: NodeJS ve Go kadar hızlı performans
- ◊ **Kolay**: Öğrenmesi ve kullanması kolay
- ◊ **Otomatik Dokümantasyon**: Swagger UI ve ReDoc otomatik oluşturulur
- ◊ **Tür İpuçları**: Python veri tipi ipuçları (**veri tipi hints**) ile güçlü tip kontrolü
- ◊ **Asenkron Destek**: **async/await** tam desteği
- ◊ **Veri Doğrulama** (**validation**): Pydantic ile otomatik veri doğrulama
- ◊ **Bağımlılık Enjeksiyonu**: Bağımlılık enjeksiyonu (**dependency injection**) sistemi

Kurulum için aşağıdaki iki komut çalıştırılır;

```
$ pip install fastapi
$ pip install "uvicorn[standard]" # ASGI sunucusu
```

15.1 İlk FastAPI Uygulaması

İlk uygulamamız aşağıdaki gibi olsun;

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def ana_sayfa():
    return {"mesaj": "Merhaba FastAPI!"}

@app.get("/selam/{isim}")
def selam_ver(isim: str):
    return {"mesaj": f"Merhaba {isim}!"}
```

Uygulamayı çalıştırma:

```
$ uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['D:\\Python\\Samples\\fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [13032] using WatchFiles
INFO: Started server process [3376]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Tarayıcıda aşağıdaki URL'ler açılır;

- ◊ <http://127.0.0.1:8000> - Ana sayfa
- ◊ <http://127.0.0.1:8000/selam/Ali> - Selamlaşma
- ◊ <http://127.0.0.1:8000/docs> - Otomatik API dokümantasyonu (Swagger UI)
- ◊ <http://127.0.0.1:8000/redoc> - Alternatif dokümantasyon (ReDoc)

15.2 Path Parametreleri

URL'den değer almak için yol (**path**) parametreleri kullanılır;

```
@app.get("/kullanici/{kullanici_id}")
def kullanici_getir(kullanici_id: int):
    return {"kullanici_id": kullanici_id, "isim": "Ali"}
```

```
@app.get("/urunler/{kategori}/{urun_id}")
def urun_detay(kategori: str, urun_id: int):
    return {
        "kategori": kategori,
        "urun_id": urun_id,
        "isim": f"{kategori} ürünü {urun_id}"
    }
```

15.3 Query Parametreleri

URL'deki query string'den değerler aşağıdaki şekilde alınabilir;

```
@app.get("/ara")
def ara(q: str, limit: int = 10, offset: int = 0):
    """
    q: arama terimi (zorunlu)
    limit: sonuç sayısı (varsayılan 10)
    offset: başlangıç noktası (varsayılan 0)
    """
    return {
        "arama": q,
        "limit": limit,
        "offset": offset,
        "sonuclar": [f"Sonuç {i}" for i in range(offset, offset + limit)]
    }

@app.get("/filtrele")
def filtre(kategori: Optional[str] = None, min_fiyat: Optional[float] = None):
    """Opsiyonel parametreler"""
    filtre = {}
    if kategori:
        filtre["kategori"] = kategori
    if min_fiyat:
        filtre["min_fiyat"] = min_fiyat
    return {"filtre": filtre}
```

Buradaki uygulama aşağıdaki örneklerde verilen query string'den değerler alabilir;

- ◊ <http://127.0.0.1:8000/ara?q=python>
- ◊ <http://127.0.0.1:8000/ara?q=python&limit=5&offset=10>
- ◊ http://127.0.0.1:8000/filtrele?kategori=kitap&min_fiyat=50

15.4 Request Body

POST isteklerinde veri göndermek için Pydantic modelleri kullanılır:

```
from typing import Optional
from pydantic import BaseModel, Field, EmailStr

class Kullanici(BaseModel):
    isim: str = Field(..., min_length=2, max_length=50)
    email: EmailStr
    yas: int = Field(..., gt=0, lt=150)
    aktif: bool = True
    aciklama: Optional[str] = None

@app.post("/kullanici")
def kullanici_olustur(kullanici: Kullanici):
    return { "mesaj": "Kullanıcı oluşturuldu", "kullanici": kullanici }

class Urunguncelle(BaseModel):
    isim: Optional[str] = None
    fiyat: Optional[float] = Field(None, gt=0)
    stok: Optional[int] = Field(None, ge=0)
```

```
@app.put("/urun/{urun_id}")
def urun_guncelle(urun_id: int, urun: UrunGuncelle):
    return {
        "urun_id": urun_id,
        # Pydantic v2 için model_dump(exclude_unset=True) kullanılır
        "guncellenen": urun.model_dump(exclude_unset=True),
    }
```

15.5 HTTP Yöntemleri

Beş adet http yöntemine ilişkin örnekler aşağıda verilmiştir;

```
from fastapi import status

# GET - Veri okuma
@app.get("/urunler")
def urunleri_listele():
    return [{"id": 1, "isim": "Ürün 1"}, {"id": 2, "isim": "Ürün 2"}]

# POST - Yeni veri oluşturma
@app.post("/urunler", status_code=status.HTTP_201_CREATED)
def urun_olustur(urun: dict):
    return {"mesaj": "Ürün oluşturuldu", "urun": urun}

# PUT - Veri güncelleme (tüm alanlar)
@app.put("/urunler/{urun_id}")
def urun_guncelle_put(urun_id: int, urun: dict):
    return {"mesaj": f"Ürün {urun_id} güncellendi", "urun": urun}

# PATCH - Kısmi güncelleme
@app.patch("/urunler/{urun_id}")
def urun_guncelle_patch(urun_id: int, urun: dict):
    return {"mesaj": f"Ürün {urun_id} kısmen güncellendi", "urun": urun}

# DELETE - Veri silme
@app.delete("/urunler/{urun_id}", status_code=status.HTTP_204_NO_CONTENT)
def urun_sil(urun_id: int):
    return None # 204 için içerik dönmeyin
```

15.6 Response Models

Dönen veriyi şekillendirmek için `response_model` kullanılır:

```
class KullaniciGiris(BaseModel):
    email: EmailStr; sifre: str

class KullaniciCikis(BaseModel):
    id: int
    isim: str
    email: EmailStr
    # sifre alanı yok!

@app.post("/giris", response_model=KullaniciCikis)
def giris_yap(kullanici: KullaniciGiris):
    # Veritabanından kullanıcı bul ve doğrula
    return {
        "id": 1,
        "isim": "Ali Veli",
        "email": kullanici.email,
        "sifre": "hash123" # Bu alan response'a dahil edilmez
    }
```

15.7 Hata Yönetimi

```
from fastapi import HTTPException

# Basit hata
@app.get("/kullanici/{kullanici_id}")
def kullanici_getir(kullanici_id: int):
    if kullanici_id > 100:
        raise HTTPException(status_code=404, detail="Kullanıcı bulunamadı")
    return {"kullanici_id": kullanici_id}

# Detaylı hata
@app.get("/urun/{urun_id}")
def urun_getir(urun_id: int):
    if urun_id < 1:
        raise HTTPException(
            status_code=400,
            detail="Ürün ID 1'den küçük olamaz",
            headers={"X-Error": "Geçersiz ID"}
        )
    return {"urun_id": urun_id}
```

15.8 Dependency Injection

Ortak kodları paylaşmak ve bağımlılıkları enjekte etmek için:

```
from fastapi import Depends, Header

# Basit dependency
def ortak_parametreler(q: Optional[str] = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
def itemleri_oku(common: dict = Depends(ortak_parametreler)):
    return common

# Token kontrolü
def token_kontrol(x_token: str = Header(...)):
    if x_token != "gizli-token":
        raise HTTPException(status_code=400, detail="Geçersiz token")
    return x_token

@app.get("/korunmus")
def korunmus_endpoint(token: str = Depends(token_kontrol)):
    return {"mesaj": "Erişim izni verildi"}

# Veritabanı bağlantısı
def veritabani_baglantisigi():
    db = {"baglanti": "aktif"}
    try:
        yield db
    finally:
        print("Veritabanı bağlantısı kapatıldı")

@app.get("/veriler")
def veri_oku(db: dict = Depends(veritabani_baglantisigi)):
    return {"veritabani": db["baglanti"], "veriler": [1, 2, 3]}
```

15.9 Asenkron Endpoint'ler

```
import asyncio
import aiohttp

@app.get("/sync")
```

```

def senkron_endpoint():
    # Normal fonksiyon
    return {"tip": "senkron"}

@app.get("/async")
async def asenkron_endpoint():
    # Asenkron fonksiyon
    await asyncio.sleep(1)
    return {"tip": "asenkron"}

@app.get("/harici-api")
async def harici_api_cagir():
    async with aiohttp.ClientSession() as session:
        async with session.get("https://api.github.com") as response:
            veri = await response.json()
            return {"durum": response.status, "veri": veri}

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            veri = await websocket.receive_text()
            await websocket.send_text(f"Mesaj alındı: {veri}")
    except Exception:
        print("Bağlantı kapatıldı")

```

15.10 Dosya Yükleme

Dosya yükleme (file upload) örneği aşağıda verilmiştir;

```

from typing import List
from fastapi import File, UploadFile

@app.post("/dosya-yukle")
async def dosya_yukle(dosya: UploadFile = File(...)):
    icerik = await dosya.read()
    return {
        "dosya_adi": dosya.filename,
        "boyut": len(icerik),
        "icerik_tipi": dosya.content_type
    }

@app.post("/coklu-dosya")
async def coklu_dosya_yukle(dosyalar: List[UploadFile] = File(...)):
    return [
        {
            "dosya_adi": dosya.filename,
            "boyut": len(await dosya.read())
        }
        for dosya in dosyalar
    ]

```

15.11 Çapraz Kaynak Paylaşımı

Çapraz kaynak paylaşımı (Cross-Origin Resource Sharing-CORS) tekniğinde ön yüz (frontend) uygulamalarından API'ye erişim kontrol altına alınır;

```

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "https://myapp.com"],
    allow_credentials=True,

```

```

allow_methods=["*"],
allow_headers=["*"],
)

# Geliştirme için tüm kaynaklara izin ver (production'da kullanma!)
# allow_origins=["*"]

```

15.12 Arka Planda Görev Çalıştırma

İstek tamamlandıktan sonra arka planda görev çalıştırma (**background task**):

```

from fastapi import BackgroundTasks

def email_gonder(email: str, mesaj: str):
    # Email gönderme simülasyonu
    import time
    time.sleep(5) # Uzun süren işlem
    print(f"{email} adresine email gönderildi: {mesaj}")

@app.post("/kayit")
async def kayit_ol(email: EmailStr, arka_plan: BackgroundTasks):
    arka_plan.add_task(email_gonder, email, "Hoş geldiniz!")
    return {"mesaj": "Kayıt başarılı, email gönderiliyor..."}

```

15.13 Veritabanı Entegrasyonu (SQLAlchemy)

```

from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

# Veritabanı kurulumu
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

#declarative_base() kullanımı SQLAlchemy 2.0 ile birlikte yerini DeclarativeBase sınıfına
→ bırakmıştır.

# Model tanımı
class KullaniciDB(Base):
    __tablename__ = "kullanicilar"
    id = Column(Integer, primary_key=True, index=True)
    isim = Column(String)
    email = Column(String, unique=True, index=True)

Base.metadata.create_all(bind=engine)

# Dependency
def veritabani_oturumu():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# CRUD (Create-Read-Update-Delete) işlemleri
@app.post("/kullanici/")
def kullanici_olustur(kullanici: Kullanici, db: Session = Depends(veritabani_oturumu)):
    db_kullanici = KullaniciDB(isim=kullanici.isim, email=kullanici.email)
    db.add(db_kullanici)
    db.commit()
    db.refresh(db_kullanici)

```

```

    return db_kullanici

@app.get("/kullanici/")
def kullanicilari_listele(skip: int = 0, limit: int = 10, db: Session =
    ↳ Depends(veritabani_oturumu)):
    kullanicilar = db.query(KullaniciDB).offset(skip).limit(limit).all()
    return kullanicilar

```

15.14 Authentication - JWT Token

```

from datetime import datetime, timedelta
from jose import JWTError, jwt
from passlib.context import CryptContext

SECRET_KEY = "gizli-anahtar-buraya"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def sifre_hash(sifre: str) -> str:
    return pwd_context.hash(sifre)

def sifre_dogrula(sifre: str, hash_sifre: str) -> bool:
    return pwd_context.verify(sifre, hash_sifre)

def token_olustur(veri: dict):
    to_encode = veri.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

@app.post("/token")
def giris(email: str, sifre: str):
    # Kullanıcı doğrulama (basitleştirilmiş)
    if email == "test@test.com" and sifre == "sifre123":
        token = token_olustur({"sub": email})
        return {"access_token": token, "token_type": "bearer"}
    raise HTTPException(status_code=401, detail="Geçersiz kimlik bilgileri")

```

15.15 Modüler Yapı

Büyük projelerde kodları modüllere ayırmak için `APIRouter` yapısı kullanılır:

```

# routers/kullanici.py
from fastapi import APIRouter

router = APIRouter(
    prefix="/kullanici",
    tags=["kullanici"]
)

@router.get("/")
def kullanicilari_listele():
    return [{"id": 1, "isim": "Ali"}]

@router.get("/{kullanici_id}")
def kullanici_detay(kullanici_id: int):
    return {"id": kullanici_id, "isim": "Ali"}

# main.py
from fastapi import FastAPI

```



```
from routers import kullanicilar
```

```
app = FastAPI()
app.include_router(kullanicilar.router)
```

15.16 Gerçek Zamanlı İletişim

Gerçek zamanlı iletişim için WebSocket yapısı kullanılır:

```
from fastapi import WebSocket

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            veri = await websocket.receive_text()
            await websocket.send_text(f"Mesaj alındı: {veri}")
    except:
        print("Bağlantı kapatıldı")
```

15.17 API'leri Test Etme

1. İlk yöntem tarayıcı (browser) üzerinden test etmektir: <http://localhost:8000/docs>
2. İkinci yöntem CURL dili ile test etmektir:
curl -X GET "http://localhost:8000/items/5"
3. Üçüncü yöntem ise Python dili ile requests kütüphanesi kullanarak test etmektir:
import requests

```
response = requests.get("http://localhost:8000/items/5")
print(response.json())
```

15.18 En İyi Uygulamalar

- ◇ Type hints her yerde kullanın.
- ◇ Response models ile veri şekillendirme yapın.
- ◇ Dependency injection ile kod tekrarını önleyin.
- ◇ Async endpoint'leri I/O işlemleri için kullanın.
- ◇ Router'larla kodları modüllere ayırın.
- ◇ Environment variables ile yapılandırma yapın.
- ◇ Logging ekleyin.
- ◇ Rate limiting uygulayın.
- ◇ HTTPS kullanın (production'da).
- ◇ Input validation'ı Pydantic'e bırakın.
- ◇ Database connection pooling kullanın.
- ◇ Background tasks ile uzun işlemleri yönetin.

15.19 Web Proje Yapısı

```
my-fastapi-app/
├── app/
│   ├── __init__.py
│   └── main.py
│       ├── models/
│       │   ├── __init__.py
│       │   └── kullanici.py
│       ├── routers/
│       │   ├── __init__.py
│       │   └── urunler.py
│       ├── schemas/
│       │   ├── __init__.py
│       │   └── kullanici.py
│       ├── database.py
│       └── dependencies.py
```

```
tests/
├── __init__.py
├── test_main.py
├── requirements.txt
└── .env
```

15.20 Düşün ve Çöz

1. **Düşün (Asenkron vs. Senkron Def Karışıklığı):** FastAPI'de bir endpoint'i `async def` olarak tanımladığımızda bloklayıcı (*blocking*) bir I/O işlemi (örneğin standart `requests.get()` veya `time.sleep()`) çalıştırırsak Olay Döngüsü (*Event Loop*) nasıl etkilenir? Bunun yerine normal `def` kullandığımızda FastAPI'nin arkadaki Thread Pool (*ThreadPoolExecutor*) yönetimi nasıl değişir?
2. **Çöz (Pydantic ile Özel Doğrulama / Custom Validation):** Yeni bir kullanıcı kaydı için Pydantic `BaseModel` kullanılarak bir `KullaniciKayit` şeması oluşturunuz.
 - ◇ Parola alanı en az 8 karakter olmalı, en az bir büyük harf ve bir rakam içermelidir.
 - ◇ Parola doğrulama (`parola_tekrar`) alanı eklenecektir. Her iki parolanın eşleşip eşleşmediğini kontrol eden bir `@field_validator` veya `@model_validator` yazınız.
3. **Çöz (Dependency Injection ile Sayfalama ve Yetkilendirme):** `Depends` yapısını kullanarak gelen istemin HTTP Başlığındaki (*Header*) `X-API-Key` değerini kontrol eden bir bağımlılık (*dependency*) yazınız. Eğer anahtar "secret-key-123" değilse `HTTPException(status_code=403)` fırlatınız. Bu bağımlılığı korumalı bir `/raporlar` endpoint'ine enjekte ediniz.
4. **Düşün (Veri Güvenliği ve Response Model):** Veritabanımızdaki `Kullanici` tablosunda `sifre_hash` niteliği bulunmaktadır. Bir endpoint'ten kullanıcı verisini döndürürken `response_model` kullanmanın, hassas verilerin (*sensitive data*) istemciye (frontend/mobile) sızmasını önlemedeki rolünü açıklayınız.

16. Bölüm

İyi Kodlama Uygulamaları ve Temiz Kod

Bu bölümde Python'da profesyonel kod yazma prensiplerini, kod kalitesini artırma yöntemlerini ve yaygın kullanılan **en iyi uygulamaları** (**best practice**) inceleyeceğiz.

16.1 Python Stil Rehberi

PEP 8, Python kodu için resmi stil rehberidir. Kodun okunabilirliğini artırır.

Girinti

```
# DOĞRU
def hesapla(x, y):
    sonuc = x + y
    return sonuc

# YANLIŞ - Tab kullanma, 4 boşluk kullan!
def hesapla(x, y):
    sonuc = x + y
    return sonuc
```

Satır Uzunluğu

```
# DOĞRU - Maksimum 79 karakter
def uzun_fonksiyon_ismi(parametre1, parametre2,
    parametre3, parametre4):
    return parametre1 + parametre2 + parametre3 + parametre4

# Uzun string'ler için
mesaj = (
    "Bu çok uzun bir mesajdır. "
    "Birden fazla satıra bölünmüştür. "
    "Okunabilirliği artırır."
)
```

Satır uzunluğu hususunda 79 karakter kuralına sadık kalınırken, **Black** biçimlendiricisinin varsayılan satır uzunluğu 88 karakter kabul edilir.

İsimlendirme Kuralları

```
# Değişkenler ve fonksiyonlar: snake_case
kullanici_adi = "Ali"
toplam_fiyat = 100.50

def hesapla_toplam(fiyat, vergi):
    return fiyat * (1 + vergi)

# Sınıflar: PascalCase
class KullaniciYoneticisi:
    pass

class OgrenciKayit:
    pass

# Sabitler: UPPER_SNAKE_CASE
MAX_BAGLANTI = 100
API_ANAHTARI = "gizli-anahtar"
PI_DEGERI = 3.14159
```

```
# Private (özel) üyeler: önünde _
class Hesap:
    def __init__(self):
        self._bakiye = 0 # Private
        self.__hesap_no = "12345" # Strongly private
```

Modülleri İçer Aktarma Düzeni

İlk önce standart kütüphaneler, sonra üçüncü parti kütüphaneler, en son kendi modüllerimizi içer aktarmalıyız (import);

```
# 1. Standart kütüphane
import os
import sys
from datetime import datetime

# 2. Üçüncü parti kütüphaneler
import numpy as np
import pandas as pd
from fastapi import FastAPI

# 3. Kendi modülleriniz
from myapp.models import Kullanici
from myapp.utils import hesapla_toplam
```

Boşluklar

```
# DOĞRU
x = 5
y = x + 10
liste = [1, 2, 3]
sozluk = {"a": 1, "b": 2}

def fonksiyon(a, b, c):
    return a + b + c

# YANLIŞ
x=5
y=x+10
liste=[1,2,3]
sozluk={"a":1,"b":2}

def fonksiyon(a,b,c):
    return a+b+c
```

Dokümantasyon Metinleri

Fonksiyon, sınıf ve modüllerin dokümantasyonu için docstring kullanılmalıdır:

```
def toplam_hesapla(sayilar: list[int]) -> int:
    """
    Verilen sayıların toplamını hesaplar.

    Args:
        sayilar (list[int]): Toplanacak sayılar listesi

    Returns:
        int: Sayıların toplamı

    Raises:
        ValueError: Liste boşsa

    Examples:
        >>> toplam_hesapla([1, 2, 3])
        6
        >>> toplam_hesapla([10, 20])
```

```

30
"""
if not sayilar:
    raise ValueError("Liste boş olamaz")
return sum(sayilar)

class Kullanici:
    """
    Kullanıcı bilgilerini yöneten sınıf.

    Attributes:
    isim (str): Kullanıcının adı
    email (str): Kullanıcının email adresi
    yas (int): Kullanıcının yaşı
    """

    def __init__(self, isim: str, email: str, yas: int):
        self.isim = isim
        self.email = email
        self.yas = yas

```

Veri Tipi İpuçlarını Kullanın

Kod okunabilirliği ve IDE desteği için veri tipi ipuçlarını (**type hint**) kullanın:

```
from typing import List, Callable, Optional, Dict, Union
```

```

def filtrele(
    sayilar: List[int],
    kosul: Callable[[int], bool]
) -> List[int]:
    """Verilen koşulu sağlayan sayıları filtreler."""
    return [s for s in sayilar if kosul(s)]

def kullanıcı_bul(
    kullanıcı_id: int
) -> Optional[Dict[str, Union[str, int]]]:
    """Kullanıcı bulunursa dict, bulunamazsa None döner."""
    if kullanıcı_id > 0:
        return {"id": kullanıcı_id, "isim": "Ali"}
    return None

```

Python 3.9+ standartları doğrultusunda doğrudan `list[int]` veya `dict[str, int]` biçiminde dahili (**built-in**) kullanıma geçilmiştir.

16.2 SOLID İlkeleri

S - Single Responsibility (Tek Sorumluluk)

Her sınıf veya fonksiyon tek bir işten sorumlu olmalı:

YANLIŞ - Birden fazla sorumluluk

```

class Kullanici:
    def __init__(self, isim, email):
        self.isim = isim
        self.email = email

    def kaydet(self):
        # Veritabanına kaydet
        pass

    def email_gonder(self):
        # Email gönder
        pass

    def rapor_olustur(self):

```

```
# Rapor oluştur
pass

# DOĞRU - Her sınıf tek bir işten sorumlu
class Kullanici:
    def __init__(self, isim, email):
        self.isim = isim
        self.email = email

class KullaniciRepository:
    def kaydet(self, kullanıcı: Kullanici):
        # Veritabanına kaydet
        pass

class EmailServisi:
    def gonder(self, alici: str, mesaj: str):
        # Email gönder
        pass

class RaporServisi:
    def olustur(self, kullanıcı: Kullanici):
        # Rapor oluştur
        pass
```

O - Open/Closed (Açık/Kapalı)

Genişletmeye açık, değişikliğe kapalı olmalı:

YANLIŞ - Yeni şekil eklemek için sınıfı değiştirmek gerekir

```
class AlanHesaplama:
    def hesapla(self, sekil):
        if sekil.tip == "daire":
            return 3.14 * sekil.yaricap ** 2
        elif sekil.tip == "kare":
            return sekil.kenar ** 2
```

DOĞRU - Abstract base class kullanımı

```
from abc import ABC, abstractmethod
```

```
class Sekil(ABC):
    @abstractmethod
    def alan_hesapla(self) -> float:
        pass

class Daire(Sekil):
    def __init__(self, yaricap: float):
        self.yaricap = yaricap

    def alan_hesapla(self) -> float:
        return 3.14 * self.yaricap ** 2

class Kare(Sekil):
    def __init__(self, kenar: float):
        self.kenar = kenar

    def alan_hesapla(self) -> float:
        return self.kenar ** 2

# Yeni şekil eklemek kolay:
class Ucgen(Sekil):
    def __init__(self, taban: float, yukseklik: float):
        self.taban = taban
        self.yukseklik = yukseklik
```

```
def alan_hesapla(self) -> float:
    return (self.taban * self.yukseklık) / 2
```

L - Liskov Substitution (Liskov İkame)

Alt sınıf, üst sınıfın yerine kullanılabilir:

DOĞRU

```
class Kus:
    def ucabilir_mi(self) -> bool:
        return True

class Serçe(Kus):
    pass # Uçabilir

class Penguen(Kus):
    def ucabilir_mi(self) -> bool:
        return False # Uçamaz ama davranış tutarlı

# Kullanım
def kus_ucur(kus: Kus):
    if kus.ucabilir_mi():
        print(f"{kus.__class__.__name__} uçuyor")
    else:
        print(f"{kus.__class__.__name__} uçamıyor")
```

I - Interface Segregation (Arayüz Ayrımı)

Sınıflar kullanmadıkları yöntemleri kodlamaya zorlanmamalı:

YANLIŞ

```
class Isci(ABC):
    @abstractmethod
    def calis(self): pass

    @abstractmethod
    def yemek_ye(self): pass

    @abstractmethod
    def uyku_mo_ver(self): pass

class Robot(Isci): # Robot yemek yemez ve uyumaz!
    def calis(self): pass
    def yemek_ye(self): pass # Gereksiz
    def uyku_mo_ver(self): pass # Gereksiz
```

DOĞRU - Küçük interface'ler

```
class Calisabilir(ABC):
    @abstractmethod
    def calis(self): pass

class Beslenebilir(ABC):
    @abstractmethod
    def yemek_ye(self): pass

class Dinlenebilir(ABC):
    @abstractmethod
    def uyku_mo_ver(self): pass

class Insan(Calisabilir, Beslenebilir, Dinlenebilir):
    def calis(self): pass
    def yemek_ye(self): pass
    def uyku_mo_ver(self): pass
```



```
class Robot(Calisabilir):
    def calis(self): pass
```

D - Dependency Inversion

Bağımlılık tersine çevirme olarak bilinen bu prensip üst seviye modüller alt seviye modüllere bağımlı olmamasını ister:

```
# YANLIŞ - Doğrudan bağımlılık
class MySQLVeritabani:
    def kaydet(self, veri):
        print("MySQL'e kaydedildi")

class KullaniciServisi:
    def __init__(self):
        self.db = MySQLVeritabani() # Sıkı bağlantı

    def kullanıcı_ekle(self, kullanıcı):
        self.db.kaydet(kullanıcı)

# DOĞRU - Abstraction kullanımı
class Veritabani(ABC):
    @abstractmethod
    def kaydet(self, veri): pass

class MySQLVeritabani(Veritabani):
    def kaydet(self, veri):
        print("MySQL'e kaydedildi")

class PostgreSQLVeritabani(Veritabani):
    def kaydet(self, veri):
        print("PostgreSQL'e kaydedildi")

class KullaniciServisi:
    def __init__(self, veritabani: Veritabani):
        self.db = veritabani # Dependency injection

    def kullanıcı_ekle(self, kullanıcı):
        self.db.kaydet(kullanıcı)

# Kullanım
mysql_db = MySQLVeritabani()
servis = KullaniciServisi(mysql_db)

postgres_db = PostgreSQLVeritabani()
servis2 = KullaniciServisi(postgres_db)
```

16.3 Tekrarlardan Kaçınma İlkesi

Bu ilke şu şekilde ifade edilir: "Her bilgi parçasının bir sistem içinde tek, açık ve güvenilir bir temsili olmalıdır". Yani aynı görevi gören kod tekrarından kaçın (Don't Repeat Yourself-DRY):

```
# YANLIŞ - Kod tekrarı
def kullanıcı_ekle(isim, email):
    if not isim:
        print("Hata: İsim boş olamaz")
        return False
    if not email:
        print("Hata: Email boş olamaz")
        return False
    if "@" not in email:
        print("Hata: Geçersiz email")
        return False
    # Kullanıcı ekle
    return True
```

```
def urun_ekle(isim, fiyat):
    if not isim:
        print("Hata: İsim boş olamaz")
        return False
    if fiyat < 0:
        print("Hata: Fiyat negatif olamaz")
        return False
    # Ürün ekle
    return True

# DOĞRU - Validation fonksiyonu
def dogrula(deger, hata_mesaji):
    if not deger:
        print(f"Hata: {hata_mesaji}")
        return False
    return True

def kullanıcı_ekle(isim, email):
    if not dogrula(isim, "İsim boş olamaz"):
        return False
    if not dogrula(email, "Email boş olamaz"):
        return False
    if "@" not in email:
        print("Hata: Geçersiz email")
        return False
    # Kullanıcı ekle
    return True
```

16.4 KISS İlkesi (Keep It Simple, Stupid)

Bu tasarım ve geliştirme ilkesi, sistemlerin, ürünlerin ve süreçlerin gereksiz karmaşıklıktan kaçınılarak basit tutulması gerektiğini savunur. Basit tutun, gereksiz karmaşıklıktan kaçın:

```
# YANLIŞ - Gereksiz karmaşık
def cift_mi(sayi):
    if sayi % 2 == 0:
        return True
    else:
        return False
```

```
# DOĞRU - Basit
def cift_mi(sayi):
    return sayi % 2 == 0
```

16.5 Code Smell'ler ve Çözümleri

Long Method (Uzun Fonksiyon)

```
# YANLIŞ
def siparis_isle(siparis):
    # 100+ satır kod
    # Stok kontrolü
    # Fiyat hesaplama
    # Kargo hesaplama
    # Email gönderme
    # Veritabanı kayıt
    pass

# DOĞRU
def siparis_isle(siparis):
    stok_kontrol(siparis)
    fiyat = fiyat_hesapla(siparis)
    kargo = kargo_hesapla(siparis)
```

```
email_gonder(siparis)
veritabanina_kaydet(siparis)
```

Magic Numbers (Sihirli Sayılar)

```
# YANLIŞ
def indirim_hesapla(fiyat):
    if fiyat > 1000:
        return fiyat * 0.15
    elif fiyat > 500:
        return fiyat * 0.10
    return fiyat * 0.05

# DOĞRU
YUKSEK_INDIRIM_SINIRI = 1000
ORTA_INDIRIM_SINIRI = 500
YUKSEK_INDIRIM_ORANI = 0.15
ORTA_INDIRIM_ORANI = 0.10
DUSUK_INDIRIM_ORANI = 0.05

def indirim_hesapla(fiyat):
    if fiyat > YUKSEK_INDIRIM_SINIRI:
        return fiyat * YUKSEK_INDIRIM_ORANI
    elif fiyat > ORTA_INDIRIM_SINIRI:
        return fiyat * ORTA_INDIRIM_ORANI
    return fiyat * DUSUK_INDIRIM_ORANI
```

16.6 Kod Kalitesi Araçları

Black Kod biçimlendirme aracı:

```
pip install black
black dosya.py
```

Pylint Kod analiz aracı:

```
pip install pylint
pylint dosya.py
```

mypy Tip kontrolü aracı:

```
pip install mypy
mypy dosya.py
```

ruff , Rust ile yazılmış son derece hızlı bir Python linter ve kod biçimlendiricidir (**formatter**):

```
pip install ruff
ruff check dosya.py
```

16.7 İz Kayıtları Oluşturma (Logging)

İz kaydı (**log**) oluşturmak için **print()** yerine **logging** kullanın:

```
import logging

# Logging ayarları
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

def kullanıcı_ekle(isim, email):
    logger.info(f"Yeni kullanıcı ekleniyor: {isim}")
    try:
        # Kullanıcı ekle
        logger.debug(f"Email doğrulanıyor: {email}")
        # ...
        logger.info("Kullanıcı başarıyla eklendi")
        return True
    except Exception as e:
```

```
logger.error(f"Kullanıcı eklenirken hata: {str(e)}")
return False
```

16.8 Konfigürasyon Yönetimi

```
# config.py
import os
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
class Config:
    DEBUG = os.getenv("DEBUG", "False") == "True"
    DATABASE_URL = os.getenv("DATABASE_URL")
    SECRET_KEY = os.getenv("SECRET_KEY")
    MAX_BAGLANTI = int(os.getenv("MAX_BAGLANTI", "100"))

    Konfigürasyona yönelik tüm bilgiler .env dosyasında olmalıdır:
```

```
# .env dosyası
DEBUG=True
DATABASE_URL=postgresql://user:pass@localhost/db
SECRET_KEY=gizli-anahtar
DB_PASSWORD=Password123
MAX_BAGLANTI=50
```

Güvenlikle ilgili kullanım farkı:

```
# YANLIŞ Kullanım:
password = "12345" # Kodda hardcoded
sql = f"SELECT * FROM users WHERE id = {user_id}" # SQL injection
```

```
# GÜVENLİ Kullanım:
import os
password = os.getenv('DB_PASSWORD')
# Parametrelili query
cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))
```

16.9 Bağlam Yöneticisi

Herhangi bir programlama dilinde, dosya işlemleri veya veri tabanı bağlantıları gibi kaynakların kullanımı çok yaygındır, ancak sınırlıdır. Bu nedenle, asıl sorun, bu kaynakların kullanımdan sonra serbest bırakılmasını sağlamaktır. Serbest bırakılmazlarsa, kaynak sızıntısına yol açar ve sistemin yavaşlamasına veya çökmesine neden olabilir.

Python'ın **bağlam yöneticileri** (**context manager**), kaynakları otomatik olarak kurmak ve temizlemek için düzgün bir yol sağlar ve hatalar oluşsa bile düzgün bir şekilde yönetilmelerini sağlar.

Bağlam Yöneticilerine Neden İhtiyacımız Var?

- ◇ **Otomatik temizleme:** Dosyalar veya bağlantılar gibi kaynakları manuel çaba gerektirmeden serbest bırakır.
- ◇ **Kaynak sızıntısını önleme:** Hatalar oluşsa bile düzgün serbest bırakmayı sağlar.
- ◇ **Daha temiz kod:** Ayrıntılı try-finally bloklarını ile değiştirir.
- ◇ **İstisna güvenli:** İstisnalarda bile her zaman temizleme işlemi gerçekleştirir.
- ◇ **Özel kontrol:** `__enter__` ve `__exit__` kullanarak herhangi bir kaynağı yönetmenizi sağlar.

```
# YANLIŞ
dosya = open("veri.txt", "r")
icerik = dosya.read()
dosya.close() # Hata olursa kapatılmayabilir!
```

```
# DOĞRU
with open("veri.txt", "r") as dosya:
    icerik = dosya.read()
# Otomatik kapatılır
```

```
# Kendi context manager'iniz
from contextlib import contextmanager

@contextmanager
def veritabani_baglanti():
    baglanti = veritabani_baglan()
    try:
        yield baglanti
    finally:
        baglanti.kapat()

with veritabani_baglanti() as db:
    db.sorgu_calistir("SELECT * FROM users")
```

16.10 Liste veya Sözlük Kavrama

Kavramalar (**comprehension**) gereksiz uzun yazılmamalıdır:

```
# YANLIŞ - Gereksiz uzun
kareler = []
for i in range(10):
    kareler.append(i ** 2)

# DOĞRU - Kısa ve öz
kareler = [i ** 2 for i in range(10)]

# Dict comprehension
sozluk = {i: i ** 2 for i in range(10)}

# Koşullu
cift_kareler = [i ** 2 for i in range(10) if i % 2 == 0]
```

16.11 En İyi Uygulamalar Özeti

- ◊ PEP 8'e uyun - Kod stili tutarlı olsun.
- ◊ Type hints kullanın - Kod okunabilirliği artar.
- ◊ Docstring yazın - Her fonksiyon dokümente edilmeli.
- ◊ DRY prensibine uyun - Kod tekrarından kaçının.
- ◊ SOLID prensiplerini uygulayın - Sürdürülebilir kod yazın.
- ◊ Test yazın - Her fonksiyon test edilmeli.
- ◊ Logging kullanın - `print()` yerine `logging`.
- ◊ Environment variables kullanın - Hassas bilgileri kodda saklamayın.
- ◊ Virtual environment kullanın - Proje bağımlılıklarını izole edin.
- ◊ Git kullanın - Versiyon kontrolü yapın.
- ◊ Code review yapın - Kodunuzu başkalarına inceletin.
- ◊ Linter ve formatter kullanın - Kod kalitesini artırın.
- ◊ CI/CD pipeline kurun - Otomatik test ve deployment.
- ◊ Exception handling yapın - Hataları yakalayın.
- ◊ Security best practices uygulayın - Güvenlik açıklarından kaçının.

16.12 Düşün ve Çöz

1. **Düşün (Liskov İkame İlkesi ve Kalıtım Problemleri):** Nesne yönelimli programlamada klasik bir örnek olan `Kare` sınıfının `Dikdortgen` sınıfından türetilmesi (**inheritance**) neden Liskov İkame İlkesini (**LSP**) ihlal eder? En-boy oranının korunması zorunluluğunun üst sınıfın davranışını nasıl bozduğunu ve bu sorunun bileşim (**composition**) ile nasıl çözülebileceğini tartışınız.
2. **Çöz (Refactoring & Code Smells):** Aşağıdaki kötü yazılmış (**smelly**) fonksiyonu inceleyiniz:

```
def hesapla(u_tipi, tutar):
    if u_tipi == 1:
        return tutar - (tutar * 0.05)
    elif u_tipi == 2:
        return tutar - (tutar * 0.10)
    elif u_tipi == 3:
```

```
    return tutar - (tutar * 0.20)
else:
    return tutar
```

Bu kodu Sihirli Sayılardan (**Magic Numbers**) arındırarak, Açık/kapalı (**open/closed**) ilkesine uygun (örneğin Strateji Tasarım Kalıbı veya **Enum** kullanarak) yeniden yapılandırınız (*refactor*).

3. **Çöz (Kendi Context Manager Class'ınızı Yazın):** Sadece `@contextmanager` dekoratörünü değil, `__enter__` ve `__exit__` sihirli metotlarını içeren bir `ZamanOlcucu` (*Timer*) sınıfı yazınız. Bu bağlam yöneticisi `with ZamanOlcucu():` bloğu içinde çalışan kodun toplam çalışma süresini saniye cinsinden hesaplayıp konsola yazdırsın.
4. **Düşün (Print vs Logging ve Ortam Değişkenleri):** Üretim ortamındaki (*production*) bir uygulamada `print()` yerine `logging` modülü kullanmanın; log seviyeleri (*INFO, ERROR, DEBUG*), izlenebilirlik ve hassas verilerin sızdırılmaması açısından sağladığı avantajları değerlendiriniz.

Dizin

- abstract class, 95
- abstraction, 84, 90
- abstraction ladder, 19
- access modifier, 89, 91
- actual parameter, 16
- annotation, 73
- arbitrary argument, 72
- argument, 16
- arithmetic operators, 43
- array, 16, 101, 106, 124
- assembler, 7
- assembly code, 7
- assertion, 134
- assignment operator, 44
- asynchronous, 140, 143
- attribute, 27, 84
- authorization, 99

- base case, 76
- binary, 14, 15, 132
- BIOS, 9
- bit, 3
- bitwise operators, 46
- boolean data type, 43
- boolean operators, 45
- broadcasting, 129
- buffer, 117
- bus, 2
- byte, 3

- caching, 99
- call, 16
- callable, 97
- camelCase, 29
- casting, 42
- class, 84
- class attribute, 85
- class method, 88
- code block, 24
- collection, 132
- comments, 26
- compiler, 5, 19
- comprehension, 114, 177
- computer program, 7
- condition, 53, 54, 59
- conditional choice, 52, 53
- conditional code, 53, 54
- constant, 30
- constant variable, 30
- constructor, 85, 101
- context manager, 146, 176
- control abstraction, 90
- control flow, 12, 38, 52, 59
- control operation, 52, 54
- control statement, 59

- control structure, 17
- coroutine, 144
- coverage, 139
- cross-cutting concerns, 100
- Cross-Origin Resource Sharing, 163
- custom exception, 118

- data, 2
- data abstraction, 90
- data structure, 16, 101
- data type, 13, 15
- decode, 3, 4
- default argument, 71
- defensive programming, 136
- dependency injection, 159
- deserialization, 132
- dict, 111
- docstring, 27, 70
- Don't Repeat Yourself, 173
- duck typing, 94

- else code, 55
- empty product, 76
- encapsulation, 84, 91
- engineer, 1
- escape sequence, 108
- exception, 116, 117
- exception chaining, 118
- exception handling, 116
- execute, 3, 4
- explicit type conversion, 42
- expression, 38, 53
- Extensible Markup Language, 122

- fetch, 3, 4
- file descriptor, 121
- first-class object, 99
- fixture, 137, 138, 140
- fixture scope, 137
- floating point number, 14
- flowchart, 52
- formal argument, 71
- frozenset, 111
- function, 15
- function block, 24
- function call, 69

- garbage collector, 19

- hash, 109
- hashable, 109, 111
- heap, 73
- high level programming language, 11

- higher-order functions, 99

- identifier, 28
- identity operators, 48
- if statement, 53
- if-else statement, 55
- immutable, 108, 111, 132
- imperative programming, 17
- implicit type conversion, 41
- import, 68, 69
- indentation, 26, 70
- index, 101
- infinity, 124
- information, 2
- inheritance, 84, 92, 118
- initialization, 30
- inner class, 95
- input-output operation, 38
- instance, 84
- instance attribute, 85
- instance method, 88
- instruction, 2, 19
- instruction code, 4, 7
- instruction cycle, 4
- instruction set, 4
- integrated circuit, 3
- interface, 117
- interpreter, 19, 20, 122
- introspection, 96
- invoke function, 69
- iteration, 76

- JavaScript Object Notation, 122, 132
- jump, 52, 63

- key-value pair, 111
- keyword argument, 71
- keywords, 25

- lambda expression, 80
- Last In First Out, 104
- lazy evaluation, 132
- linked list, 17
- list, 104
- list comprehension, 80, 105
- literal, 30, 106
- local variable, 73
- log, 116
- logging, 99
- logical sequence, 11, 52, 59
- loop, 76
- loop code, 62
- low level programming, 10, 17

- machine code, 4, 19

- machine language, 7
- magic method, 88
- main function, 16, 24
- match, 57
- membership operators, 47
- memory, 2
- method, 84, 88
- method overloading, 93
- method resolution order, 93
- mnemonic, 7
- mock, 138

- n-dimensional array, 125
- nested if, 54
- None, 28
- not a number, 124

- object, 84, 116
- object-oriented programming, 84
- opcode, 4
- operand, 4, 42, 43
- operating system, 9
- operator, 105
- operator precedence, 49
- output formatting, 39
- overriding, 89, 94

- parameter, 16
- pass-by-object-reference, 75
- pass-by-reference, 75
- pass-by-value, 75
- pattern, 57
- physical sequence, 52, 59
- placeholder, 109

- polymorphism, 84, 94
- positional argument, 71
- positional-only argument, 72
- primitive data type, 16, 27
- processor, 2
- producer-consumer pattern, 147
- programming, 7
- public, 84
- punch card, 10
- PVM, 21

- raising an exception, 117
- range, 61
- recursion, 76
- recursive, 77
- reflection, 96
- relational loop, 52, 59
- REPL, 22
- reserved words, 25
- Resource Acquisition Is Initialization, 121
- return, 15

- safe state, 116
- script, 122
- sentinel value, 63
- sequential operation, 38, 52, 54
- serialization, 132
- session, 142
- set, 109
- shared code, 92
- sign, 14
- singleton class, 96

- slicing, 103
- snake_case, 29
- snap change, 92
- spaghetti code, 12
- special method, 85
- stack, 73, 104
- stack trace, 117
- standard library, 119
- state, 84
- statement, 11, 17, 20, 25
- static method, 88
- string, 102, 106, 119
- structural programming, 16
- structure, 17
- syntactic sugar, 99
- syntax, 24

- technician, 1
- traceback, 116, 117
- tuple, 60, 106, 108
- type conversion, 41
- type hint, 170
- type hints, 159

- unique, 109
- unpacking, 109
- user defined function, 67, 69

- validation, 159
- variable, 13, 16, 101
- variable scope, 28, 73

- while statement, 59

- zip, 106